

Jan Glaubitz
Department of Mathematics (MAI)
Division of Applied Mathematics (TIMA)
Linköping University, Sweden
jan.glaubitz@liu.se

Lecture notes for the PhD course

Computational and Bayesian Inverse Problems

SeSE course, Linköping University, 26–30 October 2026

Version: October 22, 2026

Copyright © 2026 by Jan Glaubitz.
These notes are provided to the course participants for personal use.

Preface

These notes accompany the PhD course *Computational and Bayesian Inverse Problems*, given at Linköping University in the week of 26–30 October 2026 as part of the Swedish e-Science Education (SeSE) program. The course has five lectures, one per day, and the notes have one chapter per lecture:

1. Inverse problems: the basics (Monday),
2. Deterministic regularization (Tuesday),
3. Bayesian inverse problems and MAP estimation (Wednesday),
4. Uncertainty quantification by sampling (Thursday),
5. Beyond MCMC: Laplace approximation, ensemble methods, variational inference (Friday; optional modules).

Each chapter ends with exercises, of which at least one is computational and continues the two running examples introduced in the preliminary chapter (one-dimensional deconvolution and bathymetry inference from steady shallow-water flow). Appendix A is a probability refresher that participants are asked to read before the course; Appendix B describes the software used in the exercises (Python with NumPy, SciPy, and JAX).

Use of generative AI. Generative AI, specifically Claude (Anthropic), has been used to help prepare these lecture notes and the accompanying code, including drafting text, figures, exercises and their solutions, and code, as well as checking computations. All content has been reviewed by the author, who is responsible for any remaining errors.

Linköping, October 2026 Jan Glaubitz

Contents

Preface i

Notation and running examples v

1 Inverse Problems: The Basics 1
- 1.1 Examples of inverse problems . . . 2
- 1.2 Forward and inverse problems . . . 4
- 1.3 Ill-posedness . . . 5
  - 1.3.1 Hadamard's conditions . . . 6
  - 1.3.2 The continuous view: smoothing destroys information . . . 7
  - 1.3.3 The discrete view: ill-conditioning . . . 9
  - 1.3.4 Nonlinear problems . . . 10
- 1.4 What is a solution? . . . 12
- 1.5 Notes and further reading . . . 12
- 1.6 Exercises . . . 13

2 Deterministic Regularization 15
- 2.1 Variational regularization . . . 15
- 2.2 Linear problems: Tikhonov regularization . . . 17
- 2.3 Choosing the regularization parameter . . . 19
- 2.4 Sparsity: ℓ^1 regularization . . . 21
- 2.5 Nonlinear forward models . . . 23
- 2.6 What we still cannot answer . . . 25
- 2.7 Notes and further reading . . . 26
- 2.8 Exercises . . . 26

3 Bayesian Inverse Problems and MAP Estimation 28
- 3.1 Bayes' theorem and the posterior . . . 29
- 3.2 From noise models to likelihoods . . . 32
- 3.3 Priors: Gaussian, Laplace, and hierarchical . . . 34
  - 3.3.1 Gaussian priors . . . 34
  - 3.3.2 Priors on transforms of the unknown . . . 35
  - 3.3.3 Laplace priors . . . 36
  - 3.3.4 Hierarchical priors . . . 36

3.4 The linear-Gaussian case . . . 38
3.5 MAP estimation . . . 39
3.6 What the MAP estimate does not tell you . . . 44
3.7 Notes and further reading . . . 44
3.8 Exercises . . . 45

4 Uncertainty Quantification by Sampling 47
4.1 Why sample? Uncertainty quantification . . . 48
4.2 Markov chain Monte Carlo . . . 50
4.2.1 Metropolis–Hastings . . . 52
4.2.2 A map of samplers . . . 55
4.3 Gaussian posteriors: perturb-then-optimize . . . 56
4.4 Tractable conditionals: Gibbs sampling . . . 59
4.4.1 The hierarchical prior . . . 60
4.4.2 The Laplace prior as a hierarchical prior . . . 60
4.5 Everything else: Hamiltonian Monte Carlo . . . 63
4.6 Diagnostics and verification . . . 69
4.7 What sampling costs . . . 72
4.8 Notes and further reading . . . 73
4.9 Exercises . . . 73

5 Beyond MCMC: Approximate Bayesian Inference 76
5.1 Why approximate? . . . 77
5.2 The Laplace approximation . . . 78
5.2.1 When the Laplace approximation fails . . . 80
5.2.2 Using the Laplace approximation . . . 82
5.3 Variational inference . . . 82
5.3.1 Gaussian families . . . 83
5.3.2 The reparametrization trick . . . 85
5.3.3 Variational Bayes for the hierarchical prior . . . 87
5.4 Ensemble Kalman inversion . . . 89
5.5 Comparison . . . 93
5.6 Outlook: what to read next . . . 94
5.7 Notes and further reading . . . 95
5.8 Exercises . . . 96

A Probability Refresher (Pre-reading) 98
A.1 Random variables, densities, and expectations . . . 98
A.2 Conditioning and Bayes' theorem . . . 99
A.3 The multivariate normal distribution . . . 100
A.4 A catalogue of distributions used in the course . . . 101
A.5 Monte Carlo basics . . . 102
A.6 Exercises . . . 103
A.7 Solutions . . . 104

B Software for the Exercises: NumPy and JAX **105**
B.1 Installation and the course code . . . 105
B.2 A short JAX primer . . . 106
B.3 Running the samplers . . . 107
B.4 Pitfalls . . . 109

C Solutions to the Exercises **110**
C.1 Inverse problems: the basics . . . 110
C.2 Deterministic regularization . . . 114
C.3 Bayesian inverse problems and MAP estimation . . . 117
C.4 Uncertainty quantification by sampling . . . 121
C.5 Beyond MCMC: approximate Bayesian inference . . . 128

Bibliography **134**

Notation and running examples

This preliminary chapter collects the notation of the notes and describes the two running examples on which every chapter tests its methods. Skim the tables now and return to them when a symbol is unclear; read the running examples before or together with Chapter 1. Table 0.3 lists where the chapters use the running examples, and the exercises of every chapter continue them.

Notation

Conventions. Vectors are bold lower case ($\boldsymbol{x}$, $\boldsymbol{y}$, $\boldsymbol{\varepsilon}$), matrices bold upper case ($\boldsymbol{A}$, $\boldsymbol{L}$, $\boldsymbol{\Gamma}$), and their entries are not bold (x_i, A_{ij}); functions of a continuous variable, such as a signal $x(t)$ or a river bed $b(x)$, are in italics. $\mathbf{0}$ and $\mathbf{1}$ are the vectors of zeros and ones, $\boldsymbol{I}$ is the identity, $\boldsymbol{A}^\mathsf{T}$ the transpose, $\|\cdot\|_2$ the Euclidean and $\|\cdot\|_1$ the ℓ^1 norm, and log the natural logarithm. We write p for every density and let the argument say which one is meant: $p(\boldsymbol{x})$ is the prior, $p(\boldsymbol{y} \mid \boldsymbol{x})$ the likelihood, and $p(\boldsymbol{x} \mid \boldsymbol{y})$ the posterior. $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma})$ says that $\boldsymbol{x}$ has this distribution, and $\mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Gamma})$ is its density at $\boldsymbol{x}$. $f \propto g$ means $f = c\,g$ with a constant $c > 0$ that does not depend on the variable of interest, typically an unknown normalizing constant; "const" in a formula is such a constant, and after taking logarithms it is additive. A *noise level* is always a standard deviation, never a variance. Tables 0.1 and 0.2 list the symbols; the last column points to the place where each is introduced.

Table 0.1: Inverse problems and regularization (Chapters 1 and 2).

symbol	meaning	see
$\boldsymbol{x} \in \mathbb{R}^n$, x_i	the unknown and its components; n is the number of unknowns	Definition 1.4
$\boldsymbol{y} \in \mathbb{R}^m$	the data; m is the number of data	Definition 1.4
$F\colon \mathbb{R}^n \to \mathbb{R}^m$	forward operator, in general nonlinear	Definition 1.4
$\boldsymbol{A} \in \mathbb{R}^{m\times n}$	matrix of a linear forward operator, $F(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}$	Definition 1.4
$\boldsymbol{\varepsilon}$	measurement noise in the data model $\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{\varepsilon}$	(1.5)
$\boldsymbol{\Gamma}$; σ	noise covariance; noise level, with $\boldsymbol{\Gamma} = \sigma^2\boldsymbol{I}$ for independent data	(3.6)
$\boldsymbol{x}^\dagger$	the true unknown, from which synthetic data are generated	Section 2.3
$\boldsymbol{A}^\dagger$	pseudo-inverse; $\boldsymbol{A}^\dagger\boldsymbol{y}$ is the least-squares solution with minimal norm	Section 1.3
$\sigma_1 \geq \sigma_2 \geq \ldots$; $\kappa(\boldsymbol{A})$	singular values; condition number σ_1/σ_n (or σ_1/σ_r for rank r)	Lemma 1.10
$\boldsymbol{J} = \boldsymbol{J}_F(\boldsymbol{x})$	Jacobian $\partial F/\partial \boldsymbol{x} \in \mathbb{R}^{m\times n}$ of the forward operator	(1.8)
R; λ	regularizer (penalty); regularization parameter	Definition 2.1
J_λ; $\boldsymbol{x}_\lambda$	regularized functional $\frac{1}{2}\|F(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \lambda R(\boldsymbol{x})$; its minimizer	Definition 2.1
$\boldsymbol{L} \in \mathbb{R}^{(n-1)\times n}$	difference matrix, $(\boldsymbol{L}\boldsymbol{x})_i = x_{i+1} - x_i$, the increments of $\boldsymbol{x}$	Section 2.1

Table 0.2: Bayesian inference, sampling, and approximation (Chapters 3 to 5 and Appendix A).

symbol	meaning	see
$p(\boldsymbol{x})$; $p(\boldsymbol{y} \mid \boldsymbol{x})$	prior; likelihood	Definition 3.2
$p(\boldsymbol{x} \mid \boldsymbol{y})$; $p(\boldsymbol{y})$	posterior; evidence	Definition 3.2
$U(\boldsymbol{x})$	potential $-\log p(\boldsymbol{y} \mid \boldsymbol{x}) - \log p(\boldsymbol{x})$, the negative log-posterior up to a constant	(4.1)
$\boldsymbol{x}_{\mathrm{MAP}}$; $\boldsymbol{x}_{\mathrm{CM}}$	MAP estimate (minimizer of U); posterior or conditional mean $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}]$	Definition 3.3
$\boldsymbol{\Gamma}_{\mathrm{post}}$; credible interval	posterior covariance; interval holding 95 % of a marginal posterior (band: all components)	Definition 3.3
$\boldsymbol{\mu}$, $\boldsymbol{\Gamma}_{\mathrm{prior}}$; $\boldsymbol{\mu}_{\mathrm{post}}$	prior mean and covariance; mean of a Gaussian posterior	Theorem 3.10
$\boldsymbol{H}$	Hessian of U, the precision of a Gaussian posterior or of its approximation	Definition 5.1
γ	scale of a Gaussian or Laplace prior on $\boldsymbol{x}$ or on its increments $\boldsymbol{L}\boldsymbol{x}$	Section 3.3
$\boldsymbol{\theta} = (\theta_i)$; β, ϑ	local variances of the hierarchical prior; shape and scale of their hyper-prior $\mathrm{InvGamma}(\beta, \vartheta)$, with $\beta = 1$, $\vartheta = 10^{-3}$ throughout	(3.10)
$\boldsymbol{x}^{(k)}$; N; $\widehat{f}_N$	k-th sample or state of a chain; number of samples; Monte Carlo estimate of $\mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$	Definition 4.1
τ_f; N_{eff} (ESS)	integrated autocorrelation time; effective sample size N/τ_f	Definition 4.5
$\widehat{R}$	potential scale reduction factor of several chains	(4.19)
$\boldsymbol{r}$, $\boldsymbol{M}$, h, L	momentum, mass matrix, step size, and number of leapfrog steps of HMC	Algorithm 4.16
q, $\mathcal{Q}$; q_{L}	approximating density and its family; Laplace approximation $\mathcal{N}(\boldsymbol{x}_{\mathrm{MAP}}, \boldsymbol{H}^{-1})$	Definition 5.5
$\mathrm{KL}(q \,\|\, p)$; $\mathrm{ELBO}(q)$	Kullback–Leibler divergence; evidence lower bound	Definition 5.5
$\mathbb{E}$, Var, Cov, $\mathbb{P}$	expectation, variance, covariance matrix, probability; $\mathbb{E}_q$ under q, $\mathbb{E}[\cdot \mid \boldsymbol{y}]$ under the posterior	Appendix A.1
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma})$	normal distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Gamma}$	(3.2)
$\mathrm{Laplace}(0, \gamma)$, $\mathrm{Cauchy}(0, \gamma)$	Laplace and Cauchy distributions with scale γ	Table A.1
$\mathrm{Gamma}(\beta, \vartheta)$, $\mathrm{InvGamma}(\beta, \vartheta)$, $\mathrm{GG}(r, \beta, \vartheta)$	gamma, inverse gamma, and generalized gamma distributions with shape β and scale ϑ	Table A.1
$\mathcal{U}(0, 1)$	uniform distribution on $[0, 1]$	Table A.1

Letters with two meanings. Following the conventions of the fields they come from, a few letters have two meanings, which the context separates: σ is the noise level and σ_i a singular value; γ is the width of the deconvolution kernel and the scale of a prior; x is the unknown and, in the bathymetry example, the position along the channel, whose unknown is $\boldsymbol{b}$; h is the water depth and the step size of HMC; L is the length of the channel and the number of leapfrog steps, while $\boldsymbol{L}$ is the difference matrix; J is a functional (J_λ) or the number of chains or ensemble members, and $\boldsymbol{J}$ a Jacobian; κ is the condition number and, in Remark 3.16, a conductivity.

Running example 1: one-dimensional deconvolution

Setting. A signal $x\colon [0,1] \to \mathbb{R}$ is observed through an instrument that averages it with a Gaussian kernel k of width $\gamma = 0.03$, and the record carries additive noise (Example 1.1). The model (1.1) takes the signal to vanish outside $[0,1]$. The forward operator is linear and smoothing, and the inverse problem is ill-posed in the sense of stability: the blur damps the component of frequency ν by the factor $\mathrm{e}^{-2\pi^2\gamma^2\nu^2}$ (Section 1.3.2).

Discretization. The midpoint rule with $n = 128$ cells of width $1/n$, cell centers $t_j = (j - \frac{1}{2})/n$, and $m = 64$ observation points $s_i = (i - \frac{1}{2})/m$ gives the linear model $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{\varepsilon}$ with $A_{ij} = k(s_i - t_j)/n$ (1.2). Figure 0.1 shows what the matrix does. It is dense, but its entries decay like the kernel away from the diagonal $s_i = t_j$ and fall below $3.5 \cdot 10^{-5}$ where $|s_i - t_j| > 4\gamma$. A *row* holds the weights with which one datum averages the signal over a window of about $\pm 3\gamma$: in the interior they sum to one, so that y_i is a local weighted mean of $\boldsymbol{x}$ plus noise, while at the ends of the interval the window is cut off and the weights of the first row sum to 0.60. A *column* holds the data that a unit spike in one cell produces, i.e., the kernel sampled at the observation points. The matrix has full row rank 64, a null space of dimension 64, and the condition number $\kappa(\boldsymbol{A}) = 3.5 \cdot 10^7$ (Section 1.3.3).

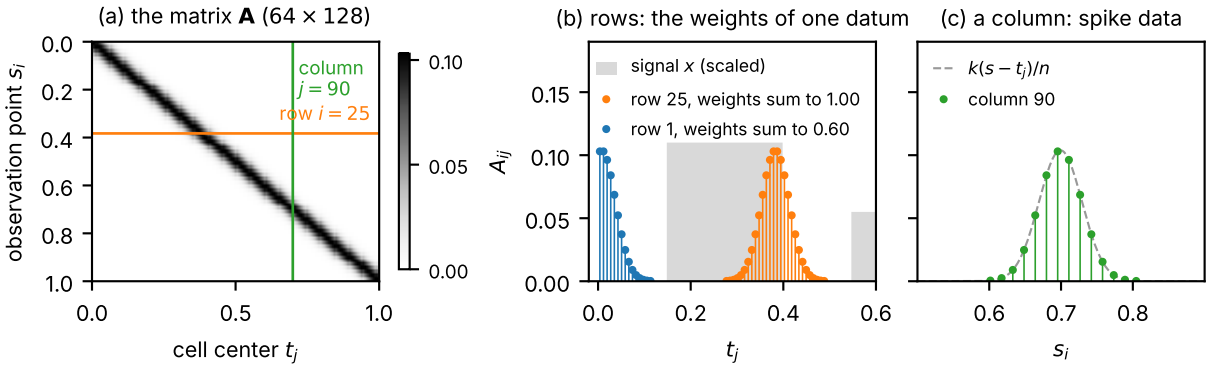


Figure 0.1: What the deconvolution matrix $\boldsymbol{A}$ of running example 1 does. (a) The 64×128 matrix as an image: a band of width about $\pm 3\gamma$ around the diagonal, with one row (orange) and one column (green) marked. (b) Two rows over the cell centers t_j (entries below 10^{-4} not drawn), with the piecewise-constant signal in gray: datum 25 ($s_{25} = 0.383$) averages the signal over a window that straddles an edge; the window of datum 1 is cut off at $t = 0$. (c) One column over the observation points s_i: the data of a unit spike at $t_{90} = 0.699$ are the kernel, scaled by $1/n$ and sampled at 64 points. Figure 1.2 shows the test signals and their data.

Noise and test signals. The noise is white and Gaussian, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$, with $\sigma = 0.05$ unless stated otherwise, i.e., five percent of the maximum of either test signal (Figure 1.2). The *smooth* signal $x(t) = \mathrm{e}^{-((t-0.3)/0.08)^2} + 0.6\,\mathrm{e}^{-((t-0.7)/0.12)^2}$ has two bumps, of heights 1 and 0.6, and the norm $\|\boldsymbol{x}\|_2 = 4.45$. The *piecewise-constant* signal is a box of height 1 on $(0.15, 0.4]$ followed by a block of height 0.5 on $(0.55, 0.65]$ and 0.8 on $(0.65, 0.85]$; its five jumps have the heights 1, -1, 0.5, 0.3, and -0.8. The synthetic data are computed with the same matrix $\boldsymbol{A}$ that is used for the inversion, which is the inverse crime of Remark 3.5, committed knowingly. In code, with `numpy` imported as `np`:

```
from running_examples import deconvolution_operator, signal_smooth, noisy_data
A, t, s = deconvolution_operator()            # n = 128, m = 64, gamma = 0.03
y = noisy_data(A, signal_smooth(t), 0.05, np.random.default_rng(1))
```

Running example 2: bathymetry from steady shallow-water flow

Setting. Water flows steadily from left to right through a channel of length $L = 25\,\mathrm{m}$ (Figure 0.2). The discharge per unit width is $q = 4.42\,\mathrm{m}^2/\mathrm{s}$, and the depth at the outflow is held at $h_{\mathrm{out}} = 2\,\mathrm{m}$; this is a classical test case for shallow-water solvers [49]. The unknown is the

bottom $b(x)$, the river bed; the water above it has the depth $h(x)$, and the free surface $\eta = h + b$ is measured at m sensors (Example 1.3).

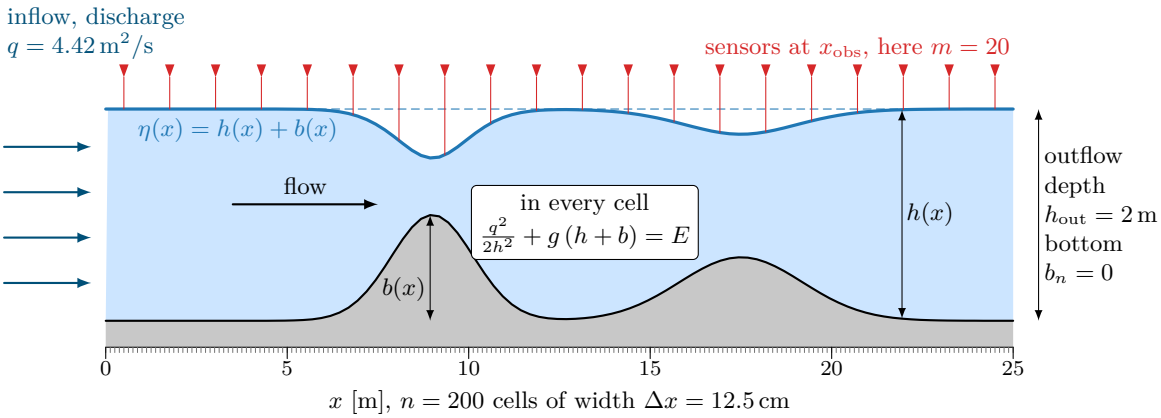


Figure 0.2: The channel of running example 2 with the smooth test bottom and the 200 cells (ticks); over the bumps the surface dips below the outflow level (dashed). Not to scale: the bottom and the dip, at most 20 and 9 cm, are drawn five times higher than the depth of 2 m.

Forward operator. For steady subcritical flow without friction the shallow-water equations reduce to Bernoulli's relation (1.4), $q^2/(2h^2) + g\,(h + b) = E$, with $g = 9.81\,\mathrm{m/s^2}$ and the energy $E = q^2/(2h_{\mathrm{out}}^2) + g\,(h_{\mathrm{out}} + b_n) = 22.06\,\mathrm{m^2/s^2}$ fixed by the outflow. In every cell the depth is therefore the largest root of the cubic $gh^3 + (gb - E)h^2 + q^2/2 = 0$, which the code computes with 40 Newton steps, and the surface at the sensors is obtained by linear interpolation of η between cell centers. The flow is subcritical if h exceeds the critical depth $(q^2/g)^{1/3} = 1.26\,\mathrm{m}$, and for both test bottoms the Froude number $\mathrm{Fr} = q/(h\sqrt{gh})$ lies between 0.50 at the outflow and 0.63 above the highest bump. A bottom that rises above 36 cm has no subcritical steady state, and the forward operator then returns `NaN`; this matters for draws from wide priors (Example 5.15). The map $\boldsymbol{b} \mapsto F(\boldsymbol{b})$ is nonlinear, cheap (a fraction of a millisecond), and local: the surface at a sensor depends only on the bottom in the two neighboring cells and on the outflow (Example 1.11).

Discretization, sensors, and noise. The channel is divided into $n = 200$ cells of width $\Delta x = 12.5\,\mathrm{cm}$ with centers $x_j = (j - \frac{1}{2})\Delta x$. Chapter 1 treats all 200 bottom values as unknown. From Chapter 2 on, the bottom in the outflow cell is the known datum $b_n = 0$, which fixes E and with it the level of the surface, and the unknown is $\boldsymbol{b} \in \mathbb{R}^{199}$. The sensors are equispaced between 0.5 and 24.5 m: $m = 20$ *sparse* sensors, 1.26 m apart, or $m = 200$ *dense* sensors, 12 cm apart, about one per cell. The noise is independent and Gaussian with $\sigma = 1\,\mathrm{cm}$. Dense sensors give a well-conditioned problem, sparse sensors an exact null space: the bottom between them is invisible (Example 1.11).

Test bottoms. The *smooth* bottom $b(x) = 0.20\,\mathrm{e}^{-((x-9)/1.6)^2} + 0.12\,\mathrm{e}^{-((x-17.5)/2.4)^2}\,\mathrm{m}$ has two bumps, of heights 20 and 12 cm. The *piecewise-constant* bottom has steps of 15 cm on $(6, 11]$, 6 cm on $(11, 14]$, and 20 cm on $(17, 20.5]$ m and is zero elsewhere. Both vanish at the outflow, and over their highest point the surface dips by 9.2 cm (Figure 0.3). The chapters use the smooth bottom; the piecewise-constant one, which default project C uses as well, is there for experiments with the edge-preserving priors of Chapter 3. In code, with the forward map in JAX (Appendix B):

```python
import swe_jax
from running_examples import make_grid, bottom_smooth
x, dx = make_grid(200)                          # cell centers, dx = 0.125 m
forward = swe_jax.make_forward(x, np.linspace(0.5, 24.5, 20))  # 20 sensors
b = bottom_smooth(x)[:-1]                       # the unknown: 199 values
y = forward(b) + 0.01 * np.random.default_rng(1).standard_normal(20)
```

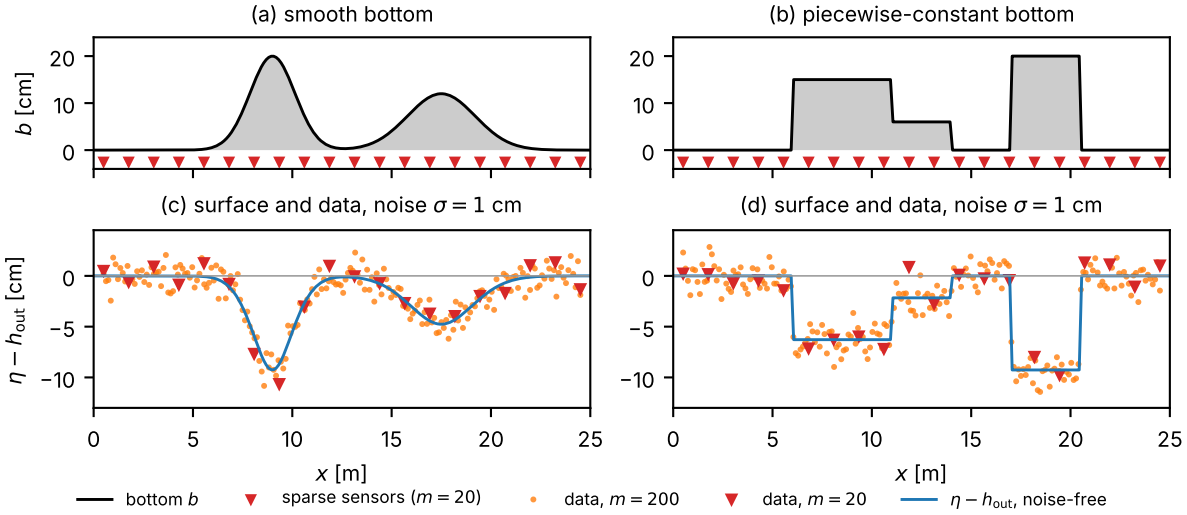


Figure 0.3: The two test bottoms of running example 2 and their data. (a, b) The bottoms and the positions of the 20 sparse sensors. (c, d) The deviation $\eta - h_{\text{out}}$ of the surface from the outflow level, noise-free and measured by 200 and 20 sensors: every rise of the bottom becomes a dip of a third to a half of its height (Figure 1.4 shows the flow).

Table 0.3: The running examples in the chapters.

chapter	deconvolution	bathymetry
Chapter 1	ill-posedness, the naive solution, singular values (Example 1.1)	nonlinearity, sensitivity, null space of sparse sensors (Examples 1.3 and 1.11)
Chapter 2	Tikhonov regularization, parameter choice, total variation (Sections 2.2 to 2.4)	Gauss–Newton with JAX Jacobians (Example 2.9)
Chapter 3	credible bands, the IAS for the hierarchical prior (Examples 3.12 and 3.15)	likelihood, linearized credible bands (Examples 3.7 and 3.12)
Chapter 4	random walk, perturb-then-optimize, Gibbs, diagnostics (Examples 4.8, 4.12, 4.14 and 4.20)	HMC and other samplers, choice of prior (Examples 4.18 and 4.19)
Chapter 5	variational inference, variational Bayes (Examples 5.9 and 5.11)	Laplace approximation, ensemble Kalman methods (Examples 5.4 and 5.15)

Software

All exercises use Python with NumPy and SciPy, and the nonlinear and sampling parts use JAX for automatic differentiation. The folder `code/` contains the two running examples, the solvers and samplers of the chapters, and the scripts that produce every figure, e.g., `ch00_figures.py` for this chapter; Appendix B explains how to install and use them.

Chapter 1

Inverse Problems: The Basics

Imagine standing in a corridor and hearing two people talk behind a closed door (Figure 1.1). The door muffles the voices: consonants disappear, the words blur into each other, and the hum of the ventilation adds a hiss on top. From what reaches your ear you try to work out what was said. This is an inverse problem. The *forward* direction—how a given sentence sounds after passing through the door—is governed by physics and is easy to simulate. The *inverse* direction—which sentence produced the sound you hear—is what you actually want, and it is hard for a reason that this chapter makes precise: the door has destroyed information, and no amount of cleverness on your side of it can bring that information back. What you *can* do is use what you know about language, and about who is in the room, to fill the gaps.

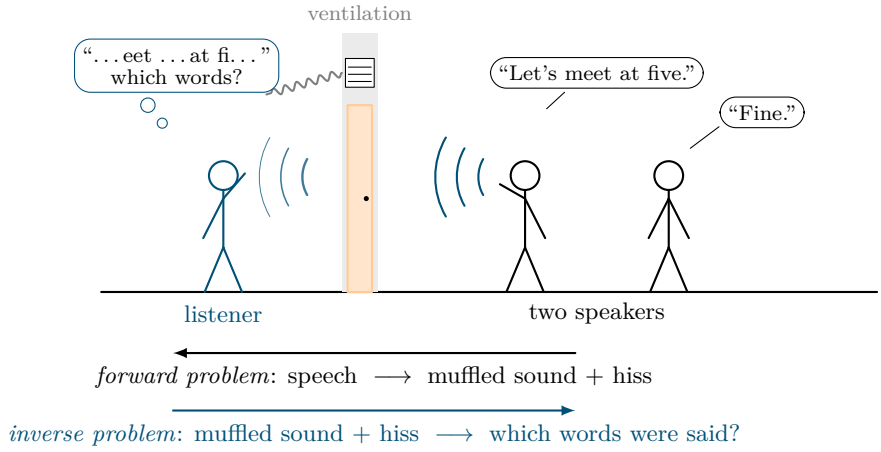


Figure 1.1: The inverse problem of the corridor. Speech passes through the door and arrives muffled, with the hiss of the ventilation added (forward problem). The listener wants the words (inverse problem). The door has removed information for good; the listener's knowledge of the language supplies the rest.

Almost every measurement in science and engineering is such a door. Lenses blur, detectors integrate, tissue scatters, sensors are sparse, and every record carries noise. This chapter introduces the vocabulary that lets us say precisely what has gone wrong, and why no computation can undo it. We start with examples (Section 1.1), fix the common structure of forward and inverse problems (Section 1.2), define ill-posedness and show that it is a fact and not a numerical accident (Section 1.3), and end with the question that the rest of the course answers in three different ways (Section 1.4).

1.1 Examples of inverse problems

We begin with three examples. The first two are linear and come from signal processing and medical imaging; the third is nonlinear and comes from the modeling of physical systems. The first and the third accompany us through the whole course as *running examples*; data and code for them are provided (see the preliminary chapter and Appendix B).

Example 1.1: Deconvolution and denoising (running example 1)

A signal $x\colon [0,1] \to \mathbb{R}$ is recorded by an instrument with finite resolution. Every instrument averages over a small window: a camera pixel collects light from a small patch, a thermometer needs time to react, a seismometer has inertia. If the window is described by a *kernel* k, the instrument records

$$y(s) = \int_0^1 k(s-t)\,x(t)\,\mathrm{d}t + \varepsilon(s), \quad k(t) = \frac{1}{\gamma\sqrt{2\pi}}\,\mathrm{e}^{-t^2/(2\gamma^2)}, \tag{1.1}$$

a *convolution* of x with k, plus measurement noise ε. The width γ of the Gaussian kernel is the resolution of the instrument. Recovering x from y is called *deconvolution*; the special case $k = \delta$ (no blur, only noise) is *denoising*.

On a computer we represent x by its values $\boldsymbol{x} \in \mathbb{R}^n$ at n equidistant points t_j and the data by $\boldsymbol{y} \in \mathbb{R}^m$ at m observation points s_i. The midpoint rule turns (1.1) into the linear system

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{\varepsilon}, \quad A_{ij} = \tfrac{1}{n}\,k(s_i - t_j), \tag{1.2}$$

with a dense matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$. Throughout these notes we use $n = 128$ unknowns, $m = 64$ observation points, $\gamma = 0.03$, white Gaussian noise $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2\boldsymbol{I})$ with $\sigma = 0.05$ (five percent of the signal maximum), and two test signals, a smooth one and a piecewise-constant one (Figure 1.2). Whenever we speak of a *noise level* in these notes, we mean the standard deviation σ of the noise, never its variance. Look at the figure: the blur rounds the corners and lowers the peaks, the noise is visible but modest, and one can still see where the bumps and boxes are. It is hard to believe that recovering $\boldsymbol{x}$ from $\boldsymbol{y}$ could be a serious problem. It is.

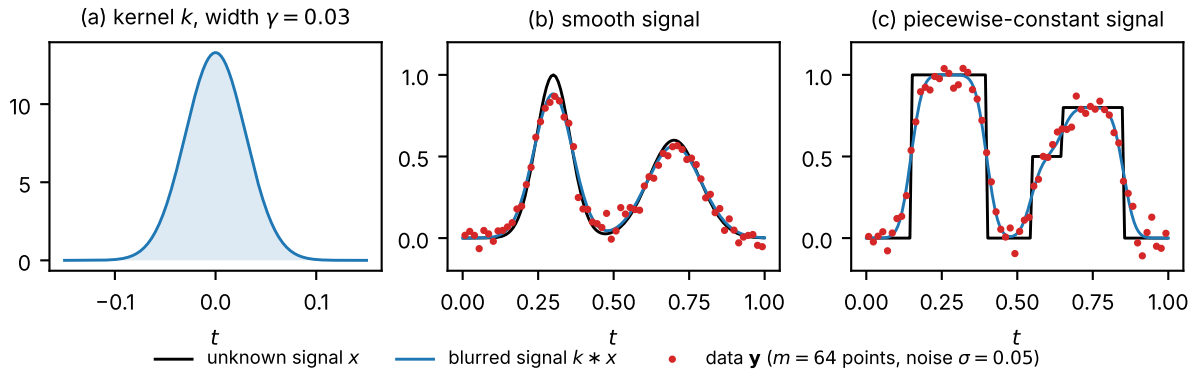


Figure 1.2: Running example 1. (a) The Gaussian kernel k. (b, c) The two test signals x, the blurred signals $k * x$, and the data $\boldsymbol{y}$: $m = 64$ noisy samples of the blurred signal. The blur rounds the corners of the piecewise-constant signal and lowers the peaks; the noise has standard deviation $\sigma = 0.05$, five percent of the signal maximum.

Example 1.2: X-ray computed tomography

In computed tomography (CT) the measurement is not a blurred copy of the unknown but a different object altogether. An X-ray beam travelling along a line ℓ through the body is attenuated; by the Beer–Lambert law the logarithm of the ratio of emitted to detected intensity equals the line integral of the attenuation coefficient μ along the ray,

$$y(\ell) = \int_\ell \mu \, \mathrm{d}s + \varepsilon(\ell). \tag{1.3}$$

The scanner collects these integrals for many parallel rays at many angles; arranged by angle and detector position they form the *sinogram* (Figure 1.3b). The map $\mu \mapsto y$ is the *Radon transform*. Each projection flattens the object to a shadow, and the inverse problem is to rebuild the object from its shadows. The problem is linear, since (1.3) is linear in μ, and on the face of it benign: with rays from all directions and little noise, a textbook formula (filtered back-projection) reconstructs the image well. The difficulty appears when the dose is reduced to protect the patient (more noise) or when fewer angles are available, because the scan has to be fast or the geometry of the scanner limits the angular range. Figure 1.3c shows what the textbook formula does then: the anatomy is still visible, but so are streaks and grain that are not in the patient. Reducing the dose without losing diagnostic quality is one of the driving applications of the methods of this course.

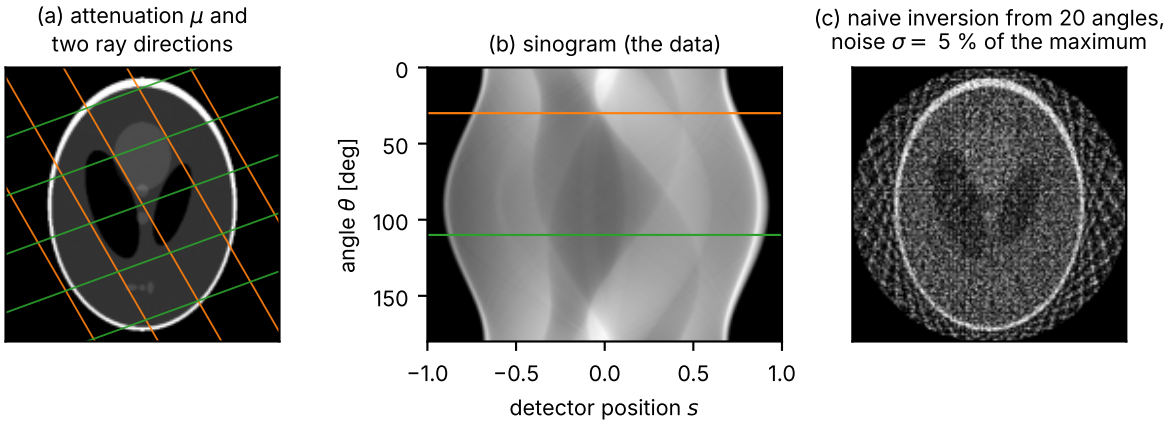


Figure 1.3: X-ray tomography. (a) The unknown attenuation μ (the Shepp–Logan phantom, a cartoon of a head) and two families of parallel rays. (b) The sinogram: each row collects the line integrals (1.3) for one angle; the two marked rows belong to the ray directions in (a). (c) Filtered back-projection applied naively to data from only 20 angles with five percent noise.

Example 1.3: Parameter identification in differential equations (running example 2)

In many fields the forward model is a differential equation whose coefficients are the unknown. A chemist measures the concentrations of species in a reactor over time and wants the rate constants and the stoichiometry of the reactions, i.e., the right-hand side of a system of ordinary differential equations [63]. A hydrologist measures the free surface of a river and wants the shape of the river bed, which enters the shallow-water equations as a coefficient [39]. In both cases, evaluating the forward model means solving the differential

equation for a candidate parameter, and the data depend *nonlinearly* on the parameter even if the equation is linear in the state.

Our second running example is the river bed (Figure 1.4). Water flows steadily through a channel of length $L = 25\,\mathrm{m}$ with discharge $q = 4.42\,\mathrm{m}^2/\mathrm{s}$ and a depth of $h_{\mathrm{out}} = 2\,\mathrm{m}$ at the outflow [49]. Above an unknown bottom $b(x)$, the water has depth $h(x)$ and the free surface is $\eta = h + b$; it is the surface that can be measured, at m sensors, with noise of standard deviation about one centimeter. For steady subcritical flow the depth is determined pointwise by Bernoulli's relation,

$$\frac{q^2}{2h^2} + g\,(h + b) = E = \text{const}, \tag{1.4}$$

where E is fixed by the outflow condition and $g = 9.81\,\mathrm{m/s}^2$. For a given $b(x)$, $h(x)$ is thus the largest root of a cubic. The forward operator $F\colon \boldsymbol{b} \mapsto \eta(\boldsymbol{x}_{\mathrm{obs}})$ is therefore nonlinear but cheap (half a millisecond for $n = 200$ cells). At the same time, the surface is a distorted, damped, and inverted image of the bottom: over a hill the water gets shallower and faster, and the surface *dips* (Figure 1.4b). This makes it challenging to recover the bottom topography from surface measurements.

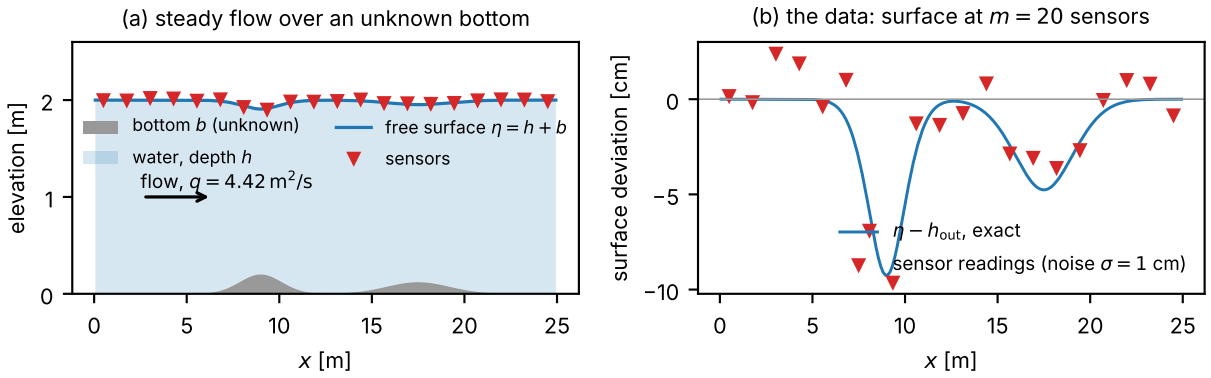


Figure 1.4: Running example 2. (a) Steady flow over the unknown bottom b (gray): the water (blue) has depth h, the free surface $\eta = h + b$ is measured at $m = 20$ sensors. (b) The data, drawn as the deviation of the surface from the outflow level: where the bottom rises by 20 cm, the surface dips by about 9 cm; the sensor readings carry Gaussian noise with standard deviation 1 cm.

1.2 Forward and inverse problems

Different as they are, the three examples share one skeleton, which we now fix as our definition.

Definition 1.4: Inverse problem

Let $\mathcal{X}$ be the set of possible unknowns and $\mathcal{Y}$ the set of possible data (in this course, $\mathcal{X} = \mathbb{R}^n$ and $\mathcal{Y} = \mathbb{R}^m$ after discretization). The *forward operator* $F\colon \mathcal{X} \to \mathcal{Y}$ describes how the unknown produces noise-free data. The *data model* is

$$\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{\varepsilon}, \tag{1.5}$$

where $\boldsymbol{\varepsilon}$ is the measurement noise. The *forward problem* is: given $\boldsymbol{x}$, compute $F(\boldsymbol{x})$. The *inverse problem* is: given $\boldsymbol{y}$, find $\boldsymbol{x}$—or, as we will learn to say, find out what $\boldsymbol{y}$ tells us about $\boldsymbol{x}$. The problem is *linear* if $F(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}$ for a matrix $\boldsymbol{A}$, and *nonlinear* otherwise.

Table 1.1: The examples of Section 1.1 in the language of Definition 1.4.

Problem	Unknown $\boldsymbol{x}$	Forward operator F	Data $\boldsymbol{y}$	Linear
Deconvolution	signal on $[0,1]$	convolution with k	noisy blurred signal	yes
CT	attenuation μ	Radon transform (line integrals)	sinogram	yes
Reaction kinetics	rate constants	solve the ODE, read off concentrations	time series	no
Bathymetry	river bed b	solve (1.4), read off η at sensors	surface at sensors	no

Figure 1.5 summarizes the situation. The forward problem runs with the flow of causality: causes produce effects, and physics (or chemistry, or biology) tells us how. Simulating it is what most of computational science does: integrate the equations, evaluate the integral, run the model. The inverse problem runs against the flow: from the effects we have observed, we want the causes. On the way back we meet two obstacles. The forward map has mixed and damped the information (the door in the corridor), and the data have been corrupted by noise (the hiss of the ventilation). Both obstacles are harmless in the forward direction and, as the next section shows, fatal in the inverse direction.

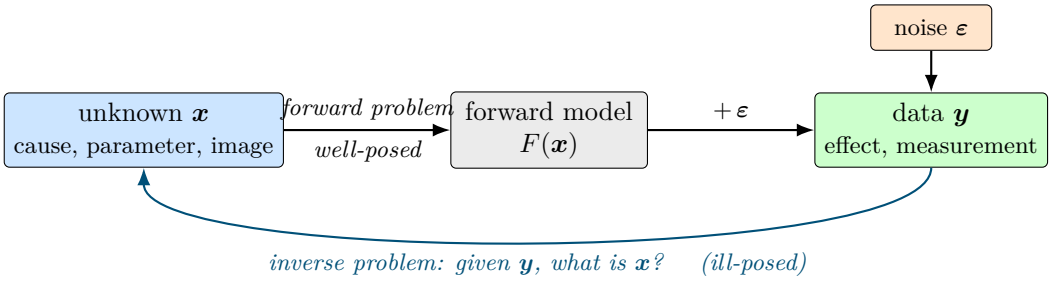


Figure 1.5: The forward problem follows causality and is well-posed; the inverse problem runs against it and is, almost always, ill-posed.

Remark 1.5: Three kinds of unknowns

It helps to notice what kind of thing the unknown is. It can be a *source or state* that drives a known system (the signal in deconvolution, the attenuation in CT, the initial condition of a weather model); a *coefficient or parameter* of the system itself (rate constants, the river bed, the conductivity in impedance tomography); or a *geometry* (the shape of an obstacle, the position of a crack). Source problems with linear physics are linear inverse problems; coefficient and geometry problems are nonlinear even when the physics is linear, because the state depends nonlinearly on the coefficients. The nonlinear problems are the ones most participants of this course meet in their own research, and they are the reason we insist on gradients and automatic differentiation later on.

1.3 Ill-posedness

1.3.1 Hadamard's conditions

What "harmless" and "fatal" mean was made precise by Hadamard [53].

Definition 1.6: Well-posed and ill-posed problems (Hadamard)

The problem of finding $\boldsymbol{x} \in \mathcal{X}$ with $F(\boldsymbol{x}) = \boldsymbol{y}$ is *well-posed* if
(i) *(existence)* for every admissible $\boldsymbol{y} \in \mathcal{Y}$ a solution exists;
(ii) *(uniqueness)* the solution is unique; and
(iii) *(stability)* the solution depends continuously on the data, i.e., small changes of $\boldsymbol{y}$ cause small changes of $\boldsymbol{x}$.
If one of the conditions fails, the problem is *ill-posed*.

Conditions (i) and (ii) say that the question has exactly one answer; condition (iii) says that a small nudge to the data must not fling the answer across the room. Figure 1.6 shows the three ways in which the conditions fail. Note that the definition speaks about the *problem*—the operator F, the sets $\mathcal{X}$ and $\mathcal{Y}$, and the notion of "small" on each—and not about any method for solving it. Ill-posedness is a property of the question, not of the algorithm.

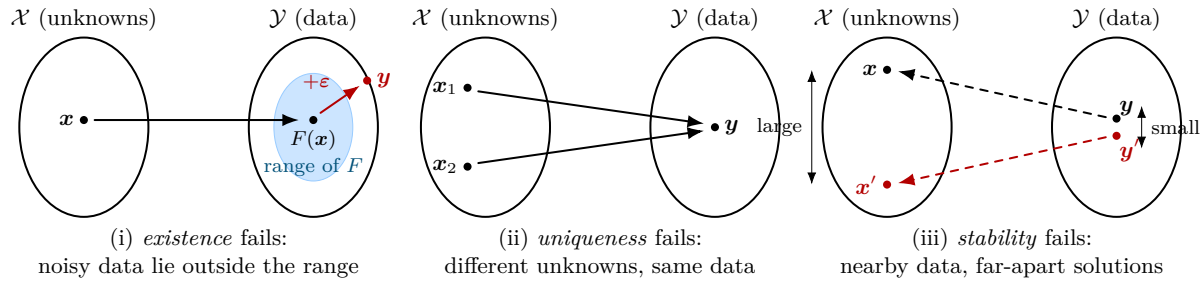


Figure 1.6: Hadamard's three conditions and how they fail. (i) The noise pushes the data out of the range of F: no unknown reproduces $\boldsymbol{y}$ exactly. (ii) Two different unknowns produce the same data. (iii) Two data sets that differ by less than the noise belong to unknowns that are far apart.

Which condition fails, and where? All three conditions in Definition 1.6 can fail in our examples, and it pays to know which one does, because the remedies differ.

- *Existence* fails as soon as there is noise. A convolution with a smooth kernel produces a smooth function, so a jagged, noisy $\boldsymbol{y}$ is not in the range of $\boldsymbol{A}$: there is no signal that, blurred, gives exactly the data. For square invertible matrices, and for matrices with full row rank such as our 64×128 deconvolution matrix, this failure is hidden, because every $\boldsymbol{y}$ is $\boldsymbol{Ax}$ for some $\boldsymbol{x}$; this is one reason why discretized problems can look better than they are.

- *Uniqueness* fails when the forward operator has a null space, i.e., when different unknowns produce the same data. In Example 1.3 with sparse sensors this is literally true: the surface at a sensor depends, by (1.4), only on the bottom in the two neighboring cells and, through E, on the outflow cell, so a bump in the bed between two sensors is invisible (Figure 1.9c). In Example 1.1 with $m = 64$ observations of $n = 128$ unknowns the matrix has a null space of dimension 64 as well, and even for $m = n$ there is an approximate one: no component of the signal is exactly invisible, but the rapidly oscillating ones nearly are, as the next subsection quantifies.

- *Stability* fails in deconvolution, in CT, and in most problems in which the forward operator smooths. This failure is the subtle one, and the next two subsections are devoted to it.

Motivation : Why not just invert?

The deconvolution problem (1.2) is a linear system. Why not solve it for $\boldsymbol{x}$ and go home? Figure 1.7 answers. We write $\boldsymbol{x} = \boldsymbol{A}^{\dagger}\boldsymbol{y}$ for the least-squares solution of $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{y}$ with minimal norm ($\boldsymbol{A}^{\dagger}$ is the pseudo-inverse; for a square invertible matrix it is $\boldsymbol{A}^{-1}$). The notation is a formula, not a recipe: one never computes an inverse matrix, which is expensive and numerically unstable, but solves the linear system or the least-squares problem with a stable solver. For exact data the least-squares solution recovers the smooth signal to within the plotting accuracy. With a noise level of $\sigma = 10^{-3}$—a tenth of a percent of the signal, far below what a real instrument achieves—the "solution" oscillates with an amplitude of several thousand. With the realistic $\sigma = 0.05$ the amplitude is $5 \cdot 10^5$. The data and the blurred signal in Figure 1.2b are close; the solutions are useless. The information about the fine structure of $\boldsymbol{x}$ has been overwritten by noise, and the solver, which cannot know this, amplifies the noise as faithfully as it would amplify the signal.

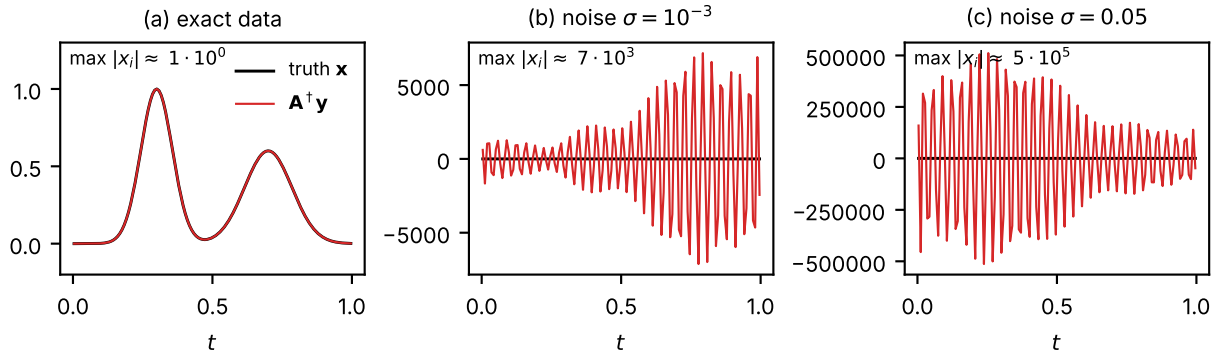


Figure 1.7: The naive solution $\boldsymbol{A}^{\dagger}\boldsymbol{y}$ of the deconvolution problem for the smooth signal. (a) Exact data: the least-squares solution is exact. (b) Noise of 10^{-3}: oscillations with an amplitude of $7 \cdot 10^3$. (c) Noise of 0.05: amplitude $5 \cdot 10^5$. The vertical scales differ by orders of magnitude; the true signal is invisible in (b) and (c).

Remark 1.7: Not every inverse problem is ill-posed in the same way

Hadamard's conditions are a checklist, not a verdict. The bathymetry problem with dense sensors turns out to be stable (Section 1.3.4); its difficulty lies in the sparse sensors, i.e., in uniqueness, and the cure for a missing uniqueness (add information about the bed between the sensors) differs from the cure for a missing stability (damp the amplified noise). Before choosing a method, find out which condition fails.

1.3.2 The continuous view: smoothing destroys information

Why is the inverse in Figure 1.7 so sensitive? Feed the convolution (1.1) a pure oscillation $x(t) = \sin(2\pi\nu t)$ of frequency ν (and ignore the boundaries of the interval). The output is the same oscillation with a smaller amplitude,

$$(k * x)(s) = \widehat{k}(\nu)\,\sin(2\pi\nu s), \quad \widehat{k}(\nu) = \mathrm{e}^{-2\pi^2\gamma^2\nu^2}, \tag{1.6}$$

where $\widehat{k}$ is the Fourier transform of the kernel. For $\gamma = 0.03$ the attenuation factors are

$$\nu = 5\colon\ 0.64, \quad \nu = 10\colon\ 0.17, \quad \nu = 20\colon\ 8 \cdot 10^{-4}, \quad \nu = 30\colon\ 10^{-7}, \quad \nu = 40\colon\ 5 \cdot 10^{-13}.$$

Think of frosted glass: coarse shapes pass, fine detail is washed out. Ten oscillations per unit length survive with a sixth of their amplitude; forty are reduced to nothing. To undo the blur, the inverse has to *divide* by $\widehat{k}(\nu)$, i.e., multiply the component at frequency 30 by 10^7. White noise contains every frequency at the same size, so noise of size 0.05 at that frequency becomes an oscillation of amplitude $5 \cdot 10^5$. That is Figure 1.7c: the forward map is frosted glass, the inverse a magnifying glass pointed at the frost.

Nothing here is specific to Gaussian kernels. Every integral operator with a square-integrable kernel, and the solution operators of most differential equations, are *compact* operators: they map bounded sets to sets that can be covered by finitely many small balls, which is a technical way of saying that they squeeze infinitely many directions of the input to almost nothing. For such operators the instability is a theorem.

Theorem 1.8: Compact operators have unbounded inverses

Let $\mathcal{X}$, $\mathcal{Y}$ be Hilbert spaces, $A\colon \mathcal{X} \to \mathcal{Y}$ a compact linear operator, and suppose the range of A is infinite-dimensional. Then the inverse $A^{-1}\colon \operatorname{range}(A) \to \mathcal{X}$ (defined on the range, modulo the null space) is not continuous: for every $C > 0$ there are data perturbations δy with $\|\delta y\| = 1$ such that the corresponding solution perturbation satisfies $\|\delta x\| > C$. The inverse problem $Ax = y$ violates Hadamard's condition (iii) in every pair of norms on $\mathcal{X}$ and $\mathcal{Y}$ for which A is compact.

Proof outline. A compact operator between Hilbert spaces has a *singular system* $(\sigma_j, v_j, u_j)_{j \geq 1}$: orthonormal $v_j \in \mathcal{X}$ and $u_j \in \mathcal{Y}$ and numbers $\sigma_1 \geq \sigma_2 \geq \cdots > 0$ with $Av_j = \sigma_j u_j$: an orthonormal set of inputs that A maps to an orthonormal set of outputs, each scaled by a *singular value* σ_j (for a matrix this is its singular value decomposition). Since the range is infinite-dimensional, there are infinitely many σ_j. Since A is compact, the image $\{Av_j\} = \{\sigma_j u_j\}$ of the bounded sequence $\{v_j\}$ has a convergent subsequence; but the $\sigma_j u_j$ are mutually orthogonal, and a sequence of orthogonal vectors can only converge to zero, so $\sigma_j \to 0$ along that subsequence and, by monotonicity, altogether. Now perturb the data by $\delta y = u_j$. The corresponding solution perturbation is $\delta x = A^{-1}u_j = \sigma_j^{-1} v_j$ with $\|\delta x\| = 1/\sigma_j \to \infty$; choosing j large enough gives $\|\delta x\| > C$. □

Pause on what this says. Ill-posedness is not a shortcoming of our algorithms, our computers, or our discretization. It can be proven, and the proof works for essentially every forward operator that smooths. No algorithm and no engineering trick can make the inverse of a compact operator continuous. What *can* be done—and this is the whole course—is to change the question: instead of asking for the unknown that reproduces the data exactly, we ask for the unknown that is compatible with the data *and* with what we know independently of the data. The door has destroyed part of the message for good; the rest is reconstructed from the sound that came through and from our knowledge of the language.

Remark 1.9: How much information do the data contain?

The attenuation factors (1.6) also say how much the data can tell us. A component of the signal at frequency ν is recoverable only if it arrives above the noise, $\widehat{k}(\nu) \gtrsim \sigma$, i.e., for $\nu \lesssim \sqrt{2\ln(1/\sigma)}\,/(2\pi\gamma)$. For $\gamma = 0.03$ this is about 13 oscillations per unit length at the noise level $\sigma = 0.05$ and about 20 at $\sigma = 10^{-3}$: reducing the noise fiftyfold buys seven more. The information in the data grows only with the *logarithm* of the signal-to-noise ratio, and this logarithm is the currency of ill-posed problems. Better instruments help,

but slowly; prior knowledge is the other source of information, and the rest of the course is about using it.

1.3.3 The discrete view: ill-conditioning

Now return to the computer. Suppose for a moment that $m = n$ in (1.2), so that $\boldsymbol{A}$ is square and invertible. Then all three of Hadamard's conditions hold for the discretized problem: a unique solution $\boldsymbol{A}^{-1}\boldsymbol{y}$ exists for every $\boldsymbol{y}$ and depends continuously (even linearly) on $\boldsymbol{y}$. By the letter of Definition 1.6 the discrete problem is well-posed—and yet Figure 1.7. The resolution is that in finite dimensions the question is not *whether* the solution depends continuously on the data but *how strongly*, and the answer is the condition number. To state it we need the *singular values* $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n \geq 0$ of a matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, the square roots of the eigenvalues of $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$: they are the amplification factors of $\boldsymbol{A}$ along an orthonormal set of input directions, so that $\sigma_n \|\boldsymbol{x}\| \leq \|\boldsymbol{A}\boldsymbol{x}\| \leq \sigma_1 \|\boldsymbol{x}\|$ for every $\boldsymbol{x}$, with equality along the first and the last of these directions.

Lemma 1.10: Error amplification and the condition number

Let $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ be invertible with singular values $\sigma_1 \geq \cdots \geq \sigma_n > 0$, let $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{y}$ and $\boldsymbol{x} + \delta\boldsymbol{x} = \boldsymbol{A}^{-1}(\boldsymbol{y} + \delta\boldsymbol{y})$. Then

$$\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \;\leq\; \kappa(\boldsymbol{A})\,\frac{\|\delta\boldsymbol{y}\|}{\|\boldsymbol{y}\|}, \qquad \kappa(\boldsymbol{A}) := \frac{\sigma_1}{\sigma_n}, \tag{1.7}$$

and the bound is attained for suitable $\boldsymbol{y}$ and $\delta\boldsymbol{y}$. For a rectangular $\boldsymbol{A}$ of rank r and data in its range, $\boldsymbol{y} \in \operatorname{range}(\boldsymbol{A})$ (always the case if $\boldsymbol{A}$ has full row rank), the same holds for the minimum-norm least-squares solution with $\kappa(\boldsymbol{A}) = \sigma_1/\sigma_r$, where σ_r is the smallest nonzero singular value. For inconsistent data the right-hand side acquires the factor $\|\boldsymbol{y}\| \,/\, \|\boldsymbol{A}\boldsymbol{x}\| \geq 1$.

Proof. Since $\|\boldsymbol{A}\boldsymbol{z}\| \geq \sigma_n \|\boldsymbol{z}\|$ for every $\boldsymbol{z}$, applying this to $\boldsymbol{z} = \delta\boldsymbol{x}$ gives $\|\delta\boldsymbol{x}\| \leq \sigma_n^{-1} \|\boldsymbol{A}\delta\boldsymbol{x}\| = \sigma_n^{-1} \|\delta\boldsymbol{y}\|$; and $\|\boldsymbol{y}\| = \|\boldsymbol{A}\boldsymbol{x}\| \leq \sigma_1 \|\boldsymbol{x}\|$. Multiply the two inequalities. Equality holds when $\boldsymbol{x}$ points along the direction of largest amplification and $\delta\boldsymbol{x}$ along the direction of smallest amplification; the rectangular case is the same argument on the range of $\boldsymbol{A}$. □

The condition number is the worst-case amplification factor of relative errors: a microphone with gain κ. For our deconvolution matrix $\kappa(\boldsymbol{A}) \approx 3.5 \cdot 10^7$, so a relative data error of 10^{-3} can become a relative solution error of $3.5 \cdot 10^4$, and Figure 1.7b shows that the noise does find the bad directions. Figure 1.8a shows how κ depends on the discretization: for a fixed kernel it grows with the number of cells n, roughly like $\exp(c\,n^2)$, until the smallest singular values fall below the machine precision times σ_1; from then on the computed κ lies between 10^{16} and 10^{20} and fluctuates irregularly, because the computed smallest singular values are rounding errors. Smoother kernels (larger γ) are worse. Figure 1.8b explains why: the singular values of $\boldsymbol{A}_n$ for different n all lie on one curve, the singular values of the continuous operator, and refining the grid simply adds smaller ones at the end.

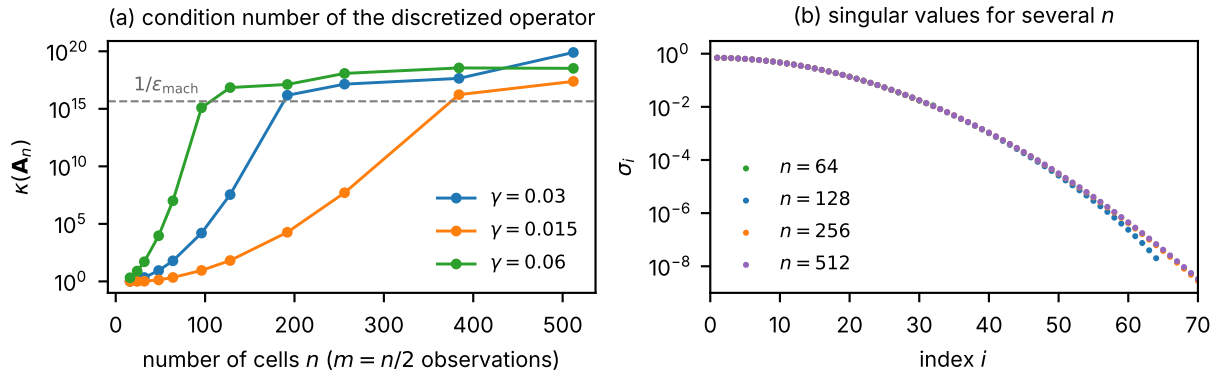


Figure 1.8: (a) The condition number of the discretized convolution operator ($m = n/2$ observation points) grows with the number of cells n and with the kernel width γ, until it saturates at or above the reciprocal machine precision. (b) The singular values of $\boldsymbol{A}_n$ for $n = 64, 128, 256, 512$ lie on one curve: a finer grid resolves more of the continuous operator, including the directions it almost annihilates.

Summary : Discretization does not cure ill-posedness, it hides it

A discretized ill-posed problem is a well-posed but ill-conditioned problem, and its condition number grows without bound as the discretization is refined. The finer the grid, the better the matrix approximates the operator, and the more of the operator's ill-posedness the matrix inherits. Conversely, a coarse grid *is* a crude form of regularization: it throws away the fine scales before the noise can be amplified in them. We will do this deliberately, and better, in Chapter 2.

1.3.4 Nonlinear problems

For a nonlinear forward operator F the notions above are applied locally. Near a point $\boldsymbol{x}$, Taylor's theorem gives

$$F(\boldsymbol{x} + \delta\boldsymbol{x}) \approx F(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})\,\delta\boldsymbol{x}, \quad \boldsymbol{J}(\boldsymbol{x}) = \frac{\partial F}{\partial \boldsymbol{x}}(\boldsymbol{x}) \in \mathbb{R}^{m\times n}, \tag{1.8}$$

and the Jacobian $\boldsymbol{J}$ plays the role of $\boldsymbol{A}$: its singular values measure how much a small change of the unknown changes the data, and its condition number how strongly data errors are amplified in the solution near $\boldsymbol{x}$. Two new phenomena appear. The degree of ill-posedness may depend on $\boldsymbol{x}$, and the equation $F(\boldsymbol{x}) = \boldsymbol{y}$ may have several isolated solutions, so that uniqueness can fail even without a null space. The question whether the data determine the parameter at all is called *identifiability*: $\boldsymbol{x}$ is *locally identifiable* if $\boldsymbol{J}(\boldsymbol{x})$ has full column rank, and a direction $\delta\boldsymbol{x}$ with $\boldsymbol{J}\delta\boldsymbol{x} = \boldsymbol{0}$ is a change of the unknown that the data do not see.

Example 1.11: Sensitivity of the free surface to the bottom

In Example 1.3 the forward operator is given implicitly by Bernoulli's relation (1.4). Differentiating it with respect to b at fixed q and E gives $(g - q^2/h^3)\,\partial h/\partial b + g = 0$, hence, with the Froude number $\mathrm{Fr}^2 = q^2/(gh^3)$,

$$\frac{\partial h}{\partial b} = -\frac{1}{1-\mathrm{Fr}^2}, \quad \frac{\partial \eta}{\partial b} = 1 + \frac{\partial h}{\partial b} = -\frac{\mathrm{Fr}^2}{1-\mathrm{Fr}^2}. \tag{1.9}$$

In our channel Fr lies between 0.5 and 0.63, so $\partial\eta/\partial b$ lies between -0.33 and -0.67 (Figure 1.9a): raising the bottom by a centimeter lowers the surface above it by a third to two thirds of a centimeter. The surface is a damped and inverted image of the bottom, but only mildly damped, and the relation is *local*: the surface at x depends on the bottom at x only (and on the outflow). In slow flow ($\mathrm{Fr} \to 0$) the sensitivity vanishes—a lake does not reveal its bottom—and near critical flow ($\mathrm{Fr} \to 1$) it blows up, together with the steady model itself.

The Jacobian of the discretized map is therefore essentially diagonal, and Figure 1.9b confirms what (1.9) predicts. With dense sensors ($m = n = 200$, one every 0.12 m) the Jacobian has rank 194. One large singular value, about 20, belongs to the outflow cell, which shifts the surface at every sensor through E; six vanish, those of the three cells upstream of the first sensor, of the two cells between the last sensor and the outflow, and of one combination of the outflow cell with the others; and the remaining ones lie between 0.07 and 0.67. On the directions that the sensors see, the problem is *well-conditioned*: once the outflow direction is excluded, the ratio of the largest to the smallest nonzero singular value is about 10 (with it, $\kappa = \sigma_1/\sigma_r \approx 280$, but only because of the single large singular value), and a centimeter of noise in the data becomes about four centimeters of error in the bed. The smallest of these singular values are due to the interpolation, not to the physics: a sensor midway between two cell centers averages them and barely sees an oscillation from cell to cell; with the sensors exactly at the cell centers they would be the local sensitivities $|\partial\eta/\partial b| \in [0.33, 0.67]$. With $m = 20$ sensors the Jacobian has only 20 nonzero singular values; the other 180 directions form a null space, and Figure 1.9c shows one of them, a 10 cm bump between two sensors for which $F(\boldsymbol{b}+\delta\boldsymbol{b}) = F(\boldsymbol{b})$ holds exactly, not approximately. This is ill-posedness of the uniqueness kind, and no amount of data from the same sensors can fix it; only more sensors, time-dependent data (the wave-flume setting of default project C), or prior knowledge about what river beds look like can.

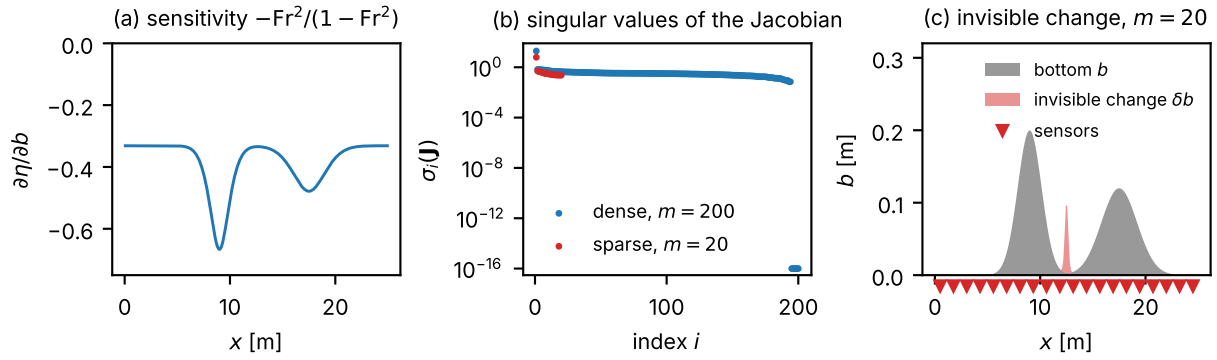


Figure 1.9: Running example 2. (a) The local sensitivity (1.9) along the channel. (b) Singular values of the Jacobian of F at the true bottom for dense sensors ($m = 200$, one every 0.12 m) and sparse sensors ($m = 20$, one every 1.26 m): dense sensors give a well-conditioned problem apart from six invisible directions (their zero singular values are drawn at 10^{-16}) and one large singular value that belongs to the outflow cell, sparse sensors a rank-deficient one with an exact null space of dimension 180. (c) A direction in that null space: a 10 cm bump between two sensors that leaves all sensor readings unchanged.

1.4 What is a solution?

We can now say what the inverse problem $\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{\varepsilon}$ asks and why "solve for $\boldsymbol{x}$" is the wrong request. The data determine some components of the unknown well, some poorly, and some not at all; a method that treats all components alike returns noise. There are three ways to turn the request into a well-posed question, and they are the three parts of this course.

1. *Pick a stable approximation.* Decide, by a rule, which components to keep and how much to damp the rest, and return one answer. This is *regularization* (Chapter 2). Regularization is fast and deterministic, and it hides its assumptions in a parameter that has to be chosen.

2. *Describe what is compatible with the data and with prior knowledge.* Treat the noise as random, so that the data are one draw of a random variable, and treat the unknown as uncertain. The data then do not single out one $\boldsymbol{x}$ but reweight the set of plausible ones; prior knowledge—smoothness, sparsity, positivity, a plausible range—does the rest. This is the *Bayesian* formulation (Chapter 3), and its most probable answer turns out to be a regularized solution with the assumptions made explicit.

3. *Quantify the uncertainty.* Instead of one $\boldsymbol{x}$, report the whole distribution of compatible unknowns, or at least its mean and spread, so that downstream decisions know how much to trust the reconstruction. This requires exploring the posterior distribution by sampling or by approximation (Chapters 4 and 5).

The three answers build on each other, and the picture of this chapter remains the right mental model for all of them: the components of the unknown that the forward operator transmits strongly are decided by the data, the components it damps below the noise are decided by the prior, and the uncertainty is large exactly where the damping is strong.

Summary : Chapter 1 in five sentences

An inverse problem infers causes from measured effects through a forward model, $\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{\varepsilon}$. It is ill-posed when existence, uniqueness, or stability fails, and for smoothing forward operators the failure of stability is a theorem, not a numerical accident. Discretization hides the instability behind a condition number that grows without bound as the grid is refined. A smoothing forward operator damps the fine-scale components of the unknown; the data determine the components that arrive above the noise level, and nothing else. The job of the rest of the course is to decide what to do with the other directions, and to say how sure we are.

1.5 Notes and further reading

Hadamard's conditions are in [53]; he considered ill-posed problems to be physically meaningless, a view that the inverse-problems community has spent a century revising. The discrete point of view of this chapter—singular values and the condition number—follows Hansen's book [56], whose first chapters are the best next reading; they also introduce the singular value decomposition and the Picard plot, which show direction by direction how the noise takes over, a natural next step after this chapter. The operator-theoretic background (compact operators, singular systems, the proof of Theorem 1.8 in full) is in Engl, Hanke, and Neubauer [31]. Mueller and Siltanen [83] develop deconvolution and tomography with code and real data, and Natterer [85] is the reference for the mathematics of CT. For the statistical point of view that the rest of these notes take, start with the first chapters of Kaipio and Somersalo [64] and Calvetti and Somersalo

[19]. The bathymetry problem and its direct solution for dense data are described in [39]; the test case is from [49].

1.6 Exercises

Exercise 1.1: Smoothing kills high frequencies

Let k be the Gaussian kernel (1.1) on the whole real line. (a) Verify (1.6), i.e., compute the convolution of k with $x(t) = \sin(2\pi\nu t)$. (b) Show the general statement behind it: for any integrable kernel k, the amplitude of the response to $\sin(2\pi\nu t)$ tends to zero as $\nu \to \infty$ (this is the Riemann–Lebesgue lemma). (c) Conclude that the inverse of the convolution operator cannot be bounded on $L^2(\mathbb{R})$, without using Theorem 1.8.

Exercise 1.2: The null space of the steady shallow-water map

In Example 1.3 the surface at a sensor is obtained by linear interpolation of $\eta = h + b$ between the two neighboring cell centers, and h in each cell is determined by (1.4) with E fixed by the outflow cell. (a) Which cells influence the reading of a given sensor? (b) For $n = 200$ cells and $m = 20$ equispaced sensors, determine the dimension of the null space of the Jacobian $\boldsymbol{J}(\boldsymbol{b})$ without computing it, and describe the null space in words. (c) Derive (1.9) and explain why the dimension of the null space, and the part of it that consists of changes away from the sensors, do not depend on $\boldsymbol{b}$ in this example, although the problem is nonlinear. Which part of the null space does depend on $\boldsymbol{b}$?

Exercise 1.3: Noise amplification (computer)

Use `code/running_examples.py`. (a) Build $\boldsymbol{A}$ for $n = 128$, $m = 64$, and $\gamma = 0.03$, compute $\kappa(\boldsymbol{A})$, and compute the least-squares solutions $\boldsymbol{A}^\dagger \boldsymbol{y}$ for the smooth signal with noise levels $\sigma = 0, 10^{-6}, 10^{-4}, 10^{-2}, 0.05$ (several noise realizations each). Plot the error $\left\| \boldsymbol{A}^\dagger \boldsymbol{y} - \boldsymbol{x} \right\|$ against σ and compare with the bound (1.7). (b) The least-squares solution for *exact* data of the piecewise-constant signal has a maximal error of 0.26 (try it). Explain this with the null space of $\boldsymbol{A}$: which part of the signal can $m = 64$ observations never see? (c) Compute $\kappa(\boldsymbol{A}_n)$ for $n = 32, 64, \ldots, 1024$ with $m = n/2$ and $\gamma \in \{0.015, 0.03, 0.06\}$ and explain the shape of the curves in Figure 1.8a, including the saturation. (d) Verify the estimate of Remark 1.9 numerically: filter the data with a low-pass filter that keeps the frequencies below ν^* and compare the result with the blurred signal.

Exercise 1.4: The Jacobian of a nonlinear forward map (computer)

Implement the forward map F of Example 1.3 in JAX (or use `jacobian_fd` from the course code) and compute its Jacobian at the smooth bottom for $n = 200$ and $m \in \{20, 50, 200\}$ sensors. (a) Plot the singular values and determine the numerical rank in each case. (b) For $m = 20$, construct a bottom perturbation $\delta\boldsymbol{b}$ with $F(\boldsymbol{b} + \delta\boldsymbol{b}) = F(\boldsymbol{b})$ to machine precision and plot it. (c) For $m = 200$, add Gaussian noise with standard deviation 1 cm to $F(\boldsymbol{b})$ and compute the minimum-norm least-squares solution $\delta\boldsymbol{b} = \boldsymbol{J}^\dagger \delta\boldsymbol{y}$ of the linearized problem ($\boldsymbol{J}$ is singular): how large is the error in the bed, and does (1.7) predict it?

Exercise 1.5: Your own inverse problem

Take a problem from your research or your field. Identify the unknown, the forward model, the data, and the noise, and write down the data model (1.5). Is the forward map linear? Which of Hadamard's conditions do you expect to fail, and why? If you can evaluate the forward map, how expensive is one evaluation, and can you compute its Jacobian? Write half a page; this is the first paragraph of your project proposal.

Chapter 2

Deterministic Regularization

Back to the corridor (Figure 2.1). The listener does not try to reconstruct the sentence from the muffled sound alone; she insists that the result be a plausible sentence in a language she knows. Where the sound is clear, the words follow the sound; where it is not, they follow the grammar. Regularization is this idea in mathematical form: we add to the requirement "fit the data" a second requirement, "be reasonable", and a parameter that says how much we trust the ear and how much the grammar. This chapter develops the idea for linear problems (Section 2.2), discusses how the parameter is chosen (Section 2.3), introduces the grammar of sparsity (Section 2.4), and extends everything to nonlinear forward models (Section 2.5). It ends with the questions that deterministic regularization leaves open, which are the questions of the rest of the course.

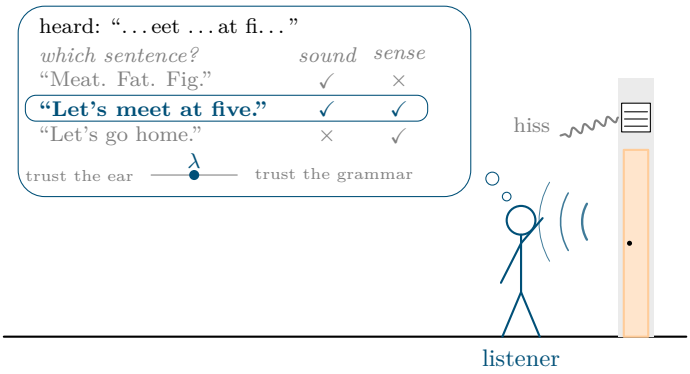


Figure 2.1: The corridor, continued from Figure 1.1. The listener weighs candidate sentences by two criteria: does the sentence fit what she heard, and is it a sentence? The first candidate explains every fragment of the sound and means nothing; the third is a fine sentence that ignores the sound; the second does both well enough and wins. The slider is the regularization parameter λ: it sets how far a poor score in one column can be bought with a good one in the other.

2.1 Variational regularization

Chapter 1 ended with the observation that the data determine some components of the unknown and not others, and that a method which treats all components alike returns noise. The remedy is to stop asking for the $\boldsymbol{x}$ that reproduces the data and to ask instead for the $\boldsymbol{x}$ that reproduces the data *well enough* while being *reasonable*.

Definition 2.1: Variational regularization

Let $F\colon \mathbb{R}^n \to \mathbb{R}^m$ be the forward operator, $\boldsymbol{y} \in \mathbb{R}^m$ the data, $R\colon \mathbb{R}^n \to [0,\infty)$ a *regularizer* (or *penalty*), and $\lambda > 0$ the *regularization parameter*. The *regularized solution* is

$$\boldsymbol{x}_\lambda \in \operatorname*{arg\,min}_{\boldsymbol{x}\in\mathbb{R}^n} J_\lambda(\boldsymbol{x}), \quad J_\lambda(\boldsymbol{x}) = \tfrac{1}{2}\,\|F(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \lambda\, R(\boldsymbol{x}). \tag{2.1}$$

The first term is the *data misfit*, measured in the Euclidean norm $\|\boldsymbol{v}\|_2 = (\sum_i v_i^2)^{1/2}$; it quantifies how well $\boldsymbol{x}$ explains the data. The second term measures how unreasonable $\boldsymbol{x}$ is; λ sets the exchange rate between the two.

The data misfit is the ear, the regularizer is the grammar, and λ is how much we trust each. For $\lambda \to 0$ we recover a least-squares solution (for $R = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ the naive solution $\boldsymbol{A}^\dagger\boldsymbol{y}$ of Chapter 1, for $R = \frac{1}{2}\|\boldsymbol{L}\boldsymbol{x}\|_2^2$ the least-squares solution with the smallest $\|\boldsymbol{L}\boldsymbol{x}\|_2$); for $\lambda \to \infty$ we ignore the data, up to the components that R does not charge, and return the most reasonable $\boldsymbol{x}$ of all, $\boldsymbol{x} = \boldsymbol{0}$ for $R = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ and the best-fitting constant for the difference matrix. Everything interesting happens in between.

The grammars What "reasonable" means is encoded in R, and the choice of R is the most important modelling decision in the whole chapter. Figure 2.2 shows the four regularizers we use, each with a signal it considers cheap and a signal it considers expensive.

- $R(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{x}\|_2^2$ prefers *small* signals. It has no opinion about the shape, only about the size.
- $R(\boldsymbol{x}) = \frac{1}{2}\|\boldsymbol{L}\boldsymbol{x}\|_2^2$, with the *difference matrix* $\boldsymbol{L} \in \mathbb{R}^{(n-1)\times n}$, $(\boldsymbol{L}\boldsymbol{x})_i = x_{i+1} - x_i$, prefers *smooth* signals: it charges for every change from one point to the next, and it charges quadratically, so that large jumps are very expensive.
- $R(\boldsymbol{x}) = \|\boldsymbol{x}\|_1 = \sum_i |x_i|$ prefers *sparse* signals, i.e., signals that are zero except at a few points.
- $R(\boldsymbol{x}) = \|\boldsymbol{L}\boldsymbol{x}\|_1$, the *total variation*, prefers *piecewise-constant* signals: it charges for the total amount of up and down, so that a single jump of height one costs the same as a slow rise of height one.

Constraints such as positivity are regularizers too: $R(\boldsymbol{x}) = 0$ if $\boldsymbol{x} \geq \boldsymbol{0}$ and $R(\boldsymbol{x}) = \infty$ otherwise. We will use the first two regularizers in Sections 2.2 and 2.3 and the last two in Section 2.4.

Remark 2.2: What regularization buys, and what it costs

For a linear forward operator and a regularizer for which J_λ is strictly convex, for example Tikhonov regularization with $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\boldsymbol{0}\}$ (Lemma 2.3), (2.1) has a unique minimizer, and it depends continuously on the data. In other words, the regularized problem satisfies all three of Hadamard's conditions, even if the original problem violates all three. For merely convex penalties such as ℓ^1 and total variation, minimizers exist under mild conditions, but uniqueness, and with it continuity, needs additional assumptions on $\boldsymbol{A}$. The price is *bias*: $\boldsymbol{x}_\lambda$ is not the true unknown even for exact data, because the penalty pulls it towards what it considers reasonable. The theory of regularization [31] shows that the bias disappears in the right limit: if λ is driven to zero together with the noise, slowly enough, then $\boldsymbol{x}_\lambda$ converges to the true unknown. For a fixed noisy data set, which is the situation in practice, the bias is simply the cost of stability.

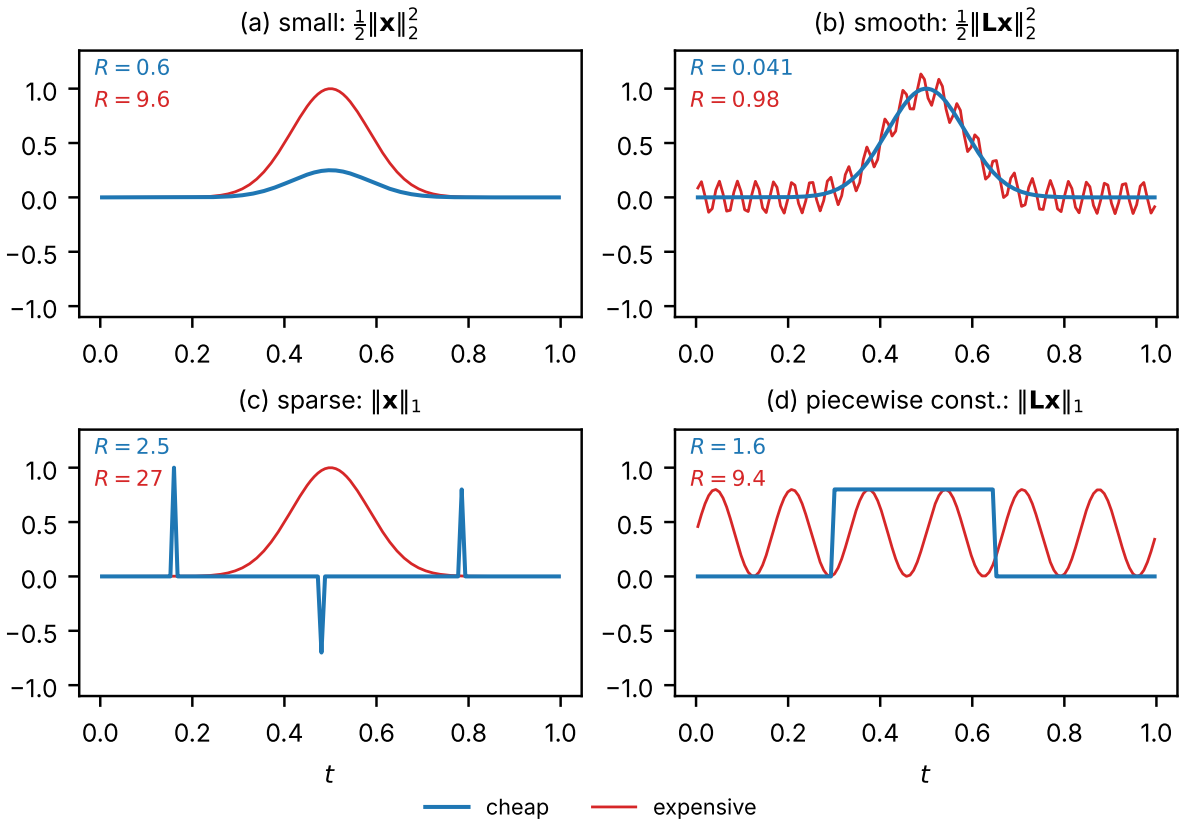


Figure 2.2: Four regularizers and what they believe. In each panel the blue signal is cheap and the red signal is expensive for the penalty in the title; the values of R are given. (a) Size: the same bump at a quarter of the amplitude costs a sixteenth. (b) Smoothness: adding small, fast oscillations to the bump multiplies the cost by twenty. (c) Sparsity: three spikes are cheaper than one bump of the same height. (d) Total variation: a box costs its two jumps; an oscillation of the same amplitude costs every rise and fall.

2.2 Linear problems: Tikhonov regularization

For a linear forward operator and a quadratic penalty, (2.1) can be solved by linear algebra. This is the oldest and still the most widely used form of regularization, named after Tikhonov, who introduced it in the 1960s.

Lemma 2.3: Tikhonov regularization and the normal equations

Let $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, $\boldsymbol{L} \in \mathbb{R}^{p\times n}$ with $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\mathbf{0}\}$, and $\lambda > 0$. The functional

$$J_\lambda(\boldsymbol{x}) = \tfrac{1}{2}\,\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \tfrac{\lambda}{2}\,\|\boldsymbol{L}\boldsymbol{x}\|_2^2 \tag{2.2}$$

has a unique minimizer, namely the solution of the *normal equations*

$$(\boldsymbol{A}^\mathsf{T}\boldsymbol{A} + \lambda\,\boldsymbol{L}^\mathsf{T}\boldsymbol{L})\,\boldsymbol{x}_\lambda = \boldsymbol{A}^\mathsf{T}\boldsymbol{y}. \tag{2.3}$$

Proof. J_λ is a quadratic function with gradient $\nabla J_\lambda(\boldsymbol{x}) = \boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}) + \lambda \boldsymbol{L}^\mathsf{T}\boldsymbol{L}\boldsymbol{x}$ and constant

Hessian $\boldsymbol{H} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A} + \lambda \boldsymbol{L}^\mathsf{T}\boldsymbol{L}$. For any $\boldsymbol{z} \neq \boldsymbol{0}$ we have $\boldsymbol{z}^\mathsf{T}\boldsymbol{H}\boldsymbol{z} = \|\boldsymbol{A}\boldsymbol{z}\|_2^2 + \lambda \|\boldsymbol{L}\boldsymbol{z}\|_2^2 > 0$, because at least one of the two terms is positive by the assumption on the null spaces. So $\boldsymbol{H}$ is positive definite, J_λ is strictly convex, and its unique minimizer is the point where the gradient vanishes, which is (2.3). □

The assumption $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\boldsymbol{0}\}$ is mild: for $\boldsymbol{L} = \boldsymbol{I}$ it is automatically satisfied, and for the difference matrix it only asks that $\boldsymbol{A}$ does not annihilate constant signals.

What Tikhonov does to each frequency For the deconvolution problem we can say exactly what (2.3) does, using the frequency picture of Section 1.3.2. Take $\boldsymbol{L} = \boldsymbol{I}$ and recall from (1.6) that the convolution multiplies the component of the signal at frequency ν by $\widehat{k}(\nu)$. Applying $\boldsymbol{A}^\mathsf{T}$ multiplies by $\widehat{k}(\nu)$ again, so (2.3) reads, frequency by frequency, $(\widehat{k}(\nu)^2 + \lambda)\,\widehat{x}_\lambda(\nu) = \widehat{k}(\nu)\,\widehat{y}(\nu)$. Inserting the data model $\widehat{y} = \widehat{k}\,\widehat{x}^\dagger + \widehat{\varepsilon}$ gives

$$\widehat{x}_\lambda(\nu) = \varphi(\nu)\,\widehat{x}^\dagger(\nu) + \frac{\varphi(\nu)}{\widehat{k}(\nu)}\,\widehat{\varepsilon}(\nu), \quad \varphi(\nu) = \frac{\widehat{k}(\nu)^2}{\widehat{k}(\nu)^2 + \lambda}. \tag{2.4}$$

Compare this with the naive solution, which has $\varphi \equiv 1$: it keeps every frequency of the true signal and divides the noise at every frequency by $\widehat{k}(\nu)$, with the consequences seen in Figure 1.7. The *filter* $\varphi(\nu)$ of (2.4) is close to one where $\widehat{k}(\nu)^2 \gg \lambda$ and close to zero where $\widehat{k}(\nu)^2 \ll \lambda$ (Figure 2.3). Tikhonov regularization is a low-pass filter whose cutoff frequency, where $\widehat{k}(\nu)^2 = \lambda$, is set by λ. Below the cutoff the signal passes and the noise is divided by $\widehat{k}$ as before, but only where $\widehat{k}$ is still large; above the cutoff the signal is lost, but so is the noise, since the amplification factor $\varphi/\widehat{k} = \widehat{k}/(\widehat{k}^2 + \lambda)$ never exceeds $1/(2\sqrt{\lambda})$. The listener turns the volume down exactly where there is only static.

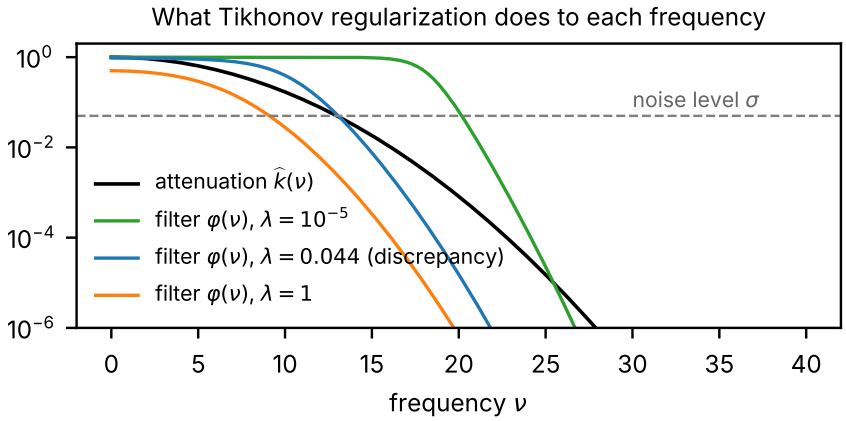


Figure 2.3: The Tikhonov filter (2.4) for the deconvolution problem and three values of λ, together with the attenuation $\widehat{k}(\nu)$ of the forward operator and the noise level. A small λ keeps frequencies far beyond the point where the attenuation has fallen below the noise; a large λ removes frequencies that the data could still have delivered.

Example 2.4: Tikhonov regularization of the deconvolution problem

Figure 2.4 shows $\boldsymbol{x}_\lambda$ with $\boldsymbol{L} = \boldsymbol{I}$ for the two test signals of Example 1.1 and three values of λ. With $\lambda = 10^{-5}$ the cutoff lies at a frequency of about 18, far above the frequencies where the attenuation falls below the noise level, and the reconstruction is an oscillation of amplitude two around the truth: better than the naive solution's $5 \cdot 10^5$, but useless. With $\lambda = 1$ the cutoff is at frequency zero, and the reconstruction is a shrunk and flattened version of the signal. With $\lambda = 0.044$, chosen by the discrepancy principle of Section 2.3,

the smooth signal is recovered with an error of 0.60 (the signal has norm 4.45), i.e., to about 13 percent. With the same λ the piecewise-constant signal is recovered less well, with an error of 1.63: its edges are smeared, and the reconstruction rings on both sides of every jump. Its own discrepancy-principle value, $\lambda = 0.016$, gives 1.51 (Figure 2.7a), which is also about the best that Tikhonov regularization with $\boldsymbol{L} = \boldsymbol{I}$ achieves for this signal. No value of λ helps, because the signal's edges live at the frequencies that the filter removes. The grammar of smallness is the wrong grammar for this signal; Section 2.4 supplies the right one.

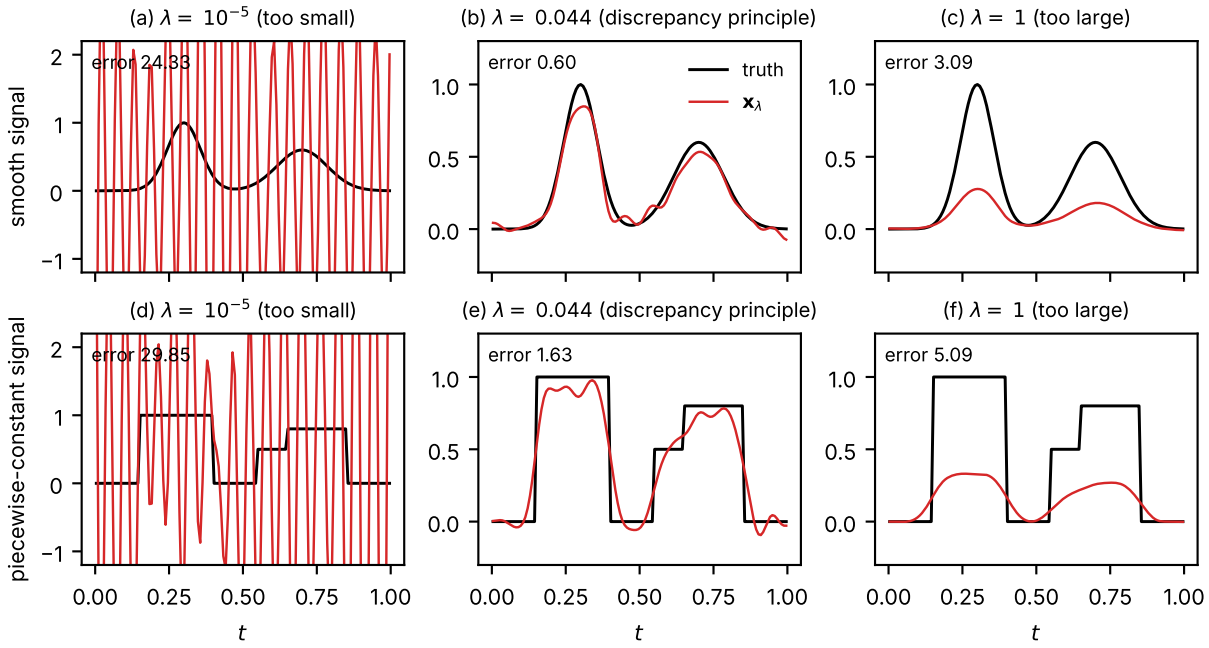


Figure 2.4: Tikhonov regularization ($\boldsymbol{L} = \boldsymbol{I}$) of the deconvolution data of Figure 1.2 for three values of λ: too small (noise passes), chosen by the discrepancy principle for the smooth signal, and too large (signal lost). Top row: the smooth signal; bottom row: the piecewise-constant signal, whose edges are smeared at every λ.

Bias and noise (2.4) also explains the shape of the error as a function of λ. The error of $\boldsymbol{x}_\lambda$ has two parts: the *bias* $(\varphi - 1)\widehat{x}^\dagger$, which is what the filter removes from the true signal and which grows with λ, and the *amplified noise* $(\varphi/\widehat{k})\widehat{\varepsilon}$, which shrinks with λ. Figure 2.5a shows both for the smooth signal: the error curve is U-shaped, with the bias dominating on the right and the noise on the left, and its minimum marks the best possible λ. That minimum is where we would like to be, and it is defined in terms of the truth, which we do not have. Choosing λ from the data alone is the subject of the next section.

2.3 Choosing the regularization parameter

Three rules are in common use. They differ in what they assume to be known, and none of them is safe.

The discrepancy principle If the noise level σ is known, we know how well the true unknown fits the data: $\left\|\boldsymbol{A}\boldsymbol{x}^\dagger - \boldsymbol{y}\right\|_2 = \|\boldsymbol{\varepsilon}\|_2 \approx \sqrt{m}\,\sigma$. A solution that fits the data better than that is fitting noise; a solution that fits worse has thrown away signal. The discrepancy principle therefore chooses λ such that

$$\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2 = \tau\sqrt{m}\,\sigma, \quad \tau \geq 1 \text{ close to one}, \tag{2.5}$$

which has a unique solution because the discrepancy increases continuously and strictly monotonically with λ (Exercise 2.2, Figure 2.5b). In our example it picks $\lambda = 0.044$ and an error within three percent of the best possible one. Its weakness is the input: σ has to be known, and an estimate of σ that is wrong by a factor of two changes λ considerably.

The L-curve Plot the size of the solution, $\|\boldsymbol{L}\boldsymbol{x}_\lambda\|_2$, against the discrepancy $\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2$ on logarithmic axes as λ varies (Figure 2.5c). For small λ the solution is huge and the discrepancy barely changes (the vertical branch); for large λ the solution is small and the discrepancy grows (the horizontal branch). The curve has the shape of an L, and the *L-curve criterion* [55] takes the λ at the corner, where the solution is as small as it can be without a large loss of fit. It needs no knowledge of σ. In our example the corner lies at $\lambda = 0.0027$, and the error there is 1.44, more than twice the best possible: the L-curve undersmooths here, which is its known weakness for smooth solutions, whose curves have no sharp corner.

Generalized cross-validation Cross-validation asks how well $\boldsymbol{x}_\lambda$ computed from all data but one predicts the one left out, averaged over the data. For Tikhonov regularization this can be evaluated without actually leaving anything out: the regularized solution is a linear function of the data, $\boldsymbol{A}\boldsymbol{x}_\lambda = \boldsymbol{S}_\lambda\boldsymbol{y}$ with the *influence matrix* $\boldsymbol{S}_\lambda = \boldsymbol{A}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A} + \lambda\boldsymbol{L}^\mathsf{T}\boldsymbol{L})^{-1}\boldsymbol{A}^\mathsf{T}$, and the *generalized cross-validation* function of Golub, Heath, and Wahba [48],

$$V(\lambda) = \frac{m\,\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2^2}{\left(m - \operatorname{tr}\boldsymbol{S}_\lambda\right)^2}, \tag{2.6}$$

is minimized over λ (Figure 2.5d). The trace of $\boldsymbol{S}_\lambda$ is the *effective number of degrees of freedom* of the fit: m for the naive solution (with $\boldsymbol{A}$ of full row rank), and for $\lambda \to \infty$ the number of directions of $\ker \boldsymbol{L}$ that $\boldsymbol{A}$ sees, i.e., 0 for $\boldsymbol{L} = \boldsymbol{I}$ and 1 for the difference matrix. GCV also needs no σ, and in our example it picks $\lambda = 0.0072$ with an error of 0.98: better than the L-curve, worse than the discrepancy principle, again on the side of too little smoothing.

Remark 2.5: What the rules cannot do

All three rules turn one knob, λ, and they do so with mixed success: in Figure 2.5a two of them land at an error twice the best possible. There is no rule that is reliable for every problem; the discrepancy principle is the most robust when σ is known, and σ can often be estimated from repeated measurements or from the high-frequency content of the data. More important is what no rule does. None of them chooses the grammar R, which matters more than λ, as Example 2.4 showed. None of them says how far $\boldsymbol{x}_\lambda$ is from the truth: the error curve in Figure 2.5a is drawn with knowledge of $\boldsymbol{x}^\dagger$, and in practice we see only the data. Both questions have answers in the Bayesian formulation of Chapter 3, where λ becomes a hyper-parameter that can be inferred together with $\boldsymbol{x}$ and the spread of the posterior replaces the error curve.

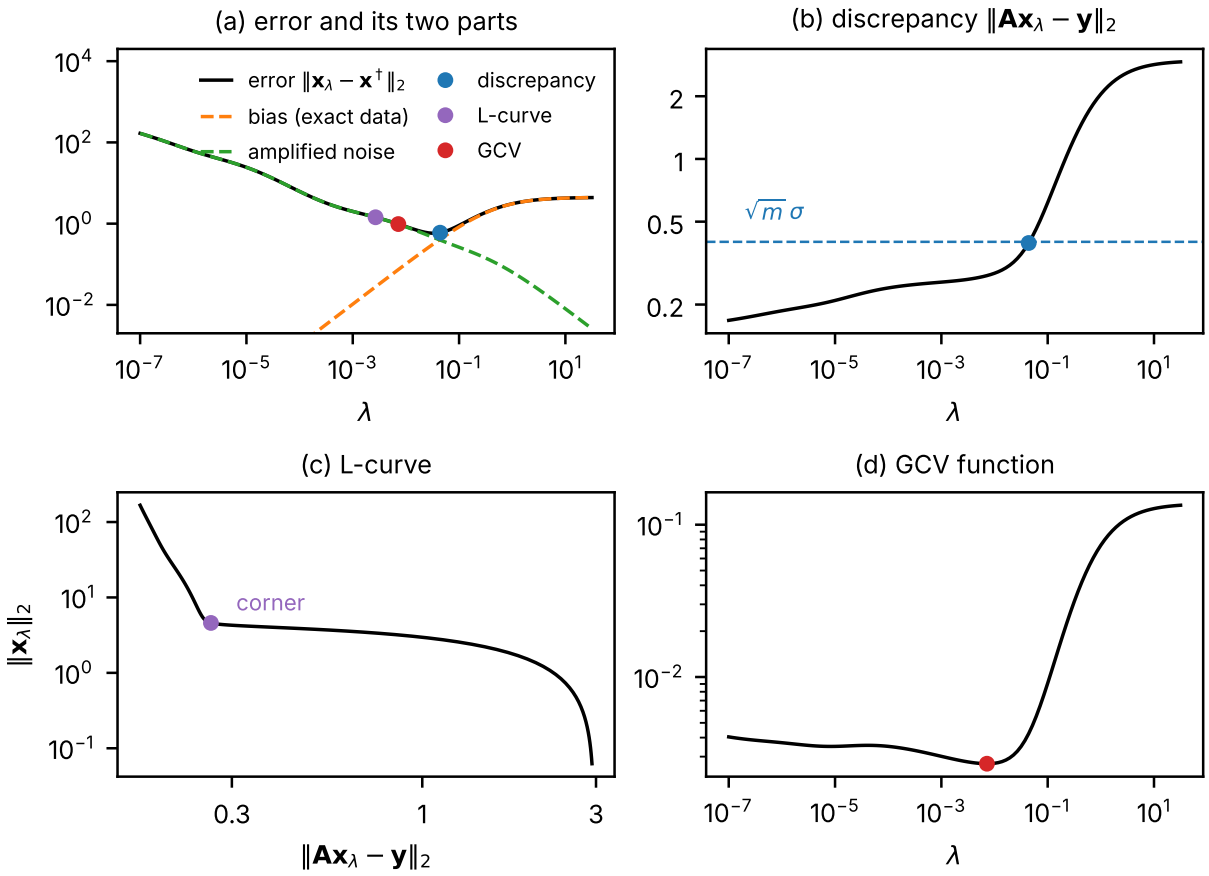


Figure 2.5: Choosing λ for the smooth signal. (a) The error $\left\|\boldsymbol{x}_\lambda - \boldsymbol{x}^\dagger\right\|_2$ and its two parts, bias and amplified noise, with the choices of the three rules marked. (b) The discrepancy against λ and the level $\sqrt{m}\,\sigma$ of the discrepancy principle. (c) The L-curve and its corner. (d) The GCV function and its minimum.

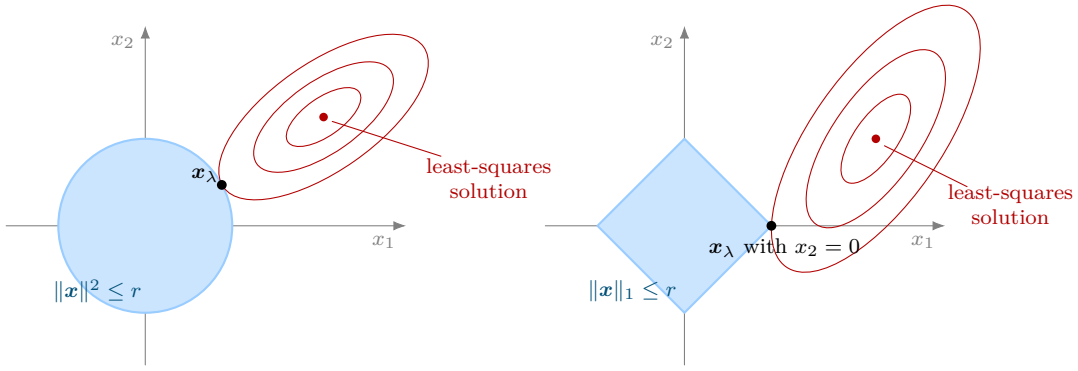
2.4 Sparsity: ℓ^1 regularization

Example 2.4 left us with a signal, the piecewise-constant one, that quadratic penalties cannot handle. The reason is geometric. A quadratic penalty charges every component of $\boldsymbol{x}$ according to its square, so that small components are almost free and nothing is ever set exactly to zero: the penalty prefers to spread a given amount of signal thinly over all components. A signal with a few large jumps and no change elsewhere is the opposite of thin. Figure 2.6 shows the same thing in two dimensions. The regularized solution is the point where the smallest level set of the data misfit, an ellipse around the least-squares solution, touches the set of points with penalty at most some value r. For the ℓ^2 penalty this set is a disc, and an ellipse touches a disc at a point with both coordinates nonzero. For the ℓ^1 penalty it is a diamond, and an ellipse almost always touches a diamond at one of its tips, where one coordinate is exactly zero. The diamond catches solutions on its tips, and in n dimensions the tips are the sparse vectors.

Definition 2.6: ℓ^1 and total-variation regularization

For $\boldsymbol{L} \in \mathbb{R}^{p\times n}$ and $\lambda > 0$, the ℓ^1-regularized solution is

$$\boldsymbol{x}_\lambda = \arg\min_{\boldsymbol{x}} \ \tfrac{1}{2}\,\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \lambda\,\|\boldsymbol{L}\boldsymbol{x}\|_1\,. \tag{2.7}$$



(a) ℓ^2 penalty: the ellipse touches the disc at a point with $x_1 \neq 0$ and $x_2 \neq 0$

(b) ℓ^1 penalty: the ellipse touches the diamond at a tip, where one coordinate is exactly zero

Figure 2.6: Why ℓ^1 penalties produce zeros and ℓ^2 penalties do not. The red ellipses are level sets of the data misfit around the least-squares solution, the blue sets are the points with penalty at most r, and the regularized solution is where the smallest ellipse touches the blue set.

With $\boldsymbol{L} = \boldsymbol{I}$ the solution is sparse: few of its entries are nonzero. With the difference matrix $\boldsymbol{L}$ the penalty is the *total variation* of $\boldsymbol{x}$ [105], and the solution is piecewise constant: few of its increments are nonzero, and edges are preserved rather than smeared.

The price of the diamond is that it has corners. The functional in (2.7) is convex, so it has a well-defined minimizer, but it is not differentiable where a component of $\boldsymbol{L}\boldsymbol{x}$ is zero, which is exactly where the solution wants to be. There are no normal equations, and the minimizer has to be found by a *nonsmooth convex optimization* method. This is a well-developed field, and we only list the names of the main tools: *proximal gradient* methods such as ISTA and its accelerated version FISTA [7], which alternate a gradient step on the misfit with a soft-thresholding step that sets small components to zero; the *alternating direction method of multipliers* (ADMM) [14], which splits the problem into a Tikhonov problem and a thresholding problem and iterates between them; and *iteratively reweighted least squares*, which replaces $\|\boldsymbol{L}\boldsymbol{x}\|_1$ by a sequence of weighted quadratic penalties, so that each step is a Tikhonov problem with weights computed from the previous iterate. The iterative alternating scheme of Chapter 3 belongs to this last family; for large problems, generalized Krylov subspace methods solve the whole sequence of weighted problems in one subspace that grows from step to step [76]. For the exercises, the course code provides a FISTA solver (`solvers.l1_regularization`) that can be used as a black box; it solves (2.7) for our problem sizes in a fraction of a second.

Figure 2.7 shows what the right grammar does. With the total-variation penalty the piecewise-constant signal is recovered with sharp edges and flat plateaus, and the error halves compared with the best Tikhonov solution. The same penalty applied to the smooth signal produces a staircase—the solution is piecewise constant because the penalty says it should be—and the error is almost twice that of Tikhonov regularization.

Remark 2.7: The penalty must match the unknown

There is no universally right regularizer. Smoothness penalties are right for smooth unknowns and wrong for unknowns with edges; sparsity penalties are the other way around. The choice is a statement about what the unknown looks like, made before the data are seen, and it affects the result more than any choice of λ. Deterministic regularization offers no language to make this statement precise or to check it against the data. Chapter 3

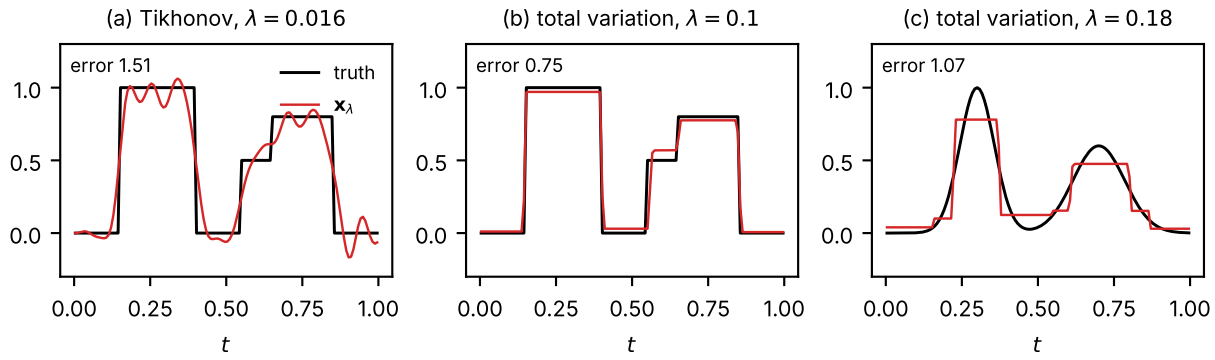


Figure 2.7: Total variation versus Tikhonov on the deconvolution data, with λ chosen by the discrepancy principle in each case. (a) Tikhonov regularization of the piecewise-constant signal smears the edges and rings. (b) Total variation recovers the edges and the plateaus; the error drops from 1.51 to 0.75. (c) Total variation applied to the smooth signal produces a staircase; the error is 1.07 against 0.60 for Tikhonov.

does: there the penalty becomes a probability distribution over unknowns, the *prior*, and the question "which grammar?" becomes "what do I believe the unknown looks like, and how strongly?"

2.5 Nonlinear forward models

Most forward models in the participants' research are nonlinear, like the bathymetry map of Example 1.3. The regularized functional

$$J_\lambda(\boldsymbol{x}) = \tfrac{1}{2}\,\|F(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \tfrac{\lambda}{2}\,\|\boldsymbol{L}\boldsymbol{x}\|_2^2 \tag{2.8}$$

is then no longer quadratic, and three things change. There are no normal equations; the minimizer has to be found iteratively. The functional may have several local minima, and an iterative method finds the one in whose basin it starts, so that the starting point is part of the method. And every iteration needs the gradient

$$\nabla J_\lambda(\boldsymbol{x}) = \boldsymbol{J}_F(\boldsymbol{x})^{\mathsf{T}}\big(F(\boldsymbol{x}) - \boldsymbol{y}\big) + \lambda\,\boldsymbol{L}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{x}, \tag{2.9}$$

where $\boldsymbol{J}_F$ is the Jacobian of F from (1.8), so that computing derivatives of the forward model becomes the central practical question. The landscape metaphor is the right one: J_λ is a mountain range over the space of unknowns, the solution is the bottom of the deepest valley, and an optimization method is a hiker in fog who can only feel the slope under her feet.

Algorithm 2.8: Gauss–Newton with line search

Given a starting point $\boldsymbol{x}_0$, repeat for $k = 0, 1, 2, \ldots$ until the decrease of J_λ becomes negligible:

1. Evaluate the residual $\boldsymbol{r}_k = F(\boldsymbol{x}_k) - \boldsymbol{y}$ and the Jacobian $\boldsymbol{J}_k = \boldsymbol{J}_F(\boldsymbol{x}_k)$.
2. Replace F by its linearization $F(\boldsymbol{x}_k + \boldsymbol{d}) \approx F(\boldsymbol{x}_k) + \boldsymbol{J}_k\boldsymbol{d}$ in (2.8) and minimize the result over the step $\boldsymbol{d}$. This is a Tikhonov problem, solved by its normal equations as in Lemma 2.3:

$$\boldsymbol{d}_k = \arg\min_{\boldsymbol{d}} \ \tfrac{1}{2}\,\|\boldsymbol{J}_k\boldsymbol{d} + \boldsymbol{r}_k\|_2^2 + \tfrac{\lambda}{2}\,\|\boldsymbol{L}(\boldsymbol{x}_k + \boldsymbol{d})\|_2^2 .$$

3. Backtracking line search: set $\alpha = 1$ and halve it until $J_\lambda(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k) \le J_\lambda(\boldsymbol{x}_k) + 10^{-4}\,\alpha\,\nabla J_\lambda(\boldsymbol{x}_k)^{\mathsf{T}} \boldsymbol{d}_k$; then $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha \boldsymbol{d}_k$.

Each iteration solves one linear inverse problem, the one obtained by pretending that F is linear near the current iterate. Close to the solution the method converges fast; far from it the line search keeps it from overshooting. Adding a multiple of the identity to the Tikhonov problem of step 2, which shortens the step when the linearization is poor, gives the Levenberg–Marquardt method. If only the gradient is available, and not the Jacobian, quasi-Newton methods such as L-BFGS build up an approximation of the Hessian from the gradients seen so far; for everything beyond this sketch see Nocedal and Wright [89].

Three ways to a gradient The Jacobian $\boldsymbol{J}_F$ in (2.9) and in Algorithm 2.8 is the one object that the user has to supply, and there are three ways to get it. *Finite differences* perturb each component of $\boldsymbol{x}$ in turn and need n extra evaluations of F per Jacobian: affordable for our $n = 200$ bathymetry cells and half-millisecond forward model, hopeless for $n = 10^5$ and a forward model that takes a minute. The *adjoint method* obtains the gradient of the misfit at the cost of about one extra forward solve, independently of n: for a forward model defined by a linear equation $\boldsymbol{K}(\boldsymbol{x})\boldsymbol{u} = \boldsymbol{f}$ with observations $\boldsymbol{P}\boldsymbol{u}$, one solves the *adjoint equation* $\boldsymbol{K}(\boldsymbol{x})^{\mathsf{T}}\boldsymbol{p} = \boldsymbol{P}^{\mathsf{T}}(F(\boldsymbol{x}) - \boldsymbol{y})$ once and reads off every component of the gradient from $\boldsymbol{p}$. It is the classical route for PDE-constrained problems and the reason why large-scale inversion is possible at all, but deriving and implementing the adjoint of a complex code is a project in itself. *Automatic differentiation* (AD) gets the same result by letting the computer apply the chain rule to the forward code, operation by operation [5, 51]. Forward mode propagates one input direction through the code and costs about one forward evaluation per column of the Jacobian; reverse mode propagates sensitivities backwards and delivers the gradient of a scalar function for a small constant multiple of the cost of one forward evaluation, independently of the number of unknowns, which is the adjoint method, derived automatically. (In our JAX implementation of the bathymetry map the measured wall-clock ratio is about 8; see Chapter 4.) In JAX the forward map of Example 1.3, written as a plain function, differentiates in one line:

```
forward = swe_jax.make_forward(x_cells, x_obs)     # b -> surface at the sensors
jacobian = jax.jit(jax.jacfwd(forward))            # Jacobian by forward mode
misfit = lambda b: 0.5 * jnp.sum((forward(b) - y) ** 2)
grad = jax.grad(misfit)                            # gradient by reverse mode
```

The only requirement is that the forward code is written in the library's array language without side effects; Appendix B lists the pitfalls.

Example 2.9: Gauss–Newton for the bathymetry problem

We apply Algorithm 2.8 to Example 1.3 with the smoothness penalty, the Jacobian from JAX, and the bottom fixed to its known value $b_n = 0$ at the outflow, which sets the datum of the water level. Starting from the flat bottom $\boldsymbol{b}_0 = \boldsymbol{0}$, the method converges in four to five iterations for both sensor layouts (Figure 2.8a); the line search accepts full steps after the first one, which is the behavior of Gauss–Newton near a well-behaved minimum. With λ chosen by the discrepancy principle, the dense sensors give a root-mean-square error of 1.3 cm and the sparse sensors of 2.1 cm, against data noise of 1 cm. The sparse case is the instructive one. In Example 1.11 we found that the data from 20 sensors determine the bottom at the sensors and nothing in between; Figure 2.8c shows what fills the gaps. Without the penalty, the solver returns spikes under the sensors and nothing elsewhere, and the error is three times larger. With the smoothness penalty, the bottom between the sensors is interpolated, smoothly, by the grammar: the null space of Chapter 1 is filled

by the regularizer, not by the data, and the reconstruction between the sensors is only as good as the assumption that the true bottom is smooth.

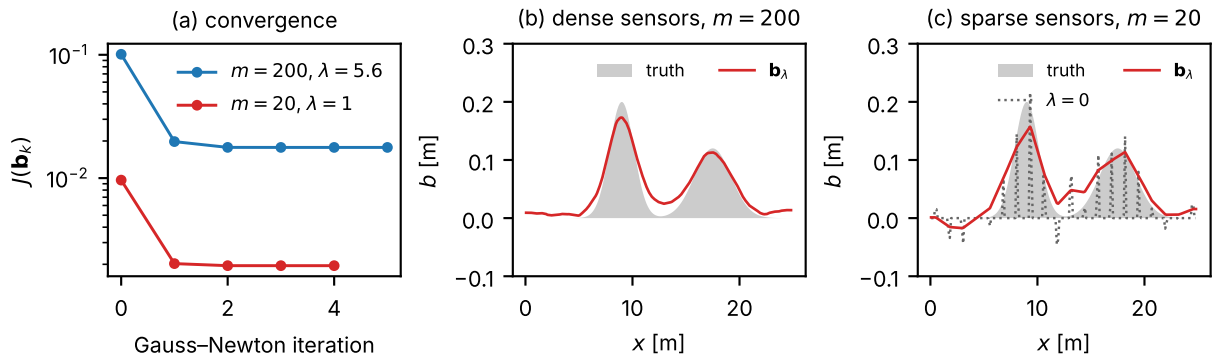


Figure 2.8: Gauss–Newton on the bathymetry problem with the smoothness penalty, λ from the discrepancy principle, Jacobians from JAX. (a) The objective J_λ against the iteration for dense ($m = 200$) and sparse ($m = 20$) sensors. (b) Reconstruction from dense sensors. (c) Reconstruction from sparse sensors with the penalty (red) and without it (dotted): without a grammar, there is nothing between the sensors.

2.6 What we still cannot answer

We can now compute stable solutions of ill-posed problems, linear or not, in a few lines of code. Three questions remain, and they are the questions a user of the reconstruction will ask. Which grammar? We chose smoothness or sparsity by looking at the truth in our examples; in practice the truth is what we are looking for. Which λ? The rules of Section 2.3 work sometimes and need knowledge, of σ, that we may not have. And how wrong is $\boldsymbol{x}_\lambda$? Between the sensors in Figure 2.8c the reconstruction is pure assumption, but nothing in the plot says so. Chapter 3 answers all three with one idea: the penalty is the logarithm of a probability distribution that says what unknowns we consider plausible, λ is its scale, and once everything is a probability, the error of the reconstruction is a probability too.

Summary : Chapter 2 in five sentences

Regularization replaces “reproduce the data” by “fit the data well enough and be reasonable”, with a regularizer that encodes what reasonable means and a parameter λ that sets the exchange rate. Quadratic penalties lead to Tikhonov regularization, a well-conditioned linear problem (the normal equations), which acts as a low-pass filter whose cutoff is set by λ. The discrepancy principle, the L-curve, and generalized cross-validation choose λ from the data, with varying success and without choosing the penalty. ℓ^1 penalties produce sparse or piecewise-constant solutions, preserve edges, and require nonsmooth optimization. For nonlinear forward models, Gauss–Newton solves a sequence of linear inverse problems, and automatic differentiation supplies the Jacobians that it needs.

2.7 Notes and further reading

The theory of regularization, including convergence as the noise goes to zero and the convergence rates that depend on the smoothness of the unknown, is in Engl, Hanke, and Neubauer [31]; the practical side, including the parameter-choice rules and the semi-convergence of iterative methods, is in Hansen [56], with the L-curve in [55] and GCV in [48]. Mueller and Siltanen [83] treat Tikhonov and total-variation regularization with code. Total variation was introduced for image denoising by Rudin, Osher, and Fatemi [105]; proximal methods are in [7], ADMM in [14]. Nocedal and Wright [89] cover Gauss–Newton, Levenberg–Marquardt, and L-BFGS; automatic differentiation is explained in Griewank and Walther [51] and, for the machine-learning reader, in [5].

2.8 Exercises

Exercise 2.1: Normal equations and the filter

(a) Derive the normal equations (2.3) from (2.2) and show that $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\boldsymbol{0}\}$ is necessary as well as sufficient for uniqueness. (b) For the convolution operator and $\boldsymbol{L} = \boldsymbol{I}$, derive the filter (2.4) from (1.6), and show that the noise amplification factor $\varphi(\nu)/\widehat{k}(\nu)$ never exceeds $1/(2\sqrt{\lambda})$. (c) What is the analogue of (2.4) for the smoothness penalty, $\boldsymbol{L}$ the difference matrix? Hint: differences multiply the component at frequency ν by a factor proportional to ν for small ν.

Exercise 2.2: Monotonicity of the discrepancy

Show that for Tikhonov regularization the discrepancy $\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2$ is non-decreasing and the penalty $\|\boldsymbol{L}\boldsymbol{x}_\lambda\|_2$ is non-increasing in λ. Hint: compare $J_\lambda(\boldsymbol{x}_\lambda)$ with $J_\lambda(\boldsymbol{x}_\mu)$ and $J_\mu(\boldsymbol{x}_\mu)$ with $J_\mu(\boldsymbol{x}_\lambda)$ for $\lambda < \mu$. Then show that the discrepancy is *strictly* increasing unless the normal equations $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{y}$ have a solution in $\ker \boldsymbol{L}$, in which case it is constant. Hint: for $\boldsymbol{L} = \boldsymbol{I}$ and the SVD $\boldsymbol{A} = \sum_i \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^{\mathsf{T}}$, $\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2^2 = \sum_{\sigma_i>0} \big(\lambda/(\sigma_i^2+\lambda)\big)^2 (\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{y})^2 + \big\|\boldsymbol{y} - \boldsymbol{A}\boldsymbol{A}^{\dagger}\boldsymbol{y}\big\|_2^2$, which is strictly increasing if and only if $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{y} \neq 0$ for some $\sigma_i > 0$, i.e., $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{y} \neq \boldsymbol{0}$; for general $\boldsymbol{L}$ the generalized SVD of $(\boldsymbol{A}, \boldsymbol{L})$ gives the same structure. Conclude that the discrepancy principle (2.5) has a unique solution whenever $\tau\sqrt{m}\,\sigma$ lies strictly between the discrepancy of the naive solution ($\lambda \to 0$) and that of the best fit within $\ker \boldsymbol{L}$ ($\lambda \to \infty$; for $\boldsymbol{L} = \boldsymbol{I}$ this is $\boldsymbol{x} = \boldsymbol{0}$).

Exercise 2.3: Tikhonov regularization and the three rules (computer)

Use `code/running_examples.py` and `code/solvers.py`. (a) For the smooth signal, compute $\boldsymbol{x}_\lambda$ with $\boldsymbol{L} = \boldsymbol{I}$ for λ between 10^{-7} and 10^{2} and plot the error, the discrepancy, the L-curve, and the GCV function as in Figure 2.5; mark the three choices and the optimum. (b) Repeat with the difference matrix as $\boldsymbol{L}$ and compare the best achievable errors. (c) Repeat (a) for ten noise realizations and report how far each rule's error is from the optimal one on average. (d) Give the discrepancy principle a noise level that is wrong by a factor of two in either direction and report what happens.

Exercise 2.4: Total variation (computer)

Use `solvers.l1_regularization` with `transform="differences"`. (a) For the piecewise-constant signal, compute the total-variation solution for a range of λ, choose λ by the discrepancy principle, and compare the error and the position of the edges with the best Tikhonov solution. (b) Do the same for the smooth signal and describe the staircasing. (c) Replace the noise level 0.05 by 0.2 and repeat (a): at which noise level does total variation lose the smaller plateau?

Exercise 2.5: Gauss–Newton on the bathymetry problem (computer)

Use `code/swe_jax.py` and `solvers.gauss_newton`. (a) Reproduce Figure 2.8 for $m = 20$ and $m = 200$ sensors. (b) Replace the smoothness penalty by $\boldsymbol{L} = \boldsymbol{I}$ and explain the difference between the sensors for $m = 20$. (c) Start Gauss–Newton from a bottom that lies 0.5 m below the true one everywhere and from one that puts the hills in the wrong places; does it find the same minimum? Why can it not be started from a bottom 0.5 m above the true one? (d) Compare the Jacobian from `jax.jacfwd` with the finite-difference Jacobian of `running_examples.jacobian_fd` in accuracy and in time.

Exercise 2.6: Your own inverse problem, regularized

For the problem you described in Exercise 1.5, write down the regularized functional (2.1): which penalty, and why? Is the forward map linear? If not, which of the three ways to a gradient is available to you, and what would one Gauss–Newton iteration cost? Which rule would you use to choose λ, and do you know your noise level?

Chapter 3

Bayesian Inverse Problems and MAP Estimation

The corridor, one last time (Figure 3.1). In Chapter 2 the listener picked the sentence that fit the sound well enough and was a sentence; she had to decide how to weigh the two requirements, and she had to pick one sentence. Now she does what people actually do—she gives odds; that human perception works this way, weighing how well a candidate explains the sensory input against how likely the candidate was to begin with, is the subject of a large literature in psychology and neuroscience [70, 90]. "Let's meet at five" is likely, "Let's eat at five" is possible, "Let's meet at nine" is unlikely. The odds come from two sources: how well a sentence explains the sound she heard and how often people say such things. Nothing has been thrown away. The odds contain the best guess, but also the second best and how far behind it is. This is the Bayesian formulation of the inverse problem, and it answers the three questions that Section 2.6 left open: the penalty of Chapter 2 becomes a probability distribution over unknowns, our belief about the structure of the unknown, which can be sampled and inspected; the parameter λ becomes a quantity with a meaning; and the spread of the odds replaces the error curve that we could only draw with the truth in hand. This chapter sets up the formulation (Section 3.1), identifies its two ingredients (Sections 3.2 and 3.3), works out the one case in which everything can be computed by hand (Section 3.4), and shows that the methods of Chapter 2 are its most probable answers (Section 3.5).

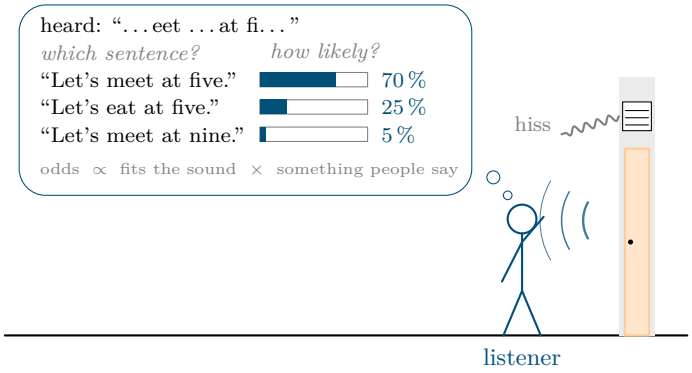


Figure 3.1: The corridor, part three. The listener no longer picks one sentence; she gives odds. The odds of a candidate are the product of two factors: how well it explains what she heard (the likelihood) and how often people say such things (the prior). "Let's eat at five" explains the sound as well as "Let's meet at five" but is said less often at an office door; "Let's meet at nine" is a common sentence that explains the sound poorly. The list of candidates with their odds is the posterior.

3.1 Bayes' theorem and the posterior

Half a page of probability. We need very little probability, and the pre-reading (Appendix A) has all of it; here is the half page we use. A random vector $\boldsymbol{x} \in \mathbb{R}^n$ has a *density* $p(\boldsymbol{x}) \geq 0$ with $\int p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = 1$, and the probability that $\boldsymbol{x}$ lies in a set B is $\int_B p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$. Two random vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ have a *joint* density $p(\boldsymbol{x}, \boldsymbol{y})$; each has a *marginal* density, $p(\boldsymbol{x}) = \int p(\boldsymbol{x}, \boldsymbol{y})\,\mathrm{d}\boldsymbol{y}$; and the *conditional* density of $\boldsymbol{x}$ given that $\boldsymbol{y}$ has been observed is $p(\boldsymbol{x} \mid \boldsymbol{y}) = p(\boldsymbol{x}, \boldsymbol{y})/p(\boldsymbol{y})$. We write p for every density and let the argument say which one is meant. Writing the joint density in both orders, $p(\boldsymbol{x}, \boldsymbol{y}) = p(\boldsymbol{y} \mid \boldsymbol{x})\,p(\boldsymbol{x}) = p(\boldsymbol{x} \mid \boldsymbol{y})\,p(\boldsymbol{y})$, turns one conditional into the other:

$$p(\boldsymbol{x} \mid \boldsymbol{y}) = \frac{p(\boldsymbol{y} \mid \boldsymbol{x})\,p(\boldsymbol{x})}{p(\boldsymbol{y})}. \tag{3.1}$$

This is *Bayes' theorem*, and it is all the theory this chapter needs. It is older than the word statistics. Thomas Bayes derived the special case of an unknown success probability in an essay published after his death in 1763 [6], to answer the question of what a number of observed successes and failures say about the unknown chance of success, i.e., an inverse problem. Pierre-Simon Laplace found the general rule independently in 1774 [73], called it the probability of causes given events, and applied it to the orbits of planets and comets and to the ratio of boy to girl births in Paris; Stigler [112] tells the story. Until well into the twentieth century the subject was called *inverse probability*, which is the name under which ours was born. The density we use most is the Gaussian:

$$\mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Gamma}) = \frac{1}{\sqrt{(2\pi)^n \det \boldsymbol{\Gamma}}} \exp\Big(-\tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}} \boldsymbol{\Gamma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu})\Big), \quad \boldsymbol{x} \in \mathbb{R}^n, \tag{3.2}$$

with mean $\boldsymbol{\mu}$ and symmetric positive definite covariance $\boldsymbol{\Gamma}$. Notably, (3.2) is the density of $\boldsymbol{\mu} + \boldsymbol{\Gamma}^{1/2}\boldsymbol{\xi}$ for a standard normal $\boldsymbol{\xi}$, which is also how one draws samples from it (Appendix A).

Remark 3.1: Randomness as a model of ignorance

Nothing in the bathymetry problem is random in the everyday sense: the bottom of the channel is what it is, one fixed function. Why, then, a density on it? Two readings of probability have coexisted for two centuries. In the *frequentist* reading, a probability is the long-run frequency of a repeatable event. Only the data are random, through the noise; the unknown is a fixed quantity and has no distribution; and statements are about procedures, not about the unknown. A frequentist 95 % *confidence* interval is a recipe that, applied to many data sets, covers the true value in 95 percent of the cases, and it says nothing about whether the one interval we computed does. In the *Bayesian* reading, a probability measures a state of knowledge, and randomness is the mathematical model of not knowing. The prior $p(\boldsymbol{x})$ does not claim that nature draws bottoms at random; it records which bottoms we consider plausible before measuring, the posterior records which we consider plausible afterwards, and a 95 % *credible* interval is a statement about this unknown given these data. For inverse problems the second reading is the useful one, for two reasons. The unknown is a single object about which we want to say something, and prior knowledge enters anyway, as Chapter 2 showed, so that it is better to write it down as a distribution than to hide it in a penalty. The price is that the answer depends on the prior, and the chapter does not hide that either. Kaipio and Somersalo [64] and Jaynes [62] discuss the two readings at length; Gelman et al. [37] is the standard account of Bayesian data analysis.

Definition 3.2: The Bayesian inverse problem

Let the unknown $\boldsymbol{x} \in \mathbb{R}^n$ and the noise $\boldsymbol{\varepsilon}$ in the data model $\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{\varepsilon}$ of Definition 1.4 be random and independent.

- The *prior* $p(\boldsymbol{x})$ says what we believe about the unknown before seeing the data.
- The *likelihood* $p(\boldsymbol{y} \mid \boldsymbol{x})$ is the density of the data for a fixed unknown: the data model read as a statement about $\boldsymbol{y}$.
- The *posterior* $p(\boldsymbol{x} \mid \boldsymbol{y})$, given by (3.1), says what we believe about the unknown after seeing the data.
- The *evidence* $p(\boldsymbol{y}) = \int p(\boldsymbol{y} \mid \boldsymbol{x})\, p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}$ is the probability of the data under the model, with the unknown integrated out.

The posterior is the solution of the Bayesian inverse problem.

Three remarks on reading (3.1). (1) The evidence does not depend on $\boldsymbol{x}$, so for everything that concerns the shape of the posterior we may write $p(\boldsymbol{x} \mid \boldsymbol{y}) \propto p(\boldsymbol{y} \mid \boldsymbol{x})\, p(\boldsymbol{x})$ and forget the denominator; it returns in Remark 3.17. (2) The posterior is a density on $\mathbb{R}^n$, and for $n \gg 1$ nobody will ever integrate it or plot it. This chapter is about what can be computed instead, namely the posterior in closed form when everything is Gaussian (Section 3.4), its most probable point (Section 3.5)—and, in Chapter 4, samples from it. (3) The word *solution* has changed its meaning for the third time: in Section 1.4 we promised to describe what is compatible with the data and with prior knowledge, and (3.1) is that description.

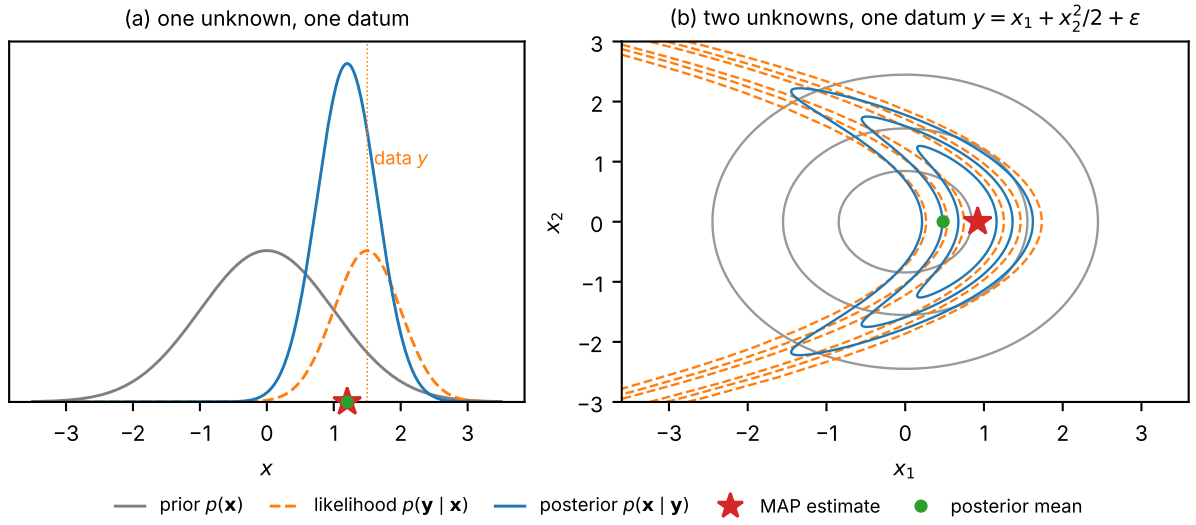


Figure 3.2: Prior times likelihood is the posterior. (a) One unknown with the prior $\mathcal{N}(0, 1)$ and one datum $y = x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.5^2)$, $y = 1.5$: the posterior is pulled towards the data but not all the way, and it is narrower than both of its factors; its MAP estimate (star) and its mean (dot) coincide. (b) Two unknowns with the prior $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and one datum $y = x_1 + x_2^2/2 + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.3^2)$, $y = 1$. The likelihood (dashed) is constant along a parabola: one equation cannot determine two unknowns. The prior (gray) decides where on the parabola, and the posterior (blue) is a bent ridge whose maximum (star) and mean (dot) are different points.

Figure 3.2 shows (3.1) in one and two dimensions. Panel (b) is the whole course in one

picture. The data concentrates the unknown on a curve, the two-dimensional version of the null space of Chapter 1; the prior decides where on the curve; and the result is a shape, not a point. Two points are nevertheless worth naming, because they are what we can compute and report.

Definition 3.3: Point estimates and spread

The *maximum a posteriori* (MAP) estimate is the most probable unknown,

$$\boldsymbol{x}_{\mathrm{MAP}} \in \arg\max_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{y}), \tag{3.3}$$

and the *posterior mean* (also *conditional mean*, CM) is the average unknown, $\boldsymbol{x}_{\mathrm{CM}} = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}] = \int \boldsymbol{x}\, p(\boldsymbol{x} \mid \boldsymbol{y})\, \mathrm{d}\boldsymbol{x}$. The *posterior covariance* $\boldsymbol{\Gamma}_{\mathrm{post}} = \mathbb{E}[(\boldsymbol{x} - \boldsymbol{x}_{\mathrm{CM}})(\boldsymbol{x} - \boldsymbol{x}_{\mathrm{CM}})^{\mathsf{T}} \mid \boldsymbol{y}]$ measures the spread; its diagonal gives the posterior standard deviation of each component. A 95 % *credible interval* for the component x_i is an interval $[a_i, b_i]$ with

$$\int_{a_i}^{b_i} p(x_i \mid \boldsymbol{y})\, \mathrm{d}x_i = 0.95, \tag{3.4}$$

where $p(x_i \mid \boldsymbol{y})$ is the marginal posterior density of x_i, i.e., the posterior with all other components integrated out. The interval is not unique; one takes the equal-tailed interval, whose endpoints are the 2.5 and 97.5 percent quantiles of $p(x_i \mid \boldsymbol{y})$, or the shortest one. For a Gaussian posterior both are $(\boldsymbol{x}_{\mathrm{CM}})_i \pm 1.96\sqrt{(\boldsymbol{\Gamma}_{\mathrm{post}})_{ii}}$. A *credible band* is the collection of the intervals of all components; it is a statement about each component separately, not about the vector as a whole.

For a Gaussian posterior, the MAP and the posterior mean coincide. In general, they do not coincide, however. This is illustrated by Figure 3.2b: the mean of a bent ridge lies inside the bend, at a point of low probability. Which estimate to report is a question about what the estimate is for, and we return to it in Section 3.5.

MAP estimation recovers deterministic regularized inverse problems. Take the negative logarithm of (3.1):

$$-\log p(\boldsymbol{x} \mid \boldsymbol{y}) = \underbrace{-\log p(\boldsymbol{y} \mid \boldsymbol{x})}_{\text{misfit}} \underbrace{-\log p(\boldsymbol{x})}_{\text{penalty}} + \log p(\boldsymbol{y}). \tag{3.5}$$

Compare with the regularized functional $J_\lambda(\boldsymbol{x}) = \frac{1}{2}\|F(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \lambda R(\boldsymbol{x})$ of (2.1). The negative log-likelihood will turn out to be the data misfit, the negative log-prior the penalty, and the most probable unknown the regularized solution. Table 3.1 is the dictionary; the rest of the chapter fills in its rows.

Table 3.1: The dictionary between Chapter 2 and this chapter.

Chapter 2	Chapter 3	where
data misfit $\frac{1}{2}\|F(\boldsymbol{x}) - \boldsymbol{y}\|_2^2$	negative log-likelihood $-\log p(\boldsymbol{y} \mid \boldsymbol{x})$	Section 3.2
regularizer $\lambda R(\boldsymbol{x})$	negative log-prior $-\log p(\boldsymbol{x})$	Section 3.3
functional $J_\lambda(\boldsymbol{x})$	$-\log p(\boldsymbol{x} \mid \boldsymbol{y})$ up to a constant	Section 3.5
regularized solution $\boldsymbol{x}_\lambda$	MAP estimate $\boldsymbol{x}_{\mathrm{MAP}}$	Section 3.5
parameter λ	ratio of variances σ^2/γ^2, a hyper-parameter	Section 3.4
error curve (needs the truth)	posterior spread (needs only the data)	Section 3.4
rules for choosing λ	choose, infer, or estimate the hyper-parameters	Remark 3.17

3.2 From noise models to likelihoods

The likelihood is the data model of Definition 1.4 read as a density of $\boldsymbol{y}$. If the noise is Gaussian with covariance $\boldsymbol{\Gamma}$ and independent of $\boldsymbol{x}$, then for fixed $\boldsymbol{x}$ the data $\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{\varepsilon}$ are Gaussian with mean $F(\boldsymbol{x})$ and the same covariance, so that

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y}; F(\boldsymbol{x}), \boldsymbol{\Gamma}) \propto \exp\Big(-\tfrac{1}{2}\|\boldsymbol{\Gamma}^{-1/2}(F(\boldsymbol{x}) - \boldsymbol{y})\|_2^2\Big). \tag{3.6}$$

For independent sensors with a common noise level, $\boldsymbol{\Gamma} = \sigma^2 \boldsymbol{I}$, the negative log-likelihood is

$$-\log p(\boldsymbol{y} \mid \boldsymbol{x}) = \frac{1}{2\sigma^2}\,\|F(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \text{const}, \tag{3.7}$$

which is the data misfit of Chapter 2, with the noise level built in as the unit in which misfit is measured. This fills the first row of Table 3.1, and it says what the ear of the corridor is: the likelihood measures how well an $\boldsymbol{x}$ explains the data, in units of the noise.

Two things about (3.6) are easy to get wrong. First, the likelihood is a function of $\boldsymbol{x}$ but not a density in $\boldsymbol{x}$. It does not integrate to one over $\boldsymbol{x}$, and for a linear forward operator with $m < n$ it is constant along the null space of $\boldsymbol{A}$, the invisible directions of Chapter 1. Maximizing the likelihood alone is the naive solution of Section 1.3 in disguise, which is why the prior is not optional. Second, $\boldsymbol{\Gamma}$ is part of the model and has to come from somewhere: from repeated measurements, from the instrument's specification, or from the high-frequency content of the data. Independent sensors give a diagonal $\boldsymbol{\Gamma}$; correlated noise, as from a drifting instrument, gives a full $\boldsymbol{\Gamma}$, and (3.6) then weights the misfit accordingly. A noise level that is wrong by a factor of two makes the data count four times too much or too little in (3.7), which is the Bayesian version of the warning in Section 2.3. Figure 3.3 shows both faces of the likelihood in two dimensions.

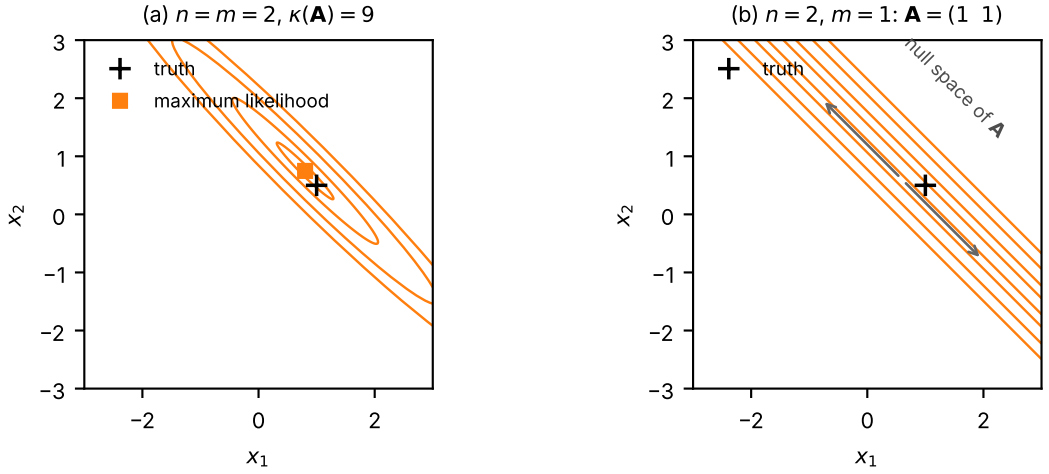


Figure 3.3: The Gaussian likelihood (3.6) as a function of the unknown, for two unknowns and $\sigma = 0.3$. (a) Two data, $\boldsymbol{A} = \left(\begin{smallmatrix} 1 & 0.8 \\ 0.8 & 1 \end{smallmatrix}\right)$ with $\kappa(\boldsymbol{A}) = 9$: the likelihood is a Gaussian in $\boldsymbol{x}$ whose contours are ellipses stretched along the weak direction of $\boldsymbol{A}$; its maximum is the least-squares solution, which lies 0.3 away from the truth along that direction. (b) One datum, $\boldsymbol{A} = (1 \;\; 1)$: the likelihood is constant along the null space of $\boldsymbol{A}$ and has no maximum; along the line only a prior can decide.

Remark 3.4: Maximum likelihood, and when it is enough

The *maximum-likelihood* estimate $\boldsymbol{x}_{\mathrm{ML}} \in \arg\max_{\boldsymbol{x}} p(\boldsymbol{y} \mid \boldsymbol{x})$ uses no prior; for Gaussian noise it is the least-squares solution, and hence the naive solution of Chapter 1. It is the right tool when the problem is well-posed. For a linear forward operator of full column rank, $\boldsymbol{x}_{\mathrm{ML}} = \boldsymbol{A}^{\dagger}\boldsymbol{y}$ has mean $\boldsymbol{x}^{\dagger}$ and covariance $\sigma^2(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}$ over the noise, and when there are many more data than unknowns, $m \gg n$, and $\boldsymbol{A}$ is well-conditioned, this covariance is small and shrinks as more data arrive (Figure 3.3a). This is the situation of classical statistics: a few parameters, many observations, and no need for a prior. Inverse problems are the opposite situation. With $m < n$ the likelihood is constant along the null space and has no maximum (Figure 3.3b); with $m \geq n$ but $\kappa(\boldsymbol{A})$ large, the covariance $\sigma^2(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})^{-1}$ has eigenvalues σ^2/σ_i^2, up to σ^2/σ_n^2, and the maximum exists but is worthless, as Figure 1.7 showed. In both cases the data do not determine the unknown, and what the data do not determine, the prior must.

Other noise. Gaussian noise is the default, not a law. Counting detectors, as in computed tomography at low dose, deliver Poisson-distributed counts, $y_i \sim \mathrm{Poisson}(F_i(\boldsymbol{x}))$, and the negative log-likelihood becomes $\sum_i \big(F_i(\boldsymbol{x}) - y_i \log F_i(\boldsymbol{x})\big)$ up to a constant, with a noise level that grows with the signal; for large counts it is well approximated by (3.6) with $\boldsymbol{\Gamma} = \operatorname{diag} F(\boldsymbol{x})$. Multiplicative noise, $y_i = F_i(\boldsymbol{x})(1+\varepsilon_i)$, is Gaussian in the logarithm of the data. Heavy-tailed noise, for instance Laplace-distributed, gives a negative log-likelihood proportional to $\sum_i |F_i(\boldsymbol{x}) - y_i|$, which is robust against outliers in the same way in which the ℓ^1 penalty is robust against small components. In every case, the likelihood is read off from the noise model, and the modeling decision is the noise model.

Remark 3.5: Model error is noise too

The data are produced by nature, $\boldsymbol{y} = F_{\mathrm{exact}}(\boldsymbol{x}) + \boldsymbol{\varepsilon}$, but we often compute with a discretized and simplified forward model F_h:

$$\boldsymbol{y} = F_h(\boldsymbol{x}) + \underbrace{\big(F_{\mathrm{exact}}(\boldsymbol{x}) - F_h(\boldsymbol{x})\big)}_{\text{model error}} + \boldsymbol{\varepsilon}.$$

The model error may be neither white nor small, and in many applications it is larger than the measurement noise. The honest treatment is to regard it as noise with its own mean and covariance, which can be estimated by running the accurate and the cheap model on samples from the prior and comparing; this is the *enhanced error model* of Kaipio and Somersalo [65]. The dishonest treatment is to generate synthetic data with the same F_h that is used for the inversion, so that the model error vanishes by construction; this is the *inverse crime*, and the test problems in these notes commit it knowingly, because their purpose is to isolate other effects.

Example 3.6: The likelihood of signal denoising

The simplest inverse problem is denoising: $F = \boldsymbol{I}$ and $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2\boldsymbol{I})$, so that

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y}; \boldsymbol{x}, \sigma^2\boldsymbol{I}) \propto \exp\Big(-\frac{1}{2\sigma^2}\,\|\boldsymbol{x} - \boldsymbol{y}\|_2^2\Big).$$

As a function of $\boldsymbol{x}$ this is a Gaussian centered at the data, as wide as the noise, and it says exactly what the data say: every component of $\boldsymbol{x}$ lies within about σ of the corresponding datum. The maximum-likelihood estimate is $\boldsymbol{y}$ itself. The problem is as well-posed as a

problem can be, $\kappa(\boldsymbol{I}) = 1$, and the maximum-likelihood estimate is as good as Remark 3.4 promises, with an error equal to the noise, no larger and no smaller; it is just not a denoised signal. The likelihood has no opinion about which part of $\boldsymbol{y}$ is signal and which is noise, and an error below the noise level is possible only for an estimate that has one, i.e., a prior: in denoising the prior does all the work.

Example 3.7: The likelihood of the bathymetry data

In Example 1.3 the free surface is measured at m sensors with independent errors of standard deviation $\sigma = 1\,\mathrm{cm}$, so that

$$-\log p(\boldsymbol{y} \mid \boldsymbol{b}) = \frac{1}{2\sigma^2}\,\|F(\boldsymbol{b}) - \boldsymbol{y}\|_2^2 + \text{const} = 5000\,\mathrm{m}^{-2} \cdot \|F(\boldsymbol{b}) - \boldsymbol{y}\|_2^2 + \text{const}.$$

The number $1/(2\sigma^2) = 5000\,\mathrm{m}^{-2}$ is the exchange rate of the data: it converts a squared misfit in meters into a log-probability. A bottom whose predicted surface is off by one centimeter at every one of 20 sensors is e^{-10} times less probable than a bottom that fits exactly, before the prior has said anything; a bottom that is off by one millimeter everywhere is $e^{-0.1}$ times less probable, i.e., essentially as probable. The likelihood does not care about the bottom between the sensors, because F does not (Example 1.11).

3.3 Priors: Gaussian, Laplace, and hierarchical

A prior is a density on $\mathbb{R}^n$ written down before the data are seen. It encodes our belief about the structure of the unknown—its size, its smoothness, whether it has edges—before the data have said anything. Each of the four penalties of Section 2.1 is the negative logarithm of a prior, and the Bayesian formulation adds something the penalties did not have: a prior can be sampled, and the samples show what it believes. Figure 3.4 shows three samples from each prior of this section, applied to the unknown itself (top row) and to its increments (bottom row). We recommend looking at it before reading on; no formula says as clearly what a prior means as a few draws from it.

3.3.1 Gaussian priors

The simplest prior is $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{0}, \gamma^2 \boldsymbol{I})$, with $-\log p(\boldsymbol{x}) = \|\boldsymbol{x}\|_2^2/(2\gamma^2)$ up to a constant: the penalty on size of Section 2.1, with $\lambda R(\boldsymbol{x})$ replaced by a term whose weight is $1/(2\gamma^2)$. Its samples (Figure 3.4a) are white noise of standard deviation γ, and γ is a pure scale: it sets the size of the samples and nothing else. Nobody believes that a signal looks like static; the merit of this prior is that it is harmless, since it says nothing but "not much larger than γ". A general Gaussian prior $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma}_{\text{prior}})$ can say more, because its covariance can correlate neighboring values. The *Gaussian process* priors of machine learning [99] are of this kind; with the squared-exponential covariance $(\boldsymbol{\Gamma}_{\text{prior}})_{ij} = \gamma^2 \exp(-(t_i - t_j)^2/(2\ell^2))$ the samples are smooth curves that vary on the scale of the correlation length ℓ. We mostly use the precision form that we already know from Chapter 2: the *smoothness prior*

$$p(\boldsymbol{x}) \propto \exp\Big(-\frac{1}{2\gamma^2}\,\|\boldsymbol{L}\boldsymbol{x}\|_2^2\Big), \quad \text{i.e., the increments } x_{i+1} - x_i \text{ are independent } \mathcal{N}(0, \gamma^2), \tag{3.8}$$

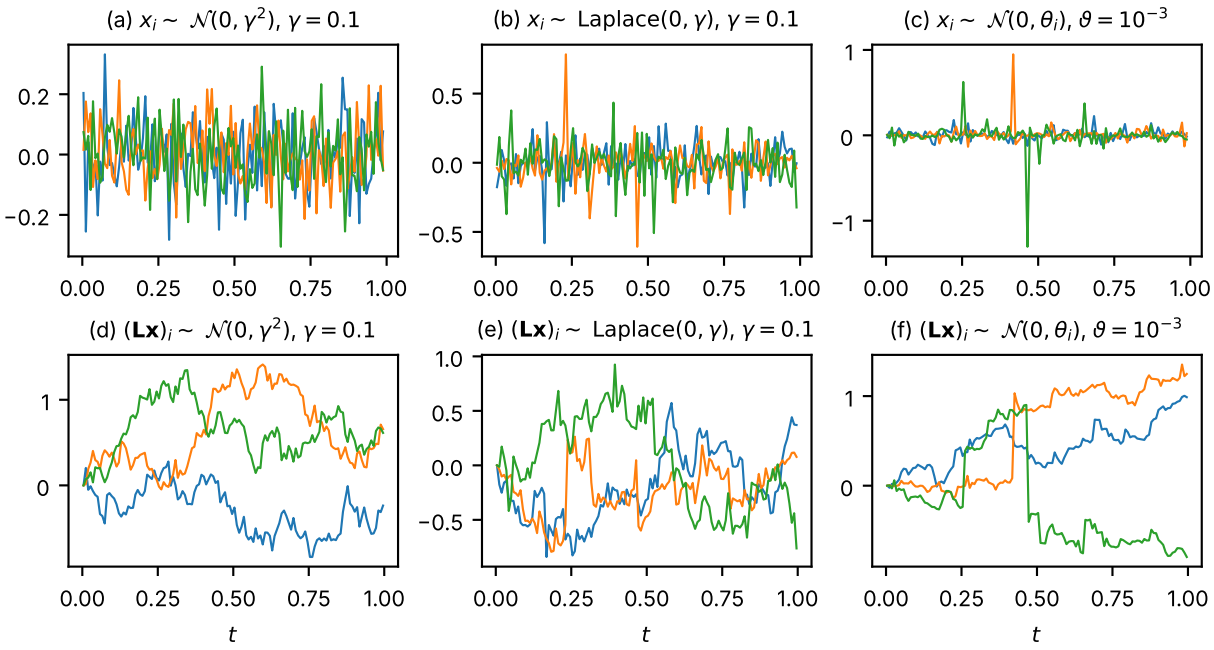


Figure 3.4: Three samples from each prior of Section 3.3 on $n = 128$ points. Top row: independent components. (a) Gaussian with $\gamma = 0.1$: static. (b) Laplace with $\gamma = 0.1$: static with outliers, not sparse. (c) Hierarchical (3.10) with $\beta = 1$ and $\vartheta = 10^{-3}$: sparse spikes. Bottom row: the same draws used as increments, started at $x_1 = 0$. (d) Gaussian increments, the smoothness prior (3.8): random walks. (e) Laplace increments, the total-variation prior: random walks with the occasional larger step, not piecewise constant. (f) Hierarchical increments: a few jumps on a nearly flat background.

with the difference matrix $\boldsymbol{L}$ of Section 2.1. Its samples (Figure 3.4d) are the cumulative sums of the samples above them: random walks, continuous but rough, and again γ only sets how far they wander. This deserves a pause. The prior on first differences does not generate smooth signals. Its MAP estimate is smooth, because the MAP is a compromise between the data and the penalty, but what the prior believes is "the next value is close to the current one", and nothing more. Smoothness in the sense of differentiability comes from penalizing second differences, $x_{i+1} - 2x_i + x_{i-1}$, whose samples are integrated random walks and look smooth, like those of the squared-exponential covariance (Exercise 3.4). Which Gaussian prior is right is a statement about the unknown, and the samples are the way to check it.

3.3.2 Priors on transforms of the unknown

The smoothness prior (3.8) is a prior on $\boldsymbol{Lx}$ rather than on $\boldsymbol{x}$, and the idea is familiar from Chapter 2: any penalty can be applied to a transform of the unknown, to increments for edges, to wavelet or Fourier coefficients for signals that are sparse in those bases. In Figure 3.4 every prior appears twice, applied to $\boldsymbol{x}$ in the top row and to $\boldsymbol{Lx}$ in the bottom row. One thing is new. The right-hand side of (3.8) does not integrate to one over $\mathbb{R}^n$, because adding a constant to $\boldsymbol{x}$ does not change $\boldsymbol{Lx}$: the density is flat along $\ker \boldsymbol{L} = \operatorname{span}\{\mathbf{1}\}$, and such a prior is called *improper*. An improper prior is harmless as long as the data see the missing direction, which is the condition $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\mathbf{0}\}$ of Lemma 2.3, but it cannot be sampled and it has no evidence. We therefore complete it: the level x_1 gets its own weak prior $\mathcal{N}(0, \gamma_0^2)$, which makes

the precision $\boldsymbol{L}^\mathsf{T}\boldsymbol{L}/\gamma^2 + \boldsymbol{e}_1\boldsymbol{e}_1^\mathsf{T}/\gamma_0^2$ invertible, and the samples in Figure 3.4 simply start at $x_1 = 0$. In the bathymetry problem the known bottom at the outflow, $b_n = 0$, is exactly a prior on the component in $\ker \boldsymbol{L}$, and we saw in Example 2.9 what happens without it. The rule is general: a prior on $\boldsymbol{L}\boldsymbol{x}$ needs a prior on $\ker \boldsymbol{L}$.

3.3.3 Laplace priors

The sparsity penalty of Section 2.4 is the negative logarithm of the *Laplace prior*,

$$p(\boldsymbol{x}) \propto \exp(-\left\|\boldsymbol{x}\right\|_1/\gamma) = \prod_{i=1}^{n} \frac{1}{2\gamma} \exp(-\left|x_i\right|/\gamma), \tag{3.9}$$

under which the components are independent and Laplace-distributed with scale γ; applied to increments, $p(\boldsymbol{x}) \propto \exp(-\left\|\boldsymbol{L}\boldsymbol{x}\right\|_1/\gamma)$, it is the *total-variation prior*. Their samples (Figure 3.4b,e) are a surprise. The Laplace samples are not sparse: a Laplace-distributed number is exactly zero with probability zero, and all the prior does is to allow larger outliers than a Gaussian of the same scale; γ is again a pure scale. The total-variation samples are random walks with the occasional larger step, not piecewise-constant signals. We will see in Section 3.5 that the MAP estimates under these priors are sparse and piecewise constant, exactly as in Chapter 2; the priors themselves do not believe it. This matters beyond aesthetics. Where the data are silent, posterior samples inherit the character of the prior, so that individual samples from the posterior under the total-variation prior will not be piecewise constant either. We generally want posterior samples to be plausible solutions of the problem, with the structure we expect of the solution, because a sample that could not be the answer is a poor way of expressing uncertainty about the answer. A method whose point estimate has the right structure but whose samples do not is only half a method.

3.3.4 Hierarchical priors

A Gaussian prior with one scale γ for all components is rigid: it has to be small to keep the plateaus flat and large to allow the jumps, and it cannot be both. The remedy is to give every component its own variance and to let the data decide which components get a large one. This is the *hierarchical* or *generalized sparse Bayesian learning* (GSBL) prior [20, 45, 117]:

$$x_i \mid \theta_i \sim \mathcal{N}(0, \theta_i), \quad \theta_i \sim \mathrm{InvGamma}(\beta, \vartheta), \quad i = 1, \dots, n, \tag{3.10}$$

where the *hyper-prior* on the local variance $\theta_i > 0$ is the inverse gamma distribution with density

$$p(\theta) = \frac{\vartheta^\beta}{\Gamma(\beta)}\, \theta^{-\beta-1} \exp(-\vartheta/\theta), \tag{3.11}$$

whose mode is $\vartheta/(\beta+1)$ and whose right tail decays only like $\theta^{-\beta-1}$. The shape parameter β controls the tail, and ϑ is a scale; we fix $\beta = 1$ and $\vartheta = 10^{-3}$ throughout these notes, so that the hyper-prior has its mode at $\vartheta/2 = 5 \cdot 10^{-4}$ and an infinite mean. The joint prior of $\boldsymbol{x}$ and $\boldsymbol{\theta}$ is $p(\boldsymbol{x}, \boldsymbol{\theta}) = \prod_i \mathcal{N}(x_i; 0, \theta_i)\, p(\theta_i)$. Figure 3.5 shows how it produces sparsity, and the heuristic is worth stating in words. A sample is drawn in two steps. First the hyper-prior decides, independently for each component, whether it is a quiet one, $\theta_i \approx \vartheta$, or a loud one, $\theta_i \gg \vartheta$; most draws land near the mode, but the heavy tail makes loud ones rare rather than impossible (Figure 3.5a). Then each component is drawn from a Gaussian of the chosen width: a spike at zero for the quiet ones, a wide bell for the loud ones (Figure 3.5b). The result is a signal

that is almost zero almost everywhere and large in a few places, i.e., sparse by construction. With the prior on increments, $(\boldsymbol{L}\boldsymbol{x})_i \mid \theta_i \sim \mathcal{N}(0, \theta_i)$, a loud increment is a jump, and the signal is piecewise constant between the rare jumps, up to a small random walk contributed by the quiet increments (Figure 3.5c). Figure 3.4c,f shows the result for whole vectors: sparse spikes, and with the prior on increments a few jumps on a nearly flat background; ϑ, being a scale, only sets their amplitude, which grows like $\sqrt{\vartheta}$. The prior is called hierarchical because its parameters have a prior of their own, a structure that Chapter 4 exploits: given $\boldsymbol{\theta}$, the prior on $\boldsymbol{x}$ is Gaussian, and everything Gaussian is computable.

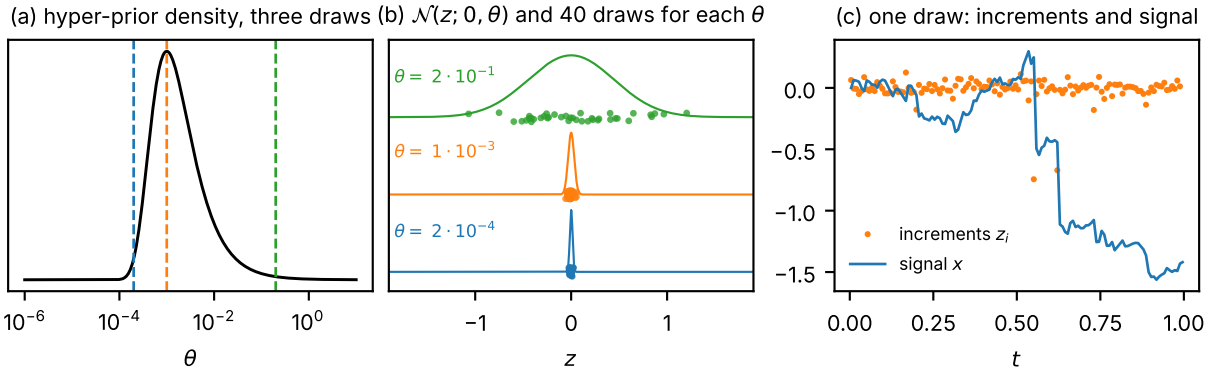


Figure 3.5: How the hierarchical prior (3.10) on increments produces piecewise-constant signals, for $\beta = 1$ and $\vartheta = 10^{-3}$. (a) The hyper-prior density (3.11) on a logarithmic axis, with three draws of θ marked: two near the mode, one in the heavy tail. (b) For each of the three θ, the conditional density $\mathcal{N}(z; 0, \theta)$ (scaled to equal height) and forty draws from it: the quiet ones stay at zero, the loud one does not. (c) One draw of all 127 increments $z_i \sim \mathcal{N}(0, \theta_i)$ and the signal with those increments: a jump wherever θ_i is large, nearly flat elsewhere.

Remark 3.8: The Cauchy prior is a member of the family

Integrating out θ_i in $p(\boldsymbol{x}, \boldsymbol{\theta})$ gives the marginal prior of a component (Exercise 3.2),

$$p(x_i) \propto \left(1 + \frac{x_i^2}{2\vartheta}\right)^{-\beta - 1/2}, \tag{3.12}$$

a Student-t density with 2β degrees of freedom and scale $\sqrt{\vartheta/\beta}$; for our $\beta = 1$ it has two degrees of freedom, and for $\beta = 1/2$ it is the Cauchy density with scale $\sqrt{2\vartheta}$. The Cauchy difference prior that Markkanen et al. [78] proposed for edge-preserving inversion, and that is the baseline in the bathymetry study of Stietz et al. [111], is therefore (3.10) on increments with $\beta = 1/2$, seen through its marginal. The hierarchical view adds the local variances $\boldsymbol{\theta}$, and gains two things: the posterior of $\boldsymbol{x}$ given $\boldsymbol{\theta}$ is Gaussian, so that it can be computed and sampled with linear algebra, and $\boldsymbol{\theta}$ itself reports where the prior has placed the jumps. The comparison in default project C is therefore a comparison of inference methods for one prior family, not of priors.

Remark 3.9: What samples say and MAP estimates do not

The Laplace prior gives sparse MAP estimates and non-sparse samples; the hierarchical prior gives both. If the prior is meant to say "the signal is sparse", only the second one says it, and the difference is invisible as long as we only compute MAP estimates. It becomes visible as soon as we sample (Chapter 4) or report uncertainties, and it is the reason why

default projects A and C use hierarchical priors. The same holds for the smoothness prior, whose samples are random walks, and in a sharper form for the total-variation prior, which Lassas and Siltanen [74] showed to lose its edge-preserving character altogether when the grid is refined. A prior should be judged by its samples.

3.4 The linear-Gaussian case

When the forward operator is linear and both the noise and the prior are Gaussian, the posterior is Gaussian and can be written down. This is the one case in which the whole program of the chapter, posterior, point estimates, and spread, is a matter of linear algebra.

Theorem 3.10: The linear-Gaussian posterior

Let $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Gamma})$ and $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma}_{\text{prior}})$, independent. Then the posterior is Gaussian, $p(\boldsymbol{x} \mid \boldsymbol{y}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_{\text{post}}, \boldsymbol{\Gamma}_{\text{post}})$, with

$$\boldsymbol{\Gamma}_{\text{post}} = \left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}\right)^{-1}, \quad \boldsymbol{\mu}_{\text{post}} = \boldsymbol{\Gamma}_{\text{post}}\left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{y} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}\boldsymbol{\mu}\right). \tag{3.13}$$

Proof. By (3.1) and (3.6), $-\log p(\boldsymbol{x} \mid \boldsymbol{y}) = \tfrac{1}{2}(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y})^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}) + \tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Gamma}_{\text{prior}}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}) + \text{const}$. This is a quadratic function of $\boldsymbol{x}$ with Hessian $\boldsymbol{H} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}$ and gradient $\boldsymbol{H}\boldsymbol{x} - (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{y} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}\boldsymbol{\mu})$, so it equals $\tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_{\text{post}})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x} - \boldsymbol{\mu}_{\text{post}}) + \text{const}$ with $\boldsymbol{\mu}_{\text{post}} = \boldsymbol{H}^{-1}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{y} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}\boldsymbol{\mu})$; this is completing the square. Comparing with (3.2) identifies the posterior as Gaussian with covariance $\boldsymbol{H}^{-1}$. □

Corollary 3.11: Tikhonov regularization is a posterior mean

Let $\boldsymbol{\Gamma} = \sigma^2\boldsymbol{I}$, $\boldsymbol{\mu} = \mathbf{0}$, and let the prior precision be $\boldsymbol{L}^{\mathsf{T}}\boldsymbol{L}/\gamma^2$, which is singular for the difference matrix, so that the prior is improper (Section 3.3.2). If $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\mathbf{0}\}$, then (3.13) holds with this precision in place of $\boldsymbol{\Gamma}_{\text{prior}}^{-1}$, and the posterior mean is

$$\boldsymbol{\mu}_{\text{post}} = \left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} + \lambda\,\boldsymbol{L}^{\mathsf{T}}\boldsymbol{L}\right)^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{y} = \boldsymbol{x}_\lambda \quad \text{with} \quad \lambda = \frac{\sigma^2}{\gamma^2}. \tag{3.14}$$

That is, the posterior mean—which is also the MAP estimate—is the Tikhonov solution of Lemma 2.3, and the regularization parameter is the ratio of the noise variance to the prior variance.

Proof. The proof of Theorem 3.10 uses only the precision and needs only that $\boldsymbol{H} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}/\sigma^2 + \boldsymbol{L}^{\mathsf{T}}\boldsymbol{L}/\gamma^2$ is invertible, which is the kernel condition. Multiply (3.13) by σ^2 and compare with (2.3). □

The deconvolution examples below use the completed precision $\boldsymbol{L}^{\mathsf{T}}\boldsymbol{L}/\gamma^2 + \boldsymbol{e}_1\boldsymbol{e}_1^{\mathsf{T}}/\gamma_0^2$ of Section 3.3.2 with $\gamma_0 = 1$, whose posterior mean differs from $\boldsymbol{x}_\lambda$ only by the weak prior on the level x_1.

The corollary fills three rows of Table 3.1 at once. The regularized solution is the most probable unknown; the exchange rate λ between the data and the prior is the ratio of two uncertainties, how noisy the data are and how large we believe the increments to be; and the discrepancy principle of Section 2.3 is the statement "σ is known, and γ is whatever makes the fit match it". What the corollary adds to Chapter 2 is the covariance $\boldsymbol{\Gamma}_{\text{post}}$, which costs one matrix

inverse of the size of the unknown and answers the question that no rule of Section 2.3 could: how far is $\boldsymbol{x}_\lambda$ from the truth?

Example 3.12: Posterior bands for the two running examples

Figure 3.6a shows the posterior of the smooth deconvolution signal under the smoothness prior (3.8) with $\gamma = 0.029$, chosen such that $\lambda = \sigma^2/\gamma^2$ is the discrepancy-principle value. The posterior mean has an error of 0.48 (the Tikhonov solution with $\boldsymbol{L} = \boldsymbol{I}$ of Example 2.4 had 0.60), the posterior standard deviation lies between 0.04 and 0.06, and the truth lies inside the 95 % band at 93 percent of the points. Over 100 noise realizations the band contains the truth at 90 percent of the points on average, close to the 95 percent it promises; the same prior applied to the piecewise-constant signal gives 58 percent. A credible band is a statement conditional on the model, and when the prior is wrong the band is wrong with it. Figure 3.6b shows the bathymetry problem with 20 sensors, linearized at the true bottom, so that the posterior mean is not a reconstruction and only the spread is of interest. With $\gamma = 3$ cm per cell the posterior standard deviation is on average 2.7 cm at the sensors (between 1.7 and 3.3 cm) and 5 cm midway between them; with $\gamma = 1$ cm it is 1.9 cm and 2.2 cm. This is the honest version of Figure 2.8c: the bottom between the sensors comes with an uncertainty, and that uncertainty is dictated by the prior, which is now written down where everyone can see it.

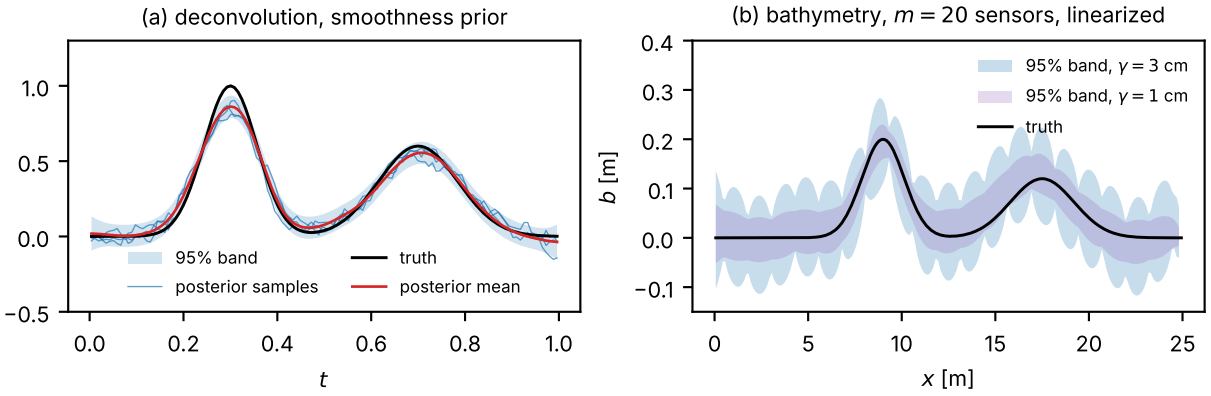


Figure 3.6: Linear-Gaussian posteriors (Theorem 3.10). (a) Deconvolution of the smooth signal with the smoothness prior: posterior mean, 95 % credible band, and three posterior samples. (b) Bathymetry with 20 sensors, linearized at the true bottom: 95 % bands for two prior scales. The band is narrow at the sensors and wide between them, and its width between the sensors is set by the prior.

One practical remark. $\boldsymbol{\Gamma}_{\text{post}}$ is an $n \times n$ matrix, trivial for $n = 128$ and impossible to form for $n = 10^6$; what remains possible in large problems is to multiply by the precision $\boldsymbol{\Gamma}_{\text{post}}^{-1}$ and to solve with it by conjugate gradients.

3.5 MAP estimation

Outside the linear-Gaussian case the posterior has no closed form, but its most probable point can still be computed, and (3.5) says how.

Definition 3.13: MAP estimation as regularization

The MAP estimate (3.3) is the minimizer of the negative log-posterior,

$$\boldsymbol{x}_{\mathrm{MAP}} \in \arg\min_{\boldsymbol{x}} \; \{-\log p(\boldsymbol{y} \mid \boldsymbol{x}) - \log p(\boldsymbol{x})\}\,, \tag{3.15}$$

which is (2.1) with the data misfit replaced by the negative log-likelihood and λR by the negative log-prior.

The dictionary of Table 3.1 runs both ways, and every method of Chapter 2 is a MAP estimate under a named prior. With Gaussian noise of level σ: the Gaussian prior $\mathcal{N}(\mathbf{0}, \gamma^2 \boldsymbol{I})$ gives Tikhonov regularization with $\lambda = \sigma^2/\gamma^2$, and the smoothness prior its $\boldsymbol{L}$-version (Corollary 3.11); the Laplace prior gives ℓ^1 regularization with $\lambda = \sigma^2/\gamma$, and the total-variation prior gives total variation; a positivity constraint is the MAP under a prior that vanishes for negative components. For a nonlinear forward operator, (3.15) is (2.8), and Algorithm 2.8 is the method of choice: Example 2.9 computed MAP estimates under the smoothness prior with $\gamma = \sigma/\sqrt{\lambda}$, i.e., 1 cm per cell for the sparse sensors. Nothing in the computations has changed. What has changed is that λ now has a meaning, that the choice of penalty is a choice of prior that can be inspected through its samples, and that the posterior around $\boldsymbol{x}_{\mathrm{MAP}}$ exists even when we do not compute it.

The hierarchical prior: the iterative alternating scheme. Under the hierarchical prior (3.10) the unknowns are $\boldsymbol{x}$ and $\boldsymbol{\theta}$ together, and the joint posterior is $p(\boldsymbol{x}, \boldsymbol{\theta} \mid \boldsymbol{y}) \propto p(\boldsymbol{y} \mid \boldsymbol{x}) \prod_i \mathcal{N}(x_i; 0, \theta_i)\, p(\theta_i)$. With Gaussian noise of level σ and the density (3.11), its negative logarithm is

$$J(\boldsymbol{x}, \boldsymbol{\theta}) = \frac{1}{2\sigma^2} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \sum_{i=1}^{n} \Big(\frac{x_i^2}{2\theta_i} + \frac{\vartheta}{\theta_i} + (\beta + \tfrac{3}{2}) \log \theta_i \Big) + \text{const}, \tag{3.16}$$

where the $\frac{1}{2}\log\theta_i$ from the normalization of $\mathcal{N}(x_i; 0, \theta_i)$ has been added to the $(\beta+1)\log\theta_i$ of the hyper-prior. The joint MAP estimate minimizes J over both variables, and the structure of J suggests how: for fixed $\boldsymbol{\theta}$ it is a quadratic function of $\boldsymbol{x}$, and for fixed $\boldsymbol{x}$ it is a sum of one-dimensional functions of the θ_i.

Algorithm 3.14: Iterative alternating scheme (IAS)

Start from the mode of the hyper-prior, $\theta_i = \vartheta/(\beta+1)$ for all i, and repeat until J stops decreasing:

1. $\boldsymbol{x}$-step: with $\boldsymbol{D}_{\boldsymbol{\theta}} = \mathrm{diag}(\theta_1, \ldots, \theta_n)$, solve the weighted Tikhonov problem

$$\boldsymbol{x} = \arg\min_{\boldsymbol{x}} \; \frac{1}{2\sigma^2} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \tfrac{1}{2}\, \boldsymbol{x}^{\mathsf{T}} \boldsymbol{D}_{\boldsymbol{\theta}}^{-1} \boldsymbol{x}, \quad \text{i.e.,} \quad (\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}/\sigma^2 + \boldsymbol{D}_{\boldsymbol{\theta}}^{-1})\boldsymbol{x} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{y}/\sigma^2,$$

   as a stacked least-squares problem.
2. $\boldsymbol{\theta}$-step: for each i, minimize the bracket in (3.16) over θ_i, which gives

$$\theta_i = \frac{x_i^2/2 + \vartheta}{\beta + 3/2}.$$

For a prior on increments, replace x_i by $(\boldsymbol{L}\boldsymbol{x})_i$ in both steps and give $\ker \boldsymbol{L}$ its own prior as in Section 3.3. Each step decreases J, and in practice the iterates settle at a stationary point of J within a few dozen steps. J is not convex in $(\boldsymbol{x}, \boldsymbol{\theta})$, and the point it converges to depends on the start. Starting at the mode of the hyper-prior means starting from "nothing jumps": the first $\boldsymbol{x}$-step is a strongly smoothed reconstruction, and the scheme then adds jumps where the data insist. Starting from large variances, "everything may

jump", ends in a worse local minimum in our example (Exercise 3.5). The scheme is implemented in `solvers.ias`.

The $\boldsymbol{x}$-step is a Tikhonov problem with a weight per component, and the $\boldsymbol{\theta}$-step adjusts the weights: a component that the data push to a large value gets a large variance and is penalized less in the next round, a component that stays small gets the floor variance $\vartheta/(\beta+\frac{3}{2})$ and is held there. Sparsity is produced by a sequence of quadratic problems; Calvetti, Somersalo, and Strang call this ℓ^2-*magic* [20], and it is the family of iteratively reweighted least squares named in Section 2.4. Lindbloom, Glaubitz, and Gelb [75] extend the scheme to sparsifying transforms with nontrivial kernels, such as $\boldsymbol{L}$, and to an unknown noise variance, and for large problems its $\boldsymbol{x}$-steps can be solved by priorconditioned Krylov methods [76].

Example 3.15: The IAS on the piecewise-constant signal

Figure 3.7 shows the scheme on the deconvolution data of the piecewise-constant signal with the hierarchical prior on increments, $\beta = 1$ and $\vartheta = 10^{-3}$, started at the mode $\theta_i = 5 \cdot 10^{-4}$. The first $\boldsymbol{x}$-step is a Tikhonov solution with a strong smoothness penalty, and already its increments are largest near the true jumps, so that the first $\boldsymbol{\theta}$-step raises the local variances there (Figure 3.7a,b). The next iterations sharpen this: wherever θ_i has grown, the penalty on the increment is weaker, the reconstruction jumps more, and θ_i grows further, while the increments elsewhere are pushed to the floor $\vartheta/(\beta+\frac{3}{2})$ (Figure 3.7c,d). After 20 iterations, each a least-squares solve, the iterates stop changing: running on to 200 iterations changes the reconstruction by less than 10^{-5}. At the end, θ_i stands more than two orders of magnitude above its floor at four places and nowhere else (Figure 3.7e,f). They are the four larger jumps, the one of height 0.5 placed one cell late, and the reconstruction has sharp edges, flat plateaus, and an error of 0.64. The small step of height 0.3 is not found; it is replaced by a ramp. Figure 3.8 compares the result with the MAP estimates under the Gaussian and the Laplace prior on increments, with the parameters of Figure 3.4. The Gaussian prior gives Tikhonov regularization with the difference matrix and $\lambda = \sigma^2/\gamma^2 = 0.25$, which spreads every edge over about seven cells and overshoots next to it (error 1.50). The Laplace prior gives total variation with $\lambda = \sigma^2/\gamma = 0.025$: sharp edges, each spread over two cells, flat plateaus, and the two small steps as a staircase (error 0.82). The hierarchical prior puts each large jump into a single cell and loses the smallest step. The reason is the shape of its penalty. Inserting the $\boldsymbol{\theta}$-step into (3.16) shows that the joint MAP estimate minimizes the data misfit plus $(\beta+\frac{3}{2})\sum_i \log(1+(\boldsymbol{L}\boldsymbol{x})_i^2/(2\vartheta))$. For increments above $\sqrt{2\vartheta} \approx 0.045$ this penalty grows only logarithmically, so that one large jump is much cheaper than a staircase of smaller jumps with the same total height, and edges stay sharp. For increments below $\sqrt{2\vartheta}$ it is quadratic, a smoothness penalty, and a ramp of height 0.3 spread over many cells is cheaper than a single jump of 0.3, which is how the smallest step is lost. The ℓ^1 penalty charges a rise by its total height, however it is split; total variation is therefore indifferent between one jump and a staircase, which is why it spreads edges over two cells and keeps the small steps as a staircase. No parameter was tuned to the data: the discrepancies of the three MAP estimates are 0.38, 0.37, and 0.38, all close to $\sqrt{m}\,\sigma = 0.4$. In exchange for λ we have chosen γ or ϑ (and β), and Remark 3.17 says how.

Remark 3.16: Three things to know about MAP estimates

Figure 3.9 illustrates each of the three with one unknown. First, the MAP estimate depends on the parametrization. If $\boldsymbol{u} = g(\boldsymbol{x})$ is a nonlinear change of variables, then the

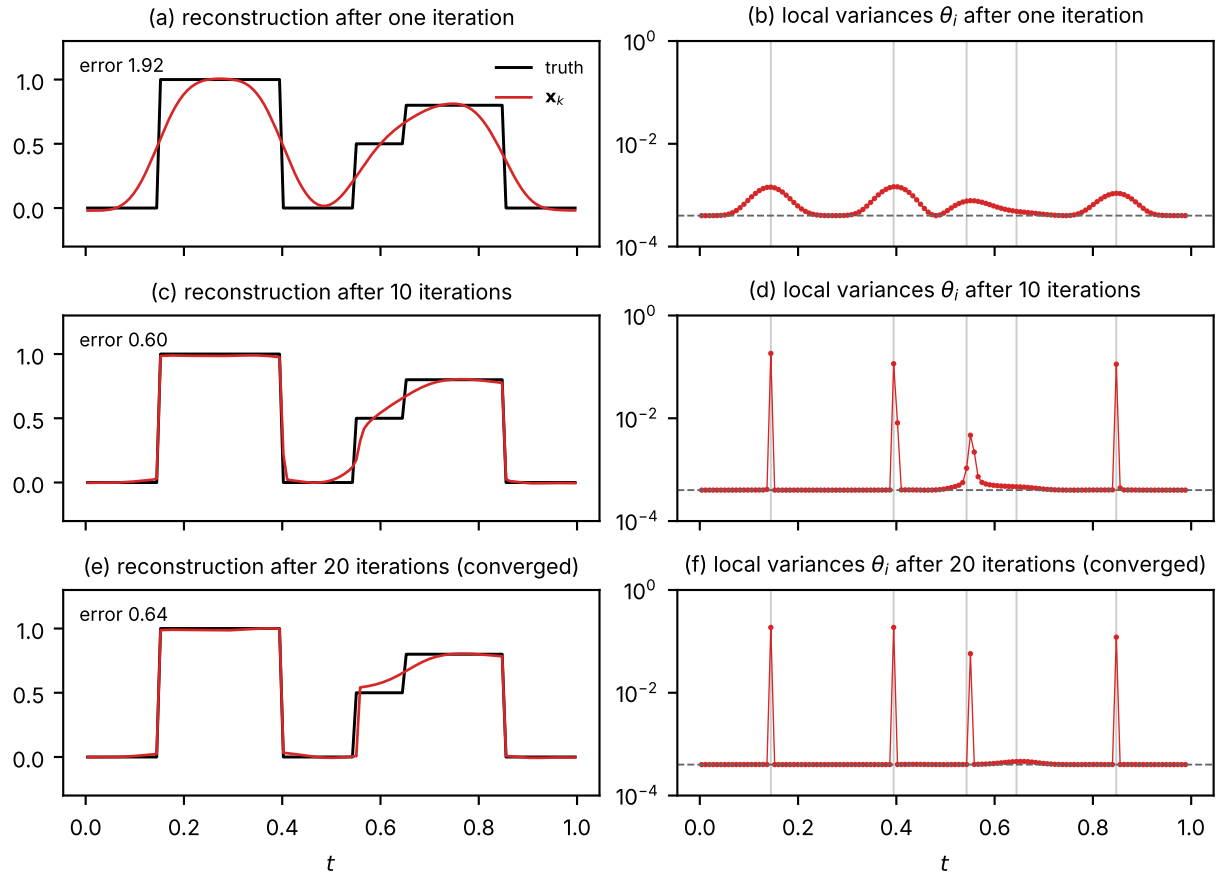


Figure 3.7: The iterative alternating scheme (Algorithm 3.14) on the piecewise-constant deconvolution data with the hierarchical prior on increments, $\beta = 1$, $\vartheta = 10^{-3}$, started at the mode of the hyper-prior. Left column: the reconstruction $\boldsymbol{x}_k$ after one iteration, after 10, and after 20 (converged; further iterations change nothing visible). Right column: the local variances θ_i at the same iterations, with the true jump positions in gray and the floor $\vartheta/(\beta + 3/2)$ dashed.

density of $\boldsymbol{u}$ carries a Jacobian factor, and the MAP estimate of $\boldsymbol{u}$ is not $g(\boldsymbol{x}_{\mathrm{MAP}})$. Take a conductivity $\kappa > 0$ whose posterior is log-normal, so that $u = \log \kappa$ has the posterior $\mathcal{N}(0, 0.6^2)$: the MAP estimate of u is 0, which corresponds to $\kappa = 1$, while the MAP estimate of κ itself is $e^{-0.36} = 0.70$, and the posterior mean of κ is 1.20 (Figure 3.9a). The posterior mean, being an integral, transforms as expected under linear maps only, and the choice of parametrization is part of the model. Second, the posterior mean under a sparsity prior is not sparse. For one datum $y = x + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$ and the Laplace prior with $\gamma = 1$, the MAP estimate is the soft threshold $\operatorname{sign}(y) \max(|y| - 1, 0)$, exactly zero for $|y| \le 1$, whereas the posterior mean is a smooth function of y that vanishes only at $y = 0$ (Figure 3.9b); in n dimensions, $\boldsymbol{x}_{\mathrm{MAP}}$ has exact zeros and $\boldsymbol{x}_{\mathrm{CM}}$ has none, for the Laplace as for the total-variation prior [50, 77]. Sparsity of a reconstruction is a property of the point estimate, not of the posterior, which is the computational side of Remark 3.9. Third, for a nonlinear forward operator the negative log-posterior is in general not convex, and $\boldsymbol{x}_{\mathrm{MAP}}$ is found by an iteration that starts somewhere. For one datum $y = x^2 + \varepsilon$ with $y = 1$, $\varepsilon \sim \mathcal{N}(0, 0.3^2)$, and the prior $\mathcal{N}(0.4, 1)$, the posterior has two peaks near ± 1 (Figure 3.9c): a descent method started at a negative value converges to the lower peak, the MAP estimate is the higher one, and the posterior mean, 0.34, lies in the valley

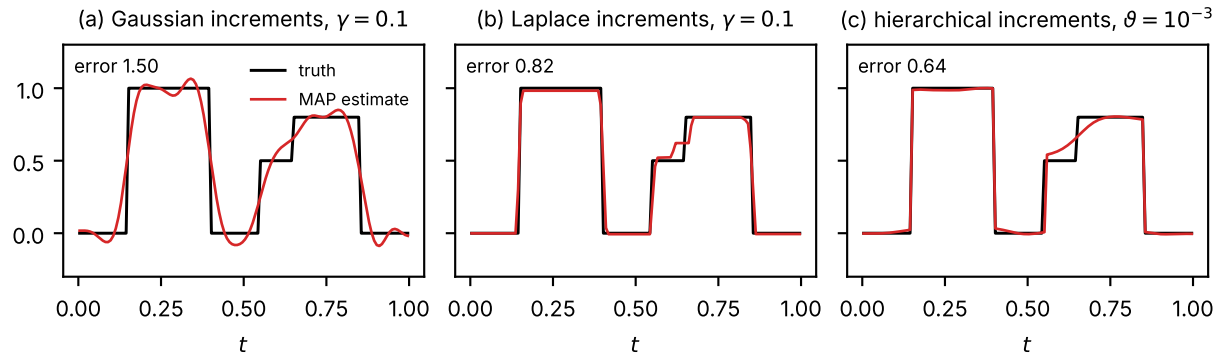


Figure 3.8: MAP estimates for the piecewise-constant deconvolution data under the three priors on increments of Figure 3.4, with the same parameters. (a) Gaussian prior, i.e., Tikhonov regularization with the difference matrix and $\lambda = \sigma^2/\gamma^2 = 0.25$: smeared edges and overshoots. (b) Laplace prior, i.e., total variation with $\lambda = \sigma^2/\gamma = 0.025$: sharp edges and a staircase at the two small steps. (c) Hierarchical prior, the result of Figure 3.7: the sharpest large edges, but the smallest step is replaced by a ramp. All three fit the data to about the noise level.

between them, where the posterior density is small. The maximum of such a posterior is well defined, but its shape is not described by it.

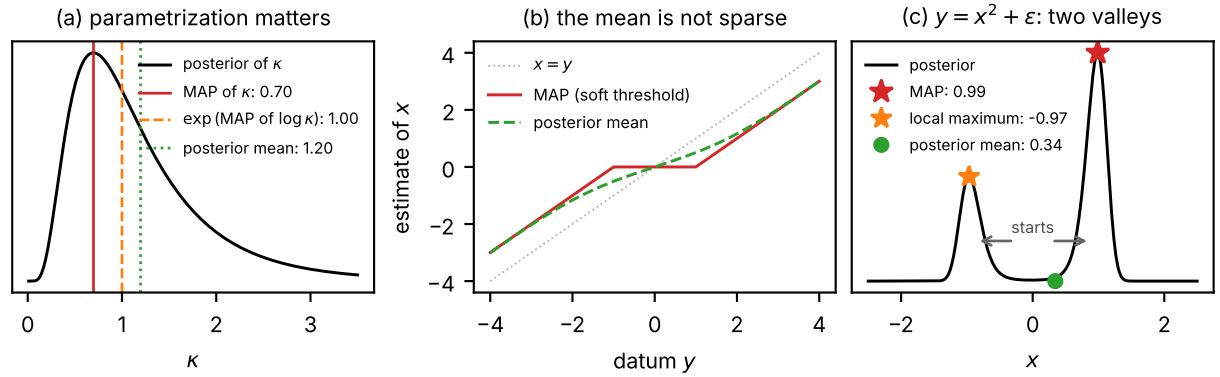


Figure 3.9: The three caveats of Remark 3.16, each with one unknown. (a) A log-normal posterior of a conductivity κ: the MAP estimate of κ, the MAP estimate of $\log \kappa$ mapped back, and the posterior mean are three different numbers. (b) Scalar denoising with a Laplace prior: the MAP estimate as a function of the datum is a soft threshold with a dead zone, the posterior mean is never exactly zero. (c) The posterior for $y = x^2 + \varepsilon$: two peaks, a descent method ends in the one it started next to, and the mean lies between them.

Remark 3.17: The free parameters of the model

The prior and the likelihood contain numbers that are not the unknown: the noise level σ, the prior scale γ, and ϑ (and β) for the hierarchical prior. These are the *hyper-parameters*, and $\lambda = \sigma^2/\gamma^2$ was the first of them. In this course we simply choose them. σ comes from the instrument or from repeated measurements, and γ or ϑ from what we know about the unknown: "the bottom changes by centimeters from one cell to the next" is a statement about γ that a hydraulic engineer can make, and a statement that the samples of the prior

let us check. Where such knowledge is missing, the rules of Section 2.3 apply with their new interpretation: the discrepancy principle fixes σ and lets the data fix γ. There are two ways not to choose, which we mention for completeness. The hierarchical way treats σ^2 and γ^2 as unknowns with priors of their own, so that they are inferred together with $\boldsymbol{x}$ [37, 64]; this is what the hierarchical prior (3.10) already does for the local variances, and the Gibbs sampler of Chapter 4 is built for such models. The empirical-Bayes way maximizes the evidence $p(\boldsymbol{y})$ of Definition 3.2 over the hyper-parameters; it is explicit only for linear forward operators with Gaussian prior and noise, costs a determinant and a solve with an $m \times m$ matrix per evaluation, and is what the sparse Bayesian learning of Tipping [117] does for the local variances of (3.10). Both are worth knowing; neither is needed for the projects.

3.6 What the MAP estimate does not tell you

The chapter has given the three answers that Section 2.6 asked for, and the linear-Gaussian case has delivered all of them in closed form. Outside that case we are left with a MAP estimate, which is a regularized solution with its assumptions written down, and with a posterior that we know how to evaluate at any point but not how to summarize. The spread has no formula. The IAS returns $\boldsymbol{\theta}$, which says where the jumps are, but the uncertainty of $\boldsymbol{x}$ given a fixed $\boldsymbol{\theta}$ is the uncertainty of a Gaussian that pretends the jump positions are known. For a nonlinear forward operator the posterior is not Gaussian, its maximum and its mean are different points, and a credible band needs the whole shape. Chapter 4 draws samples from the posterior and reads everything else off the samples; Chapter 5 discusses what to do when that is too expensive.

Summary : Chapter 3 in five sentences

The Bayesian solution of an inverse problem is the posterior density, prior times likelihood, which assigns odds to every candidate unknown instead of picking one. The likelihood is the noise model read as a density of the data, and for Gaussian noise its negative logarithm is the data misfit in units of the noise level. A prior is best understood through its samples: Gaussian priors generate static, random walks, or smooth curves depending on their covariance, the Laplace prior is not sparse, and the hierarchical prior with inverse-gamma local variances generates sparse spikes or piecewise-constant signals. For a linear forward operator with Gaussian noise and prior, the posterior is Gaussian, its mean is the Tikhonov solution with $\lambda = \sigma^2/\gamma^2$, and its covariance gives credible bands that are only as good as the prior. Every regularized solution is a MAP estimate under a prior, the iterative alternating scheme computes the joint MAP under the hierarchical prior by weighted Tikhonov steps, and the hyper-parameters, which we choose by hand in this course, could also be inferred or estimated.

3.7 Notes and further reading

Kaipio and Somersalo [64] is the standard reference for the Bayesian formulation of inverse problems in finite dimensions, and Calvetti and Somersalo [19] the modern computational account, with the hierarchical priors and the IAS of Section 3.5; the analysis of the IAS with generalized gamma hyper-priors, of which the inverse gamma is the case $r = -1$, is in [20, 21], and the generalized sparse Bayesian learning used in the default projects in [43, 45]. Hierarchical priors of

this kind have been used for sequences of images [120], for the filtering of noisy data and numerical solutions [46], for Bayesian neural networks [52], and for data assimilation with hyperbolic conservation laws [47]. Stuart [114] proves that the posterior satisfies Hadamard's conditions in function space: it exists, is unique, and depends continuously on the data, which is the precise sense in which the Bayesian formulation is well-posed. The samples of the total-variation prior and their behavior under grid refinement are in Lassas and Siltanen [74]; Cauchy difference priors in Markkanen et al. [78]; Gaussian process priors in Rasmussen and Williams [99]. The enhanced error model is in [65], and the general Bayesian background, including conjugate priors and hierarchical models, in Gelman et al. [37].

3.8 Exercises

Exercise 3.1: Completing the square

Prove Theorem 3.10 in detail, including the constant: show that $\frac{1}{2}(\boldsymbol{A}\boldsymbol{x}-\boldsymbol{y})^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}(\boldsymbol{A}\boldsymbol{x}-\boldsymbol{y})+\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Gamma}_{\text{prior}}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})=\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_{\text{post}})^{\mathsf{T}}\boldsymbol{\Gamma}_{\text{post}}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}_{\text{post}})+c$ with a c that does not depend on $\boldsymbol{x}$, and compute c. Then compute the posterior in Figure 3.2a by hand.

Exercise 3.2: The dictionary

(a) Show that the MAP estimate under Gaussian noise of level σ and the Laplace prior with scale γ is the ℓ^1-regularized solution (2.7) with $\lambda=\sigma^2/\gamma$. (b) Derive the marginal prior (3.12) by integrating $\mathcal{N}(x_i;0,\theta_i)\,p(\theta_i)$ over θ_i, and identify the degrees of freedom and the scale. (c) Derive the $\boldsymbol{\theta}$-step of Algorithm 3.14 and check that the bracket in (3.16) has a unique minimizer.

Exercise 3.3: Credible bands (computer)

Use `solvers.gaussian_posterior` and `priors.gaussian_precision`. (a) Reproduce Figure 3.6a and report the error of the posterior mean and the fraction of points at which the truth lies inside the 95 % band. (b) Repeat for 100 noise realizations and for both test signals; explain the two coverage numbers. (c) Make γ three times too large and three times too small and report what happens to the error and to the coverage.

Exercise 3.4: Priors through their samples (computer)

Use `priors.py`. (a) Draw 1000 samples from the Laplace prior with $\gamma=0.1$ and from the hierarchical prior with $\beta=1$ and $\vartheta=10^{-3}$ (Figure 3.4). For each prior, compute the fraction of components smaller than 10^{-3} in absolute value and the share of the squared norm of a sample that its largest 5 percent of components carry. Which of the two numbers tells the priors apart, and why does neither prior produce exact zeros? (b) For the three priors on increments, measure the fraction of increments larger than 0.1 in absolute value and the number of "jumps" per sample. (c) Draw samples from the Gaussian prior on second differences and from the squared-exponential covariance (`priors.se_covariance`) and compare them with Figure 3.4a,d. (d) Refine the grid from $n=128$ to $n=1024$, once with the scale $\gamma=0.1$ of each increment fixed and once with γ scaled like $n^{-1/2}$, so that the amplitude of the samples stays fixed, and describe what happens to the samples of the total-variation prior.

Exercise 3.5: The iterative alternating scheme (computer)

Use `solvers.ias`. (a) Reproduce Figure 3.7 and then vary $\vartheta \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ and $\beta \in \{1/2, 1, 2\}$; report the error, the number of iterations, and the number of θ_i above 10^{-2} in each case. (b) Start from $\boldsymbol{\theta} = \mathbf{1}$ instead of the mode of the hyper-prior and compare the objectives and the reconstructions reached. (c) Apply the scheme to the smooth signal and describe the result. (d) Replace the inverse-gamma hyper-prior by the gamma distribution $p(\theta) \propto \theta^{\beta-1} \exp(-\theta/\vartheta)$ and derive the new $\boldsymbol{\theta}$-step. Show that the joint posterior density is unbounded for $\beta < 3/2$, so that the joint MAP estimate does not exist, and that for $\beta = 3/2$ the scheme minimizes a total-variation functional. Then repeat (a) for $\beta \in \{2, 3, 4\}$ with a copy of `solvers.ias` in which the $\boldsymbol{\theta}$-step is replaced.

Exercise 3.6: Your own inverse problem, as a Bayesian

For the problem of Exercises 1.5 and 2.6: what is the noise model, and where would $\boldsymbol{\Gamma}$ come from? Which prior says what you believe about the unknown, and would you recognize its samples as plausible? Which hyper-parameters does your model have, and how would you choose each of them (Remark 3.17)? Write half a page; this is the second paragraph of your project proposal.

Chapter 4

Uncertainty Quantification by Sampling

In Chapter 2, a hiker in fog looked for the deepest valley of a landscape of which she could only feel the slope under her feet. Her colleague in this chapter has a harder task: she must survey the whole range, so that afterwards she can say how much of it lies at which altitude, how wide the valleys are, and whether there is a second valley behind the ridge. The landscape is the *potential*

$$U(\boldsymbol{x}) = -\log p(\boldsymbol{y} \mid \boldsymbol{x}) - \log p(\boldsymbol{x}) = -\log p(\boldsymbol{x} \mid \boldsymbol{y}) + \text{const}, \tag{4.1}$$

the negative log-posterior, which we can evaluate at every point but cannot draw for more than two unknowns. The survey is a list of points $\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(N)}$ that visits every region as often as its posterior probability demands: often in the deep valleys, rarely on the slopes, practically never on the peaks. Such a list is a sample from the posterior, and every question about the posterior—a mean, a credible interval, the probability that a jump is where the MAP estimate put it—becomes a question about the list. Figure 4.1 shows the survey, and it already contains the main difficulty of this chapter: in fog, the hiker cannot see where the deep valleys are, and she has to find them by walking.

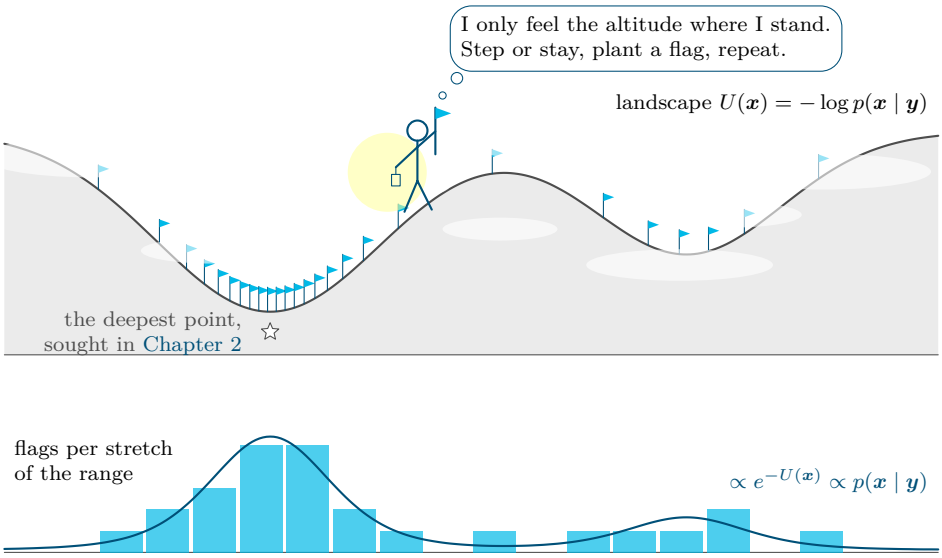


Figure 4.1: The survey of this chapter. In fog, the hiker can only feel the altitude $U(\boldsymbol{x})$ where she stands. At every step she decides whether to move or to stay, and she plants a flag. The flags end up dense where the landscape is low and sparse where it is high, and their histogram (bottom) is the posterior $p(\boldsymbol{x} \mid \boldsymbol{y}) \propto e^{-U(\boldsymbol{x})}$. The hiker of Chapter 2 was looking for the deepest point only (star).

Figure 4.2 shows three ways of walking, on the bent ridge of Figure 3.2b, which Example 4.3 defines. The random walker proposes a step in a random direction and takes it unless it leads

too far uphill; she is safe and slow. The Gibbs walker moves only parallel to the coordinate axes, but each of her moves is an exact draw from the profile of the landscape along that axis. The Hamiltonian walker rides a sled: she pushes off in a random direction, lets the slopes steer her along the valley, and stops after a fixed time, far from where she started. This chapter explains why we want such a survey (Section 4.1), how a walk can produce one (Section 4.2), which walk to use for the three kinds of posteriors in this course (Sections 4.3 to 4.5), how to tell whether the survey is finished (Section 4.6), and what it costs (Section 4.7).

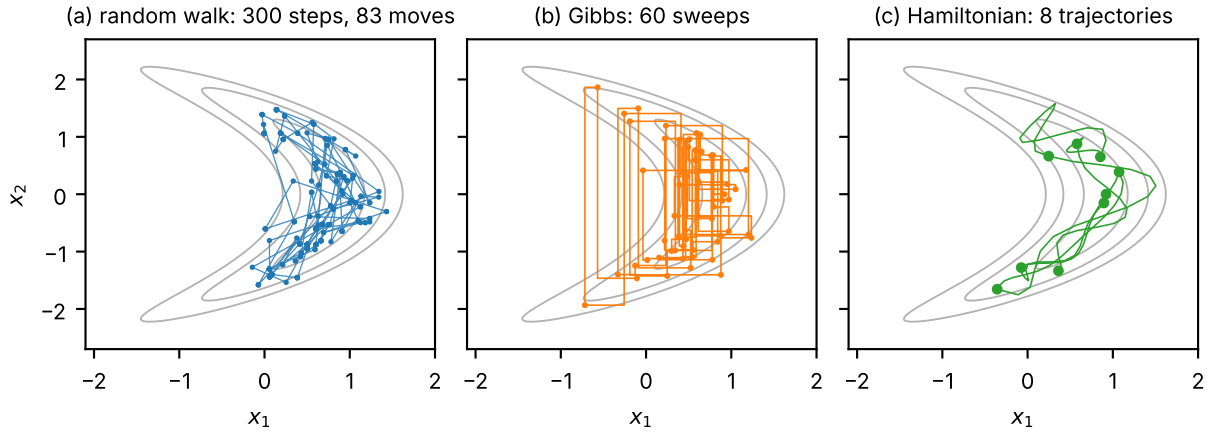


Figure 4.2: Three walkers on the bent ridge of Example 4.3 (gray contours), all started at the MAP estimate. (a) Random-walk Metropolis–Hastings (Section 4.2.1) with steps of standard deviation 0.8: 83 of 300 proposed steps are taken, the others are refused because they lead too far uphill. (b) Gibbs sampling (Section 4.4), 60 sweeps, each an exact draw of x_1 given x_2 (horizontal move) followed by one of x_2 given x_1 (vertical move). The vertical moves can jump between the two arms of the ridge; the horizontal ones are short because the ridge is narrow. (c) Hamiltonian Monte Carlo (Section 4.5), eight trajectories of 15 leapfrog steps each (lines) and their end points (dots): every trajectory follows the ridge and ends far from where it started.

4.1 Why sample? Uncertainty quantification

A posterior answers many questions at once, and the question that matters is rarely limited to "what is the most probable unknown". A weather forecaster wants the probability of more than ten millimeters of rain, not the most probable rainfall; a materials scientist wants the range of defect energies that are compatible with a measured spectrum; a river engineer wants to know whether the channel is deep enough for a barge along its whole length. Each of these is a *quantity of interest*, a function $f(\boldsymbol{x})$ of the unknown, and the posterior answers it through the *posterior expectation*

$$\mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}] = \int f(\boldsymbol{x})\, p(\boldsymbol{x} \mid \boldsymbol{y})\, \mathrm{d}\boldsymbol{x}. \tag{4.2}$$

The choice $f(\boldsymbol{x}) = x_i$ gives the posterior mean of a component, $f(\boldsymbol{x}) = (x_i - \mathbb{E}[x_i \mid \boldsymbol{y}])^2$ its variance, and the indicator of a set B, i.e., $f(\boldsymbol{x}) = 1$ for $\boldsymbol{x} \in B$ and $f(\boldsymbol{x}) = 0$ otherwise, gives the probability $\mathbb{P}(\boldsymbol{x} \in B \mid \boldsymbol{y})$, e.g., that the signal exceeds a threshold at a given point. Quantiles, and with them the credible intervals (3.4), are inverses of such probabilities. Derived quantities are no harder. The height of a peak, the position of an edge, the volume of a sediment deposit, or the prediction of a quantity that was not measured are functions $g(\boldsymbol{x})$, and their posterior distribution is the distribution of $g(\boldsymbol{x})$ when $\boldsymbol{x}$ is drawn from the posterior. This is

what *uncertainty quantification* (UQ) means in this course: computing posterior expectations, probabilities, and distributions of quantities of interest, and reporting them with their own error.

Integrals in high dimensions. For $n = 128$ unknowns, (4.2) is an integral over $\mathbb{R}^{128}$, and quadrature is out of the question: a grid with ten points per direction has 10^{128} points. Sampling replaces the integral by an average.

Definition 4.1: Monte Carlo estimate and its standard error

Let $\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(N)}$ be independent samples from the posterior, and let f have the finite posterior variance $\sigma_f^2 = \mathrm{Var}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$. The *Monte Carlo estimate* of (4.2) is the sample average

$$\widehat{f}_N = \frac{1}{N} \sum_{k=1}^{N} f(\boldsymbol{x}^{(k)}). \tag{4.3}$$

It is a random variable, because it depends on the samples, and its *Monte Carlo standard error* is its standard deviation over repeated sampling,

$$\left(\mathbb{E}\Big[\big(\widehat{f}_N - \mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}] \big)^2 \Big] \right)^{1/2} = \frac{\sigma_f}{\sqrt{N}}, \tag{4.4}$$

i.e., the root-mean-square size of the error caused by using finitely many samples. In practice, σ_f is replaced by the standard deviation of the values $f(\boldsymbol{x}^{(k)})$.

Theorem 4.2: Convergence of Monte Carlo estimates

Under the assumptions of Definition 4.1, the estimate is unbiased, $\mathbb{E}[\widehat{f}_N] = \mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$, and as $N \to \infty$:

1. $\widehat{f}_N \to \mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$ with probability one (strong law of large numbers);
2. the mean-square error is σ_f^2 / N, so that the error decays like $N^{-1/2}$ in the root-mean-square sense;
3. $\sqrt{N}\,(\widehat{f}_N - \mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}])$ converges in distribution to $\mathcal{N}(0, \sigma_f^2)$ (central limit theorem).

Proof. Unbiasedness follows from the linearity of the expectation. The samples are independent, so the variance of their sum is the sum of their variances, and $\mathrm{Var}[\widehat{f}_N] = N\sigma_f^2/N^2 = \sigma_f^2/N$, which is the mean-square error because the estimate is unbiased. The two limit theorems are the classical ones for averages of independent, identically distributed random variables with finite variance [101]. □

The third statement is what makes the standard error useful: for large N, the estimate lies within $1.96\,\sigma_f/\sqrt{N}$ of the exact value with a probability of about 95 percent, so that $\widehat{f}_N \pm 1.96\,\sigma_f/\sqrt{N}$ is an approximate 95 % confidence interval for $\mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$. This is a frequentist statement in the sense of Remark 3.1, about our own random numbers, which are random in the frequentist sense; it says how accurately we have computed the posterior, not how accurately the posterior describes the unknown.

Two features of (4.3) explain why sampling is the workhorse of UQ. First, the dimension n does not appear in the error: 100 independent samples give a posterior mean to within a tenth of a posterior standard deviation, whether $n = 2$ or $n = 10^6$. Second, the convergence is slow: every further digit costs a hundred times more samples. Monte Carlo is good at answers to two digits and bad at answers to six, and rare events are the hard case: a probability of 10^{-3} is estimated from the few samples that land in B, with a relative error of about $1/\sqrt{10^{-3}N}$, and needs $N \approx 10^5$ for an accuracy of ten percent. Two digits are enough for almost every purpose in

inverse problems, since the posterior is itself only as accurate as the noise model and the prior. Throughout the chapter, the samplers are tested on a posterior whose answers are known.

Example 4.3: A test posterior: the bent ridge

The posterior of Figure 3.2b has two unknowns, the prior $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$, and one datum $y = x_1 + x_2^2/2 + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, $\sigma = 0.3$, and $y = 1$. Its potential (4.1) is

$$U(\boldsymbol{x}) = \frac{1}{2}(x_1^2 + x_2^2) + \frac{1}{2\sigma^2}\Big(y - x_1 - \frac{x_2^2}{2}\Big)^2. \tag{4.5}$$

The likelihood is largest on the parabola $x_1 = y - x_2^2/2$, on which the datum is matched exactly, and the prior pulls toward the origin, so that the posterior is a ridge bent along the parabola and symmetric under $x_2 \mapsto -x_2$. Its MAP estimate is $(0.92, 0)$, its mean $(0.48, 0)$ lies inside the bend, and its standard deviations are 0.56 for x_1 and 0.98 for x_2. With two unknowns, these numbers can be computed by quadrature on a fine grid, which makes the bent ridge a test case with known answers: it is curved and non-Gaussian like the posterior of a nonlinear inverse problem, and small enough to check every sampler against the truth. Two of its properties are used below. The gradient is $\nabla U(\boldsymbol{x}) = (x_1 - r/\sigma^2,\ x_2 - x_2 r/\sigma^2)$ with the residual $r = y - x_1 - x_2^2/2$. The conditional of x_1 given x_2 is Gaussian, because U is quadratic in x_1, with precision $1 + 1/\sigma^2$ and mean $(y - x_2^2/2)/(1 + \sigma^2)$; the conditional of x_2 given x_1 is not Gaussian, and for x_1 well below y it has two modes, one on each arm of the ridge.

The difficulty lies elsewhere: except for Gaussian posteriors (Section 4.3), nobody knows how to draw independent samples from $p(\boldsymbol{x} \mid \boldsymbol{y})$. The way out is to give up independence.

4.2 Markov chain Monte Carlo

A *Markov chain* is a random sequence $\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \ldots$ in which every element is drawn from a *transition density* $K(\boldsymbol{x}' \mid \boldsymbol{x})$ that depends only on the previous element, $\boldsymbol{x}^{(k+1)} \sim K(\cdot \mid \boldsymbol{x}^{(k)})$. The idea of *Markov chain Monte Carlo* (MCMC) is to construct K such that the chain, run for long enough, visits the space of unknowns with the frequencies of the posterior, wherever it started. The property that makes this possible is *invariance*: if $\boldsymbol{x}^{(k)}$ is distributed according to the posterior, then so is $\boldsymbol{x}^{(k+1)}$,

$$\int K(\boldsymbol{x}' \mid \boldsymbol{x})\, p(\boldsymbol{x} \mid \boldsymbol{y})\, \mathrm{d}\boldsymbol{x} = p(\boldsymbol{x}' \mid \boldsymbol{y}) \quad \text{for all } \boldsymbol{x}'. \tag{4.6}$$

Invariance is easiest to check in the stronger form of *detailed balance*,

$$p(\boldsymbol{x} \mid \boldsymbol{y})\, K(\boldsymbol{x}' \mid \boldsymbol{x}) = p(\boldsymbol{x}' \mid \boldsymbol{y})\, K(\boldsymbol{x} \mid \boldsymbol{x}') \quad \text{for all } \boldsymbol{x}, \boldsymbol{x}', \tag{4.7}$$

which says that in equilibrium the chain moves from $\boldsymbol{x}$ to $\boldsymbol{x}'$ as often as from $\boldsymbol{x}'$ to $\boldsymbol{x}$.

Lemma 4.4: Detailed balance implies invariance

A transition density K that satisfies (4.7) leaves the posterior invariant, i.e., satisfies (4.6).

Proof. Integrate (4.7) over $\boldsymbol{x}$. The left-hand side becomes the left-hand side of (4.6), and the right-hand side becomes $p(\boldsymbol{x}' \mid \boldsymbol{y}) \int K(\boldsymbol{x} \mid \boldsymbol{x}')\, \mathrm{d}\boldsymbol{x} = p(\boldsymbol{x}' \mid \boldsymbol{y})$, because from $\boldsymbol{x}'$ the chain moves somewhere with probability one. For a chain that stays put with positive probability, such as

the one of Algorithm 4.6 below, the integral includes this point mass, and the argument is the same. □

Invariance alone does not guarantee convergence. If, in addition, the chain can reach every region of positive posterior probability and does not cycle periodically, the ergodic theorem guarantees that the averages (4.3) along the chain converge to (4.2) from any starting point, with probability one [101]. Figure 4.3 illustrates these notions for a one-dimensional target with two modes. A transition density is a whole family of densities, one for every current state: for the random-walk chain of Section 4.2.1 and the current state $x = 0.9$, the chain moves according to the blue density in panel (a) and stays where it is with probability 0.20. Panel (b) shows what convergence means. Started near $x = 3.2$, far in the tail of the target, the density of $x^{(k)}$ first fills the right mode, then the left one, and after 100 steps it cannot be distinguished from the target; the distance to the target decays geometrically (c). In practice, the first iterations are spent walking from the start to the region where the posterior lives; they are discarded as *burn-in*, and the rest of the chain is used as the sample.

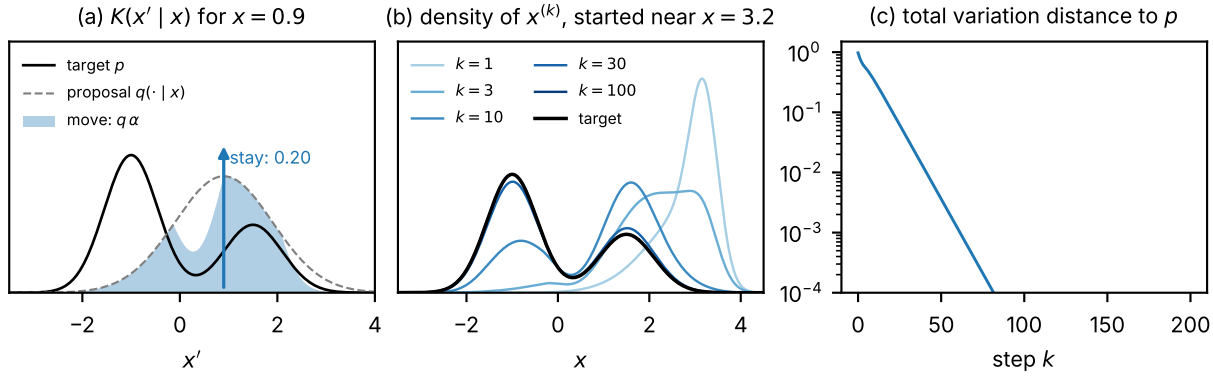


Figure 4.3: A Markov chain for a one-dimensional target p with two modes: the random walk of Algorithm 4.6 with steps of standard deviation 1. (a) The transition density $K(\cdot \mid x)$ for $x = 0.9$: the chain moves according to the proposal density (dashed) times the acceptance probability (blue area) and stays at x with probability 0.20 (arrow). (b) The density of $x^{(k)}$ after k steps, computed exactly on a grid, for a chain started near $x = 3.2$. (c) The total variation distance $\frac{1}{2}\int |p_k(x) - p(x)|\,\mathrm{d}x$ between the density p_k of $x^{(k)}$ and the target.

The price of correlation. Consecutive elements of a chain are correlated, and correlated samples carry less information than independent ones.

Definition 4.5: Integrated autocorrelation time and effective sample size

Let $f_k = f(\boldsymbol{x}^{(k)})$ along a chain in equilibrium, i.e., started from the posterior, so that the mean $\mathbb{E}[f_k] = \mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$ and the variance σ_f^2 of f_k do not depend on k. The *autocorrelation at lag* j is the correlation of two elements of the chain that are j steps apart,

$$\rho_f(j) = \frac{\mathrm{Cov}(f_k, f_{k+j})}{\sigma_f^2} = \frac{\mathbb{E}[(f_k - \mathbb{E}[f_k])(f_{k+j} - \mathbb{E}[f_{k+j}])]}{\sigma_f^2}, \quad j = 0, 1, 2, \ldots, \tag{4.8}$$

which in equilibrium does not depend on k either; $\rho_f(0) = 1$, and for independent samples $\rho_f(j) = 0$ for all $j \geq 1$. The *integrated autocorrelation time* of f is

$$\tau_f = 1 + 2\sum_{j=1}^{\infty} \rho_f(j), \tag{4.9}$$

and for large N the chain average (4.3) has the variance

$$\operatorname{Var}[\widehat{f}_N] \approx \frac{\tau_f\, \sigma_f^2}{N} = \frac{\sigma_f^2}{N_{\text{eff}}} \quad \text{with} \quad N_{\text{eff}} = \frac{N}{\tau_f}. \tag{4.10}$$

The number N_{eff} is the *effective sample size*: for estimating $\mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}]$, the chain of length N is worth N_{eff} independent samples.

The approximation (4.10) follows by writing the variance of the average as a double sum over the correlations of all pairs f_k, f_l (Exercise 4.1). The integrated autocorrelation time is the number of steps the chain needs to forget where it was, as seen through f, and it is the single most important number about a sampler. It depends on f: a chain can forget quickly in some quantities and slowly in others, i.e., *mix* well in the first and badly in the second, as we will see in Section 4.6. Figure 4.4 shows (4.10) at work for the three walkers of Figure 4.2. Their errors decay like $N^{-1/2}$, as for independent samples, but with constants that differ by the factor $\sqrt{\tau_f}$: the random walker needs 22 steps for one independent sample, the Gibbs walker 9, and the Hamiltonian walker one, at the price of 15 gradient evaluations per step.

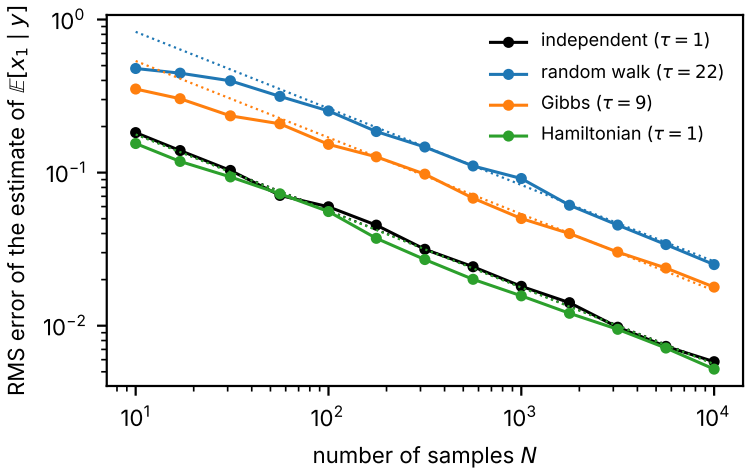


Figure 4.4: The Monte Carlo error (4.10) of the posterior mean of x_1 for the bent ridge of Example 4.3, whose exact value 0.481 is known from quadrature: root-mean-square error over 200 chains of each walker, all started in equilibrium. The errors decay like $N^{-1/2}$ and follow $\sigma_f\sqrt{\tau_f/N}$ (dotted) with the integrated autocorrelation time τ_f of each walker; independent samples have $\tau_f = 1$.

4.2.1 Metropolis–Hastings

The oldest and most general way to build a chain with detailed balance is to propose a move and then decide whether to make it [57, 81].

Algorithm 4.6: Metropolis–Hastings

Choose a *proposal density* $q(\boldsymbol{x}' \mid \boldsymbol{x})$ and a start $\boldsymbol{x}^{(0)}$. For $k = 0, 1, 2, \ldots$:

1. Propose $\boldsymbol{x}' \sim q(\cdot \mid \boldsymbol{x}^{(k)})$.
2. Compute the *acceptance probability*

$$\alpha = \min\Big\{1, \frac{p(\boldsymbol{x}' \mid \boldsymbol{y})\, q(\boldsymbol{x}^{(k)} \mid \boldsymbol{x}')}{p(\boldsymbol{x}^{(k)} \mid \boldsymbol{y})\, q(\boldsymbol{x}' \mid \boldsymbol{x}^{(k)})}\Big\} = \min\Big\{1, e^{U(\boldsymbol{x}^{(k)}) - U(\boldsymbol{x}')} \frac{q(\boldsymbol{x}^{(k)} \mid \boldsymbol{x}')}{q(\boldsymbol{x}' \mid \boldsymbol{x}^{(k)})}\Big\}. \tag{4.11}$$

3. With probability α *accept*, $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}'$; otherwise *reject*, $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)}$.

The posterior enters only through the ratio in (4.11), in which the evidence cancels: all we need is the potential (4.1), i.e., the data misfit plus the negative log-prior, the same function that MAP estimation minimizes. For the *random walk* $\boldsymbol{x}' = \boldsymbol{x}^{(k)} + s\boldsymbol{C}\boldsymbol{\xi}$ with $\boldsymbol{\xi} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, a step size $s > 0$, and a matrix $\boldsymbol{C}$ that shapes the steps, the proposal is symmetric and the ratio of the q's is one. The rule then reads: always accept a step downhill, and accept a step uphill by ΔU with probability $e^{-\Delta U}$. It is implemented in `samplers.rw_metropolis`.

That the chain satisfies detailed balance takes three lines (Exercise 4.1): for $\boldsymbol{x}' \neq \boldsymbol{x}$, both sides of (4.7) equal $\min\{p(\boldsymbol{x} \mid \boldsymbol{y})\, q(\boldsymbol{x}' \mid \boldsymbol{x}), p(\boldsymbol{x}' \mid \boldsymbol{y})\, q(\boldsymbol{x} \mid \boldsymbol{x}')\}$. The rejection is essential. A chain that took every step downhill and refused every step uphill would be an optimizer and end in a valley; a chain that took every step would be a random walk on $\mathbb{R}^n$ and know nothing about the posterior. The acceptance rule sits in between, at exactly the right place: uphill steps are taken just often enough for the time spent in every region to match its posterior probability. Figure 4.5a shows the rule for a one-dimensional target: a proposal to a point of higher density is always taken, a proposal to a point of lower density with the ratio of the two densities as probability.

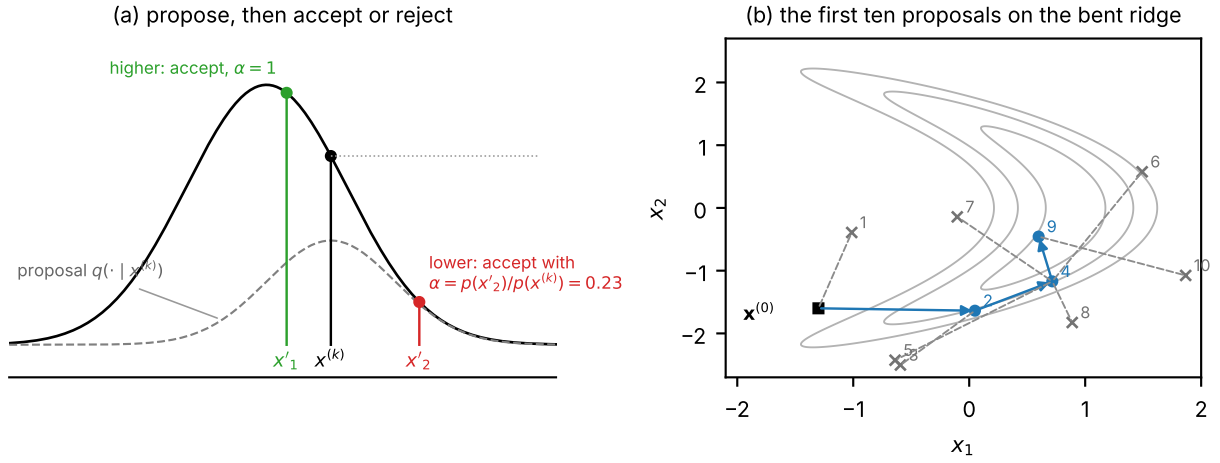


Figure 4.5: Metropolis–Hastings with a symmetric proposal. (a) For a one-dimensional target (black) and the current state $x^{(k)}$, a proposal x'_1 to a point of higher density is accepted, a proposal x'_2 to a point of lower density with probability $\alpha = p(x'_2)/p(x^{(k)}) = 0.23$; the dashed curve is the proposal density. (b) The first ten proposals of the random walk on the bent ridge, numbered in the order in which they are made and started at the square (Example 4.7): accepted proposals (blue arrows and dots) and rejected ones (gray crosses).

Example 4.7: Metropolis–Hastings on the bent ridge, step by step

We run the random walk with $\boldsymbol{C} = \boldsymbol{I}$ and $s = 0.8$ on the posterior of Example 4.3, started at $\boldsymbol{x}^{(0)} = (-1.3, -1.6)$, far outside the ridge, where $U = 7.90$. Table 4.1 lists the first six proposals and Figure 4.5b shows the first ten. The first proposal leads far uphill, to $U = 21.42$, and is accepted with probability $e^{7.90-21.42} \approx 10^{-6}$, i.e., practically never: the chain stays where it is, $\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)}$. The second leads downhill, to $U = 2.19$ on the lower arm of the ridge, and is accepted with probability one. The chain then rejects a proposal far uphill, moves on along the arm, and rejects the sixth proposal by a hair: its acceptance probability is 0.16, and the uniform number drawn is 0.18. After ten proposals, three of them accepted, the chain has arrived on the ridge; these first iterations are burn-in. Every iteration costs one evaluation of U, i.e., one forward solve, whether the proposal is accepted or not.

Table 4.1: The first six proposals of Example 4.7: current state, proposal, their potentials, the acceptance probability α of (4.11), the uniform number u, and the decision; a proposal is accepted if $u < \alpha$.

proposal	current state	proposal $\boldsymbol{x}'$	U (current)	$U(\boldsymbol{x}')$	α	u	decision
1	$(-1.30, -1.60)$	$(-1.01, -0.39)$	7.90	21.42	$1 \cdot 10^{-6}$	0.71	reject
2	$(-1.30, -1.60)$	$(0.05, -1.64)$	7.90	2.19	1	0.98	accept
3	$(0.05, -1.64)$	$(-0.59, -2.50)$	2.19	16.52	$6 \cdot 10^{-7}$	0.96	reject
4	$(0.05, -1.64)$	$(0.72, -1.17)$	2.19	1.84	1	0.67	accept
5	$(0.72, -1.17)$	$(-0.64, -2.43)$	1.84	12.64	$2 \cdot 10^{-5}$	0.85	reject
6	$(0.72, -1.17)$	$(1.49, 0.58)$	1.84	3.68	0.16	0.18	reject

Step size and shape. The random walk has a dilemma. Small steps are almost always accepted but go nowhere, and large steps would go far but are almost always rejected, because in n dimensions almost every direction leads uphill. For posteriors that resemble a product of n independent components, Roberts, Gelman, and Gilks [104] showed that the best compromise is a step size $s \approx 2.38/\sqrt{n}$, measured in posterior standard deviations, with an acceptance rate of about 0.234, and that τ_f then grows linearly with n, to about $3n$ for a Gaussian posterior. The steps must shrink as the dimension grows, and a walker with short random steps needs about n of them to cross the posterior once: the random walker diffuses, she does not travel. The shape of the steps matters as much as their size. A posterior that is narrow in some directions and wide in others forces steps small enough for the narrowest direction, and the walker then crawls along the widest one. The remedy is to shape the steps like the posterior, with $\boldsymbol{C}$ a Cholesky factor of the posterior covariance, if it is known, or of the inverse Hessian of U at the MAP estimate.

Example 4.8: Random-walk Metropolis–Hastings for deconvolution

We sample the Gaussian posterior of Example 3.12, the smooth deconvolution signal with $n = 128$ unknowns under the smoothness prior, for which the exact answer is known, with $2 \cdot 10^5$ iterations and two proposals (Figure 4.6). With isotropic steps, $\boldsymbol{C} = \boldsymbol{I}$, the step size must be as small as $s = 0.004$, a tenth of the smallest posterior standard deviation, for an acceptance rate of 0.28. The chain drifts slowly and has $\tau \approx 5800$ for the value $x(0.3)$ on the first bump, so that the $1.8 \cdot 10^5$ iterations after burn-in are worth 31 independent samples. With $\boldsymbol{C}$ the Cholesky factor of the posterior covariance and the optimal $s = 2.38/\sqrt{128} = 0.21$, the acceptance rate is 0.24, as predicted, and $\tau \approx 456 = 3.6\,n$. This is thirteen times better, and still only 394 effective samples from $1.8 \cdot 10^5$ evaluations of the posterior. Perturb-then-optimize (Section 4.3) produces 4000

exact and independent samples of this posterior in 0.03 seconds.

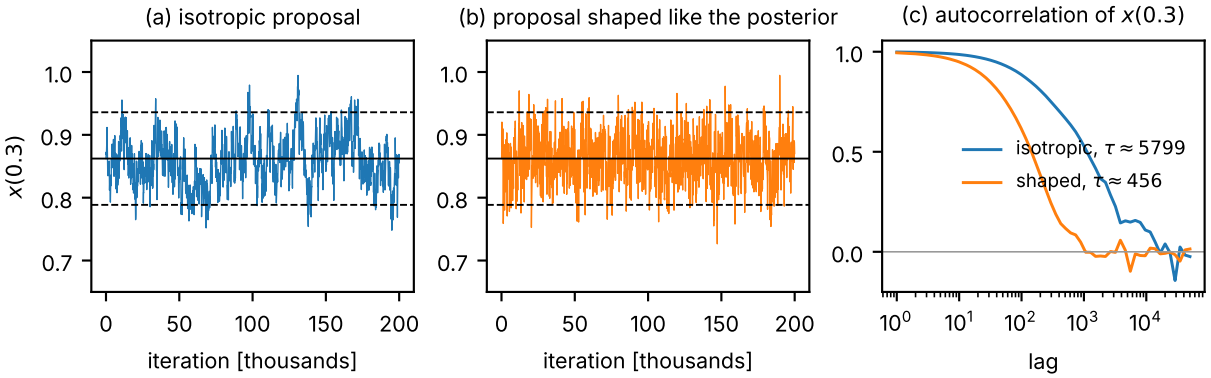


Figure 4.6: Random-walk Metropolis–Hastings on the Gaussian deconvolution posterior with $n = 128$ (Example 4.8). (a) Trace of $x(0.3)$ with isotropic steps, every hundredth iteration, with the posterior mean (solid) and the 95 % credible interval (dashed) from the closed form. (b) The same with steps shaped like the posterior covariance. (c) Autocorrelations of the two chains.

Remark 4.9: The baseline of default project C

Stietz et al. [111] sample 64 bottom values of the flume problem with random-walk Metropolis–Hastings, because gradients of their forward model were not available: one chain with a squared-exponential proposal covariance and 4000 samples after burn-in. By the rule of thumb $\tau \approx 3n$, 4000 steps of a random walk in 64 dimensions are worth about twenty independent samples, even with a perfectly shaped proposal. That is enough for a posterior mean to within about a quarter of a posterior standard deviation, and not enough for the tails of a 95 % interval. Whether their reported intervals survive four chains and the diagnostics of Section 4.6 is one of the questions of project C.

4.2.2 A map of samplers

The random walk uses nothing but values of U. Better samplers use more information about the posterior, in the same way in which Gauss–Newton uses more information than a search along random directions, and Kim et al. [67] sort them by the information they use (Figure 4.7). *Gradient information* adds a drift down the slope. The Metropolis-adjusted Langevin algorithm (MALA) proposes $\boldsymbol{x}' = \boldsymbol{x} - \frac{h^2}{2}\nabla U(\boldsymbol{x}) + h\boldsymbol{\xi}$, a gradient step plus noise, and corrects it by (4.11) [103]; Hamiltonian Monte Carlo (Section 4.5) follows the gradient for many steps. *Hessian information* shapes the proposal like the posterior, either once, with the Hessian at the MAP estimate as in Example 4.8, or at every step: stochastic Newton is MALA preconditioned with the inverse Hessian at the current point, i.e., a Newton step plus noise [79]. *Dimension independence* matters when $\boldsymbol{x}$ is a discretized function and the mesh is refined. The step sizes of the random walk and of MALA must then shrink like $n^{-1/2}$ and $n^{-1/6}$, i.e., the proposal variances like n^{-1} and $n^{-1/3}$ [102, 104], and the samplers come to a halt in the limit. The *preconditioned Crank–Nicolson* (pCN) proposal $\boldsymbol{x}' = \sqrt{1 - s^2}\,\boldsymbol{x} + s\,\boldsymbol{\xi}$, with $\boldsymbol{\xi}$ drawn from a Gaussian prior with mean zero, leaves the prior invariant, so that only the likelihood enters the acceptance probability, and the acceptance rate does not degrade as the mesh is refined [25]. The three kinds of information combine into the eight corners of a cube, and the dimension-independent likelihood-informed

(DILI) samplers [27] use all three, together with a reduction to the few directions that the data inform. The corners in blue are those we use. In our problems $n \leq 200$ is fixed, and dimension independence is not our concern, with one exception: the elliptical slice sampler of default projects A and C (Algorithm 4.17) belongs to the back face of the cube.

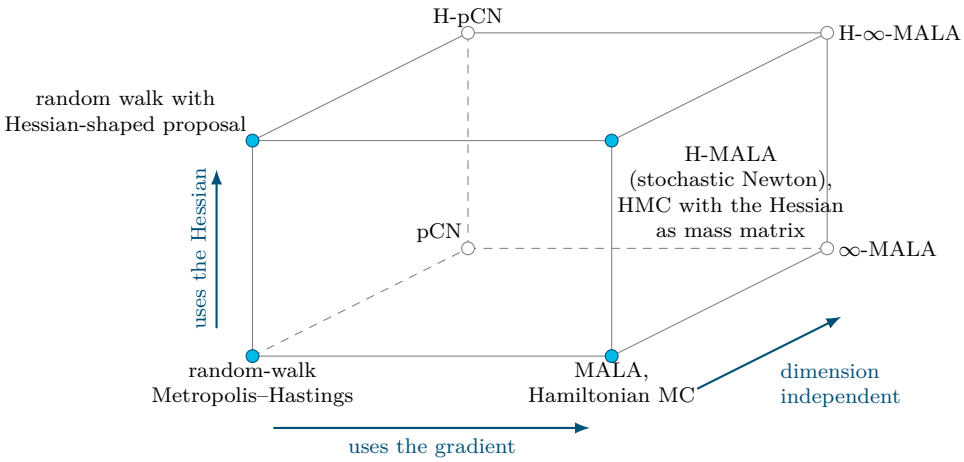


Figure 4.7: Markov chain Monte Carlo methods sorted by the information they use, after [67, Figure 1]: the gradient of U (left to right), its Hessian (bottom to top), and whether the proposal remains efficient when the mesh of a discretized function is refined (front to back). MALA stands for Metropolis-adjusted Langevin algorithm, pCN for preconditioned Crank–Nicolson, and H- for Hessian-informed. Blue: methods used in this chapter.

Three cases. The rest of the chapter does not walk through the cube corner by corner. It sorts the posteriors of this course by their structure, because the structure, not the generality of the method, decides what sampling costs:

1. a Gaussian posterior, as for a linear forward operator with a Gaussian prior, is sampled exactly, with independent samples and no Markov chain (Section 4.3);
2. a posterior whose conditionals can be sampled exactly, as for a linear forward operator with a hierarchical or Laplace prior, is sampled by Gibbs sampling (Section 4.4);
3. everything else, in particular a nonlinear forward operator, is sampled by Hamiltonian Monte Carlo if gradients are available, and by elliptical slice sampling if they are not (Section 4.5).

4.3 Gaussian posteriors: perturb-then-optimize

A Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma})$ is sampled as $\boldsymbol{\mu} + \boldsymbol{R}\boldsymbol{\xi}$ with $\boldsymbol{\xi} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ and any factor $\boldsymbol{R}\boldsymbol{R}^\mathsf{T} = \boldsymbol{\Gamma}$ (Appendix A). For the linear-Gaussian posterior of Theorem 3.10, $\boldsymbol{\Gamma}$ is the inverse of the precision $\boldsymbol{H} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}/\sigma^2 + \boldsymbol{B}^\mathsf{T}\boldsymbol{B}$, where $\boldsymbol{B}^\mathsf{T}\boldsymbol{B}$ is the prior precision, e.g., $\boldsymbol{B} = \boldsymbol{L}/\gamma$ for the smoothness prior, completed by a row for $\ker \boldsymbol{L}$ as in Section 3.3.2. A Cholesky factorization of $\boldsymbol{H}$ is cheap for $n = 128$ and impossible for $n = 10^6$, where only products with $\boldsymbol{A}$, $\boldsymbol{A}^\mathsf{T}$, and $\boldsymbol{B}$ are available. There is a way around the factorization, and it only needs the solver for the MAP estimate.

Theorem 4.10: Perturb-then-optimize

Let $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$, and let the prior be $\mathcal{N}(\boldsymbol{0}, (\boldsymbol{B}^\mathsf{T}\boldsymbol{B})^{-1})$ with $\boldsymbol{B} \in \mathbb{R}^{p \times n}$ such that $\boldsymbol{H} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A}/\sigma^2 + \boldsymbol{B}^\mathsf{T}\boldsymbol{B}$ is invertible. For independent $\boldsymbol{\xi} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_m)$ and $\boldsymbol{\eta} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_p)$, the solution of the perturbed Tikhonov problem

$$\boldsymbol{x}^* = \arg\min_{\boldsymbol{x}} \left\{ \frac{1}{2\sigma^2} \left\| \boldsymbol{A}\boldsymbol{x} - (\boldsymbol{y} + \sigma\boldsymbol{\xi}) \right\|_2^2 + \frac{1}{2} \left\| \boldsymbol{B}\boldsymbol{x} - \boldsymbol{\eta} \right\|_2^2 \right\} \tag{4.12}$$

is an exact sample from the posterior $\mathcal{N}(\boldsymbol{\mu}_{\text{post}}, \boldsymbol{H}^{-1})$.

Proof. The normal equations of (4.12) give $\boldsymbol{x}^* = \boldsymbol{H}^{-1}(\boldsymbol{A}^\mathsf{T}(\boldsymbol{y}+\sigma\boldsymbol{\xi})/\sigma^2 + \boldsymbol{B}^\mathsf{T}\boldsymbol{\eta})$, an affine function of the Gaussian vector $(\boldsymbol{\xi}, \boldsymbol{\eta})$ and hence Gaussian. Its mean is $\boldsymbol{H}^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{y}/\sigma^2 = \boldsymbol{\mu}_{\text{post}}$ by (3.13), and its covariance is $\boldsymbol{H}^{-1}(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}/\sigma^2 + \boldsymbol{B}^\mathsf{T}\boldsymbol{B})\boldsymbol{H}^{-1} = \boldsymbol{H}^{-1}$. □

The recipe is to perturb the data by a fresh noise realization, to perturb the prior mean by a fresh draw, and to compute the MAP estimate of the perturbed problem. Figure 4.8 shows it for two unknowns. The objective of (4.12) is a quadratic bowl, whose shape is set by the precision $\boldsymbol{H}$ and whose lowest point, without the perturbations, is the posterior mean (a). Perturbing the data and the prior mean moves the bowl without changing its shape (b), and the lowest points of many such bowls scatter exactly like the posterior: 94.6 percent of 500 of them lie inside its 95 % ellipse (c).

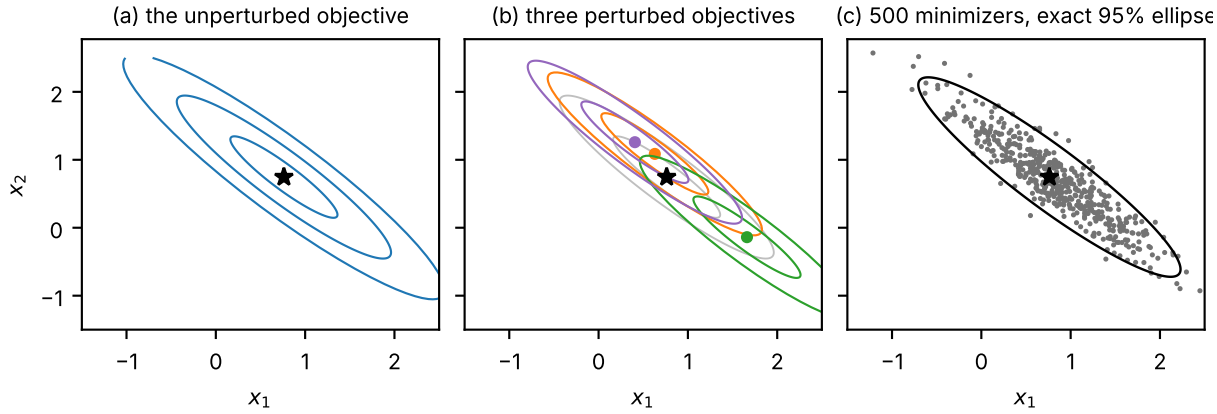


Figure 4.8: Perturb-then-optimize for the two-unknown problem of Figure 3.3a with the prior $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. (a) Contours of the objective of (4.12) without perturbations, and its minimizer, the posterior mean (star). (b) Three perturbed objectives (colors) with their minimizers (dots), and the unperturbed one (gray). (c) The minimizers of 500 perturbed objectives and the 95 % ellipse of the posterior.

Each sample costs one regularized solution, computed with whatever solver computes the MAP estimate, and the samples are independent: there is no Markov chain, no burn-in, and no autocorrelation. For our problem sizes, `samplers.perturb_optimize` computes one QR factorization of the stacked matrix $\left(\begin{smallmatrix} \boldsymbol{A}/\sigma \\ \boldsymbol{B} \end{smallmatrix}\right)$ and reuses it for all samples; for large problems, every sample is one run of conjugate gradients on the normal equations of (4.12). The idea has been found several times, under names such as perturbation-optimization [92] and Gaussian sampling by local perturbations [94]. For a nonlinear forward operator, perturbed optimization gives only approximate samples, and randomize-then-optimize uses them as proposals of a Metropolis–Hastings step [4].

Remark 4.11: Solving (4.12) only approximately

In large problems, (4.12) is solved by an iterative method that is stopped after finitely many steps, typically conjugate gradients for the normal equations $\boldsymbol{H}\boldsymbol{x} = \boldsymbol{b}$ with the perturbed right-hand side $\boldsymbol{b} = \boldsymbol{A}^{\mathsf{T}}(\boldsymbol{y} + \sigma\boldsymbol{\xi})/\sigma^2 + \boldsymbol{B}^{\mathsf{T}}\boldsymbol{\eta}$. The samples are then no longer exact, and not even Gaussian, since the iterates of conjugate gradients depend nonlinearly on $\boldsymbol{b}$. The direction of the error is predictable, however. Started at $\boldsymbol{x}_0$, the k-th iterate is the $\boldsymbol{H}$-orthogonal projection of the exact sample $\boldsymbol{x}^*$ onto $\boldsymbol{x}_0$ plus a Krylov space, so that it is never farther from $\boldsymbol{x}_0$ than $\boldsymbol{x}^*$ in the norm $(\boldsymbol{v}^{\mathsf{T}}\boldsymbol{H}\boldsymbol{v})^{1/2}$. Started at zero, truncated conjugate gradients therefore pull the samples toward zero. Started at the MAP estimate, they keep the mean right, because the iterate is an odd function of the perturbation, but they make the samples too concentrated around it: the solver underestimates exactly the uncertainty that the samples were meant to quantify. For the smooth deconvolution signal of Example 4.12, whose precision has the condition number 44, five iterations started at zero shift the mean by up to 0.9 posterior standard deviations, five iterations started at the MAP estimate shrink the posterior standard deviations by up to 40 percent, and 20 iterations give samples that cannot be distinguished from exact ones. The number of iterations needed grows like $\sqrt{\kappa(\boldsymbol{H})}$, so that a good preconditioner pays twice, and the safe practice is to check that doubling the number of iterations does not change the sample statistics. Exact alternatives exist: conjugate gradients can produce samples from the Krylov space that they build [97], and an approximate solution can be accepted or rejected by a Metropolis–Hastings step that makes the result exact for any accuracy of the solver [41].

Example 4.12: What the samples answer for the smooth signal

Figure 4.9 uses 4000 perturb-then-optimize samples of the posterior of Example 3.12, which take 0.03 seconds. Panel (a) is a test: the sample mean and standard deviation converge to the closed form at the rate $N^{-1/2}$, and after 4000 samples the largest error of the mean is 0.034 posterior standard deviations, which is the Monte Carlo error and nothing else. Panel (b) answers a question that the band of Figure 3.6a does not: where does the signal exceed 0.5? The posterior probability is one on the first bump and reaches 0.9 on the second, whose true height is 0.6, and it changes from zero to one within a few cells at the edges of the bumps. Panel (c) recomputes Figure 3.6a from the samples: the 95 % band estimated from the samples differs from the closed form by at most 0.004. The samples also give what no band gives, the posterior of derived quantities. The height $\max_{t<0.5} x(t)$ of the first bump has the posterior mean 0.88 and the 95 % credible interval $[0.82, 0.95]$, and two lessons hide in these numbers. First, the peak of the posterior mean, 0.86, is not the posterior mean of the peak, 0.88: the maximum of an average is smaller than the average of the maxima, and a quantity of interest must be evaluated sample by sample and averaged afterwards, not evaluated at the mean. Second, the true height 1.0 lies far outside the interval. The smoothness prior, with the scale chosen by the discrepancy principle, flattens the peak, and the posterior inherits this bias with a confidence that the band, which contains the truth at 93 percent of the points, does not reveal at first sight; panel (c) shows the truth leaving the band at the top of the first bump. Sampling is exact here; the prior is not.

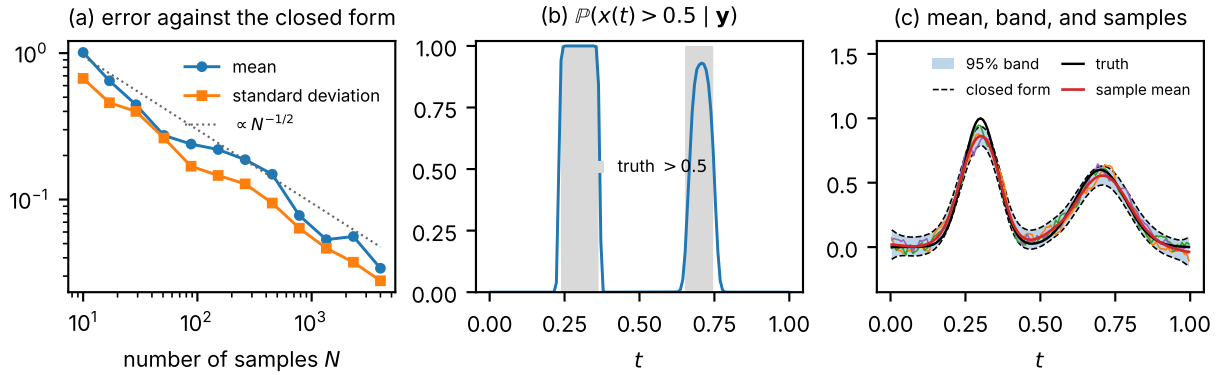


Figure 4.9: Perturb-then-optimize (Theorem 4.10) for the smooth deconvolution signal with the smoothness prior of Example 3.12 (Example 4.12). (a) Largest error of the sample mean and of the sample standard deviation, in units of the posterior standard deviation, against the number of samples. (b) The posterior probability that $x(t) > 0.5$, with a band of two Monte Carlo standard errors; gray: where the truth exceeds 0.5. (c) Posterior mean and 95 % band estimated from the samples, three samples, and the band of Figure 3.6a from the closed form (dashed).

4.4 Tractable conditionals: Gibbs sampling

The hierarchical prior was built such that $\boldsymbol{x}$ is Gaussian given $\boldsymbol{\theta}$ (Section 3.3.4), and the iterative alternating scheme of Algorithm 3.14 exploited this structure for optimization by alternating between $\boldsymbol{x}$ and $\boldsymbol{\theta}$. The Gibbs sampler exploits it for sampling, by alternating between exact draws from the two conditionals [38].

Algorithm 4.13: Gibbs sampling

Split the unknowns into blocks, here $(\boldsymbol{x}, \boldsymbol{\theta})$, and start from some $\boldsymbol{\theta}^{(0)}$. For $k = 0, 1, 2, \ldots$:

1. draw $\boldsymbol{x}^{(k+1)} \sim p(\boldsymbol{x} \mid \boldsymbol{\theta}^{(k)}, \boldsymbol{y})$;
2. draw $\boldsymbol{\theta}^{(k+1)} \sim p(\boldsymbol{\theta} \mid \boldsymbol{x}^{(k+1)}, \boldsymbol{y})$.

Each step leaves the joint posterior $p(\boldsymbol{x}, \boldsymbol{\theta} \mid \boldsymbol{y})$ invariant, because it replaces one block by a draw from its correct conditional and leaves the other block alone; it is a Metropolis–Hastings step whose proposal is accepted with probability one. There is nothing to tune, and nothing is rejected.

Figure 4.10a shows one sweep on the bent ridge of Example 4.3, with the two unknowns as blocks. The conditional of x_1 given x_2 is a narrow Gaussian across the ridge, and the conditional of x_2 given x_1 has two modes, one on each arm, so that a single draw can carry the walker from one arm to the other, as in Figure 4.2b. The price of moving only parallel to the axes shows when the unknowns are strongly coupled. For a Gaussian with correlation ρ, the conditionals have the standard deviation $\sqrt{1-\rho^2}$, and the walker crawls along the diagonal in small steps (b); the chain of x_1 is the autoregressive chain of Exercise 4.1 with $a = \rho^2$, whose integrated autocorrelation time is $(1+\rho^2)/(1-\rho^2) \approx 50$ for $\rho = 0.98$. Strongly coupled unknowns should therefore be drawn together, in one block, which is what the $\boldsymbol{x}$-step below does with all components of $\boldsymbol{x}$.

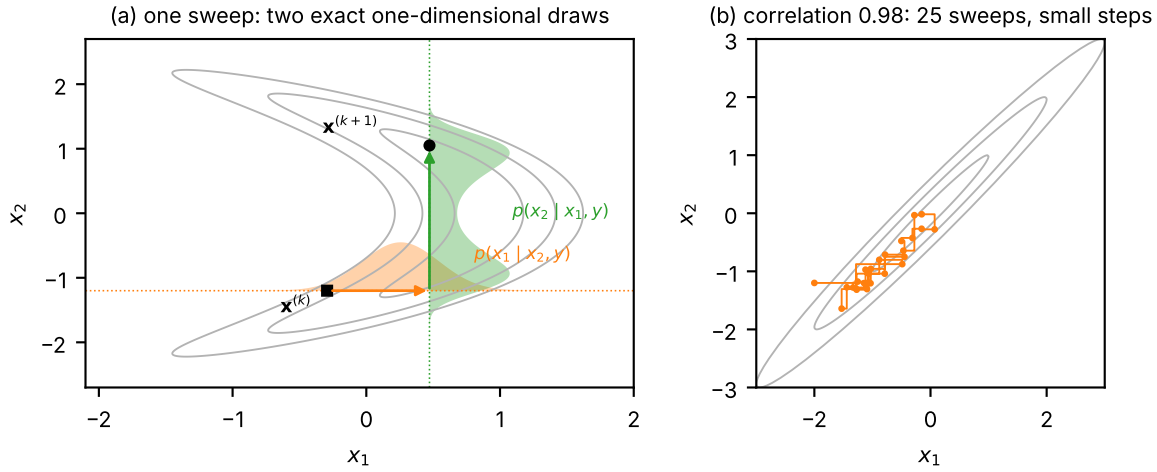


Figure 4.10: Gibbs sampling. (a) One sweep on the bent ridge: from $\boldsymbol{x}^{(k)}$ (square), x_1 is drawn from its conditional (orange, along the dotted horizontal line), and then x_2 from its conditional (green, along the dotted vertical line), which has two modes. (b) Twenty-five sweeps for a Gaussian with correlation 0.98: the conditionals are narrow, and the walker moves along the diagonal in small steps.

4.4.1 The hierarchical prior

For a linear forward operator and the hierarchical prior (3.10) on the increments $\boldsymbol{z} = \boldsymbol{L}\boldsymbol{x}$, both conditionals are standard distributions. Given $\boldsymbol{\theta}$, the prior is Gaussian with the precision $\boldsymbol{L}^{\mathsf{T}}\boldsymbol{D}_{\boldsymbol{\theta}}^{-1}\boldsymbol{L}$, completed for $\ker \boldsymbol{L}$, and $p(\boldsymbol{x} \mid \boldsymbol{\theta}, \boldsymbol{y})$ is the linear-Gaussian posterior of Theorem 3.10, which Theorem 4.10 samples with $\boldsymbol{B} = \boldsymbol{D}_{\boldsymbol{\theta}}^{-1/2}\boldsymbol{L}$. Given $\boldsymbol{x}$, the local variances are independent of each other and of the data, with

$$p(\theta_i \mid \boldsymbol{x}, \boldsymbol{y}) \propto \mathcal{N}(z_i; 0, \theta_i)\, p(\theta_i) \propto \theta_i^{-(\beta+1/2)-1} \exp\Big(-\frac{\vartheta + z_i^2/2}{\theta_i}\Big), \tag{4.13}$$

i.e., $\theta_i \mid \boldsymbol{x} \sim \mathrm{InvGamma}(\beta + \frac{1}{2}, \vartheta + \frac{1}{2}z_i^2)$ by (3.11). The comparison with the IAS is instructive. The $\boldsymbol{\theta}$-step of Algorithm 3.14, $\theta_i = (\vartheta + z_i^2/2)/(\beta + 3/2)$, is the mode of (4.13), and the $\boldsymbol{x}$-step is the mean of the Gaussian conditional, which perturb-then-optimize turns into a draw. The Gibbs sampler is the IAS with dice: where the IAS takes the most probable value of each conditional, the Gibbs sampler draws one. The function `samplers.gibbs_hierarchical` implements it in the variables $(x_1, \boldsymbol{z})$, the level and the increments, in which the prior precision is diagonal; a sweep costs one Cholesky factorization of an $n \times n$ matrix and $n - 1$ draws from inverse gamma distributions, about one millisecond for $n = 128$.

4.4.2 The Laplace prior as a hierarchical prior

The Laplace prior has no Gaussian conditional in sight, but it has one in disguise. Its density is a *scale mixture* of Gaussians [3],

$$\frac{1}{2\gamma}\, e^{-|z|/\gamma} = \int_0^\infty \mathcal{N}(z; 0, \theta)\, \frac{1}{2\gamma^2}\, e^{-\theta/(2\gamma^2)}\, \mathrm{d}\theta, \tag{4.14}$$

i.e., $z \mid \theta \sim \mathcal{N}(0, \theta)$ with an exponential hyper-prior of mean $2\gamma^2$ on the variance (Exercise 4.2). The Laplace prior is therefore a hierarchical prior with the structure of (3.10) and a different hyper-prior, and the Gibbs sampler carries over with a new $\boldsymbol{\theta}$-step: given z_i, the precision $1/\theta_i$

follows an inverse Gaussian distribution with mean $1/(\gamma\,|z_i|)$ and shape parameter $1/\gamma^2$. This is the Bayesian lasso of Park and Casella [96], implemented in `samplers.gibbs_laplace`. The representation also explains the samples of Figure 3.4 in one sentence: the exponential hyper-prior has a light tail, so that the variances of all increments are of the same order and none is much louder than the others, whereas the heavy tail of the inverse gamma hyper-prior lets a few variances exceed the typical one by orders of magnitude, and their increments become jumps.

Example 4.14: Three posteriors for the piecewise-constant signal

Figure 4.11 is the sampling counterpart of Figure 3.8: the posteriors of the piecewise-constant deconvolution problem under the three priors on increments, with the parameters of Example 3.15. They are computed from 18000 perturb-then-optimize samples for the Gaussian prior and from $2 \cdot 10^4$ Gibbs sweeps with a burn-in of 2000 for the Laplace and the hierarchical prior, in 35 seconds altogether.
Posterior means (top row). The errors of the posterior means are 1.50, 1.39, and 0.99; the MAP estimates of Example 3.15 had 1.50, 0.82, and 0.64. For the two edge-preserving priors, the mean is worse than the MAP estimate, because it averages over samples whose jumps sit in different cells and is therefore smooth; this is the second caveat of Remark 3.16 in action, and it shows that the purpose of sampling is not a better point estimate.
Credible bands (top row). The truth lies inside the 95 % band at 87, 95, and 98 percent of the points. The Gaussian band is equally wide everywhere, about 0.36, and misses the truth at the four large edges. The Laplace band widens at the edges and misses fewer points. The hierarchical band is narrowest on the plateaus, 0.23, and at the large edges as wide as the jump itself, because the posterior is unsure in which of two or three cells the jump happens, so that the value there is either low or high. Its few misses are all at the small step of height 0.3, where the posterior is as sure as the MAP estimate that the signal is a ramp: no sampler can repair what the prior believes, and Example 3.15 explained why this prior prefers a ramp.
Samples (middle row). Only the hierarchical samples look like the truth, with flat plateaus and sharp jumps. A sample has on average 44, 46, and 13 increments larger than 0.1 in absolute value, against five in the truth, and its total variation is 10.8, 12.7, and 7.5, against 3.6. The Laplace samples are as rough as the Gaussian ones, as Remark 3.9 predicted from the samples of the prior.
Jump probabilities (bottom row). The probability that an increment exceeds 0.1 is about 0.3 everywhere for the Gaussian and the Laplace prior, with peaks of about 0.6 at the large edges. For the hierarchical prior, it is about 0.05 on the plateaus and rises to between 0.3 and 0.5 at the four large jumps, spread over two to four cells: this is the uncertainty of the jump positions, which no MAP estimate reports. At the small step it stays below 0.1.

Remark 4.15: Funnels, and why default project C changes variables

The Gibbs sampler of the hierarchical prior mixes well on the plateaus and slowly at the jumps (Example 4.20): the integrated autocorrelation time of $\log\theta_i$ is about 2 sweeps on a plateau and 80 at the first jump. The reason is the coupling between z_i and θ_i. A large θ_i allows a large z_i, and a large z_i makes the next θ_i large, so that the chain switches only rarely between "the jump is in this cell" and "the jump is in the neighboring cell". In the plane of z_i and $\log\theta_i$, the prior $\mathcal{N}(z_i; 0, \theta_i)\,p(\theta_i)$ has the shape of a funnel, wide at the top, where θ_i is large, and narrow at the bottom (Figure 4.12a). Neal's funnel [86] is the standard example of such a geometry, on which every sampler with a fixed step size struggles, because a step that suits the neck is far too small for the top. Figure 4.12 shows the effect in the smallest problem that has it: two increments z_1, z_2

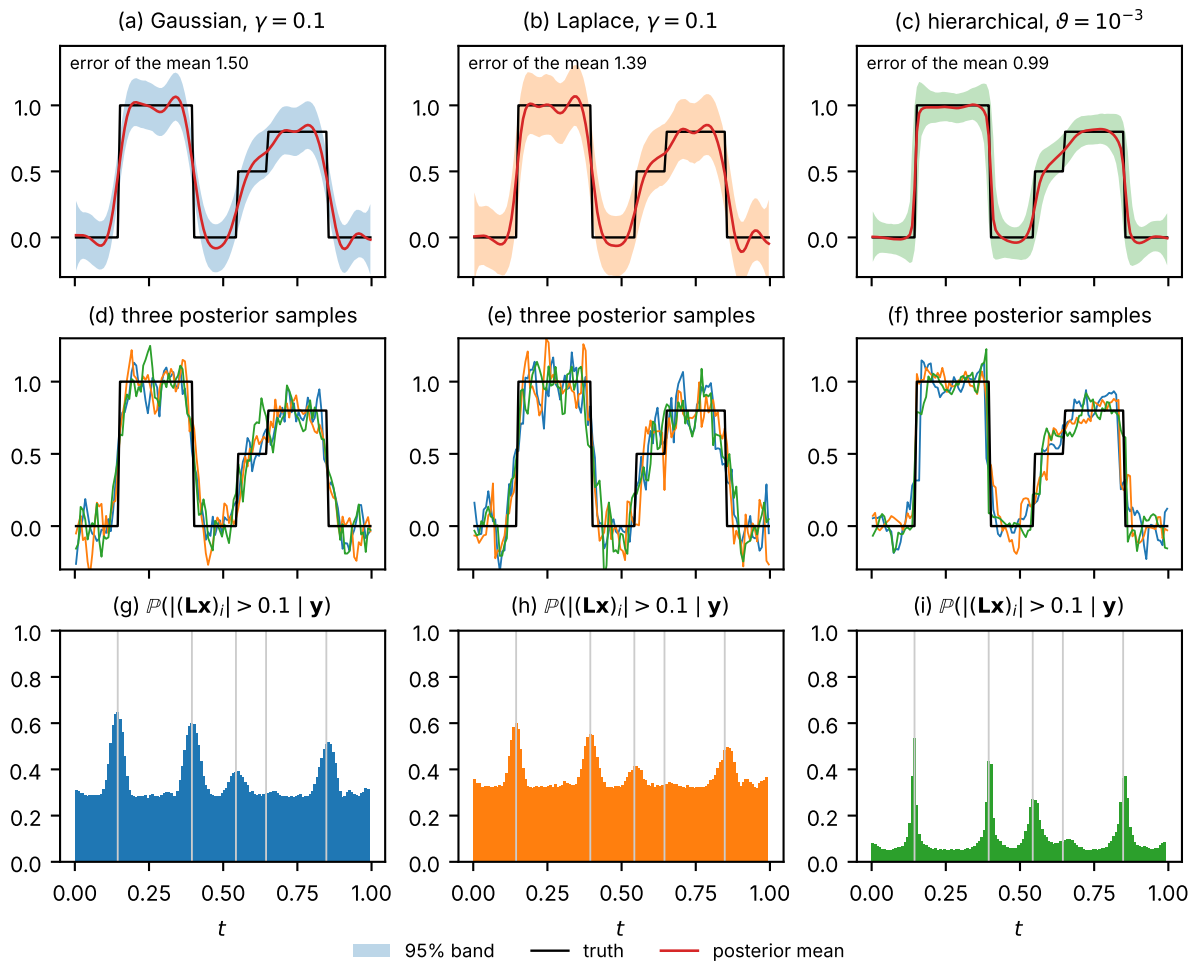


Figure 4.11: Posteriors of the piecewise-constant deconvolution problem under the three priors on increments of Figure 3.8 (Example 4.14). Columns: Gaussian, Laplace, and hierarchical prior. Top row: posterior mean and 95 % credible band. Middle row: three posterior samples. Bottom row: the posterior probability that the increment $(\boldsymbol{Lx})_i$ exceeds 0.1 in absolute value; gray lines: true jump positions.

with the hierarchical prior and one datum of their sum, $y = z_1 + z_2 + \varepsilon$ with $y = 0.5$ and $\varepsilon \sim \mathcal{N}(0, 0.05^2)$, which says that the signal rises by 0.5 in one of two cells but not in which. The Gibbs sampler moves the jump from one cell to the other only about every 60 sweeps (b), and the integrated autocorrelation time of z_1 is 52. For a nonlinear forward operator there is a second problem: $\boldsymbol{x}$ is no longer Gaussian given $\boldsymbol{\theta}$, and the Gibbs sampler is lost. Both problems are attacked by changes of variables. The non-centered parametrization $z_i = \sqrt{\theta_i}\, w_i$ with independent $w_i \sim \mathcal{N}(0, 1)$ moves the coupling from the prior into the likelihood [95]. The hierarchical prior normalization of Glaubitz and Marzouk [44] goes further: it writes the hierarchical prior as the image of a standard normal distribution under an explicit map, here $\theta_i = F^{-1}(\Phi(v_i))$ and $z_i = \sqrt{\theta_i}\, u_i$ with the distribution functions F of the hyper-prior and Φ of $\mathcal{N}(0, 1)$. In the variables $(\boldsymbol{u}, \boldsymbol{v})$, the prior is standard normal and the funnel is gone (c), and the posterior is a standard normal prior times a likelihood, the form for which elliptical slice sampling (Algorithm 4.17) is made, for linear and nonlinear forward operators alike. The coupling does not disappear, however; it moves into the likelihood. In the toy problem, elliptical slice sampling in the

normalized variables switches between the cells less often than the Gibbs sampler, with $\tau = 191$ for z_1; its advantages are that it needs no linear solves and that it works for nonlinear forward operators. Default project C takes this route, and part of its work is to find out what it costs.

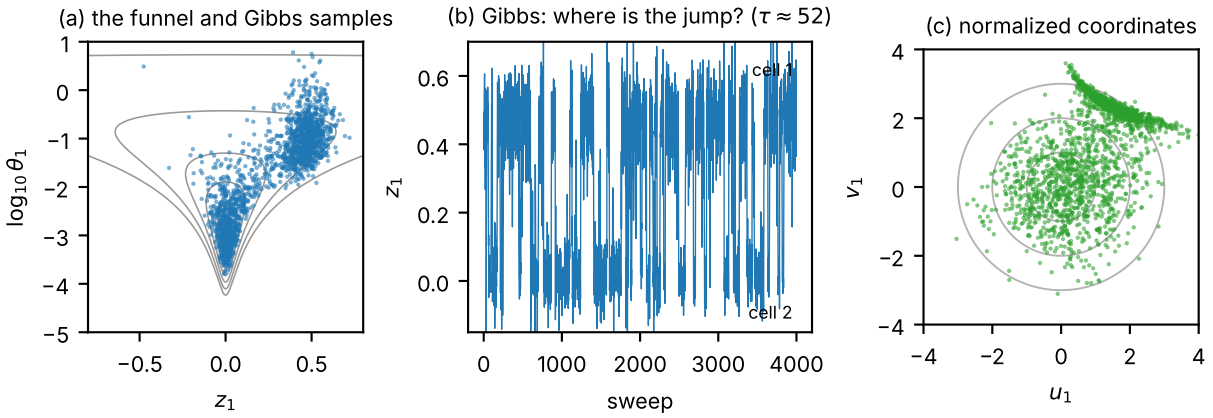


Figure 4.12: The funnel of the hierarchical prior in the toy problem of Remark 4.15: two increments and one datum of their sum. (a) Contours of the logarithm of the prior density of $(z_1, \log_{10}\theta_1)$ and Gibbs samples, which sit either in the wide top (the jump is in cell 1) or in the narrow neck (it is in cell 2). (b) The trace of z_1 under the Gibbs sampler: the jump switches between the cells. (c) The posterior in the normalized variables (u_1, v_1), in which the prior is standard normal (contours), sampled by elliptical slice sampling.

4.5 Everything else: Hamiltonian Monte Carlo

For a nonlinear forward operator, the posterior is neither Gaussian nor conditionally Gaussian, and we are back to Metropolis–Hastings with a general proposal. The random walk fails in high dimensions because its proposals are blind: a random direction in $\mathbb{R}^n$ leads uphill, and the step must be tiny to be accepted. Hamiltonian Monte Carlo (HMC) [29, 87] makes proposals that are far away and still likely to be accepted, by letting the gradient of U steer them.

The sled. Think of the landscape U as a frictionless surface and of the walker as a sled at the position $\boldsymbol{x}$. She gives herself a random push, a *momentum* $\boldsymbol{r} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{M})$ with a symmetric positive definite *mass matrix* $\boldsymbol{M}$, and slides. Going downhill she gains speed, going uphill she loses it, and after a while she is far away, at a point of the same total energy

$$H(\boldsymbol{x}, \boldsymbol{r}) = U(\boldsymbol{x}) + \tfrac{1}{2}\,\boldsymbol{r}^{\mathsf{T}}\boldsymbol{M}^{-1}\boldsymbol{r}, \tag{4.15}$$

potential plus kinetic. The motion obeys Hamilton's equations

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t} = \boldsymbol{M}^{-1}\boldsymbol{r}, \quad \frac{\mathrm{d}\boldsymbol{r}}{\mathrm{d}t} = -\nabla U(\boldsymbol{x}), \tag{4.16}$$

and their flow has three properties that make it a perfect proposal: it conserves H, it preserves volume in the space of all $(\boldsymbol{x}, \boldsymbol{r})$, and it is reversible, i.e., flipping the momentum and following the flow again leads back to the start [87]. The joint density $e^{-H(\boldsymbol{x},\boldsymbol{r})} \propto p(\boldsymbol{x} \mid \boldsymbol{y})\,\mathcal{N}(\boldsymbol{r}; \boldsymbol{0}, \boldsymbol{M})$ has the posterior as its $\boldsymbol{x}$-marginal and is left invariant by the flow, so that a proposal made by

following the exact flow for any time would be accepted with probability one. Figure 4.13 shows both pictures for the one-dimensional target of Figure 4.3. In panel (a), the sled is pushed at the bottom of the left valley with enough kinetic energy to cross the ridge; it is fast where U is low and slow where U is high, and it turns around where U reaches its total energy. Panel (b) shows the same dynamics in the plane of position and momentum, the *phase space*, in which trajectories run along level sets of H: closed curves around one valley at low energy and around both valleys at high energy. HMC alternates random kicks, which change the momentum and with it the level set, with flows along level sets; the positions at the ends of the flows are the samples, and only a large kick lets the chain leave its valley. The exact flow is not available, but a discretization that keeps two of the three properties is.

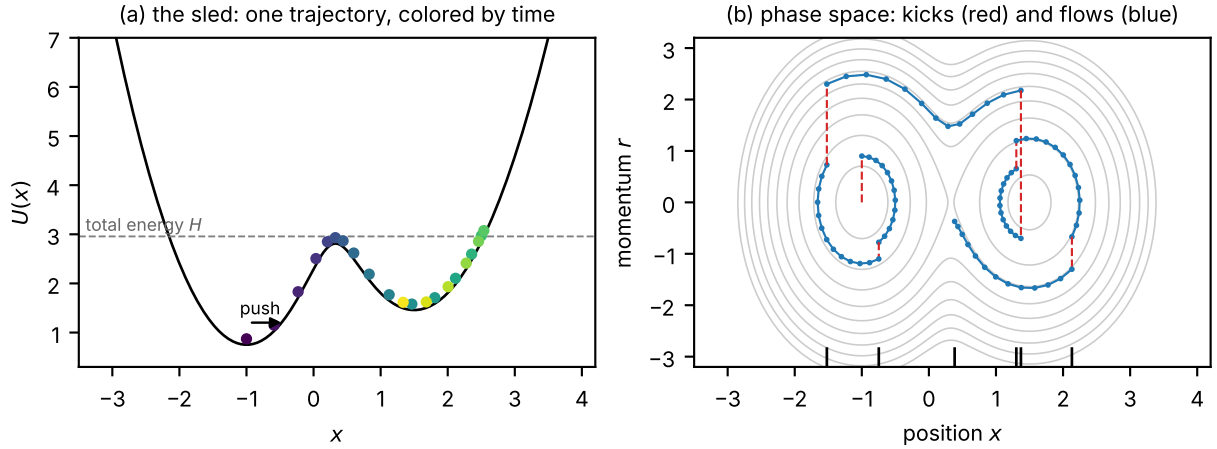


Figure 4.13: Hamiltonian Monte Carlo for the one-dimensional target of Figure 4.3. (a) The potential $U = -\log p$ and one trajectory of the sled (dots colored by time), pushed at the bottom of the left valley; the dashed line is its total energy. (b) Phase space: level sets of H (gray), six iterations of HMC with hand-picked momentum kicks (red) and leapfrog flows (blue), and the samples (ticks); the third kick is large enough to cross the ridge.

The leapfrog scheme. One step of size h of the *leapfrog* or Störmer–Verlet scheme is

$$\boldsymbol{r}_{1/2} = \boldsymbol{r} - \tfrac{h}{2}\,\nabla U(\boldsymbol{x}), \quad \boldsymbol{x}' = \boldsymbol{x} + h\,\boldsymbol{M}^{-1}\boldsymbol{r}_{1/2}, \quad \boldsymbol{r}' = \boldsymbol{r}_{1/2} - \tfrac{h}{2}\,\nabla U(\boldsymbol{x}'), \tag{4.17}$$

a half kick, a drift, and a half kick. Each of the three substeps changes one variable by an amount that depends only on the other, so that each preserves volume, and the symmetric arrangement makes the step reversible. Energy is conserved only approximately, with an error of order h^2 that stays bounded over long trajectories, and the Metropolis–Hastings step corrects it exactly.

Algorithm 4.16: Hamiltonian Monte Carlo

Choose a step size h, a number of steps L, a mass matrix $\boldsymbol{M}$, and a start $\boldsymbol{x}^{(0)}$. For $k = 0, 1, 2, \ldots$:

1. Draw a momentum $\boldsymbol{r} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{M})$.
2. Starting from $(\boldsymbol{x}^{(k)}, \boldsymbol{r})$, take L leapfrog steps (4.17), which end at $(\boldsymbol{x}', \boldsymbol{r}')$.
3. Accept $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}'$ with probability $\min\{1, \exp(H(\boldsymbol{x}^{(k)}, \boldsymbol{r}) - H(\boldsymbol{x}', \boldsymbol{r}'))\}$; otherwise set $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)}$.

The momentum is discarded after every iteration. Consecutive half kicks merge into full kicks, and an iteration costs L gradients of U. The functions `samplers.hmc` (NumPy) and `samplers.make_hmc_jax` (JAX) implement the algorithm in about thirty lines each;

both draw the step size in every iteration at random from $[0.8h, 1.2h]$, which prevents trajectories that return exactly to where they started.

With a single leapfrog step and $\boldsymbol{M} = \boldsymbol{I}$, the proposal of HMC is that of MALA, and HMC with $L = 1$ is MALA; the gain of HMC comes from long trajectories.

Tuning. HMC has three parameters, and each has a clear meaning. The step size h sets the accuracy of the leapfrog scheme: if it is too large, the energy error causes rejections, or the scheme becomes unstable and the trajectory explodes; acceptance rates between 0.6 and 0.9 are a good target [8]. The trajectory length hL sets how far a proposal goes: if it is too short, HMC degenerates to a random walk, and if it is too long, the trajectory turns around and comes back. The mass matrix $\boldsymbol{M}$ is a preconditioner, and its best choice is the Hessian of U, i.e., the posterior precision. To see why, take a Gaussian posterior with precision $\boldsymbol{H}$ and $\boldsymbol{M} = \boldsymbol{H}$. Then (4.16) becomes $\ddot{\boldsymbol{x}} = -(\boldsymbol{x} - \boldsymbol{\mu}_{\text{post}})$, a harmonic oscillator with the same period 2π in every direction, whose solution

$$\boldsymbol{x}(t) - \boldsymbol{\mu}_{\text{post}} = \cos t\,(\boldsymbol{x}(0) - \boldsymbol{\mu}_{\text{post}}) + \sin t\,\boldsymbol{H}^{-1}\boldsymbol{r} \tag{4.18}$$

is, at $t = \pi/2$, an exact sample from the posterior, independent of the start (Exercise 4.5). A trajectory of length $hL \approx \pi/2$ thus produces a nearly independent sample, one of length π the mirror image of the start, and one of length 2π the start itself, which is why the step size is varied at random. For a non-Gaussian posterior, the Gauss–Newton Hessian at the MAP estimate is a good mass matrix as long as the posterior is not too far from Gaussian.

The no-U-turn sampler. NUTS, the no-U-turn sampler of Hoffman and Gelman [59], chooses the trajectory length by itself: it extends the trajectory forwards and backwards in time, doubling its length, until its two ends begin to approach each other, and then draws the new state from the points of the trajectory in a way that preserves the posterior. During a warm-up phase, it also adapts h to a target acceptance rate and estimates a diagonal or dense mass matrix from the warm-up samples. NUTS is the default sampler of Stan, NumPyro [98], and BlackJAX [17]; we use BlackJAX, which is written in JAX and runs on any log-density written in JAX.

Gradients. Every leapfrog step needs $\nabla U(\boldsymbol{x}) = \boldsymbol{J}_F(\boldsymbol{x})^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}(F(\boldsymbol{x}) - \boldsymbol{y}) - \nabla \log p(\boldsymbol{x})$, the gradient of Chapter 2, and the three ways to a gradient of Section 2.5 apply. With the forward model in JAX, the potential takes a few lines and its gradient one more:

```
def U(b):                                   # potential: misfit + smoothness prior
    r = forward(b) - y                      # forward map of Chapter 2
    z = L @ b                               # increments of the bottom
    return 0.5 * r @ r / sigma**2 + 0.5 * z @ z / gamma**2
grad_U = jax.jit(jax.grad(U))               # reverse mode
step = samplers.make_hmc_jax(U, eps=0.3, n_leap=5, Minv_chol=C)   # C C^T = M^{-1}
```

We count the cost of gradient-based samplers in gradients and that of gradient-free ones in forward solves, and convert only where a comparison needs it. For the steady shallow-water model with 199 unknowns and 20 sensors, one reverse-mode gradient of U costs about 8 forward solves and one forward-mode Jacobian (`jax.jacfwd`) about 200, i.e., about one per unknown, like finite differences; these are medians of timings inside jitted loops, and they depend on the machine.

Without gradients: elliptical slice sampling. When gradients of the forward model are not available, as for the spectral solver of [111], the gradient corners of Figure 4.7 are closed, but the structure of the prior can still be used. Suppose that the posterior has the form $p(\boldsymbol{w} \mid$

$\boldsymbol{y}) \propto \mathcal{L}(\boldsymbol{w})\,\mathcal{N}(\boldsymbol{w}; \mathbf{0}, \boldsymbol{I})$ with a standard normal prior and the likelihood $\mathcal{L}(\boldsymbol{w}) = p(\boldsymbol{y} \mid \boldsymbol{w})$. Every Gaussian prior can be brought into this form by whitening, $\boldsymbol{x} = \boldsymbol{\mu} + \boldsymbol{R}\boldsymbol{w}$ with $\boldsymbol{R}\boldsymbol{R}^\mathsf{T} = \boldsymbol{\Gamma}_\text{prior}$, and the hierarchical prior by the prior normalization of Remark 4.15. Elliptical slice sampling [84] moves on the ellipse through the current point and a fresh prior draw, which the prior leaves invariant, and searches along the ellipse for a point that the likelihood accepts.

Algorithm 4.17: Elliptical slice sampling

Given the current state $\boldsymbol{w}$:

1. Draw $\boldsymbol{\nu} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ and $u \sim \mathcal{U}(0, 1)$, and set the threshold $\ell = \log \mathcal{L}(\boldsymbol{w}) + \log u$.
2. Draw an angle $\varphi \sim \mathcal{U}(0, 2\pi)$ and set the bracket $[\varphi_\text{min}, \varphi_\text{max}] = [\varphi - 2\pi, \varphi]$.
3. Set $\boldsymbol{w}' = \boldsymbol{w}\cos\varphi + \boldsymbol{\nu}\sin\varphi$. If $\log \mathcal{L}(\boldsymbol{w}') > \ell$, return $\boldsymbol{w}'$ as the new state.
4. Otherwise shrink the bracket towards $\varphi = 0$, which is the current state: set $\varphi_\text{min} = \varphi$ if $\varphi < 0$ and $\varphi_\text{max} = \varphi$ otherwise; draw $\varphi \sim \mathcal{U}(\varphi_\text{min}, \varphi_\text{max})$ and go to step 3.

Every iteration ends with a move, there is no step size to tune, and only values of the likelihood are needed, typically a handful per iteration.

Figure 4.14 shows one iteration for the bent ridge of Example 4.3, whose prior is already standard normal, so that $\boldsymbol{w} = \boldsymbol{x}$. The ellipse through $\boldsymbol{w}$ and $\boldsymbol{\nu}$ crosses the ridge four times, and the slice, the part of the ellipse on which $\log \mathcal{L} > \ell$, consists of four arcs. The first proposal, at the ends of the initial bracket, is rejected; the second is rejected as well and becomes the new right end of the bracket; the third lands on an arc that does not contain the current state and is accepted. One iteration can thus carry the chain from one arm of the ridge to the other, a move that the random walk of Example 4.7 makes only in many small steps. Since the bracket always contains the current state at $\varphi = 0$, the shrinking stops after finitely many proposals, and the resulting chain leaves the posterior invariant [84].

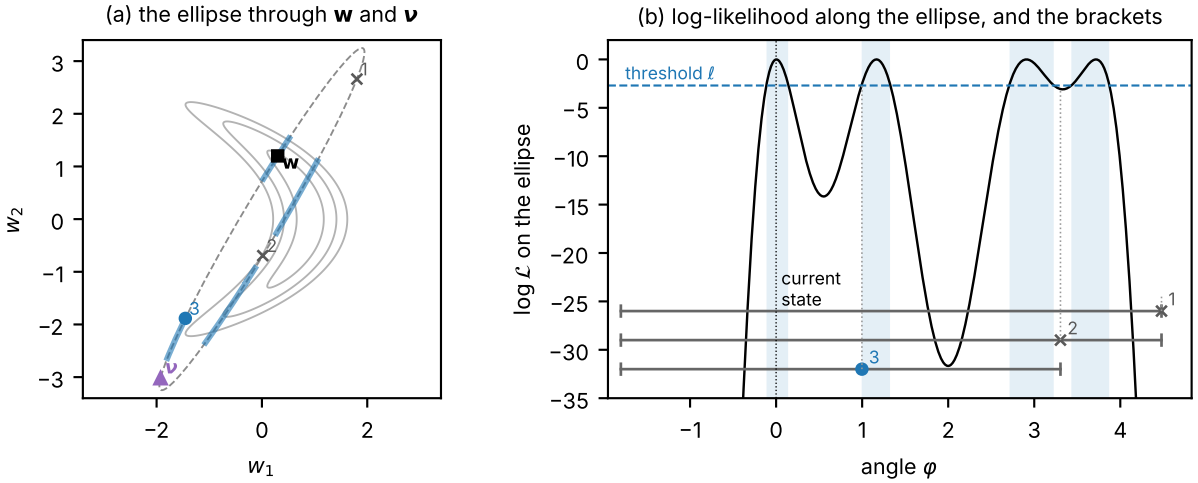


Figure 4.14: Elliptical slice sampling (Algorithm 4.17) on the bent ridge of Example 4.3, whose prior is standard normal. (a) The ellipse (dashed) through the current state $\boldsymbol{w}$ (square) and the prior draw $\boldsymbol{\nu}$ (triangle), the slice (blue arcs), and the three proposals of one iteration, rejected (crosses) and accepted (dot). (b) The log-likelihood along the ellipse as a function of the angle φ, the threshold ℓ (dashed), the slice (shaded), and the bracket from which each proposal is drawn; the brackets shrink towards the current state at $\varphi = 0$.

The prior is only one choice of the Gaussian reference. Any Gaussian approximation of the posterior, say with mean $\boldsymbol{m}$ and covariance $\boldsymbol{R}\boldsymbol{R}^\mathsf{T}$, can take its place: with $\boldsymbol{x} = \boldsymbol{m} + \boldsymbol{R}\boldsymbol{w}$, the posterior of $\boldsymbol{w}$ is $\mathcal{N}(\boldsymbol{w}; \mathbf{0}, \boldsymbol{I})$ times $\mathcal{L}(\boldsymbol{w}) = p(\boldsymbol{m} + \boldsymbol{R}\boldsymbol{w} \mid \boldsymbol{y})/\mathcal{N}(\boldsymbol{w}; \mathbf{0}, \boldsymbol{I})$, and Algorithm 4.17

applies unchanged [88]. The closer the reference is to the posterior, the more nearly constant $\mathcal{L}$ is and the longer the arcs that the sampler accepts. The Laplace approximation of Chapter 5, the Gaussian with the MAP estimate as mean and the inverse Hessian as covariance, is the natural choice; with it, elliptical slice sampling uses Hessian information and moves to the corner of H-pCN in Figure 4.7.

Figure 4.15 shows why the choice matters. When the data are informative, the posterior occupies a small part of the prior, the ellipse through a prior draw crosses it on a short arc, and the bracket must shrink many times before a proposal is accepted: 14 proposals in the iteration of (a), against one with the Laplace approximation as reference in (b). Worse, the point finally accepted lies close to the current one, so that the chain also moves slowly. With the Laplace approximation, $\mathcal{L}$ is nearly constant, the first proposal is usually accepted, and the new state is nearly uncorrelated with the old one, since $\mathbb{E}[\cos\varphi] = 0$. For two unknowns, the prior reference costs 27 forward solves per effective sample and the Laplace reference 1.1; with 64 unknowns, all informed by the data, the first grows to about 20 000, while the second still needs about four (c). Example 4.18 shows the same contrast for 199 unknowns.

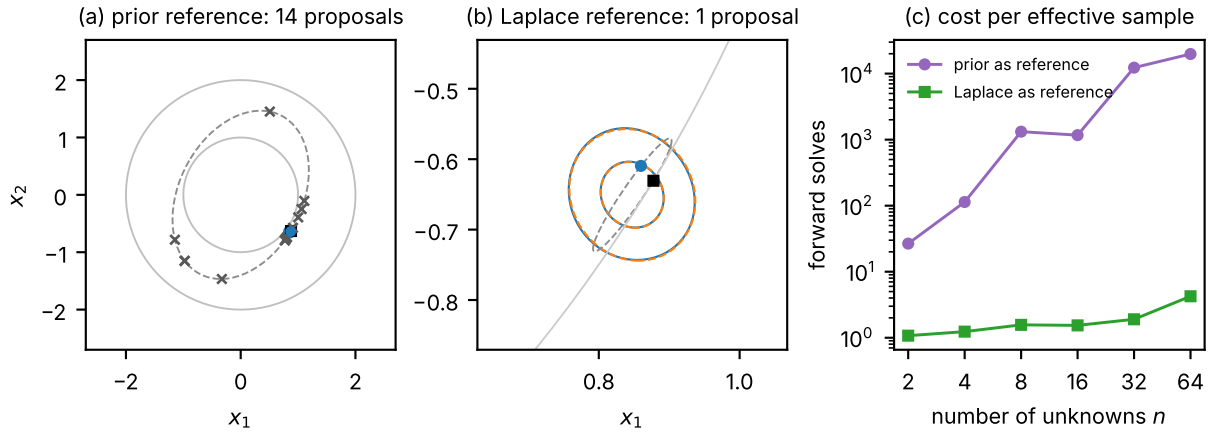


Figure 4.15: Elliptical slice sampling with two references for a posterior that is much narrower than the prior: prior $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$, forward operator $F(\boldsymbol{x})_i = x_i + 0.3\,x_{i+1}^2$ (cyclically), and noise $\sigma = 0.05$. (a) Two unknowns with the prior as reference: prior (gray circles at one and two standard deviations), posterior (blue), the ellipse of one iteration (dashed), and its proposals, rejected (crosses) and accepted (dot), starting next to the MAP estimate (square). (b) One iteration from the same state with the Laplace approximation (orange dashed, nearly on top of the posterior) as reference, in a window of ±0.22 around the MAP estimate; gray: the ellipse of (a). (c) Forward solves per effective sample of the worst component against the number of unknowns; geometric mean over two data sets.

Example 4.18: Sampling the bathymetry posterior

We sample the posterior of the bathymetry problem of Example 2.9 with 20 sensors, noise of 1 cm, and the smoothness prior with $\gamma = 3$ cm per cell on the 199 unknown bottom values, i.e., the nonlinear version of Figure 3.6b, with five samplers (Figure 4.16). Gauss–Newton finds the MAP estimate in five iterations, and its Hessian $\boldsymbol{H} = \boldsymbol{J}^\mathsf{T}\boldsymbol{J}/\sigma^2 + \boldsymbol{L}^\mathsf{T}\boldsymbol{L}/\gamma^2$ at the MAP estimate serves as mass matrix and to shape proposals. *HMC* with $h = 0.3$ and $L = 5$, i.e., $hL = 1.5 \approx \pi/2$, accepts 87 percent of its proposals; the integrated autocorrelation time of the bottom at $x = 9$ m is 1.6, and the smallest effective sample size over all 199 components is 1107 in 2500 iterations. The slowest component thus has $\tau \approx 2.3$, which makes 11 gradients per effective sample, or about 90 forward solves, and

six seconds for the whole run. *Random-walk Metropolis–Hastings* with the Hessian-shaped proposal and the optimal step size has $\tau \approx 1060 = 5.3\,n$ for its slowest component in a run of $2 \cdot 10^5$ iterations (710 at $x = 9\,\mathrm{m}$), i.e., it needs about 1060 forward solves per effective sample, about 12 times as many as HMC. A run of $2 \cdot 10^4$ iterations, as in Figure 4.16c, is too short for this sampler: its smallest effective sample size is 11, and it underestimates τ. *NUTS* from BlackJAX, which knows nothing about the Hessian and adapts during 500 warm-up iterations, takes about 50 leapfrog steps per iteration and needs 93 gradients per effective sample with a diagonal and 66 with a dense mass matrix: six to eight times as many as the hand-tuned HMC, at no effort. *Elliptical slice sampling* with the prior as reference needs 9 likelihood evaluations per iteration, and in a run of $2 \cdot 10^5$ iterations its slowest component has $\tau \approx 3300$ (1540 at $x = 9\,\mathrm{m}$), i.e., about 30 000 forward solves per effective sample. Even this run has a smallest effective sample size of only 55, so these numbers are rough, and probably still too low. The data are much more informative than the prior, whose standard deviation grows to 42 cm at the inflow, so that the ellipses through prior draws must be shrunk to short arcs before a point is accepted. With the Laplace approximation as reference, the same sampler needs 1.65 likelihood evaluations per iteration and has $\tau = 2.4$ at $x = 9\,\mathrm{m}$ and $\tau \approx 3.2$ for its slowest component, i.e., 5 forward solves per effective sample: the cheapest of the five samplers, about 17 times cheaper than HMC. This sampler, HMC, and the shaped random walk all need the Jacobian at the MAP estimate once, about 200 forward solves, which Gauss–Newton has computed anyway. The samples also answer a question about the posterior itself. Its standard deviations, between a minimum of 1.8 cm at a sensor and 6.2 cm at the inflow, agree with those of the Gaussian linearized at the MAP estimate to within six percent, and the posterior mean differs from the MAP estimate by at most 0.55 cm. The posterior is nearly Gaussian, which is why the Laplace approximation is such a good reference, and why it would have given the same band at the cost of one Hessian (Chapter 5); it took samples to find this out. The truth lies inside the 95 % band at 99 percent of the cells.

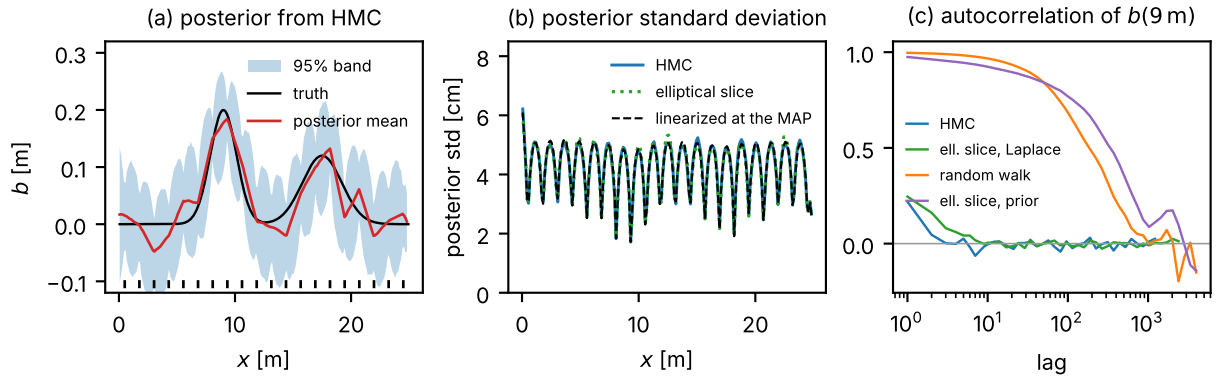


Figure 4.16: Sampling the bathymetry posterior with 20 sensors (ticks) and the smoothness prior (Example 4.18). (a) Posterior mean and 95 % band from 2500 HMC iterations. (b) Posterior standard deviation from HMC, from elliptical slice sampling with the Laplace approximation as reference, and from the Gaussian linearized at the MAP estimate; the minima are at the sensors. (c) Autocorrelation of $b(9\,\mathrm{m})$ for HMC with the Gauss–Newton Hessian as mass matrix ($\tau = 1.6$), elliptical slice sampling with the Laplace approximation (2.4) and with the prior (1226) as reference, and the random walk with the Hessian-shaped proposal (521), from runs of $2 \cdot 10^4$ iterations; the runs of $2 \cdot 10^5$ iterations in the text give 1540 and 710 for the last two.

Example 4.19: Why the posterior mean looks rough, and how to fix it

The posterior mean in Figure 4.16a is rough: it has kinks at the sensors, a dip near $x = 3\,\mathrm{m}$, and wiggles between the two bumps. The samplers are not to blame, since they agree on this posterior; the prior is. Figure 4.17 compares three priors for the same data. (a) The prior of Example 4.18, with $\gamma = 3\,\mathrm{cm}$ per cell, is too weak for 20 sensors: the MAP estimate fits the data to 1.1 cm in the Euclidean norm, a quarter of the noise level $\sqrt{m}\,\sigma = 4.5\,\mathrm{cm}$, so that the posterior mean follows the noise. (b) With $\gamma = 1\,\mathrm{cm}$, the value of the discrepancy principle (misfit 4.3 cm), the mean no longer follows the noise and the band narrows, but the peaks are cut, to 0.16 m instead of 0.20 m, and the mean is still nearly piecewise linear between the sensors: a prior on increments believes in random walks (Section 3.3.1). (c) A prior on second differences, $b_{i+1} - 2b_i + b_{i-1} \sim \mathcal{N}(0, \gamma_2^2)$, believes in smooth curves. Its two-dimensional null space, the linear functions, is fixed by the known outflow bottom and a weak prior $\mathcal{N}(0, (3\,\mathrm{cm})^2)$ on the last increment, and its scale $\gamma_2 = 0.2\,\mathrm{cm}$ is again chosen by the discrepancy principle (misfit 3.9 cm). The posterior mean is smooth, reaches 0.18 m at the first peak, and is as rough as the truth: its root-mean-square second difference is $4.6 \cdot 10^{-4}$, against $6.3 \cdot 10^{-4}$ for the truth and $2.6 \cdot 10^{-3}$ in (a). The root-mean-square error improves less, from 2.1 to 1.9 cm, because it is dominated by a few places where the noise pulls every estimate away from the truth, such as the dip near the inflow, which all three posterior means share; the truth lies inside the band at 97 percent of the cells. More sensors would help more than any prior (Example 2.9).

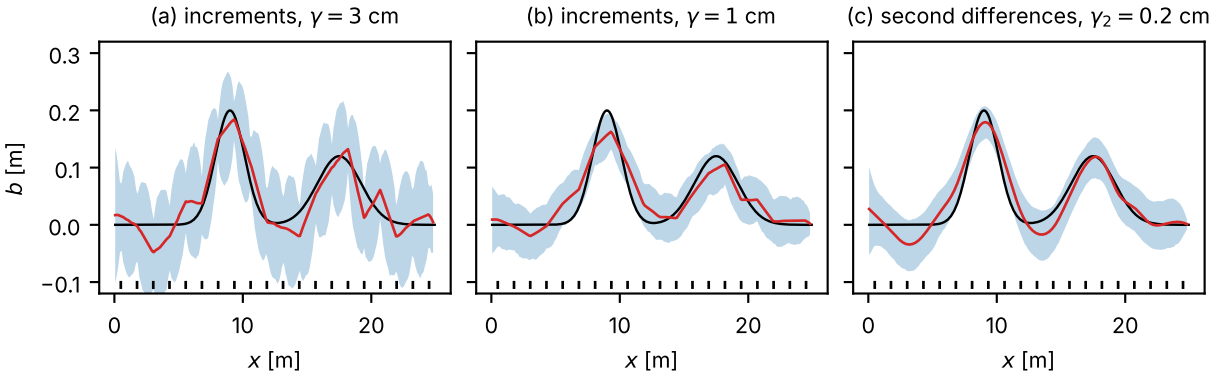


Figure 4.17: Posterior means (red) and 95 % bands for the bathymetry data of Figure 4.16 under three priors, from HMC (Example 4.19); truth in black, sensors as ticks. (a) Increments with $\gamma = 3\,\mathrm{cm}$ per cell, the prior of Figure 4.16. (b) Increments with $\gamma = 1\,\mathrm{cm}$, chosen by the discrepancy principle. (c) Second differences with $\gamma_2 = 0.2\,\mathrm{cm}$, chosen by the discrepancy principle.

4.6 Diagnostics and verification

A sampler never reports its own failure. A chain that is stuck in one valley looks exactly like a chain that has converged to a narrow posterior, and its averages are just as smooth. Three questions must therefore be answered separately: has the chain forgotten its start, how many independent samples is it worth, and does it sample the right posterior? Diagnostics answer the first two, and verification answers the third.

Trace plots and burn-in. The first thing to look at is the trace of a few quantities against the iteration, as in Figure 4.6a,b: the components that matter, the hyper-parameters, and U itself. A chain in equilibrium looks like stationary noise around a fixed level. A trend or a slow drift means that the chain is still on its way or mixes slowly, and the initial transient is the burn-in to be discarded.

Estimating the autocorrelation time. The autocorrelations (4.8) must be estimated from the chain itself, by

$$\widehat{\rho}_f(j) = \frac{\sum_{k=1}^{N-j}(f_k - \bar{f})(f_{k+j} - \bar{f})}{\sum_{k=1}^{N}(f_k - \bar{f})^2} \quad \text{with} \quad \bar{f} = \frac{1}{N}\sum_{k=1}^{N} f_k,$$

and these estimates are noisy at large lags, where few pairs are available; summing all of them gives an estimate of τ_f that is dominated by noise. Geyer's *initial positive sequence* [40] sums the estimated autocorrelations in consecutive pairs, $\rho_f(2j) + \rho_f(2j+1)$, and stops at the first pair whose sum is negative, which the exact pair sums of a reversible chain never are; `samplers.iat` and `samplers.ess` implement it. The effective sample size should be reported for every quantity of interest, and at least its smallest value over all components. A few hundred effective samples suffice for means and standard deviations; 95 % intervals need more, since they are determined by the 2.5 percent of the samples in each tail.

Several chains. A single chain cannot reveal that it has missed a part of the posterior; several chains from dispersed starts can, because they disagree. The *potential scale reduction factor* $\widehat{R}$ of Gelman and Rubin [36] compares the variance between J chains with the variance within them. For chains of length N with means $\bar{f}_j$, overall mean $\bar{f}$, and variances s_j^2, it is

$$\widehat{R} = \sqrt{\frac{\frac{N-1}{N}W + \frac{1}{N}B}{W}} \quad \text{with} \quad W = \frac{1}{J}\sum_{j=1}^{J} s_j^2, \quad B = \frac{N}{J-1}\sum_{j=1}^{J}(\bar{f}_j - \bar{f})^2, \tag{4.19}$$

and $\widehat{R}$ approaches one as the chains converge, while $\widehat{R} > 1$ signals that they have not yet forgotten their starts. The modern version [118] splits every chain into halves, to detect trends within a chain, and computes (4.19) for the ranks of the values instead of the values themselves, which makes it robust against heavy tails; it is implemented in `samplers.rank_rhat`. The current recommendation is to run at least four chains and to require $\widehat{R} < 1.01$ and $N_{\text{eff}} > 400$ for every reported quantity [118].

Example 4.20: Diagnosing the Gibbs sampler of the hierarchical prior

We run four chains of the Gibbs sampler of Section 4.4.1 on the piecewise-constant problem, started from $\theta_i = 5 \cdot 10^{-4}$ (the mode of the hyper-prior), 1, 10^{-6}, and 10^{-2} for all i, with 6000 sweeps each and a burn-in of 1000 (Figure 4.18). On the plateaus, the chains mix well: the integrated autocorrelation time of $\log\theta_i$ is 1.8 sweeps, and the median effective sample size over the components of $\boldsymbol{x}$ is 10^4 of the $2 \cdot 10^4$ retained sweeps. At the jumps, they mix slowly. At the first jump, θ_i switches between a quiet and a loud level, which is the chain deciding whether the jump happens in this cell or in its neighbor; the effective sample size of $\log\theta_i$ over the four chains is 244, the smallest of all components, i.e., its integrated autocorrelation time of $\log\theta_i$ is about $2 \cdot 10^4/244 \approx 80$ sweeps. $\widehat{R}$ is below 1.01 everywhere except at three of the jumps, where it reaches 1.024: the diagnostic finds exactly the places where the funnel of Remark 4.15 acts. The remedy is a longer run or a change of variables, and the diagnosis would have been invisible in a single chain.

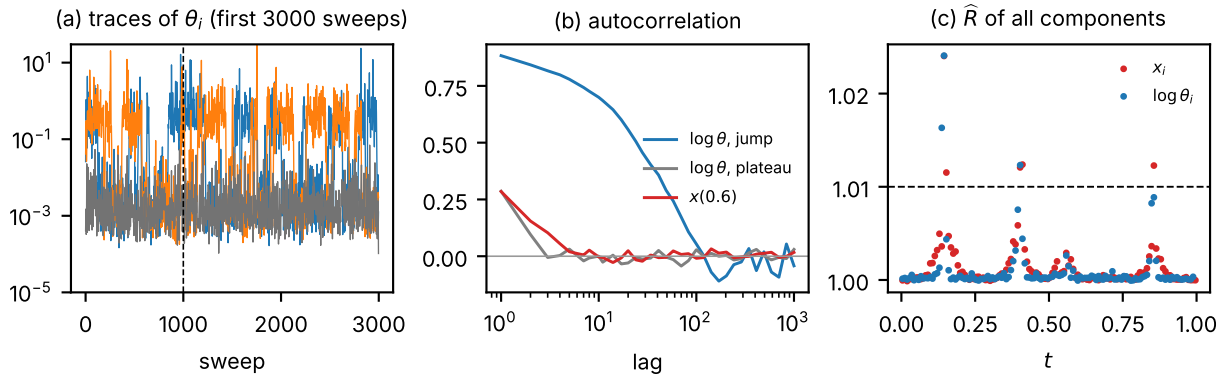


Figure 4.18: Diagnostics of the Gibbs sampler for the hierarchical prior, four chains from dispersed starts (Example 4.20). (a) Traces of θ_i at the first jump for chains 1 (blue) and 2 (orange) and on a plateau for chain 1 (gray), every third sweep; the dashed line ends the burn-in. (b) Autocorrelations of $\log\theta_i$ at the jump and on the plateau, and of $x(0.6)$. (c) Rank-normalized $\widehat{R}$ of all components of $\boldsymbol{x}$ and $\log\boldsymbol{\theta}$, with the threshold 1.01 (dashed).

Verification. Diagnostics check that a chain has converged, not that it has converged to the right distribution. A sign error in the gradient, a missing Jacobian factor after a change of variables, or a wrong conditional in a Gibbs sampler produce perfectly mixing chains with wrong answers. Three tests catch such errors, in increasing order of strength. First, run the sampler on a problem with a known posterior: a Gaussian one, whose mean and covariance are known in closed form (Figures 4.6 and 4.9a), or a low-dimensional one that can be integrated by quadrature, like the bent ridge of Figure 4.4. Second, sample the same posterior with two different samplers and compare their estimates within their Monte Carlo errors, e.g., the Gibbs sampler with elliptical slice sampling. Third, check the *coverage*: draw an unknown from the prior and data from the model, compute the posterior, record whether the 95 % interval contains the drawn unknown, and repeat; if the prior, the likelihood, and the sampler are all correct, the frequency is 95 percent, and the ranks of the drawn unknowns among the posterior samples are uniformly distributed [24, 115]. Coverage for a fixed truth that was not drawn from the prior, such as our test signals, tests the prior along with the sampler. The 87, 95, and 98 percent of Example 4.14, and the missed small step, are results of this kind, and they are statements about the priors.

Remark 4.21: What to report about a sampler

Every project report that contains samples states:

1. the sampler, its tuning parameters, and how they were chosen;
2. the number of chains, their starting points, the number of iterations, and the burn-in;
3. $\widehat{R}$ and N_{eff} for every reported quantity, and their worst values over all components;
4. trace plots of the slowest quantities;
5. the cost: wall-clock time, and forward and gradient evaluations per effective sample;
6. the verification: a test on a known posterior and, where possible, coverage on synthetic data;
7. the results with their Monte Carlo errors: posterior means and MAP estimates, credible bands, a few samples, and the posterior distributions of the quantities of interest.

A result without items 2 to 5 cannot be checked, and a result without item 6 cannot be trusted.

4.7 What sampling costs

The cost of a sampler is the cost of one effective sample: the number of forward-model evaluations, gradients, or linear solves per iteration times the integrated autocorrelation time of the slowest quantity of interest. Table 4.2 collects it for the experiments of this chapter.

Table 4.2: What one effective sample costs, from the experiments of this chapter.

sampler	requires	one iteration	τ	one effective sample
perturb-then-optimize	linear F, Gaussian prior and noise	one least-squares solve	1	one solve
Gibbs sampling	linear F, conditionally Gaussian prior	one Cholesky factorization	2 on plateaus, about 80 at jumps	about 80 factorizations
random walk, shaped	values of U, a covariance	one forward solve	$3.6\,n$ to $5.3\,n$	$3.6\,n$ to $5.3\,n$ forward solves
HMC, Hessian as mass matrix	gradients, a Hessian	5 gradients	2.3	11 gradients, about 90 forward solves
NUTS (BlackJAX)	gradients	about 50 gradients	1.3 to 1.9	66 to 93 gradients
elliptical slice, prior as reference	a Gaussian prior	9 forward solves	about 3300 (rough)	about 30 000 forward solves (rough)
elliptical slice, Laplace approximation as reference	a Gaussian approximation of the posterior	1.7 forward solves	3.2	5 forward solves

Four lessons can be read off the table. First, structure beats generality: the exact samplers of the first two rows produce an effective sample for the price of one or a few linear solves, and whenever the posterior is Gaussian or conditionally Gaussian, they are the methods of choice. Second, gradients pay: on the bathymetry problem, HMC needs about 12 times fewer forward-solve equivalents per effective sample than the random walk, although a gradient costs about eight forward solves, and the advantage grows with the dimension, since the cost per effective sample grows like n for the random walk and only like $n^{1/4}$ for HMC [8]; for HMC, the growth comes from the number of leapfrog steps per trajectory, not from τ. The same arithmetic applies to the flume of default project C, where a forward solve costs 60 ms in JAX, and the cost of a gradient in forward solves has to be measured for that model. Third, knowledge beats adaptation, but not by much: the Hessian at the MAP estimate, a by-product of Gauss–Newton, made our HMC six to eight times cheaper than NUTS, which learns the geometry during its warm-up. Fourth, a sampler that uses only the prior pays for every bit of information that the data add: elliptical slice sampling with the prior as reference was the most expensive method on the bathymetry problem, and with the Laplace approximation as reference the cheapest. This works because that posterior is nearly Gaussian; under the hierarchical prior it is not, no single Gaussian is a good reference, and the prior normalization of Remark 4.15 is the starting point. Before a long run, multiply the cost of one forward solve by the number of forward solves per effective sample and by the number of effective samples needed, a few hundred for means and standard deviations and a few thousand for the tails. When the product is too large even for the best sampler, the methods of Chapter 5 replace the posterior by an approximation that is cheaper to compute, and the samplers of this chapter are what such approximations are checked against.

Summary : Chapter 4 in five sentences

Uncertainty quantification means computing posterior expectations, probabilities, and distributions of quantities of interest, and samples from the posterior compute all of them by averaging, with an error $\sigma_f/\sqrt{N_{\text{eff}}}$ that does not depend on the dimension. Markov chain Monte Carlo produces correlated samples, and the integrated autocorrelation time τ_f, the number of steps per independent sample, is the most important number about a sampler: about $3n$ for a well-shaped random walk and of order one for HMC with a good mass matrix. The structure of the posterior decides the method: Gaussian posteriors are sampled exactly by perturb-then-optimize, conditionally Gaussian ones by Gibbs sampling, which is the iterative alternating scheme with dice, and everything else by Hamiltonian Monte Carlo with gradients from automatic differentiation, or by elliptical slice sampling without them, which works best with a Gaussian reference close to the posterior. Samples reveal what MAP estimates hide: posterior means are not better reconstructions, only the hierarchical prior gives posterior samples with flat plateaus and sharp jumps, and credible intervals inherit the biases of the prior, as for the height of the smooth bump and the small step. A sampler never reports its own failure, and every result needs several chains, $\widehat{R}$, effective sample sizes, and a verification against a known answer.

4.8 Notes and further reading

Robert and Casella [101] is the standard reference for Monte Carlo methods and the Markov chain theory behind them, Neal [87] the classical account of HMC, and Betancourt [10] explains why HMC works in terms of the geometry of high-dimensional distributions. Kaipio and Somersalo [64] and Calvetti and Somersalo [19] treat sampling for inverse problems, including Gibbs samplers for hierarchical models; Gelman et al. [37] and Vehtari et al. [118] the diagnostics. The optimal scaling results for the random walk, MALA, and HMC are in [8, 102, 104], and the samplers for functions, which remain efficient under mesh refinement, in [9, 25, 27]; hIPPYlib-MUQ [67] implements them for PDE-constrained problems. Sampling Gaussians by optimization is treated in [92, 94], with iterative solvers in [41, 97], and its nonlinear extension in [4]; elliptical slice sampling with a general Gaussian reference is in [88]. Transport maps, which push a reference distribution to the posterior and can be used either instead of MCMC or to precondition it, are surveyed in [80]; the hierarchical prior normalization of default projects A and C is in [44].

4.9 Exercises

Exercise 4.1: Metropolis–Hastings and Gibbs

(a) Show that the Metropolis–Hastings chain of Algorithm 4.6 satisfies detailed balance (4.7). (b) Show that a Gibbs step, which replaces $\boldsymbol{x}$ by a draw from $p(\boldsymbol{x} \mid \boldsymbol{\theta}, \boldsymbol{y})$, is a Metropolis–Hastings step with acceptance probability one. (c) Derive (4.10) from $\operatorname{Var}[\frac{1}{N}\sum_k f_k] = \sigma_f^2 N^{-2} \sum_{k,l} \rho_f(|k-l|)$, and compute τ_f for the autoregressive chain $f_{k+1} = a f_k + \sqrt{1-a^2}\,\xi_k$ with $|a| < 1$ and independent $\xi_k \sim \mathcal{N}(0,1)$. How many steps of the chain with $a = 0.9$ are worth one independent sample?

Exercise 4.2: Scale mixtures and conditionals

(a) Verify the scale mixture (4.14) with the integral $\int_0^\infty \theta^{-1/2} e^{-a/\theta - b\theta}\,\mathrm{d}\theta = \sqrt{\pi/b}\, e^{-2\sqrt{ab}}$ for $a, b > 0$. (b) Derive the conditional (4.13), and show that the mode of (4.13) is the $\boldsymbol{\theta}$-step of Algorithm 3.14. (c) Derive the inverse Gaussian conditional of $1/\theta_i$ for the Laplace prior; the inverse Gaussian density with mean μ and shape λ is proportional to $w^{-3/2} \exp(-\lambda(w-\mu)^2/(2\mu^2 w))$.

Exercise 4.3: Random walks in high dimensions (computer)

Use `samplers.rw_metropolis` and `samplers.iat`. (a) Sample $\mathcal{N}(\mathbf{0}, \boldsymbol{I}_n)$ for $n \in \{2, 8, 32, 128\}$ with step sizes $s = c/\sqrt{n}$, $c \in [0.5, 5]$; plot the acceptance rate and τ of x_1 against c, and confirm the optimum $c \approx 2.38$, the acceptance rate of about 0.23, and $\tau \approx 3n$. (b) Repeat (a) for $n \in \{2, 8, 32\}$ and the Gaussian with the standard deviations $\sigma_j = 10^{-2j/(n-1)}$, $j = 0, \dots, n-1$, i.e., between 1 and 0.01, once with shaped steps, $\boldsymbol{C} = \mathrm{diag}(\sigma_1, \dots, \sigma_n)$ and $s = c/\sqrt{n}$, and once with isotropic steps, $\boldsymbol{C} = \boldsymbol{I}$ and $s = c/(\sum_j \sigma_j^{-2})^{1/2}$, which reduces to $c/\sqrt{n}$ for equal standard deviations. Compare the acceptance rates and τ of the widest and of the narrowest component with (a), and explain the difference; the isotropic chains need about 10^6 iterations. (c) Reproduce Example 4.8, and estimate how many iterations the isotropic random walk would need for 400 effective samples.

Exercise 4.4: Perturb-then-optimize and Gibbs sampling (computer)

Use the functions `perturb_optimize`, `gibbs_laplace`, and `gibbs_hierarchical` of `samplers.py`. (a) Extend Theorem 4.10 to a prior with nonzero mean. Reproduce Figure 4.9 and compute the posterior distribution of the position of the maximum of the first bump. (b) Reproduce Figure 4.11 and the numbers of Example 4.14. (c) Repeat (b) for the hierarchical prior with $\vartheta \in \{10^{-4}, 10^{-2}\}$: does the posterior find the small step, and what happens to the coverage and to the samples? (d) Run four chains and compute $\widehat{R}$ and the effective sample sizes as in Example 4.20; how long must the chains be for $N_{\mathrm{eff}} > 400$ in every component?

Exercise 4.5: Hamiltonian Monte Carlo (computer)

(a) Verify that (4.18) solves (4.16) for a Gaussian posterior with $\boldsymbol{M} = \boldsymbol{H}$, and show that $\boldsymbol{x}(\pi/2)$ is distributed according to the posterior and independent of $\boldsymbol{x}(0)$. (b) Implement the leapfrog scheme (4.17), and check numerically that it is reversible and that the energy error over a trajectory of fixed length decreases like h^2. (c) Run `samplers.hmc` without the random variation of the step size on a two-dimensional Gaussian with $\boldsymbol{M} = \boldsymbol{H}$ and $hL \in \{\pi/2, \pi, 2\pi\}$, and explain the autocorrelations you observe. (d) Reproduce Example 4.18 with `samplers.make_hmc_jax` and `samplers.elliptical_slice`, and compare HMC, NUTS from BlackJAX (Appendix B), and elliptical slice sampling with both references in effective samples per forward solve and per second. (e) Repeat (d) for the prior on second differences of Example 4.19.

Exercise 4.6: Your own inverse problem, sampled

For the problem of Exercises 1.5, 2.6 and 3.6: which of the three cases of Section 4.2 does your posterior belong to, and which sampler would you use? What does one forward evaluation cost, and one gradient? How many effective samples does your quantity of

interest need, and what does that cost with the numbers of Table 4.2? How will you verify the sampler? Write half a page; this is the third paragraph of your project proposal.

Chapter 5

Beyond MCMC: Approximate Bayesian Inference

In Chapter 4, the hiker surveyed the whole range by walking it, and her survey was exact up to a Monte Carlo error that she could estimate and reduce. It was also expensive: thousands of steps, each a forward solve or a gradient (Table 4.2). This chapter is about three colleagues who have been asked for a report by tomorrow (Figure 5.1). The first walks down to the deepest point, as in Chapter 2, measures how the ground curves there, and reports the valley as a bowl of that curvature: the *Laplace approximation*. The second inflates a balloon in the valley: it takes the shape of the valley as well as a balloon can, fills it from within, and never pushes through a wall: *variational inference*. The third sends out a search party whose members cannot feel the slope; they radio their altitudes, fit a straight line through the reports, and move as a team: *ensemble Kalman inversion*. Each report is a probability density q that stands in for the posterior, and each is cheap because it gives something up: the bowl and the balloon are Gaussian, and the search party is a few hundred points that see the landscape only through straight lines. None of them sees the second valley in Figure 5.1, and none of them says how wrong it is. The question of this chapter is therefore not only how to compute such reports, but when to trust them.

A word on names. *Approximate Bayesian computation* (ABC) is a different subject: it is for models whose likelihood cannot be evaluated, and it keeps those draws from the prior whose simulated data come close to the observed ones [109]. In our problems the likelihood can always be evaluated, and the approximations of this chapter use it. Section 5.1 says what we approximate and why, Sections 5.2 to 5.4 treat the three methods, Section 5.5 compares them on the test problems of Chapter 4, and Section 5.6 connects them to the projects and to what lies beyond this course.

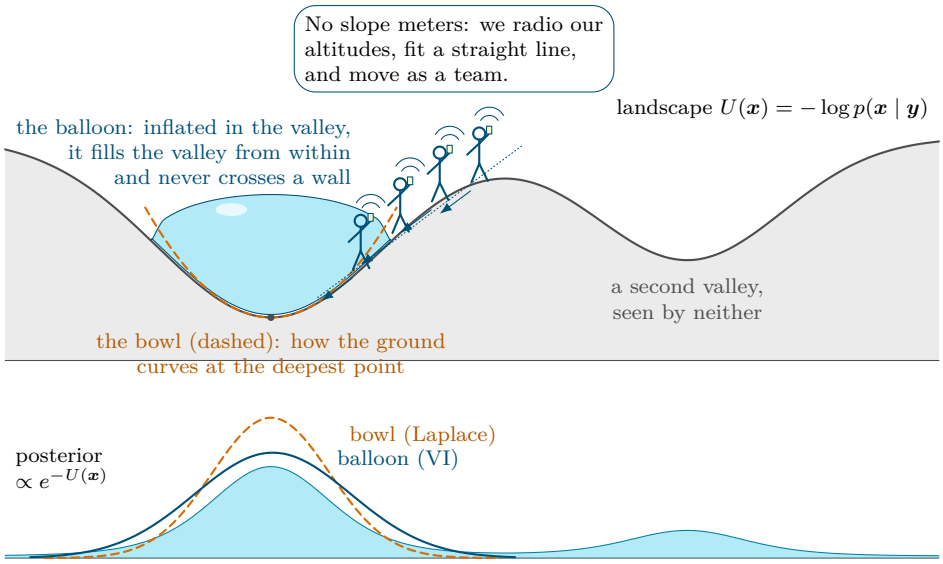


Figure 5.1: The landscape of Figure 4.1 and three reports that need no survey. The bowl (dashed) has the curvature of the landscape at its deepest point: the Laplace approximation of Section 5.2. The balloon is the Gaussian that fits best into the valley: variational inference, Section 5.3. The search party measures only altitudes and moves along the straight line that fits them, as the ensemble Kalman methods of Section 5.4 do. Bottom: the posterior and the two Gaussians, computed for this landscape. The bowl is narrower than the valley (standard deviation 0.82 against 1.11), the balloon has the width of the valley (1.09), and both miss the second valley, which holds a quarter of the probability.

5.1 Why approximate?

The cost of a sampler is the number of forward solves per effective sample times the number of effective samples (Section 4.7). For the bathymetry problem, HMC needs about 10 gradients, i.e., about 80 forward solves, per effective sample, and 400 effective samples cost about $3 \cdot 10^4$ forward solves, less than a minute in JAX. For a three-dimensional flow model with ten minutes per forward solve, the same numbers mean seven months. An *approximate posterior* is a probability density q that is cheaper to compute, to sample, and to integrate than the posterior, and close to it in a sense that each method makes precise. Table 5.1 lists the three methods of this chapter with what they need and what they deliver.

Table 5.1: Three approximate posteriors at a glance.

method	needs	delivers	typical failure
Laplace approximation	a MAP estimate and the Hessian there	a Gaussian at the MAP estimate	non-Gaussian posteriors: curved, skewed, several peaks, kinks
variational inference	gradients of U, a family of densities	the best member of the family	too narrow, one peak only, depends on the parametrization
ensemble Kalman methods	forward solves only, many in parallel	an ensemble of points	straight-line updates: approximate for nonlinear F, limited by the ensemble size

An approximation reports its answer with the same confidence whether it is right or wrong,

and a cheap answer is only useful if we know how far to trust it. Every approximation must therefore be checked, in one of two ways: against a sampler, on the problem itself or on a smaller version of it, as in Section 4.6; or by the importance weights of Section 5.2.2, which check the approximation where it puts its draws, without sampling the posterior. The samplers of Chapter 4 remain the reference, and the test problems of that chapter, whose posteriors we know, are where we check.

5.2 The Laplace approximation

Definition 5.1: The Laplace approximation

Let the potential U of (4.1) be twice continuously differentiable with a minimizer $\boldsymbol{x}_{\mathrm{MAP}}$ at which the Hessian $\boldsymbol{H} = \nabla^2 U(\boldsymbol{x}_{\mathrm{MAP}})$ is positive definite. The Laplace approximation of the posterior is the Gaussian

$$q_{\mathrm{L}}(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}; \boldsymbol{x}_{\mathrm{MAP}}, \boldsymbol{H}^{-1}). \tag{5.1}$$

The derivation takes one line. Since $\nabla U(\boldsymbol{x}_{\mathrm{MAP}}) = \mathbf{0}$, the second-order Taylor polynomial of the potential at the MAP estimate is $U(\boldsymbol{x}) \approx U(\boldsymbol{x}_{\mathrm{MAP}}) + \frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}_{\mathrm{MAP}})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x}-\boldsymbol{x}_{\mathrm{MAP}})$, and exponentiating gives $p(\boldsymbol{x} \mid \boldsymbol{y}) \propto e^{-U(\boldsymbol{x})} \approx e^{-U(\boldsymbol{x}_{\mathrm{MAP}})} \exp\!\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}_{\mathrm{MAP}})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x}-\boldsymbol{x}_{\mathrm{MAP}})\right) \propto q_{\mathrm{L}}(\boldsymbol{x})$. The same approximation inside the integral of the evidence gives $p(\boldsymbol{y}) \approx e^{-U(\boldsymbol{x}_{\mathrm{MAP}})}(2\pi)^{n/2}(\det \boldsymbol{H})^{-1/2}$, if U includes the normalizing constants of the likelihood and the prior. The idea is as old as Bayesian inference: in the memoir of 1774 in which he stated Bayes' theorem, Laplace expanded the logarithm of the posterior of a success probability to second order around its maximum, which gave a normal density, to show that the posterior concentrates at the observed frequency [73, 113]. The Laplace approximation is exact when U is quadratic, i.e., for the linear-Gaussian posterior of Theorem 3.10, and in general it is as good as the quadratic Taylor polynomial is over the region where the posterior has its mass.

The Gauss–Newton Hessian. For Gaussian noise and a Gaussian prior, the potential and its Hessian are

$$\begin{aligned} U(\boldsymbol{x}) &= \tfrac{1}{2}\left\|\boldsymbol{\Gamma}^{-1/2}(F(\boldsymbol{x}) - \boldsymbol{y})\right\|_2^2 + \tfrac{1}{2}\left\|\boldsymbol{\Gamma}_{\mathrm{prior}}^{-1/2}(\boldsymbol{x} - \boldsymbol{\mu})\right\|_2^2, \\ \nabla^2 U(\boldsymbol{x}) &= \underbrace{\boldsymbol{J}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{J} + \boldsymbol{\Gamma}_{\mathrm{prior}}^{-1}}_{\text{Gauss–Newton}} + \sum_{i=1}^{m} \left[\boldsymbol{\Gamma}^{-1}(F(\boldsymbol{x}) - \boldsymbol{y})\right]_i \nabla^2 F_i(\boldsymbol{x}), \end{aligned} \tag{5.2}$$

with the Jacobian $\boldsymbol{J}$ of F at $\boldsymbol{x}$. In practice one drops the last term, which vanishes for a linear forward operator and is small when the residuals are small or F is nearly linear, and uses the Gauss–Newton matrix, which is positive definite everywhere and is computed by Algorithm 2.8 anyway. The Laplace approximation then costs a MAP estimate and one more Jacobian: no sampling, no Monte Carlo error, no tuning.

Example 5.2: A positive unknown

Take the conductivity $\kappa > 0$ of Remark 3.16, whose logarithm $u = \log \kappa$ has the posterior $\mathcal{N}(0, 0.6^2)$ (Figure 5.2). The potential of κ is $U(\kappa) = \log \kappa + (\log \kappa)^2/(2 \cdot 0.6^2)$ up to a constant; its minimizer is $\kappa_{\mathrm{MAP}} = e^{-0.36} = 0.70$, and its curvature there is $U''(\kappa_{\mathrm{MAP}}) = (0.6\,\kappa_{\mathrm{MAP}})^{-2}$. The Laplace approximation in κ is therefore $\mathcal{N}(0.70, 0.42^2)$ (a, c). It is centered at the mode, not at the posterior mean 1.20; it is symmetric, where the posterior

is skewed; and it puts 4.8 percent of its probability on negative conductivities. In the variable u, the potential is a parabola, and the Laplace approximation is exact (b); mapped back to κ, it is the posterior (c).

The example generalizes. The Laplace approximation, like the MAP estimate, depends on the parametrization, and it is good in the parametrization in which the posterior is closest to Gaussian: positive quantities such as conductivities, rate constants, and variances should be approximated in their logarithms.

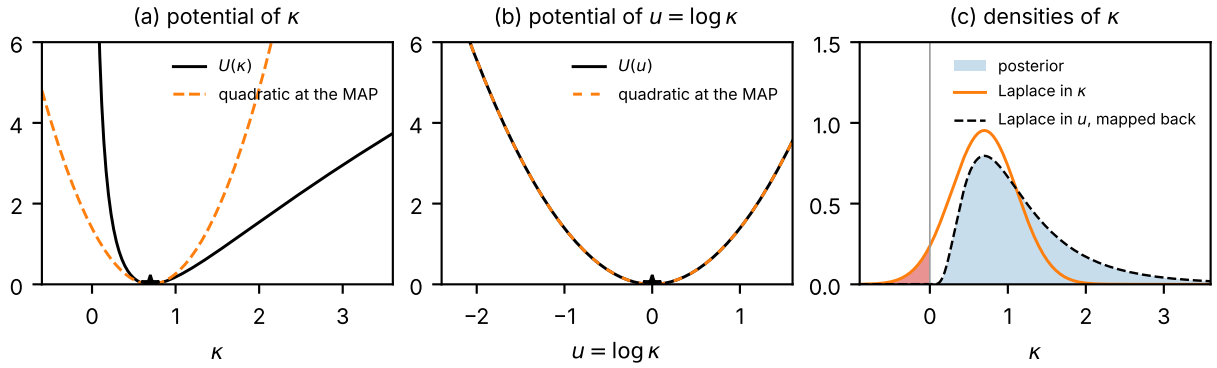


Figure 5.2: The Laplace approximation of a log-normal posterior (Example 5.2). (a) The potential of κ and its second-order Taylor polynomial at the MAP estimate (star). (b) The potential of $u = \log \kappa$ is a parabola, and its Taylor polynomial is exact. (c) The posterior density of κ, the Laplace approximation in κ, which puts 4.8 percent of its probability on $\kappa < 0$ (red), and the Laplace approximation in u, mapped back to κ, which is the posterior.

Remark 5.3: How accurate is the Laplace approximation?

The approximation becomes exact when the data concentrate the posterior around a single point. Schillings, Sprungk, and Wacker [108] make this precise: if, as the noise level σ tends to zero, the potential keeps a unique minimizer with a positive definite Hessian, and smoothness and growth conditions hold, then the Hellinger distance between the posterior and its Laplace approximation is of the order σ; if the data leave directions undetermined, convergence needs more structure, e.g., a Gaussian prior in those directions. Helin and Kretschmann [58] give bounds for a fixed noise level that can be evaluated. In inverse problems, the noise is rarely small enough for these results to be quantitative, and the posterior is rarely concentrated in all directions; the practical test is the comparison with a sampler or the importance weights of Section 5.2.2.

Example 5.4: The Laplace approximation of the bathymetry posterior

We use the bathymetry problem with 20 sensors and the prior on increments with $\gamma = 1\,\mathrm{cm}$ per cell of case (b) in Example 4.19, which the discrepancy principle selects: the MAP estimate fits the data to 4.3 cm, at the noise level 4.5 cm. Gauss–Newton converges in four iterations, which cost four Jacobians and 13 forward solves; with one more Jacobian for the Hessian, and a Jacobian costing about 200 forward solves, one per unknown, also in JAX (Section 4.7), the Laplace approximation costs about 1000 forward solves. This is the price of about a dozen effective samples of HMC, which needs 9.8 gradients, about 80 forward solves, per effective sample on this posterior. The 95 % bands of the

Laplace approximation and of an HMC reference (5000 iterations, mass matrix $\boldsymbol{H}$) are indistinguishable (Figure 5.3a). The posterior standard deviation is between 0.9 and 3.1 cm, against a prior standard deviation that grows from 1 cm at the outflow, where the bottom is known, to 14 cm at the inflow (b). The two approximations of it agree within five percent in every component (c), which is the accuracy of the reference itself: its smallest effective sample size, 2551, means a relative error of 1.4 percent per standard deviation and up to three times that over 199 components. The HMC mean differs from the MAP estimate by at most 0.11 posterior standard deviations: the posterior is nearly Gaussian, as in Example 4.18, and the Laplace approximation is all one needs, for the price of about a dozen effective samples.

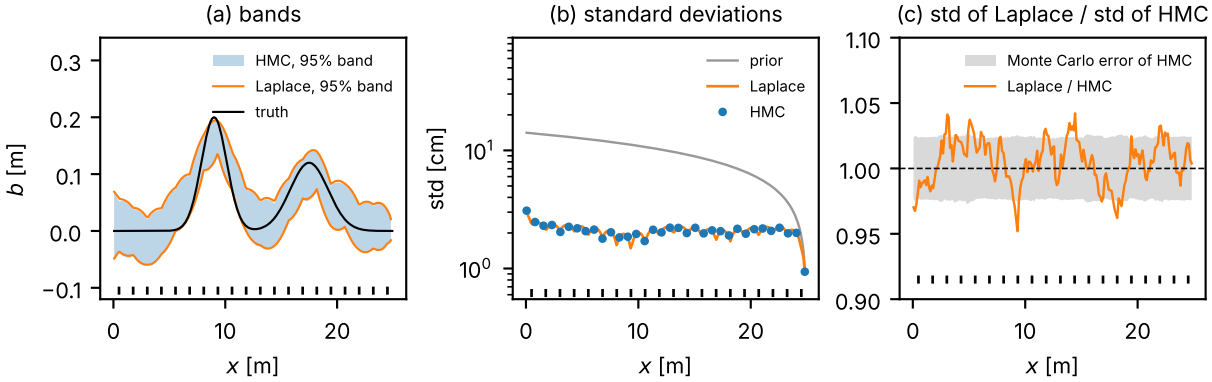


Figure 5.3: The Laplace approximation of the bathymetry posterior with 20 sensors (Example 5.4). (a) The 95 % bands of HMC and of the Laplace approximation; truth in black, sensors as ticks. (b) Posterior standard deviations of the Laplace approximation and of HMC, and the prior standard deviation. (c) The ratio of the two posterior standard deviations and the 95 % Monte Carlo error band of the HMC reference, $1 \pm 1.96/\sqrt{2\,\mathrm{ESS}}$ per component.

Large problems. For $n = 10^6$ unknowns, the Hessian can be neither stored nor inverted, but the data inform only a few directions. With a factor $\boldsymbol{R}\boldsymbol{R}^\mathsf{T} = \boldsymbol{\Gamma}_\text{prior}$ of the prior covariance, the covariance $\boldsymbol{H}^{-1}$ of the Laplace approximation is the prior covariance minus a correction of rank at most m, which is computed from the leading eigenpairs of the *prior-preconditioned data-misfit Hessian* $\boldsymbol{R}^\mathsf{T}\boldsymbol{J}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{J}\boldsymbol{R}$ (Exercise 5.1c). For the bathymetry problem, only 8 of its eigenvalues exceed one, i.e., the 20 sensors determine 8 of the 199 directions better than the prior does; 200 sensors determine 24. The leading eigenpairs are found by Lanczos or randomized methods from products with $\boldsymbol{J}$ and $\boldsymbol{J}^\mathsf{T}$, one linearized forward and one adjoint solve each [54], and the truncated correction is optimal in a precise sense [110]. The cost of the Laplace approximation then scales with the number of informed directions rather than with n, which makes it the method of choice for PDE-constrained problems with millions of unknowns [15, 34].

5.2.1 When the Laplace approximation fails

The Laplace approximation sees the posterior through a keyhole: the value, the slope, and the curvature of U at one point. Figure 5.4 shows three posteriors for which the view through the keyhole is misleading, all from Chapters 3 and 4.

A curved posterior. At the MAP estimate $(0.92, 0)$ of the bent ridge of Example 4.3, the exact Hessian is $\operatorname{diag}(12.1, 0.083)$: the ridge is steep across and almost flat along x_2, because the MAP estimate lies at the bend, where the ridge turns, and the residual term of (5.2), -0.92, nearly cancels the prior curvature 1 in x_2. The Laplace approximation with this Hessian has the standard deviations 0.29 and 3.48, while the posterior has 0.56 and 0.98. Its width across the ridge is right, since U is quadratic in x_1 with the curvature 12.1 everywhere, but a Gaussian cannot follow the bend: it is centered at the MAP estimate instead of the posterior mean $(0.48, 0)$, half as wide in x_1 as the posterior, whose marginal in x_1 is widened by the bend, and 3.6 times too wide in x_2, where it reaches far beyond the arms. The Gauss–Newton Hessian, $\operatorname{diag}(12.1, 1)$, drops the residual term and gives the standard deviation 1.00 in x_2, close to the posterior's 0.98 because the data say little about x_2; it does not follow the bend either. The curvature at one point says nothing about where the arms go.

Several peaks. For the posterior of $y = x^2 + \varepsilon$ in Remark 3.16, the Laplace approximation describes the peak at the MAP estimate and is blind to the other peak, which holds 32 percent of the probability. MAP estimates from several starts find well-separated peaks, and a mixture of their Laplace approximations, weighted by their approximate evidences, is a remedy when the peaks are few and narrow.

A kink. For scalar denoising with the Laplace prior of Remark 3.16 and the datum $y = 0.8$, the MAP estimate is zero, inside the dead zone of the soft threshold, where $U(x) = (x - 0.8)^2/2 + |x|$ has a kink and no Hessian. The Gaussian with the curvature on either side of the kink, $\mathcal{N}(0, 1)$, puts half of its probability below zero, where the posterior has 30 percent, and it is centered at zero, where the posterior mean is 0.39. The same happens in n dimensions for the ℓ^1 and the total-variation priors, whose MAP estimates have exact zeros where U is not differentiable: their Laplace approximations are undefined, and smoothing the absolute value makes them defined but not accurate.

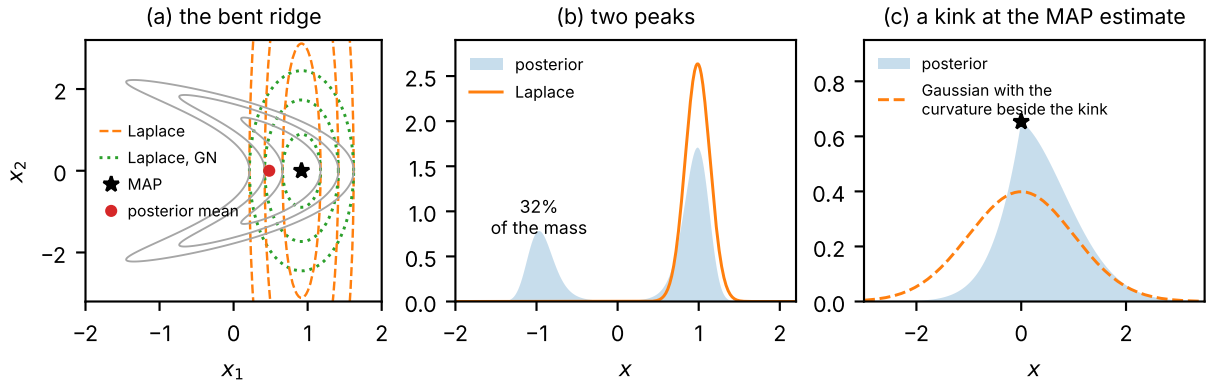


Figure 5.4: Three posteriors that the Laplace approximation (orange) misrepresents. (a) The bent ridge of Example 4.3 (gray contours) and the contours of its Laplace approximations with the exact Hessian (orange, dashed) and the Gauss–Newton Hessian (green, dotted) at the same heights: both are centered at the MAP estimate (star) instead of the posterior mean (dot), neither follows the bend, and the exact Hessian makes the Gaussian far too wide in x_2. (b) The posterior of $y = x^2 + \varepsilon$ of Remark 3.16: the Laplace approximation misses the second peak. (c) Denoising with a Laplace prior, $y = 0.8$: the MAP estimate is at the kink of U, where no Hessian exists; the Gaussian with the curvature on either side of the kink is a poor substitute.

Two further failures matter for the projects. The joint MAP estimate of the hierarchical prior (3.10), computed by the IAS of Algorithm 3.14, lies in the neck of the funnel of Remark 4.15,

where the density is highest and most sharply curved: the Laplace approximation there, in $\boldsymbol{\theta}$ or in $\log \boldsymbol{\theta}$, is much too narrow, and as a Gaussian it cannot represent the funnel, in which the spread of an increment depends on its variance. This is one reason why default project A compares variational approximations of the hierarchical posterior with samplers. And posteriors with an exact symmetry, such as the AC-CRNN of default project B, whose reactions can be numbered in any order, have as many equivalent peaks as there are permutations; a Laplace approximation can describe only one of them, which is correct once the symmetry has been broken, e.g., by ordering the rate constants.

5.2.2 Using the Laplace approximation

A Laplace approximation can be reported as the result, as in Example 5.4, but it is also useful when it is not good enough. It is the natural reference of elliptical slice sampling, which with it needed five forward solves per effective sample on the bathymetry problem, and the natural mass matrix of HMC (Example 4.18 and Figure 4.15). And it is a proposal for *importance sampling*: if $\boldsymbol{x}_1, \dots, \boldsymbol{x}_S$ are independent draws from any density q, then

$$\mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}] \approx \frac{\sum_{k=1}^{S} w_k f(\boldsymbol{x}_k)}{\sum_{k=1}^{S} w_k}, \quad w_k = \frac{e^{-U(\boldsymbol{x}_k)}}{q(\boldsymbol{x}_k)}, \tag{5.3}$$

converges to the posterior expectation as $S \to \infty$ for every f with a finite posterior expectation, whatever q is, as long as $q > 0$ wherever the posterior is positive. The unknown normalizing constants cancel in the ratio, and the effective sample size $(\sum_k w_k)^2 / \sum_k w_k^2$ measures how many draws the weighted sample is worth. The catch is the speed of convergence: if q is too narrow somewhere, rare draws get huge weights, the weights can have infinite variance, and the estimate converges too slowly to be useful. The *Pareto* $\hat{k}$ *diagnostic* turns this into a number [119]: it fits a generalized Pareto distribution to the largest weights, whose shape parameter $\hat{k}$ measures how heavy their tail is. For $\hat{k} < 0.5$ the weights have a finite variance and (5.3) converges at the usual rate; for $0.5 < \hat{k} < 0.7$ their variance is infinite, but a version of (5.3) that replaces the largest weights by quantiles of the fitted distribution, Pareto-smoothed importance sampling, is still reliable; for $\hat{k} > 0.7$ neither is, for any practical S (`approximations.pareto_khat`). The diagnostic therefore also checks q itself, with one blind spot: it sees only where the draws of q land, and a part of the posterior that they never reach goes unnoticed. For the bathymetry posterior, 4000 draws from the Laplace approximation give $\hat{k}$ between 0.22 and 0.45 in three repetitions and an effective sample size of about 3500: the approximation passes the check, and the reweighted standard deviations agree with HMC within about five percent. For the bent ridge, $\hat{k}$ is between 0.71 and 0.91 in five repetitions with the exact Hessian and between 0.52 and 0.75 with the Gauss–Newton Hessian, and the reweighted posterior mean of x_1 lies between 0.59 and 0.71 in all ten, above the true 0.48: draws from a Gaussian that does not follow the bend reach the arms too rarely for any practical S.

5.3 Variational inference

The Laplace approximation is a Gaussian chosen by a local criterion. Variational inference chooses a density from a family by a global one: the member of the family that is closest to the posterior in the Kullback–Leibler divergence.

Definition 5.5: Variational inference

Let $\mathcal{Q}$ be a family of probability densities on $\mathbb{R}^n$. The *Kullback–Leibler divergence* of $q \in \mathcal{Q}$ from the posterior is

$$\mathrm{KL}(q \,\|\, p(\cdot \mid \boldsymbol{y})) = \int q(\boldsymbol{x}) \log \frac{q(\boldsymbol{x})}{p(\boldsymbol{x} \mid \boldsymbol{y})} \,\mathrm{d}\boldsymbol{x}, \tag{5.4}$$

and variational inference computes $q^* \in \arg\min_{q \in \mathcal{Q}} \mathrm{KL}(q \,\|\, p(\cdot \mid \boldsymbol{y}))$. Equivalently, q^* maximizes the *evidence lower bound*

$$\mathrm{ELBO}(q) = \mathbb{E}_q[\log p(\boldsymbol{y} \mid \boldsymbol{x}) + \log p(\boldsymbol{x})] - \mathbb{E}_q[\log q(\boldsymbol{x})] = -\mathbb{E}_q[U(\boldsymbol{x})] + \mathcal{H}[q], \tag{5.5}$$

where $\mathcal{H}[q] = -\mathbb{E}_q[\log q(\boldsymbol{x})]$ is the entropy of q and U includes the normalizing constants of the likelihood and the prior.

The two problems are equivalent because of the identity

$$\log p(\boldsymbol{y}) = \mathrm{ELBO}(q) + \mathrm{KL}(q \,\|\, p(\cdot \mid \boldsymbol{y})), \tag{5.6}$$

which follows by inserting $\log p(\boldsymbol{x} \mid \boldsymbol{y}) = \log p(\boldsymbol{y} \mid \boldsymbol{x}) + \log p(\boldsymbol{x}) - \log p(\boldsymbol{y})$ into (5.4). Since the divergence is nonnegative and zero only for $q = p(\cdot \mid \boldsymbol{y})$, the ELBO is a lower bound of the logarithm of the evidence, whence its name, and maximizing it minimizes the divergence without the evidence, which is the one number we cannot compute. The two terms of (5.5) are the two forces on the balloon of Figure 5.1: $-\mathbb{E}_q[U]$ rewards q for staying where the potential is low, in the valleys, and the entropy rewards it for spreading out. The walls push the balloon in, the gas pushes it out.

The direction of the divergence. The divergence is not symmetric, and its direction matters. $\mathrm{KL}(q \,\|\, p)$ averages $\log(q/p)$ over q: it grows without bound when q puts probability where the posterior has almost none, but when q misses a region where the posterior has much, it charges only the logarithm of the probability that q does cover. Its minimizers are therefore *mode-seeking*. On the two-peak posterior of Remark 3.16, the best Gaussian in this sense sits on one peak (Figure 5.5a): on the one at the MAP estimate, with divergence 0.42, of which $-\log 0.68 = 0.39$ is the price of missing the other peak, or, if the optimization starts on the other side, on the other one, a local minimum with divergence 1.17. The best Gaussian in the reverse direction, $\mathrm{KL}(p \,\|\, q)$, has the mean and the variance of the posterior (Exercise 5.2): it straddles both peaks and has its maximum in the valley between them, where the posterior has almost no probability. Within one peak, the asymmetry has the same effect. For a Gaussian posterior with correlation 0.9 (Figure 5.5b), the best mean-field approximation in the direction $\mathrm{KL}(q \,\|\, p)$ has the standard deviation $\sqrt{1 - 0.9^2} = 0.44$ in both coordinates, where the marginals of the posterior have 1. Variational inference uses $\mathrm{KL}(q \,\|\, p)$ because its average is over q, which we can sample, and not over the posterior, which we cannot; expectation propagation minimizes local versions of the other direction [82].

5.3.1 Gaussian families

The two standard families are the *mean-field* Gaussians $\mathcal{N}(\boldsymbol{m}, \mathrm{diag}(s_1^2, \dots, s_n^2))$, with $2n$ parameters, which treat the components as independent, and the *full-rank* Gaussians $\mathcal{N}(\boldsymbol{m}, \boldsymbol{C}\boldsymbol{C}^\mathsf{T})$ with a lower triangular $\boldsymbol{C}$ with positive diagonal, with $n + n(n+1)/2$ parameters; low-rank-plus-diagonal covariances lie in between. For Gaussian families, the optimality conditions of variational inference can be written down, and they explain what the method does.

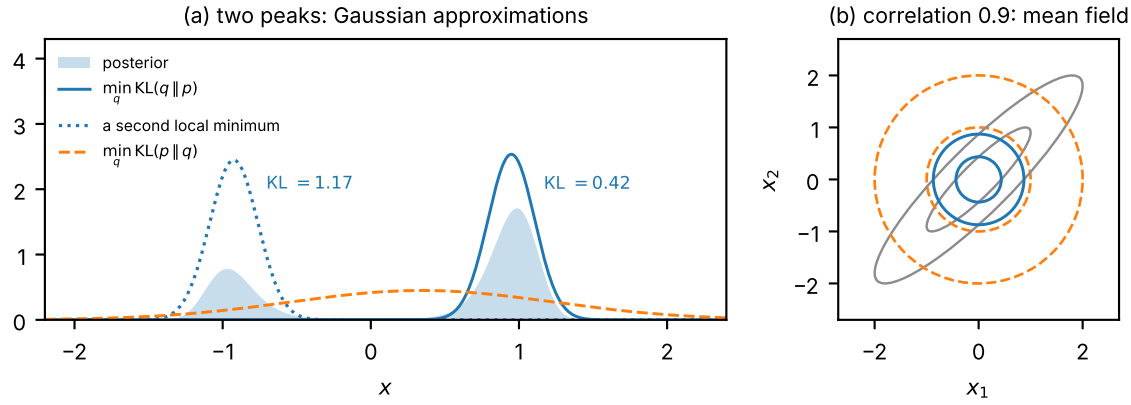


Figure 5.5: The two directions of the Kullback–Leibler divergence. (a) The two-peak posterior of Remark 3.16 and three Gaussians: the minimizer of $\mathrm{KL}(q \,\|\, p)$ (solid), a second local minimizer (dotted), and the minimizer of $\mathrm{KL}(p \,\|\, q)$, which has the posterior mean and variance (dashed). (b) A Gaussian posterior with correlation 0.9 (gray, contours at one and two standard deviations) and its best mean-field approximations in $\mathrm{KL}(q \,\|\, p)$ (blue), which has the conditional standard deviations, and in $\mathrm{KL}(p \,\|\, q)$ (orange, dashed), which has the marginal ones.

Theorem 5.6: Gaussian variational inference

Let U be twice continuously differentiable, with U, ∇U, and $\nabla^2 U$ integrable against every Gaussian density.

(a) If $q = \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$ maximizes the ELBO over all Gaussians, or is any other stationary point of it, then

$$\mathbb{E}_q[\nabla U(\boldsymbol{x})] = \mathbf{0} \quad \text{and} \quad \boldsymbol{\Sigma}^{-1} = \mathbb{E}_q[\nabla^2 U(\boldsymbol{x})]. \tag{5.7}$$

(b) If the posterior is Gaussian, $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{H}^{-1})$, then the best Gaussian is the posterior itself, and the best mean-field Gaussian is $\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag}(1/H_{11}, \dots, 1/H_{nn}))$, with

$$\frac{1}{H_{ii}} \le (\boldsymbol{H}^{-1})_{ii}, \quad i = 1, \dots, n, \tag{5.8}$$

and equality if and only if $H_{ij} = 0$ for all $j \neq i$. Its divergence from the posterior is $\frac{1}{2}(\sum_i \log H_{ii} - \log\det \boldsymbol{H})$.

Proof. (a) By (5.5), $\mathrm{ELBO}(\boldsymbol{m}, \boldsymbol{\Sigma}) = -\mathbb{E}_q[U] + \frac{1}{2}\log\det\boldsymbol{\Sigma} + \frac{n}{2}(1 + \log 2\pi)$. The derivative of $\mathbb{E}_q[U]$ with respect to $\boldsymbol{m}$ is $\mathbb{E}_q[\nabla U]$. The Gaussian density φ with covariance $\boldsymbol{\Sigma}$ satisfies $\frac{\mathrm{d}}{\mathrm{d}t}\varphi(\boldsymbol{x}; \boldsymbol{\Sigma} + t\boldsymbol{\Delta})|_{t=0} = \frac{1}{2}\operatorname{tr}(\boldsymbol{\Delta}\nabla^2_{\boldsymbol{x}}\varphi(\boldsymbol{x}; \boldsymbol{\Sigma}))$ for every symmetric $\boldsymbol{\Delta}$, as one checks by differentiating (3.2), and two integrations by parts give the derivative $\frac{1}{2}\operatorname{tr}(\boldsymbol{\Delta}\,\mathbb{E}_q[\nabla^2 U])$ of $\mathbb{E}_q[U]$ in the direction $\boldsymbol{\Delta}$; the derivative of $\frac{1}{2}\log\det\boldsymbol{\Sigma}$ is $\frac{1}{2}\operatorname{tr}(\boldsymbol{\Delta}\boldsymbol{\Sigma}^{-1})$. Setting the derivatives of the ELBO to zero gives (5.7) [91]. (b) For $U(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x} - \boldsymbol{\mu}) + \text{const}$, we have $\nabla U(\boldsymbol{x}) = \boldsymbol{H}(\boldsymbol{x} - \boldsymbol{\mu})$ and $\nabla^2 U = \boldsymbol{H}$, so that (a) gives $\boldsymbol{m} = \boldsymbol{\mu}$ and $\boldsymbol{\Sigma} = \boldsymbol{H}^{-1}$, the posterior. For the mean-field family, $\mathbb{E}_q[U] = \frac{1}{2}(\boldsymbol{m} - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{m} - \boldsymbol{\mu}) + \frac{1}{2}\sum_i H_{ii}s_i^2 + \text{const}$, and the ELBO, $-\mathbb{E}_q[U] + \sum_i \log s_i + \text{const}$, is maximal for $\boldsymbol{m} = \boldsymbol{\mu}$ and $s_i^2 = 1/H_{ii}$. By the Cauchy–Schwarz inequality, $1 = \left((\boldsymbol{H}^{1/2}\boldsymbol{e}_i)^{\mathsf{T}}(\boldsymbol{H}^{-1/2}\boldsymbol{e}_i)\right)^2 \le (\boldsymbol{e}_i^{\mathsf{T}}\boldsymbol{H}\boldsymbol{e}_i)(\boldsymbol{e}_i^{\mathsf{T}}\boldsymbol{H}^{-1}\boldsymbol{e}_i) = H_{ii}(\boldsymbol{H}^{-1})_{ii}$, with equality if and only if $\boldsymbol{H}^{1/2}\boldsymbol{e}_i$ and $\boldsymbol{H}^{-1/2}\boldsymbol{e}_i$ are parallel, i.e., if $\boldsymbol{e}_i$ is an eigenvector of $\boldsymbol{H}$. The divergence of $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{D}^{-1})$, $\boldsymbol{D} = \operatorname{diag}(H_{11}, \dots, H_{nn})$, from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{H}^{-1})$ is $\frac{1}{2}(\operatorname{tr}(\boldsymbol{H}\boldsymbol{D}^{-1}) - n + \log\det\boldsymbol{D} - \log\det\boldsymbol{H})$, and $\operatorname{tr}(\boldsymbol{H}\boldsymbol{D}^{-1}) = n$. □

Part (a) says that Gaussian variational inference is a Laplace approximation averaged over q.

The Laplace approximation sets the gradient of U to zero and inverts its curvature at one point; variational inference sets the average gradient to zero and inverts the average curvature over the region that q itself occupies. The balloon feels the walls where it touches them, the bowl only at its bottom. Part (b) says that a mean-field approximation reports $1/H_{ii}$, which is the variance of x_i given all other components: the uncertainty that x_i would have if all other components were known. Whenever the components are correlated, as neighboring values of a discretized function nearly always are, mean-field variational inference is overconfident, and the divergence of the best mean-field Gaussian says by how much; by Hadamard's inequality $\det \boldsymbol{H} \le \prod_i H_{ii}$, it vanishes only for a diagonal $\boldsymbol{H}$.

5.3.2 The reparametrization trick

For a nonlinear forward operator, $\mathbb{E}_q[U]$ is an n-dimensional integral without a closed form, and the ELBO must be maximized with noisy estimates. The *reparametrization trick* [69, 71] writes a draw from $q = \mathcal{N}(\boldsymbol{m}, \boldsymbol{C}\boldsymbol{C}^\mathsf{T})$ as $\boldsymbol{x} = \boldsymbol{m} + \boldsymbol{C}\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, so that

$$\mathrm{ELBO}(\boldsymbol{m}, \boldsymbol{C}) = -\mathbb{E}_{\boldsymbol{\epsilon}}\big[U(\boldsymbol{m} + \boldsymbol{C}\boldsymbol{\epsilon})\big] + \sum_{i=1}^{n} \log C_{ii} + \frac{n}{2}(1 + \log 2\pi) \tag{5.9}$$

is an expectation over a distribution that does not depend on the parameters. The gradient can therefore be taken inside the expectation,

$$\begin{aligned} \nabla_{\boldsymbol{m}} \,\mathrm{ELBO} &= -\mathbb{E}_{\boldsymbol{\epsilon}}\big[\nabla U(\boldsymbol{m} + \boldsymbol{C}\boldsymbol{\epsilon})\big], \\ \nabla_{\boldsymbol{C}} \,\mathrm{ELBO} &= -\mathbb{E}_{\boldsymbol{\epsilon}}\big[\nabla U(\boldsymbol{m} + \boldsymbol{C}\boldsymbol{\epsilon})\, \boldsymbol{\epsilon}^\mathsf{T}\big] + \mathrm{diag}(1/C_{11}, \dots, 1/C_{nn}), \end{aligned}$$

restricted to the lower triangle, and estimated without bias from a few draws of $\boldsymbol{\epsilon}$.

Algorithm 5.7: Gaussian variational inference with the reparametrization trick

Given a differentiable potential U, starting values $\boldsymbol{m}$ and $\boldsymbol{C}$, e.g., the Laplace approximation, a number of draws S, and step sizes α_k, repeat for $k = 1, 2, \dots$:

1. Draw $\boldsymbol{\epsilon}_1, \dots, \boldsymbol{\epsilon}_S \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ and set $\boldsymbol{x}_s = \boldsymbol{m} + \boldsymbol{C}\boldsymbol{\epsilon}_s$.
2. Estimate the gradients of $-\,\mathrm{ELBO}$: $\boldsymbol{g} = \frac{1}{S}\sum_s \nabla U(\boldsymbol{x}_s)$ and $\boldsymbol{G} = \frac{1}{S}\sum_s \nabla U(\boldsymbol{x}_s)\boldsymbol{\epsilon}_s^\mathsf{T} - \mathrm{diag}(1/C_{11}, \dots, 1/C_{nn})$, restricted to the lower triangle, or to the diagonal for the mean-field family.
3. Update $\boldsymbol{m} \leftarrow \boldsymbol{m} - \alpha_k \boldsymbol{g}$ and $\boldsymbol{C} \leftarrow \boldsymbol{C} - \alpha_k \boldsymbol{G}$.

Each iteration costs S gradients of U. In practice, the update uses Adam [68] on $\boldsymbol{m}$, the logarithms of the diagonal entries of $\boldsymbol{C}$, and its other entries, with step sizes that decrease with k, which removes the noise at the end.

In JAX, the core of `approximations.gaussian_vi` takes ten lines:

```
def neg_elbo(p, eps):                              # eps: S draws from N(0, I)
    C = jnp.diag(jnp.exp(p["logd"])).at[tril].set(p["off"])
    x = p["m"] + eps @ C.T                         # reparametrization
    return jnp.mean(jax.vmap(U)(x)) - jnp.sum(p["logd"])
value_and_grad = jax.jit(jax.value_and_grad(neg_elbo))
for k in range(n_iter):
    key, sub = jax.random.split(key)
    val, g = value_and_grad(p, jax.random.normal(sub, (S, n)))
    p = adam_step(p, g, k)                         # descent on -ELBO
```

Example 5.8: Gaussian variational inference on the bent ridge

The best Gaussian for the bent ridge of Example 4.3, computed with the ELBO evaluated by quadrature, has the mean $(0.78, 0)$, the standard deviations 0.29 and 0.54, and the divergence 0.55 from the posterior (Figure 5.6a). It sits in the bend, and like the balloon it fills the ridge from within: both of its standard deviations are smaller than those of the posterior, 0.56 and 0.98, and it does not reach the arms (c). The Laplace approximation with the exact Hessian, by comparison, reaches far beyond the ridge along x_2; its ELBO is -610, i.e., its divergence from the posterior is 610: in the sense of variational inference it is a very bad approximation, because it puts probability where the posterior has none. With the Gauss–Newton Hessian, whose Gaussian stays within the ridge (Figure 5.4a), the divergence is 3.9, still seven times that of the best Gaussian. Algorithm 5.7 with $S = 16$ draws, started at $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$, gets within 0.01 of the best ELBO after 560 iterations (b), and after 6000 iterations, with Adam and step sizes from 10^{-2} down to 10^{-3}, it returns the mean $(0.78, -0.004)$ and the standard deviations 0.29 and 0.53. The noise of the ELBO estimates is larger than the remaining error, which is why the step sizes must decrease. Importance sampling with 4000 draws from this Gaussian does not repair it, and it shows the blind spot of $\hat{k}$: in five repetitions, $\hat{k}$ is between 0.58 and 0.85, but the reweighted mean of x_1 is between 0.66 and 0.70 in all of them, also in those with $\hat{k} < 0.7$, against the true 0.48, because the draws hardly ever reach the arms.

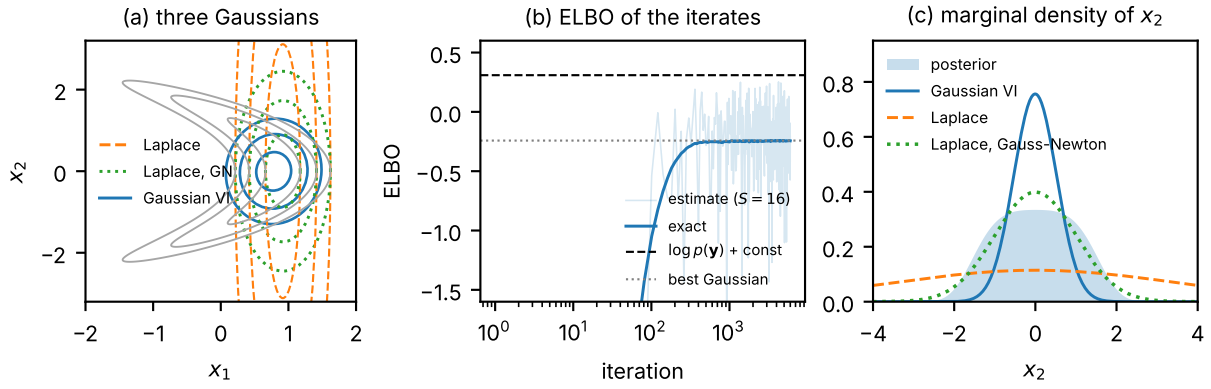


Figure 5.6: Gaussian variational inference on the bent ridge (Example 5.8). (a) Contours of the posterior (gray), of the Laplace approximations with the exact Hessian (orange, dashed) and the Gauss–Newton Hessian (green, dotted), and of the Gaussian found by Algorithm 5.7 (blue), all at the same heights relative to their maxima. (b) The ELBO of the iterates, exact (by quadrature) and estimated from 16 draws, against the logarithm of the evidence and the ELBO of the best Gaussian; their gap is the divergence 0.55 that no Gaussian can close. (c) The marginal density of x_2 and its three Gaussian approximations.

Example 5.9: Mean-field and full-rank variational inference for deconvolution

The posterior of the smooth deconvolution signal of Example 3.12 is Gaussian, so that Theorem 5.6 gives the answer in closed form, and the computation checks the theorem and the algorithm (Figure 5.7). Neighboring values are strongly correlated, with correlation 0.72, and the best mean-field Gaussian has standard deviations $1/\sqrt{H_{ii}}$ between 0.40 and 0.55 times the posterior ones (a); its 95 % band contains the truth at 76 percent of the points, the posterior band at 93 percent (b). Algorithm 5.7 reproduces both answers: with $S = 8$ draws per iteration, the mean-field standard deviations are within two percent of

$1/\sqrt{H_{ii}}$ after 4000 iterations, and the full-rank ones within 1.5 percent of the posterior ones after 10^4 iterations, i.e., after $8{\cdot}10^4$ gradients. The divergence (c) shows the difference between the families: the full-rank family, with 8384 parameters instead of 256, needs more iterations, but its divergence tends to zero (0.43 after 10^4 iterations), while the mean-field divergence stalls at 29.5, the divergence of the best mean-field Gaussian.

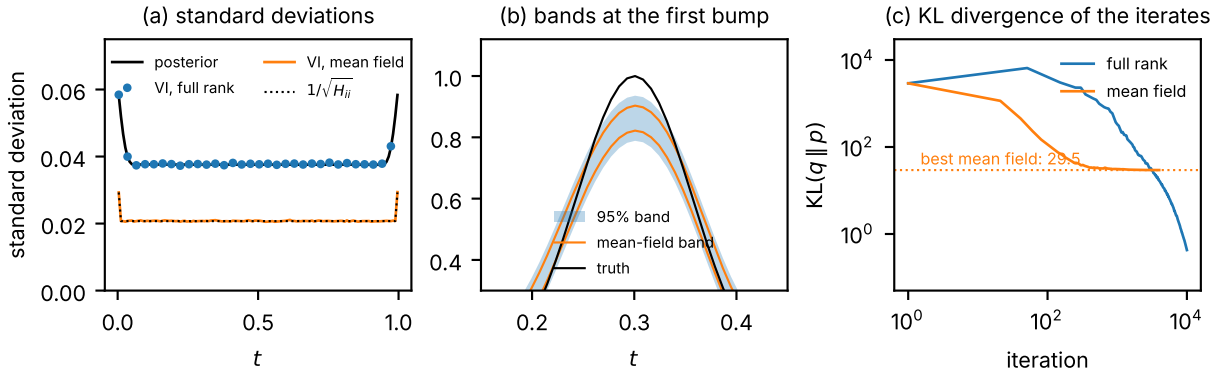


Figure 5.7: Variational inference for the Gaussian posterior of the smooth deconvolution signal (Example 5.9). (a) Posterior standard deviations, those of full-rank and of mean-field variational inference by Algorithm 5.7, and $1/\sqrt{H_{ii}}$ of Theorem 5.6. (b) The 95 % bands of the posterior and of the mean-field approximation at the first bump, with the truth. (c) The divergence $\mathrm{KL}(q \,\|\, p)$ of the iterates of both families, computed exactly, and that of the best mean-field Gaussian (dotted).

5.3.3 Variational Bayes for the hierarchical prior

For conditionally Gaussian models such as the hierarchical prior (3.10), there is a classical alternative to stochastic gradients. Let the family factorize between the unknown and the local variances, $q(\boldsymbol{x}, \boldsymbol{\theta}) = q(\boldsymbol{x})\, q(\boldsymbol{\theta})$, but leave both factors free. With one factor fixed, the ELBO is maximized over the other by $q(\boldsymbol{x}) \propto \exp(\mathbb{E}_{q(\boldsymbol{\theta})}[\log p(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{\theta})])$, and likewise for $q(\boldsymbol{\theta})$, and alternating the two updates is *coordinate ascent variational inference*, or *variational Bayes* [12, 13]. For the hierarchical prior on increments $\boldsymbol{z} = \boldsymbol{L}\boldsymbol{x}$, both updates are explicit.

Algorithm 5.10: Variational Bayes for the hierarchical prior

Start from $\mathbb{E}[1/\theta_i] = (\beta + 1)/\vartheta$, the inverse of the mode of the hyper-prior, and repeat until the mean stops changing:

1. $\boldsymbol{x}$-step: $q(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$ with

$$\boldsymbol{\Sigma} = (\boldsymbol{A}^\mathsf{T}\boldsymbol{A}/\sigma^2 + \boldsymbol{L}^\mathsf{T}\boldsymbol{D}\boldsymbol{L})^{-1}, \quad \boldsymbol{m} = \boldsymbol{\Sigma}\boldsymbol{A}^\mathsf{T}\boldsymbol{y}/\sigma^2, \quad \boldsymbol{D} = \operatorname{diag}(\mathbb{E}[1/\theta_1], \dots, \mathbb{E}[1/\theta_{n-1}]),$$

   with the prior on $\ker \boldsymbol{L}$ of Section 3.3.2 added to $\boldsymbol{L}^\mathsf{T}\boldsymbol{D}\boldsymbol{L}$.
2. $\boldsymbol{\theta}$-step: $q(\theta_i) = \mathrm{InvGamma}(\beta + \tfrac{1}{2}, \vartheta + \tfrac{1}{2}\mathbb{E}[z_i^2])$ with $\mathbb{E}[z_i^2] = (\boldsymbol{L}\boldsymbol{m})_i^2 + (\boldsymbol{L}\boldsymbol{\Sigma}\boldsymbol{L}^\mathsf{T})_{ii}$, so that

$$\mathbb{E}[1/\theta_i] = \frac{\beta + 1/2}{\vartheta + \mathbb{E}[z_i^2]/2}.$$

Compare with the IAS of Algorithm 3.14. The $\boldsymbol{x}$-step is the same weighted Tikhonov problem, with the precision $1/\theta_i$ replaced by its average. The $\boldsymbol{\theta}$-step differs twice: the average of $1/\theta_i$

replaces the mode of θ_i, which turns $\beta + \frac{3}{2}$ into $\beta + \frac{1}{2}$, and the average of z_i^2 replaces z_i^2, which adds the posterior variance of the increment. Variational Bayes is the IAS with uncertainty: an increment that the current $q(\boldsymbol{x})$ is unsure about gets a larger variance in the next round. The price is the variances of the increments, the diagonal of $\boldsymbol{L}\boldsymbol{\Sigma}\boldsymbol{L}^\mathsf{T}$, in every iteration, for large problems by Lanczos or by sampling. Agrawal et al. [1] develop and analyze the analogous scheme for gamma hyper-priors.

Example 5.11: Variational Bayes on the piecewise-constant signal

On the deconvolution data of Example 4.14 with $\beta = 1$ and $\vartheta = 10^{-3}$, Algorithm 5.10 converges in 38 iterations and a few hundredths of a second, against 14 seconds for the $2 \cdot 10^4$ Gibbs sweeps of Example 4.14 (Figure 5.8). Its mean lies within 0.02 of the IAS solution and has the error 0.61, smaller than that of the posterior mean from Gibbs sampling, 0.99: it is a reconstruction with sharp jumps, while the posterior mean averages over the positions of the jumps and rounds them off (c); the two differ by up to 1.5 posterior standard deviations. Its band is the band of a Gaussian that knows where the jumps are: it is 0.16 to 0.24 wide at the five jumps, where the Gibbs band is 0.26 to 1.1 wide (a). Variational Bayes is certain that each of the four large jumps happens in exactly one cell, with probability 1.00, where the Gibbs sampler spreads the probability over neighboring cells, between 0.27 and 0.54 in the most likely one (b). Its standard deviations are between 0.17 and 0.73 times those of the Gibbs sampler, smallest at the jumps, and the coverage drops from 98 to 93 percent. This is the failure of Theorem 5.6b in another guise: a factorized family ignores the coupling between its factors and is overconfident; by separating $\boldsymbol{x}$ from $\boldsymbol{\theta}$, variational Bayes ignores that the position of a jump and the local variances are coupled, which is the funnel of Remark 4.15, and it reports the uncertainty of the increments as if their variances were known. Default project A asks whether this overconfidence is the same for other hyper-priors, such as the generalized gamma and the horseshoe hyper-priors, and whether it survives the linearization that a nonlinear forward model requires.

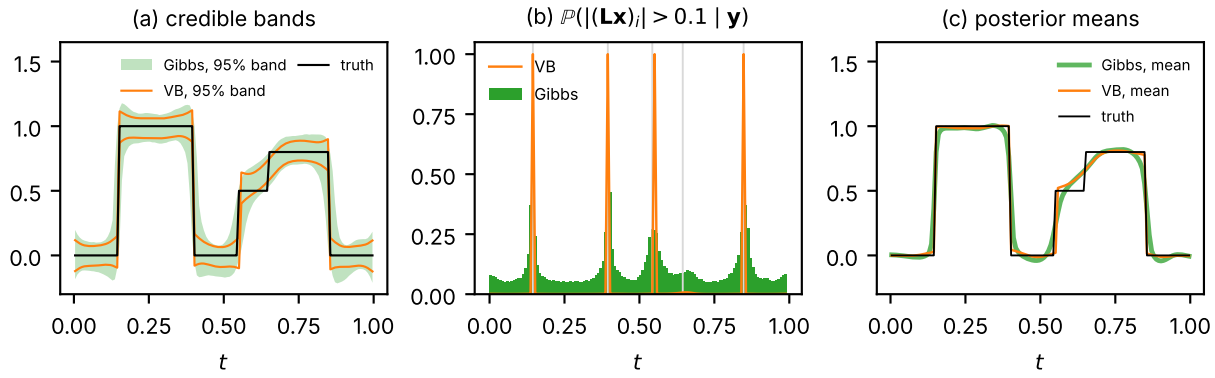


Figure 5.8: Variational Bayes (Algorithm 5.10) and Gibbs sampling for the hierarchical prior on the piecewise-constant deconvolution data (Example 5.11). (a) The 95 % bands, with the truth. (b) The posterior probability of an increment larger than 0.1; gray lines: true jump positions. (c) The posterior means of variational Bayes and of the Gibbs sampler, with the truth.

Checking a variational approximation. The ELBO says how good q is compared with other members of the family, not compared with the posterior, since $\log p(\boldsymbol{y})$ in (5.6) is unknown. Two checks remain: the comparison with a sampler on a problem that can be sampled, as in

Example 5.11, and the importance weights of draws from q with the Pareto $\hat{k}$ of Section 5.2.2 [121]. A value $\hat{k} > 0.7$ says that q misses a part of the posterior; a value $\hat{k} < 0.7$ says that q passes the check, which cannot see the parts of the posterior that the draws of q do not reach (Example 5.8). Variational inference depends on the starting point, through local optima as in Figure 5.5a, and on the parametrization, as the Laplace approximation does; several starts and the logarithms of positive quantities are cheap insurance.

Beyond Gaussians. A richer family can close the gap that a Gaussian leaves. *Normalizing flows* push a standard normal through an invertible neural network whose Jacobian determinant is cheap, and the reparametrization trick works unchanged [93, 100]; *transport maps* do the same with triangular polynomial maps [80]. The hierarchical prior normalization of Remark 4.15 is an explicit transport map, and a Gaussian family in its coordinates is a non-Gaussian family in the original ones.

5.4 Ensemble Kalman inversion

The search party of Figure 5.1 has no slope meters. When the forward model is a black box without adjoints or automatic differentiation, a Jacobian costs $n + 1$ forward solves by finite differences, and for large n gradients are out of reach. Ensemble Kalman methods use forward solves only, and the forward solves of one update are independent of each other, so that a whole ensemble is evaluated in parallel in the time of one solve. They come from data assimilation in weather forecasting, where the ensemble Kalman filter has been in use since the 1990s [33]; we use them for inverse problems [61].

Algorithm 5.12: The ensemble Kalman update

Given an ensemble $\boldsymbol{x}^{(1)}, \dots, \boldsymbol{x}^{(J)}$ and the forward values $\boldsymbol{f}^{(j)} = F(\boldsymbol{x}^{(j)})$:

1. Compute the ensemble means $\bar{\boldsymbol{x}}$ and $\bar{\boldsymbol{f}}$ and the covariances

$$\boldsymbol{C}^{xf} = \frac{1}{J-1}\sum_{j=1}^{J}(\boldsymbol{x}^{(j)} - \bar{\boldsymbol{x}})(\boldsymbol{f}^{(j)} - \bar{\boldsymbol{f}})^{\mathsf{T}}, \quad \boldsymbol{C}^{ff} = \frac{1}{J-1}\sum_{j=1}^{J}(\boldsymbol{f}^{(j)} - \bar{\boldsymbol{f}})(\boldsymbol{f}^{(j)} - \bar{\boldsymbol{f}})^{\mathsf{T}}.$$

2. Draw $\boldsymbol{\eta}^{(j)} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Gamma})$ independently and move every member,

$$\boldsymbol{x}^{(j)} \leftarrow \boldsymbol{x}^{(j)} + \boldsymbol{K}(\boldsymbol{y} + \boldsymbol{\eta}^{(j)} - \boldsymbol{f}^{(j)}), \quad \boldsymbol{K} = \boldsymbol{C}^{xf}(\boldsymbol{C}^{ff} + \boldsymbol{\Gamma})^{-1}. \tag{5.10}$$

One update costs J forward solves, which can run in parallel, and one solve with an $m \times m$ matrix; no derivatives are needed.

Theorem 5.13: The ensemble Kalman update is perturb-then-optimize

Let $F = \boldsymbol{A}$ be linear, let the prior be $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma}_{\text{prior}})$ and the noise $\mathcal{N}(\boldsymbol{0}, \boldsymbol{\Gamma})$, and replace the ensemble covariances by their limits $\boldsymbol{C}^{xf} = \boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^{\mathsf{T}}$ and $\boldsymbol{C}^{ff} = \boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^{\mathsf{T}}$. If $\boldsymbol{x}^{(j)} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma}_{\text{prior}})$ and $\boldsymbol{\eta}^{(j)} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Gamma})$ are independent, then the updated member is the minimizer of

$$\frac{1}{2}\left\|\boldsymbol{\Gamma}^{-1/2}(\boldsymbol{A}\boldsymbol{x} - (\boldsymbol{y} + \boldsymbol{\eta}^{(j)}))\right\|_2^2 + \frac{1}{2}\left\|\boldsymbol{\Gamma}_{\text{prior}}^{-1/2}(\boldsymbol{x} - \boldsymbol{x}^{(j)})\right\|_2^2, \tag{5.11}$$

and it is an exact sample from the posterior.

Proof. The normal equations of (5.11) are $(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1})\boldsymbol{x} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}(\boldsymbol{y} + \boldsymbol{\eta}^{(j)}) + \boldsymbol{\Gamma}_{\text{prior}}^{-1}\boldsymbol{x}^{(j)}$,

i.e.,

$$\boldsymbol{x} = \boldsymbol{x}^{(j)} + (\boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1}(\boldsymbol{y} + \boldsymbol{\eta}^{(j)} - \boldsymbol{A}\boldsymbol{x}^{(j)}),$$
$$(\boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1})^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1} = \boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^\mathsf{T} + \boldsymbol{\Gamma})^{-1},$$

where the second identity is checked by multiplying both of its sides by $\boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}$ from the left and by $\boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^\mathsf{T} + \boldsymbol{\Gamma}$ from the right. Its right-hand side is the gain $\boldsymbol{K}$, so that the minimizer is the update (5.10). (5.11) is the perturbed Tikhonov problem (4.12) of Theorem 4.10, with the data perturbed by $\boldsymbol{\eta}^{(j)}$ and the prior mean perturbed by $\boldsymbol{x}^{(j)} - \boldsymbol{\mu} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Gamma}_{\text{prior}})$, and its minimizer is an exact posterior sample by the argument of that proof. □

The gain $\boldsymbol{K}$ is the Kalman gain of the Kalman filter, and the perturbation of the data by $\boldsymbol{\eta}^{(j)}$ is what gives the updated ensemble the spread of the posterior: without it, its covariance would be too small by $\boldsymbol{K}\boldsymbol{\Gamma}\boldsymbol{K}^\mathsf{T}$ [16]. With J members, the ensemble covariance takes the place of $\boldsymbol{\Gamma}_{\text{prior}}$, and the update is exact only in the limit $J \to \infty$, with errors of the order $J^{-1/2}$: for the Gaussian posterior of the smooth deconvolution signal, one update from $J = 100$, 1000, and 10^4 prior draws gives standard deviations between 0.77 and 1.12, 0.92 and 1.04, and 0.98 and 1.02 times the posterior ones. A small ensemble has a second limitation, which holds for every forward operator.

Lemma 5.14: The subspace property

After any number of updates (5.10), every member lies in the affine space $\bar{\boldsymbol{x}}_0 + \operatorname{span}\{\boldsymbol{x}_0^{(1)} - \bar{\boldsymbol{x}}_0, \ldots, \boldsymbol{x}_0^{(J)} - \bar{\boldsymbol{x}}_0\}$ of the initial ensemble, whose dimension is at most $J - 1$.

Proof. The update adds $\boldsymbol{C}^{xf}\boldsymbol{d}^{(j)} = \frac{1}{J-1}\sum_k (\boldsymbol{x}^{(k)} - \bar{\boldsymbol{x}})(\boldsymbol{f}^{(k)} - \bar{\boldsymbol{f}})^\mathsf{T}\boldsymbol{d}^{(j)}$ with $\boldsymbol{d}^{(j)} = (\boldsymbol{C}^{ff} + \boldsymbol{\Gamma})^{-1}(\boldsymbol{y} + \boldsymbol{\eta}^{(j)} - \boldsymbol{f}^{(j)})$ to $\boldsymbol{x}^{(j)}$, a linear combination of the current deviations from the mean. The members therefore stay in the affine space of the current ensemble, and by induction in that of the initial one. □

An ensemble of 50 members explores at most 49 directions, whatever the number of unknowns, and it can only represent the uncertainty in those directions. This is a restriction and a regularization at once: the data cannot move the members into directions that the prior ensemble does not contain.

Straight lines through a curved landscape. For a nonlinear forward operator, the covariances of the ensemble describe F by the straight line that fits it best on the ensemble. If the matrix $\boldsymbol{G}$ minimizes $\sum_j \left\| \boldsymbol{f}^{(j)} - \bar{\boldsymbol{f}} - \boldsymbol{G}(\boldsymbol{x}^{(j)} - \bar{\boldsymbol{x}}) \right\|_2^2$, then $\boldsymbol{C}^{xf} = \boldsymbol{C}^{xx}\boldsymbol{G}^\mathsf{T}$ and $\boldsymbol{C}^{ff} = \boldsymbol{G}\boldsymbol{C}^{xx}\boldsymbol{G}^\mathsf{T} + \boldsymbol{C}^{rr}$, where $\boldsymbol{C}^{rr}$ is the covariance of the residuals of the fit: the gain is that of the linear problem with the forward operator $\boldsymbol{G}$ and the residuals of the straight line as additional noise. This is *statistical linearization*, and it is why the members need no derivatives, but also why the update is no longer exact. Figure 5.9 shows it on the bent ridge, whose single datum makes $\boldsymbol{K}$ a single vector shared by all members. For an ensemble of $2 \cdot 10^4$ members from the prior, $\boldsymbol{K} = (0.62, 0.00)^\mathsf{T}$, and every member moves along x_1 only (a): the ensemble covariance of x_2 and $F(\boldsymbol{x}) = x_1 + x_2^2/2$ vanishes by symmetry, and a straight line cannot see a parabola. The updated ensemble is nevertheless curved (b), because each member moves by its own misfit, which depends on x_2. But its mean, 0.31 in x_1, lies too deep in the bend (posterior 0.48), and for a given x_2 its spread across the ridge, 0.42, is larger than that of the posterior, 0.29.

Smaller steps: multiple data assimilation. The update can be split into N_a smaller ones, each with the noise covariance inflated to $N_a\boldsymbol{\Gamma}$ and fresh perturbations. Each of them assimilates the likelihood to the power $1/N_a$, and in the linear-Gaussian case with exact covariances the N_a updates give the same posterior as one [30]; for a nonlinear forward operator, the gain is re-estimated from the moving ensemble, which reduces the error of the straight line. This *ensemble smoother with multiple data assimilation* (ES-MDA) is the workhorse of reservoir engineering. On the bent ridge, eight updates move the mean of x_1 to 0.38 (Figure 5.9c) and 64 updates to 0.40, still short of 0.48. For a single update, Ernst et al. [32] show that the ensemble converges as $J \to \infty$ to a distribution that is related to the best linear estimate rather than to the posterior; in our example, more updates reduce this error but do not remove it.

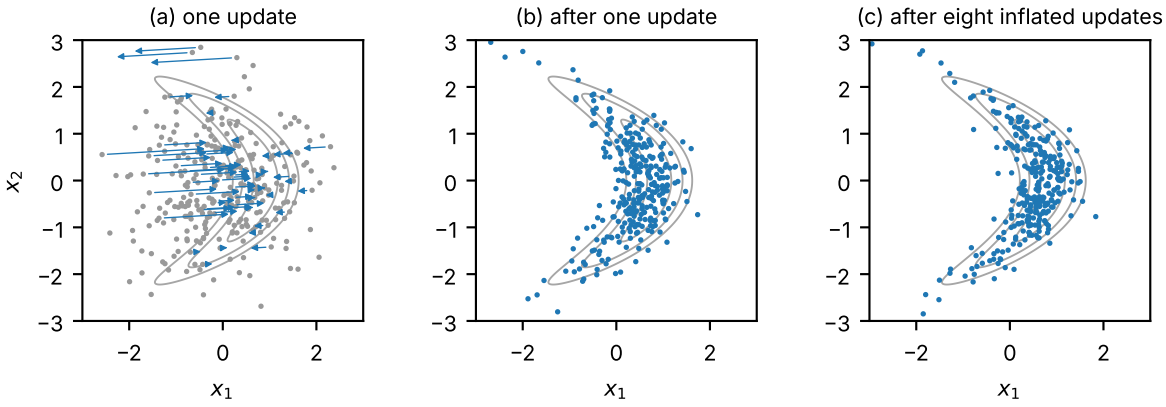


Figure 5.9: Ensemble Kalman updates on the bent ridge of Example 4.3 (gray contours), 300 members drawn from the prior $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$; the numbers in the text are from $2 \cdot 10^4$ members. (a) One update (5.10): every member moves along the same vector $\boldsymbol{K}$, here parallel to x_1 (arrows for every fifth member). (b) The ensemble after the update: curved, but too wide across the ridge and too deep in the bend. (c) After eight updates with the noise inflated eightfold (ES-MDA): closer to the posterior, but not on it.

Iterating: an optimizer. Repeating the update with the same data and without inflation assimilates the data again and again, as if every iteration brought new measurements. The ensemble contracts, and its mean minimizes the data misfit within the subspace of Lemma 5.14: this is *ensemble Kalman inversion* (EKI) [61, 107]. For a linear forward operator it behaves like the iterative regularization methods of Chapter 2: the misfit decreases with the iterations, the ensemble collapses, and the iteration must be stopped, e.g., by the discrepancy principle [60]. Alternatively, the prior is appended to the data, $\widetilde{F}(\boldsymbol{x}) = (F(\boldsymbol{x}), \boldsymbol{\Gamma}_{\text{prior}}^{-1/2}\boldsymbol{x})$, $\widetilde{\boldsymbol{y}} = (\boldsymbol{y}, \boldsymbol{\Gamma}_{\text{prior}}^{-1/2}\boldsymbol{\mu})$, with the noise covariance $\operatorname{diag}(\boldsymbol{\Gamma}, \boldsymbol{I})$, so that the misfit of the augmented problem is the potential U; this *Tikhonov EKI* converges to the MAP estimate within the ensemble subspace for linear forward operators [22]. Either way, the final ensemble has collapsed to a point and says nothing about the uncertainty: EKI is a derivative-free optimizer, not a sampler. Its MAP estimate can be the starting point of a Laplace approximation, and variants with noise in the iteration, such as the ensemble Kalman sampler [35], are designed to sample.

Example 5.15: Ensemble methods for the bathymetry problem

We use the setting of Example 5.4 and draw the initial ensembles from the prior. A draw for which no subcritical steady state exists, so that the forward model returns NaN, is replaced by a new one. This happens for 0.9 percent of the draws with $\gamma = 1\,\text{cm}$ per cell,

but for 37 percent with the prior of Example 4.18, $\gamma = 3\,\text{cm}$, whose standard deviation of 42 cm at the inflow lifts the bottom above the 36 cm at which the flow becomes critical. The samplers of Chapter 4, started at the MAP estimate, never noticed; an ensemble starts from the prior and notices at once, which makes it a prior predictive check for free. ES-MDA with eight updates and $J = 400$ members (Figure 5.10b) costs 3200 forward solves, in eight rounds of 400 parallel ones, and gives standard deviations within 10 percent of HMC and a mean within 0.57 HMC standard deviations of the HMC mean. With $J = 200$ (a, b), the standard deviations are within 13 percent and the mean within 0.87 standard deviations; with $J = 50$, the standard deviations lie between 0.68 and 1.19 times those of HMC, and the mean is up to 2.8 standard deviations off. The deviations of the standard deviations are mostly the sampling error of J members, about $(2J)^{-1/2}$ per component and up to three times that over 199 components, whereas the errors of the mean are larger than their sampling error: they are the bias of the straight lines. A single update with $J = 400$ overestimates the standard deviations by up to a factor of two upstream: one straight line is not enough for this nonlinearity. Tikhonov EKI (c) stalls at 2.2 and 1.4 cm from the MAP estimate with $J = 50$ and 100: by Lemma 5.14, the best it can reach is the minimizer of U on the span of the initial ensemble, which lies 2.1 and 1.2 cm from the MAP estimate. With $J = 200$ the span is the whole space, and the mean is 0.54 cm away after 40 iterations, i.e., 8200 forward solves, while the spread of the ensemble shrinks roughly like the inverse square root of the number of iterations and the steps with it. Gauss–Newton with a Jacobian by finite differences, 200 forward solves each, reaches the MAP estimate to 0.003 cm after three iterations and 610 forward solves. Plain EKI with $J = 200$ fits the data to the noise level after two iterations and overfits afterwards, to a misfit of 0.5 cm after 30 iterations, the semi-convergence of Chapter 2.

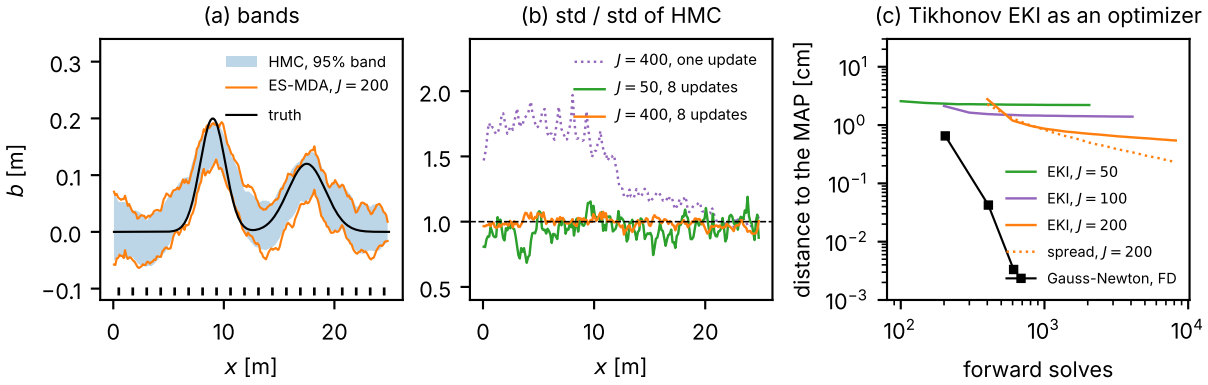


Figure 5.10: Ensemble Kalman methods for the bathymetry posterior with 20 sensors (Example 5.15). (a) The 95 % band of HMC and the band (mean ±1.96 standard deviations) of ES-MDA with $J = 200$ members and eight updates; truth in black, sensors as ticks. (b) Standard deviations of the ensembles divided by those of HMC: one update with $J = 400$, and eight updates with $J = 50$ and $J = 400$. (c) Distance of the ensemble mean of Tikhonov EKI to the MAP estimate against the number of forward solves, the spread of the ensemble for $J = 200$, and Gauss–Newton with a Jacobian by finite differences.

Remark 5.16: When ensemble methods pay off

With 199 unknowns, the ensemble methods lose to Gauss–Newton even when its Jacobian is computed by finite differences: Gauss–Newton and the Laplace approximation then cost

about 800 forward solves, three iterations and one more Jacobian, for standard deviations within five percent of HMC, ES-MDA with $J = 400$ costs 3200 for ten percent. Ensembles pay off when the unknown has so many components that $n+1$ forward solves per Jacobian are out of reach while the data inform only a few directions, when the forward model is a black box, and when many forward solves can run in parallel, since an update takes the wall-clock time of one solve. In weather forecasting all three hold, with states of 10^8 components or more and ensembles of the order of a hundred members; there the ensemble Kalman filter is a standard tool, with *localization*, which suppresses the spurious correlations that a small ensemble produces between distant unknowns and data, and *inflation*, which counteracts the collapse of the ensemble [18, 33].

5.5 Comparison

Table 5.2 collects the results of this chapter on the four test problems, each against a reference that we trust: quadrature for the bent ridge, the closed form for the Gaussian deconvolution posterior, the Gibbs sampler for the hierarchical prior, and HMC for the bathymetry problem.

Table 5.2: Approximate posteriors against references. Error of the mean: largest difference between the approximate and the reference posterior mean, in reference posterior standard deviations. Standard deviations: range of the ratios of approximate to reference posterior standard deviations over all components. The HMC reference has Monte Carlo errors of about 0.02 posterior standard deviations in the mean and 1.4 percent in the standard deviations per component, and up to three times that over its 199 components; smaller differences are not significant, and the ranges of the ensembles are mostly their own sampling error, about $(2J)^{-1/2}$ per component.

method	cost	error of the mean	standard deviations
bent ridge, $n = 2$; reference: quadrature			
Laplace, exact Hessian	one Hessian	0.78	0.51 to 3.57
Laplace, Gauss–Newton Hessian	one Jacobian	0.78	0.51 to 1.03
Gaussian VI, full rank	$9.6 \cdot 10^4$ gradients	0.54	0.52 to 0.54
ES-MDA, eight updates, $J = 2 \cdot 10^4$	$1.6 \cdot 10^5$ forward solves	0.19	1.03 to 1.14
smooth deconvolution, smoothness prior, $n = 128$; reference: closed form			
Laplace	one Cholesky factorization	exact	exact
Gaussian VI, mean field	$3.2 \cdot 10^4$ gradients	0.02	0.40 to 0.56
Gaussian VI, full rank	$8 \cdot 10^4$ gradients	0.02	0.99 to 1.02
one ensemble update, $J = 1000$	1000 forward solves	0.76	0.92 to 1.04
piecewise-constant deconvolution, hierarchical prior, $n = 128$; reference: Gibbs, $2 \cdot 10^4$ sweeps			
variational Bayes	38 factorizations	1.5	0.17 to 0.73
bathymetry, 20 *sensors*, $n = 199$; reference: HMC, about 80 forward solves per effective sample			
Laplace, Jacobians by JAX	five Jacobians, about 1000 forward solves	0.11	0.95 to 1.04
Laplace, finite differences	four Jacobians, about 800 forward solves	0.11	0.95 to 1.04
Gaussian VI, full rank, from Laplace	$3.2 \cdot 10^4$ gradients	0.05	0.97 to 1.04
ES-MDA, eight updates, $J = 200$	1600 forward solves	0.87	0.87 to 1.13
ES-MDA, eight updates, $J = 400$	3200 forward solves	0.57	0.90 to 1.08

Five lessons can be read off the table. First, on a nearly Gaussian posterior, the Laplace approximation is the cheapest method and as accurate as any other: on the bathymetry problem

it costs about as much as a dozen effective samples of HMC, where a reliable estimate needs hundreds. Gaussian variational inference, started from it in its own coordinates, moves the mean by a tenth of a posterior standard deviation toward the HMC mean and leaves the standard deviations within the Monte Carlo error of the reference, but its ELBO ends 0.6 below that of its starting point, because the noise of the stochastic gradients in the 19 900 entries of $\boldsymbol{C}$ costs more than the better mean gains; and its $3.2 \cdot 10^4$ gradients would have paid for 3000 effective samples of HMC. Check the Laplace approximation once, with a sampler or with $\hat{k}$, and then use it. Second, mean-field families are overconfident by construction (Theorem 5.6): by a factor of about two in the standard deviations for deconvolution, and about the positions of the jumps for variational Bayes with the hierarchical prior. Third, no Gaussian is good for a posterior that is far from Gaussian: on the bent ridge, both Laplace approximations are centered at the MAP estimate and too narrow in x_1, the one with the exact Hessian is also far too wide in x_2, and variational inference is too narrow in both; the importance weights flag this, but not reliably, with $\hat{k}$ between 0.52 and 0.91. The ensemble gets the means and standard deviations closest there, but its members spread across the ridge (Figure 5.9): summary statistics can hide a wrong shape. Fourth, ensembles need neither gradients nor tuning and run in parallel, but they need hundreds of members for 199 unknowns and are exact only for linear forward operators. Fifth, no method in the table reports its own error; the samplers of Chapter 4 are how we know, and the importance weights help where the draws of the approximation reach.

5.6 Outlook: what to read next

Default project A. Section 5.3.3 is its starting point: the project generalizes Algorithm 5.10 to other hyper-priors, whose $\boldsymbol{\theta}$-step needs only $\mathbb{E}[1/\theta_i]$ under a one-dimensional density, and compares it with Gibbs samplers by the measures of Example 5.11 and the Pareto $\hat{k}$. Theorem 5.6 says what to expect from a factorized family. Its paper track replaces the $\boldsymbol{x}$-step by a Gauss–Newton approximation for the nonlinear Burgers problem, i.e., the Laplace approximation of Section 5.2 inside variational Bayes.

Default project B. Task M4 compares a Laplace approximation with MCMC for 40 kinetic parameters, and Section 5.2.1 lists what to look for: rate constants in their logarithms, the permutation symmetry broken before the approximation, sparsity priors on the stoichiometric coefficients, whose kinks leave the Hessian undefined at zero, and the importance weights with $\hat{k}$ as a check. The ensemble of restarts in M5 is related to perturb-then-optimize when the data are perturbed for every restart [23], and to the deep ensembles of machine learning [72], which Habscheid and Glaubitz [52] combine with the hierarchical priors of Chapter 3 for Bayesian neural networks.

Default project C. The Laplace approximation of the posterior under the Gaussian increment prior is the reference that makes elliptical slice sampling cheap (Figure 4.15). Task M2 asks for the singular values of the Jacobian; whitened by the noise and the prior, the singular values of $\boldsymbol{\Gamma}^{-1/2}\boldsymbol{J}\boldsymbol{R}$ that exceed one count the directions that the three sensors determine better than the prior (Exercise 5.1c). Ensemble methods are the option for the flume model without JAX, at 0.7 s per forward solve, if the forward solves can run in parallel; the hierarchical ensemble Kalman methods of Kim et al. [66] combine them with the hierarchical prior. The ensemble Kalman filter of Glaubitz et al. [47] builds the same prior into every update, which keeps the shocks of hyperbolic conservation laws sharp and their smooth parts free of oscillations.

What this course did not cover. Sequential Monte Carlo moves an ensemble of weighted particles from the prior to the posterior through a sequence of tempered distributions and is exact as the number of particles grows [28]; it is the sampling counterpart of ES-MDA. Multilevel Monte Carlo combines many cheap coarse-grid solves with a few expensive fine-grid ones [42]. Surrogate models replace the forward model by a cheap emulator, and simulation-based inference learns the posterior from simulated pairs of unknowns and data, which also works when the likelihood is not available [26]. Integrated nested Laplace approximations make the Laplace approximation accurate for latent Gaussian models with few hyper-parameters [106]. Optimal experimental design asks where to put the sensors before the data are taken, and its basic quantity is the covariance of the Laplace approximation as a function of the sensor positions [2]. And the function-space view of [114], in which the unknown is a function and the methods are required to work under mesh refinement, is the natural next step for anyone whose unknown lives on a grid.

Summary : Chapter 5 in five sentences

When sampling is too expensive, an approximate posterior replaces it: a Gaussian at the MAP estimate with the inverse Hessian as covariance (Laplace), the member of a family that is closest in the Kullback–Leibler divergence (variational inference), or an ensemble moved by Kalman updates that need only forward solves (ensemble Kalman methods). The Laplace approximation costs a MAP estimate and a Hessian, is exact for linear-Gaussian problems, and scales with the number of directions the data inform, but it fails for posteriors that are curved, skewed, multimodal, or not differentiable at their mode, and it depends on the parametrization. Variational inference maximizes the ELBO by stochastic gradients with the reparametrization trick; it is mode-seeking, and a mean-field Gaussian reports the conditional variances $1/H_{ii}$ instead of the marginal ones, which makes it overconfident whenever the components are correlated. The ensemble Kalman update is perturb-then-optimize with ensemble covariances, exact for linear forward operators as the ensemble grows and approximate otherwise; iterated, it becomes a derivative-free optimizer whose ensemble collapses, and its members never leave the span of the initial ensemble. No approximation reports its own error: check it against a sampler on the problem or a smaller version of it, or with importance weights and their Pareto $\hat{k}$, which see only where the approximation puts its draws.

5.7 Notes and further reading

The Laplace approximation of posterior moments and marginal densities is treated by Tierney and Kadane [116], its convergence for inverse problems in the small-noise limit by Schillings, Sprungk, and Wacker [108] and non-asymptotically by Helin and Kretschmann [58], and its low-rank computation for large PDE-constrained problems in [15, 34], with the optimality of the truncation in [110] and the randomized eigensolvers in [54]. Blei, Kucukelbir, and McAuliffe [13] review variational inference for statisticians; the reparametrization trick is from [69], automatic differentiation variational inference in [71], the fixed-point conditions of Gaussian variational inference in [91], and the underestimated variances of mean-field approximations in Section 10.1.2 of Bishop [11]. The diagnostic $\hat{k}$ is in [119, 121], with the fit of the generalized Pareto distribution of [122]; normalizing flows are reviewed in [93] and transport maps in [80]; variational Bayes for hierarchical priors is in [1, 12]. Evensen [33] is the standard reference for the ensemble Kalman filter, with the perturbed observations of [16]; ensemble Kalman inversion is introduced in [61] and analyzed in [107], with early stopping in [60] and the prior as data in [22]; ES-MDA is in [30] and the related ensemble randomized maximum likelihood method in [23]; ensemble Kalman

methods with hierarchical sparsity-promoting priors are in [47, 66]. The limits of ensemble Kalman methods for nonlinear problems are analyzed in [32], the ensemble Kalman sampler in [35], and Calvello, Reich, and Stuart [18] give a unified account of ensemble Kalman methods.

5.8 Exercises

Exercise 5.1: The Laplace approximation by hand

(a) Compute the Laplace approximation of Example 5.2 in κ and in $u = \log \kappa$, and its probability of $\kappa < 0$. (b) Show that the evidence approximation $e^{-U(\boldsymbol{x}_{\mathrm{MAP}})}(2\pi)^{n/2}(\det \boldsymbol{H})^{-1/2}$ is exact for the linear-Gaussian posterior of Theorem 3.10. (c) Let $\boldsymbol{\Gamma}_{\mathrm{prior}} = \boldsymbol{R}\boldsymbol{R}^{\mathsf{T}}$, and let $\widetilde{\boldsymbol{H}} = \boldsymbol{R}^{\mathsf{T}}\boldsymbol{J}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{J}\boldsymbol{R} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\mathsf{T}}$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq 0$ and orthonormal eigenvectors $\boldsymbol{v}_i$. Show that the Gauss–Newton Hessian $\boldsymbol{H} = \boldsymbol{J}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{J} + \boldsymbol{\Gamma}_{\mathrm{prior}}^{-1}$ has the inverse

$$\boldsymbol{H}^{-1} = \boldsymbol{\Gamma}_{\mathrm{prior}} - \sum_{i=1}^{n} \frac{\lambda_i}{1+\lambda_i}\,(\boldsymbol{R}\boldsymbol{v}_i)(\boldsymbol{R}\boldsymbol{v}_i)^{\mathsf{T}},$$

that at most m of the λ_i are nonzero and they are the squared singular values of the whitened Jacobian $\boldsymbol{\Gamma}^{-1/2}\boldsymbol{J}\boldsymbol{R}$, and that dropping the terms with $\lambda_i < 0.1$ overestimates every posterior variance by less than ten percent. Compute the λ_i for the bathymetry problem with `approximations.lowrank_laplace`. (d) For the two-peak posterior of Figure 5.4b, compute the Laplace approximations at both peaks and their evidence approximations, and compare the weights of the resulting mixture with the true probabilities of the two peaks, 0.68 and 0.32.

Exercise 5.2: The two directions of the divergence

(a) Show that the minimizer of $\mathrm{KL}(p \,\|\, q)$ over all Gaussians q has the mean and the covariance of p, and that the minimizer over mean-field Gaussians has the marginal means and variances of p. (b) For $p = \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma} = \left(\begin{smallmatrix} 1 & \rho \\ \rho & 1 \end{smallmatrix}\right)$, compute both mean-field approximations of Figure 5.5b and both divergences as functions of ρ. (c) Prove Hadamard's inequality $\det \boldsymbol{H} \leq \prod_i H_{ii}$ for a symmetric positive definite $\boldsymbol{H}$ from the divergence formula of Theorem 5.6.

Exercise 5.3: Gaussian variational inference (computer)

Use `approximations.gaussian_vi`. (a) Reproduce Example 5.8, and vary $S \in \{1, 4, 16, 64\}$ and the step sizes; plot the exact ELBO against the number of gradients. (b) Reproduce Example 5.9 for the mean-field and the full-rank family. Then discretize the same deconvolution problem with $n \in \{32, 64, 128\}$ cells, i.e., `deconvolution_operator(n, 64)`, with data simulated from the same signal and noise realization and the smoothness prior with $\gamma = 0.0294\sqrt{128/n}$ per cell, which describes the same random function for every n. For the full-rank family with the settings of Example 5.9, how does the number of iterations to a divergence below one depend on n, and what is the limit of the mean-field divergence for these n? (c) Compute $\hat{k}$ for draws from both approximations of (b) with `approximations.pareto_khat`, and explain the result with Theorem 5.6.

Exercise 5.4: Variational Bayes and the IAS

(a) Derive the two updates of Algorithm 5.10 from $q(\boldsymbol{x}) \propto \exp(\mathbb{E}_{q(\boldsymbol{\theta})}[\log p(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{\theta})])$ and the analogous formula for $q(\boldsymbol{\theta})$. (b) Reproduce Example 5.11 with `approximations.cavi_hierarchical`, and repeat it for $\vartheta \in \{10^{-4}, 10^{-2}\}$: does variational Bayes find the small step, and how do its bands compare with those of the Gibbs sampler of Exercise 4.4c? (c) Replace $\mathbb{E}[z_i^2]$ by $(\boldsymbol{Lm})_i^2$ in the $\boldsymbol{\theta}$-step, and $\beta + \frac{1}{2}$ by $\beta + \frac{3}{2}$: which algorithm do you get?

Exercise 5.5: Ensemble Kalman methods (computer)

Use `approximations.ensemble_kalman_step`, `es_mda`, and `eki`. (a) Check Theorem 5.13 numerically for the smooth deconvolution posterior: plot the largest errors of the ensemble mean and standard deviation after one update against $J \in \{50, 100, 200, 500, 1000, 5000\}$, and compare with $J^{-1/2}$. (b) Check Lemma 5.14: start with $J = 10$ and verify that the members stay in the span of the initial ensemble. (c) Reproduce Example 5.15; then use 200 sensors instead of 20 and explain what changes. (d) Draw 1000 bottoms from the prior of Example 4.18 and plot those without a subcritical steady state: what does this say about the prior?

Exercise 5.6: Your own inverse problem, approximated

For the problem of Exercises 1.5, 2.6, 3.6 and 4.6: how many directions do your data inform, and how would you find out? Which of the three approximations of this chapter could you afford, and which failure of Section 5.2.1 or Section 5.3 do you expect? How would you check the approximation against a sampler or with importance weights? Write half a page; this is the fourth paragraph of your project proposal.

Appendix A

Probability Refresher (Pre-reading)

This appendix collects the probability that the course uses, and nothing else. Please read it before the course. If most of it is familiar, read Appendices A.3 and A.4 anyway, because the chapters use their formulas and parametrizations without comment. We work with densities throughout: every random quantity of the course has one, and every statement about it is an integral.

A.1 Random variables, densities, and expectations

A random vector $\boldsymbol{x} \in \mathbb{R}^n$ with *density* p takes values in a set $B \subseteq \mathbb{R}^n$ with the probability

$$\mathbb{P}(\boldsymbol{x} \in B) = \int_B p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}, \quad p \geq 0, \quad \int_{\mathbb{R}^n} p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} = 1.$$

A density is not a probability: it can exceed one, and only its integrals over sets are probabilities. A useful picture is a heap of sand of total mass one, spread over $\mathbb{R}^n$: the density is the height of the heap, and the probability of a set is the mass of the sand on it. For a scalar x, the *distribution function* $F(t) = \mathbb{P}(x \leq t)$ increases from zero to one, its derivative is the density, and its inverse F^{-1} is the *quantile function*: $F^{-1}(0.5)$ is the median, and $[F^{-1}(0.025), F^{-1}(0.975)]$ is the central 95 % interval.

Expectations. The *expectation* of a function f of $\boldsymbol{x}$ is $\mathbb{E}[f(\boldsymbol{x})] = \int f(\boldsymbol{x}) \, p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}$, provided that the integral converges absolutely. The mean $\boldsymbol{\mu} = \mathbb{E}[\boldsymbol{x}]$ is the center of mass of the heap, and the *covariance matrix*

$$\operatorname{Cov}(\boldsymbol{x}) = \mathbb{E}[(\boldsymbol{x} - \boldsymbol{\mu})(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}}] \in \mathbb{R}^{n \times n}$$

describes its spread. It is symmetric and positive semidefinite, its diagonal holds the variances $\operatorname{Var}(x_i)$, whose square roots are the standard deviations, and $\operatorname{Cov}(\boldsymbol{x})_{ij} / \sqrt{\operatorname{Var}(x_i) \operatorname{Var}(x_j)} \in [-1, 1]$ is the *correlation* of x_i and x_j. Expectations are linear, and for a matrix $\boldsymbol{B}$ and a vector $\boldsymbol{c}$,

$$\mathbb{E}[\boldsymbol{B}\boldsymbol{x} + \boldsymbol{c}] = \boldsymbol{B}\boldsymbol{\mu} + \boldsymbol{c}, \quad \operatorname{Cov}(\boldsymbol{B}\boldsymbol{x} + \boldsymbol{c}) = \boldsymbol{B} \operatorname{Cov}(\boldsymbol{x}) \boldsymbol{B}^{\mathsf{T}}. \tag{A.1}$$

In particular, $\operatorname{Var}(\boldsymbol{v}^{\mathsf{T}}\boldsymbol{x}) = \boldsymbol{v}^{\mathsf{T}} \operatorname{Cov}(\boldsymbol{x}) \boldsymbol{v}$ is the variance in the direction $\boldsymbol{v}$. Not every distribution has a mean or a variance: for the Cauchy and some Student-t distributions of Appendix A.4, the defining integrals diverge, because the density decays too slowly.

Change of variables. If $\boldsymbol{x}$ has the density $p_{\boldsymbol{x}}$ and $\boldsymbol{y} = T(\boldsymbol{x})$ with a differentiable bijection T whose inverse is differentiable, then

$$p_{\boldsymbol{y}}(\boldsymbol{y}) = p_{\boldsymbol{x}}(T^{-1}(\boldsymbol{y})) \, |\det \nabla T^{-1}(\boldsymbol{y})|. \tag{A.2}$$

The determinant accounts for the stretching of space: where T spreads the sand out, the heap becomes lower. Two consequences are used again and again. First, expectations do not depend on the variables in which they are computed, $\mathbb{E}[g(\boldsymbol{y})] = \mathbb{E}[g(T(\boldsymbol{x}))]$, but densities, and therefore their maxima, do: the mode of $\boldsymbol{y}$ is in general not the image of the mode of $\boldsymbol{x}$. Second, a scalar x with a continuous and increasing distribution function F is turned into a uniform variable by $u = F(x)$, and conversely $x = F^{-1}(u)$ with $u \sim \mathcal{U}(0,1)$ has the distribution function F. This is *inverse transform sampling*, and composed with the distribution function Φ of the standard normal distribution it gives the hierarchical prior normalization of Chapter 4, $\theta = F^{-1}(\Phi(v))$ with $v \sim \mathcal{N}(0,1)$.

Example A.1: The log-normal distribution

Let $u \sim \mathcal{N}(m, s^2)$ and $\kappa = e^u$. By (A.2) with $T^{-1}(\kappa) = \log\kappa$, the density of $\kappa > 0$ is $p(\kappa) = \mathcal{N}(\log\kappa; m, s^2)/\kappa$. Its median is e^m, its mode is e^{m-s^2}, and its mean is $e^{m+s^2/2}$ (Exercise A.1). For $m = 0$ and $s = 0.6$, these are 1, 0.70, and 1.20: the conductivity of Remark 3.16 and Example 5.2, whose most probable value, typical value, and average are three different numbers.

A.2 Conditioning and Bayes' theorem

Two random vectors $\boldsymbol{x} \in \mathbb{R}^n$ and $\boldsymbol{y} \in \mathbb{R}^m$ have a joint density $p(\boldsymbol{x}, \boldsymbol{y})$ on $\mathbb{R}^{n+m}$. The *marginal* densities integrate out the other variable, $p(\boldsymbol{x}) = \int p(\boldsymbol{x}, \boldsymbol{y})\,\mathrm{d}\boldsymbol{y}$ and $p(\boldsymbol{y}) = \int p(\boldsymbol{x}, \boldsymbol{y})\,\mathrm{d}\boldsymbol{x}$, and the *conditional* density of $\boldsymbol{x}$ given $\boldsymbol{y}$ is the joint density at the observed $\boldsymbol{y}$, renormalized:

$$p(\boldsymbol{x} \mid \boldsymbol{y}) = \frac{p(\boldsymbol{x}, \boldsymbol{y})}{p(\boldsymbol{y})} \quad \text{for } p(\boldsymbol{y}) > 0.$$

In the picture of the heap of sand, conditioning cuts the heap along the slice of the observed $\boldsymbol{y}$ and rescales the cross-section to mass one. Writing the joint density in both orders gives Bayes' theorem (3.1), $p(\boldsymbol{x} \mid \boldsymbol{y}) = p(\boldsymbol{y} \mid \boldsymbol{x})\,p(\boldsymbol{x})/p(\boldsymbol{y})$, with the *evidence* $p(\boldsymbol{y}) = \int p(\boldsymbol{y} \mid \boldsymbol{x})\,p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$ as the normalizing constant. Since the evidence does not depend on $\boldsymbol{x}$, the posterior is known up to this constant as soon as likelihood and prior are, which is all that MAP estimation and Markov chain Monte Carlo need.

The vectors are *independent* if $p(\boldsymbol{x}, \boldsymbol{y}) = p(\boldsymbol{x})\,p(\boldsymbol{y})$, i.e., if $p(\boldsymbol{x} \mid \boldsymbol{y}) = p(\boldsymbol{x})$: observing one says nothing about the other. Components $x_1, \dots, x_n$ are *conditionally independent* given θ if $p(\boldsymbol{x} \mid \theta) = \prod_i p(x_i \mid \theta)$, and this does not make them independent: if all components share one variance, $x_i \mid \theta \sim \mathcal{N}(0, \theta)$, then a large x_1 suggests a large θ and thereby large other components. The hierarchical prior (3.10) gives every component its own variance θ_i, and its components stay independent after $\boldsymbol{\theta}$ is integrated out; in the posterior, the data couple them.

Example A.2: A rare condition and a good test

A condition affects one person in a hundred, and a test detects it in 95 percent of the cases and raises a false alarm for 5 percent of the healthy. With $x = 1$ for the condition and $y = 1$ for a positive test, Bayes' theorem with sums in place of integrals gives

$$\mathbb{P}(x = 1 \mid y = 1) = \frac{0.95 \cdot 0.01}{0.95 \cdot 0.01 + 0.05 \cdot 0.99} = 0.16.$$

Five out of six positive tests are false alarms, because the healthy are so many more. The prior matters as much as the likelihood, and a likelihood that looks decisive can be overruled by a prior that is just as decisive in the other direction.

Iterated expectations. A conditional expectation $\mathbb{E}[f(\boldsymbol{x}) \mid \boldsymbol{y}] = \int f(\boldsymbol{x})\, p(\boldsymbol{x} \mid \boldsymbol{y})\, \mathrm{d}\boldsymbol{x}$ is a function of $\boldsymbol{y}$, and averaging it over $\boldsymbol{y}$ gives back the unconditional one:

$$\mathbb{E}[\boldsymbol{x}] = \mathbb{E}\big[\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}]\big], \quad \operatorname{Cov}(\boldsymbol{x}) = \mathbb{E}\big[\operatorname{Cov}(\boldsymbol{x} \mid \boldsymbol{y})\big] + \operatorname{Cov}\big(\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}]\big). \tag{A.3}$$

The second identity, the *law of total variance*, splits the spread of $\boldsymbol{x}$ into the average spread within the slices and the spread of the centers of the slices. For the scale mixtures $z \mid \theta \sim \mathcal{N}(0, \theta)$ of Appendix A.4, it gives $\operatorname{Var}(z) = \mathbb{E}[\theta]$.

A.3 The multivariate normal distribution

The Gaussian density (3.2) with mean $\boldsymbol{\mu}$ and symmetric positive definite covariance $\boldsymbol{\Gamma}$ is the workhorse of the course, because everything about it can be computed with linear algebra. Its level sets are ellipsoids $\{\boldsymbol{x} : (\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) = r^2\}$, whose axes point along the eigenvectors of $\boldsymbol{\Gamma}$ and have the lengths r times the square roots of its eigenvalues. The inverse $\boldsymbol{P} = \boldsymbol{\Gamma}^{-1}$ is the *precision matrix*, and many Gaussians of the course are given by their precision, e.g., the posterior of Theorem 3.10, whose precision is $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\text{prior}}^{-1}$.

Affine maps and sampling. If $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma})$, then $\boldsymbol{B}\boldsymbol{x} + \boldsymbol{c} \sim \mathcal{N}(\boldsymbol{B}\boldsymbol{\mu} + \boldsymbol{c}, \boldsymbol{B}\boldsymbol{\Gamma}\boldsymbol{B}^{\mathsf{T}})$ for every matrix $\boldsymbol{B}$ with full row rank: affine images of Gaussians are Gaussian, with the moments (A.1). Read backwards, this is how Gaussians are sampled. If $\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}} = \boldsymbol{\Gamma}$, e.g., with the lower triangular Cholesky factor $\boldsymbol{R}$, and $\boldsymbol{\xi} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ has independent standard normal components, then $\boldsymbol{\mu} + \boldsymbol{R}\boldsymbol{\xi} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Gamma})$. If the precision is given, $\boldsymbol{P} = \boldsymbol{C}\boldsymbol{C}^{\mathsf{T}}$, then $\boldsymbol{\mu} + \boldsymbol{C}^{-\mathsf{T}}\boldsymbol{\xi}$ has the covariance $\boldsymbol{C}^{-\mathsf{T}}\boldsymbol{C}^{-1} = \boldsymbol{P}^{-1}$, and a sample costs one triangular solve:

```
C = np.linalg.cholesky(P)                                   # P = C C^T
x = mu + scipy.linalg.solve_triangular(C.T, rng.standard_normal(n), lower=False)
```

Marginals and conditionals. Split $\boldsymbol{x} = (\boldsymbol{x}_a, \boldsymbol{x}_b)$, and $\boldsymbol{\mu}$, $\boldsymbol{\Gamma}$, and $\boldsymbol{P}$ accordingly. The marginal of $\boldsymbol{x}_a$ is $\mathcal{N}(\boldsymbol{\mu}_a, \boldsymbol{\Gamma}_{aa})$: one simply drops the other components. The conditional of $\boldsymbol{x}_a$ given $\boldsymbol{x}_b$ is Gaussian with

$$\begin{aligned} \mathbb{E}[\boldsymbol{x}_a \mid \boldsymbol{x}_b] &= \boldsymbol{\mu}_a + \boldsymbol{\Gamma}_{ab}\boldsymbol{\Gamma}_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b) = \boldsymbol{\mu}_a - \boldsymbol{P}_{aa}^{-1}\boldsymbol{P}_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b), \\ \operatorname{Cov}(\boldsymbol{x}_a \mid \boldsymbol{x}_b) &= \boldsymbol{\Gamma}_{aa} - \boldsymbol{\Gamma}_{ab}\boldsymbol{\Gamma}_{bb}^{-1}\boldsymbol{\Gamma}_{ba} = \boldsymbol{P}_{aa}^{-1}. \end{aligned} \tag{A.4}$$

Marginals are simple in the covariance, conditionals in the precision. For a single component, the conditional variance of x_i given all other components is $1/P_{ii}$, and it is never larger than the marginal variance Γ_{ii}: by (A.3), knowing the other components can only reduce the uncertainty on average, and for a Gaussian the conditional variance does not depend on what is known. This inequality is behind the overconfidence of mean-field variational inference in Theorem 5.6.

Completing the square. If a density is proportional to $\exp(-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{H}\boldsymbol{x} + \boldsymbol{b}^{\mathsf{T}}\boldsymbol{x})$ with a symmetric positive definite $\boldsymbol{H}$, then it is $\mathcal{N}(\boldsymbol{H}^{-1}\boldsymbol{b}, \boldsymbol{H}^{-1})$, because

$$-\tfrac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{H}\boldsymbol{x} + \boldsymbol{b}^{\mathsf{T}}\boldsymbol{x} = -\tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{H}^{-1}\boldsymbol{b})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x} - \boldsymbol{H}^{-1}\boldsymbol{b}) + \tfrac{1}{2}\boldsymbol{b}^{\mathsf{T}}\boldsymbol{H}^{-1}\boldsymbol{b}.$$

The quadratic part is the precision, and the linear part, multiplied by the covariance, is the mean. Every Gaussian posterior of the course is found this way: write down the negative logarithm of likelihood times prior, collect the quadratic and the linear terms in $\boldsymbol{x}$, and read off the precision and the mean.

Where the samples are. In high dimensions, the samples of a Gaussian are not near its mode. For $\boldsymbol{\xi} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_n)$, the squared length $\|\boldsymbol{\xi}\|_2^2$ is a sum of n independent squares with mean one and variance two, so that $\|\boldsymbol{\xi}\|_2 \approx \sqrt{n}$ with fluctuations of order one. For $n = 128$, all but one in a thousand samples lie at distances between 9 and 14 from the mode, where the density is about e^{-64} times its maximum. The mode is the single most probable point, but the region around it is too small to hold much probability, like the summit of a mountain range that holds only a tiny fraction of its area. This is why MAP estimates do not look like typical samples, and why the samplers of Chapter 4 have to explore a thin shell rather than the neighborhood of the mode.

A.4 A catalogue of distributions used in the course

Table A.1 lists the distributions of the course in the parametrizations that we use. Software libraries often use others.

In `scipy.stats`, the distributions of the table are `norm`, `laplace`, `t`, `cauchy`, `gamma`, `invgamma`, `gengamma`, and `lognorm`, our scale parameters are their `scale` arguments, e.g., InvGamma(β, ϑ) is `invgamma(beta, scale=vartheta)` and GG(r, β, ϑ) is `gengamma(beta, r, scale=vartheta)`, and the log-normal distribution is `lognorm(s, scale=np.exp(m))`. Each of them has the methods `pdf`, `logpdf`, `cdf`, `ppf` (the quantile function), and `rvs`.

The modes of the hyper-priors are $(\beta - 1)\vartheta$ for Gamma(β, ϑ) with $\beta \geq 1$ and $\vartheta/(\beta + 1)$ for InvGamma(β, ϑ); the generalized gamma distribution contains both, GG$(1, \beta, \vartheta) =$ Gamma(β, ϑ) and GG$(-1, \beta, \vartheta) =$ InvGamma(β, ϑ). If $\theta \sim$ InvGamma(β, ϑ), then $1/\theta$ is gamma distributed with shape β and scale $1/\vartheta$, which is how inverse gamma draws are made.

Table A.1: Distributions of the course. In the last column, $\xi \sim \mathcal{N}(0,1)$, $u \sim \mathcal{U}(0,1)$, $e_1, e_2 \sim$ Exp(1) (e.g., $e = -\log u$), and $g \sim$ Gamma$(\beta, 1)$ (`rng.gamma(beta)` in NumPy) are independent; the Student-t draw uses $g \sim$ Gamma$(\nu/2, 1)$.

distribution	density	mean; variance	a draw
$\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sqrt{2\pi}\,\sigma}\, e^{-(x-\mu)^2/(2\sigma^2)}$	μ; σ^2	$\mu + \sigma\xi$
Laplace$(0, \gamma)$	$\frac{1}{2\gamma}\, e^{-\lvert x\rvert/\gamma}$	0; $2\gamma^2$	$\gamma(e_1 - e_2)$
Student-t, ν degrees of freedom, scale s	$\frac{\Gamma((\nu+1)/2)}{\Gamma(\nu/2)\sqrt{\nu\pi}\,s}\left(1 + \frac{x^2}{\nu s^2}\right)^{-(\nu+1)/2}$	0 if $\nu > 1$; $\frac{\nu s^2}{\nu - 2}$ if $\nu > 2$	$s\,\xi\sqrt{\nu/(2g)}$
Cauchy$(0, \gamma)$	$\frac{\gamma}{\pi(x^2 + \gamma^2)}$	none; none	$\gamma \tan\left(\pi(u - \frac{1}{2})\right)$
Gamma(β, ϑ), shape β, scale ϑ	$\frac{1}{\Gamma(\beta)\,\vartheta^\beta}\, \theta^{\beta-1} e^{-\theta/\vartheta}$, $\theta > 0$	$\beta\vartheta$; $\beta\vartheta^2$	ϑg
InvGamma(β, ϑ)	$\frac{\vartheta^\beta}{\Gamma(\beta)}\, \theta^{-\beta-1} e^{-\vartheta/\theta}$, $\theta > 0$	$\frac{\vartheta}{\beta - 1}$ if $\beta > 1$; $\frac{\vartheta^2}{(\beta-1)^2(\beta-2)}$ if $\beta > 2$	ϑ/g
GG(r, β, ϑ), $r \neq 0$	$\frac{\lvert r\rvert}{\Gamma(\beta)\,\vartheta}\left(\frac{\theta}{\vartheta}\right)^{r\beta-1} e^{-(\theta/\vartheta)^r}$, $\theta > 0$	$\vartheta\, \frac{\Gamma(\beta + 1/r)}{\Gamma(\beta)}$ if $\beta + \frac{1}{r} > 0$	$\vartheta g^{1/r}$
log-normal, $\log\kappa \sim \mathcal{N}(m, s^2)$	$\frac{1}{\kappa}\, \mathcal{N}(\log\kappa; m, s^2)$, $\kappa > 0$	$e^{m + s^2/2}$; $(e^{s^2} - 1)\, e^{2m + s^2}$	$e^{m + s\xi}$

Figure A.1 compares the four symmetric densities. They look alike near zero and differ completely in their tails, and the tails decide what a prior believes. With scale one, a value beyond three has the probability 0.003 under the normal distribution, 0.05 under the Laplace distribution, 0.10 under the Student-t distribution with two degrees of freedom, and 0.20 under

the Cauchy distribution: as a prior on increments, the first rules out jumps of three scales, and the last expects one in five increments to be such a jump.

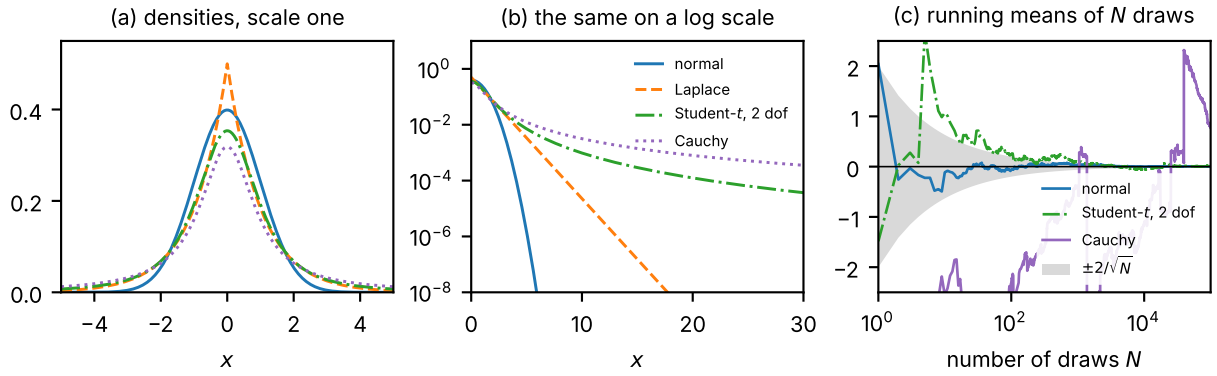


Figure A.1: Four symmetric distributions with scale one. (a) Their densities look alike near zero. (b) On a logarithmic axis, their tails differ completely: the logarithm of the density decreases quadratically (normal), linearly (Laplace), and logarithmically (Student-t and Cauchy). (c) Running means of N independent draws: for the normal distribution they settle at the rate $N^{-1/2}$ of the central limit theorem (gray band), for the Student-t distribution with two degrees of freedom, which has a mean but no variance, more slowly, and for the Cauchy distribution, which has no mean, not at all.

Scale mixtures of normals. Three of the symmetric distributions are Gaussians with a random variance. If $z \mid \theta \sim \mathcal{N}(0, \theta)$, then the marginal distribution of z is

- the Laplace distribution Laplace$(0, \gamma)$ for an exponential variance with mean $2\gamma^2$, i.e., $\theta \sim \text{Gamma}(1, 2\gamma^2)$, as in (4.14);
- for $\theta \sim \text{InvGamma}(\nu/2, \nu s^2/2)$, the Student-$t$ distribution with ν degrees of freedom and scale s, and therefore the Cauchy distribution for $\nu = 1$;
- in particular, for the hyper-prior InvGamma(β, ϑ) of (3.10), the Student-t distribution of Remark 3.8, with 2β degrees of freedom and the scale $\sqrt{\vartheta/\beta}$.

This is the mechanism of the hierarchical prior: a heavy tail of the variance makes a heavy tail of the increment. By (A.3), $\text{Var}(z) = \mathbb{E}[\theta]$, which is infinite for $\beta \leq 1$. The prior of the course, with $\beta = 1$ and $\vartheta = 10^{-3}$, therefore has increments of infinite variance, although 91 percent of them are smaller than 0.1 in absolute value (Exercise A.3).

A.5 Monte Carlo basics

Most expectations of the course have no closed form and are estimated from samples. If $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ are independent draws from p, then the *Monte Carlo estimate*

$$\widehat{f}_N = \frac{1}{N} \sum_{k=1}^{N} f(\boldsymbol{x}_k) \approx \mathbb{E}[f(\boldsymbol{x})] \tag{A.5}$$

converges to the expectation as $N \to \infty$ whenever $\mathbb{E}\,|f(\boldsymbol{x})| < \infty$; this is the *law of large numbers*. If moreover $\sigma_f^2 = \text{Var}(f(\boldsymbol{x})) < \infty$, the *central limit theorem* says how fast: for large N, $\widehat{f}_N$ is approximately distributed as $\mathcal{N}(\mathbb{E}[f(\boldsymbol{x})], \sigma_f^2/N)$. The *Monte Carlo standard error* $\sigma_f/\sqrt{N}$ is estimated by replacing σ_f with the standard deviation of the values $f(\boldsymbol{x}_k)$, and a result is

reported as $\widehat{f}_N \pm 2\sigma_f/\sqrt{N}$. Three features of this error matter for the course. It decreases like $N^{-1/2}$: one more correct digit costs a hundred times more samples. It does not depend on the dimension n of $\boldsymbol{x}$, which is why sampling works in high dimensions, where quadrature on a grid with k points per direction would need k^n points. And it needs a finite variance: in Figure A.1c, the running means of draws settle at the rate $N^{-1/2}$ for the normal distribution, more slowly for the Student-t distribution with two degrees of freedom, whose variance is infinite, and not at all for the Cauchy distribution, which has no mean.

Probabilities and quantiles. A probability is the expectation of an indicator function, $\mathbb{P}(\boldsymbol{x} \in B) = \mathbb{E}[\mathbf{1}_B(\boldsymbol{x})]$. Its estimate, the fraction of the draws in B, has the standard error $\sqrt{q(1-q)/N}$, where $q = \mathbb{P}(\boldsymbol{x} \in B)$. The probability of a jump in Chapters 4 and 5, $\mathbb{P}(|z_i| > 0.1 \mid \boldsymbol{y})$, is computed this way. Quantiles are estimated from the sorted draws: the central 95 % credible interval of a component lies between the 2.5 and the 97.5 percent quantiles of its draws.

Correlated and weighted draws. The draws of a Markov chain are not independent, and N correlated draws are worth fewer independent ones: the *effective sample size* of Section 4.6 takes the place of N in the standard error. Weighted draws from another density, as in the importance sampling of Section 5.2.2, have an effective sample size of their own.

A.6 Exercises

Exercise A.1: The log-normal distribution

Let $u \sim \mathcal{N}(m, s^2)$ and $\kappa = e^u$. (a) Derive the density of κ from (A.2). (b) Show that the median of κ is e^m, its mode e^{m-s^2}, and its mean $e^{m+s^2/2}$. (c) Check the median and the mean for $m = 0$ and $s = 0.6$ with 10^6 draws, and give the Monte Carlo standard error of the estimated mean.

Exercise A.2: Conditioning a Gaussian

Let $(x_1, x_2) \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Gamma})$ with $\boldsymbol{\Gamma} = \left(\begin{smallmatrix} 1 & \rho \\ \rho & 1 \end{smallmatrix}\right)$ and $|\rho| < 1$. (a) Compute the conditional density of x_1 given x_2 by completing the square, and compare it with (A.4). (b) Verify that the conditional variance is $1/P_{11}$ for the precision $\boldsymbol{P} = \boldsymbol{\Gamma}^{-1}$, and check the law of total variance (A.3) for x_1.

Exercise A.3: The hierarchical prior by Monte Carlo (computer)

Draw $N = 10^5$ samples of z from $z \mid \theta \sim \mathcal{N}(0, \theta)$ and $\theta \sim \text{InvGamma}(1, 10^{-3})$ with the recipes of Table A.1. (a) Estimate $\mathbb{P}(|z| > 0.1)$ with its standard error, and compare the estimate with the exact value from the Student-t distribution of Remark 3.8. (b) Plot the running mean of z^2 against N. What does the law of large numbers say, and what do you see?

A.7 Solutions

Exercise A.1. (a) With $T^{-1}(\kappa) = \log\kappa$ and $(T^{-1})'(\kappa) = 1/\kappa$, (A.2) gives

$$p(\kappa) = \frac{1}{\sqrt{2\pi}\, s\kappa} \exp\Big(-\frac{(\log\kappa - m)^2}{2s^2}\Big), \quad \kappa > 0.$$

(b) The median: $\mathbb{P}(\kappa \le e^m) = \mathbb{P}(u \le m) = \frac{1}{2}$. The mode: the derivative of $\log p(\kappa) = -\log\kappa - (\log\kappa - m)^2/(2s^2) + \text{const}$ is $-(1 + (\log\kappa - m)/s^2)/\kappa$, which vanishes for $\log\kappa = m - s^2$. The mean: $\mathbb{E}[e^u] = \int e^u\, \mathcal{N}(u; m, s^2)\, du$, and completing the square, $u - (u-m)^2/(2s^2) = -(u - m - s^2)^2/(2s^2) + m + s^2/2$, gives $e^{m+s^2/2}$. (c) The draws are $\kappa = e^{0.6\xi}$:

```
k = np.exp(0.6 * np.random.default_rng(0).standard_normal(10**6))
print(np.median(k), k.mean(), k.std() / np.sqrt(k.size))     # 1.0006 1.1982 0.0008
```

The estimated mean lies within three standard errors of $e^{0.18} = 1.1972$.

Exercise A.2. (a) Since $\mathbf{\Gamma}^{-1} = \frac{1}{1-\rho^2}\left(\begin{smallmatrix} 1 & -\rho \\ -\rho & 1 \end{smallmatrix}\right)$, the joint density is proportional to $\exp(-(x_1^2 - 2\rho x_1 x_2 + x_2^2)/(2(1-\rho^2)))$, and as a function of x_1 the exponent is $-(x_1 - \rho x_2)^2/(2(1-\rho^2)) + \text{const}$. Hence $x_1 \mid x_2 \sim \mathcal{N}(\rho x_2, 1 - \rho^2)$, which is (A.4) with $\mathbf{\Gamma}_{ab}\mathbf{\Gamma}_{bb}^{-1} = \rho$ and $\mathbf{\Gamma}_{aa} - \mathbf{\Gamma}_{ab}\mathbf{\Gamma}_{bb}^{-1}\mathbf{\Gamma}_{ba} = 1 - \rho^2$. (b) $P_{11} = 1/(1-\rho^2)$, so the conditional variance is $1/P_{11}$, and $\mathbb{E}[\mathrm{Var}(x_1 \mid x_2)] + \mathrm{Var}(\mathbb{E}[x_1 \mid x_2]) = (1 - \rho^2) + \rho^2\,\mathrm{Var}(x_2) = 1 = \mathrm{Var}(x_1)$. For $\rho = 0.9$, knowing x_2 reduces the standard deviation of x_1 from 1 to 0.44.

Exercise A.3. (a) With $\theta = 10^{-3}/g$, $g \sim \mathrm{Gamma}(1, 1)$:

```
rng = np.random.default_rng(1)
theta = 1e-3 / rng.gamma(1.0, size=10**5)                    # InvGamma(1, 1e-3)
z = np.sqrt(theta) * rng.standard_normal(10**5)
q = np.mean(np.abs(z) > 0.1)
print(q, np.sqrt(q * (1 - q) / z.size))                      # 0.0869 0.0009
```

The marginal of z is the Student-t distribution with two degrees of freedom and scale $\sqrt{10^{-3}}$, whose distribution function is $F(t) = \frac{1}{2} + \frac{a}{2\sqrt{2+a^2}}$ with $a = t/\sqrt{10^{-3}}$; hence $\mathbb{P}(|z| > 0.1) = 1 - a/\sqrt{2+a^2} = 0.0871$ for $a = 3.16$, within the standard error of the estimate. (b) Here $\mathbb{E}[z^2] = \mathbb{E}[\theta] = \infty$, so the law of large numbers does not apply, and the running mean does not converge. It jumps up whenever a draw from the far tail arrives and drifts upward in between, roughly like $10^{-3}\log N$, from 0.005 after 10^3 draws to 0.016 after 10^5 in the run above. The prior cannot be summarized by its variance, but its probabilities and quantiles, e.g., the 91 percent of increments below 0.1, describe it well.

Appendix B

Software for the Exercises: NumPy and JAX

All exercises use Python with NumPy, SciPy, and Matplotlib; the nonlinear and the sampling parts use JAX, which differentiates Python functions automatically and compiles them. This appendix says how to install the software, what the course code contains, how to write a posterior so that the samplers of Chapter 4 and those of an existing library can run on it, and which mistakes cost the most time. The course code is written to be read: it favors clarity over speed, and every function has a docstring.

B.1 Installation and the course code

Any Python from version 3.10 on works. In a fresh virtual environment,

```
pip install numpy scipy matplotlib jax blackjax
```

installs everything; BlackJAX is needed only for the NUTS sampler, and no GPU is needed. The code was tested with Python 3.13, NumPy 2.5, SciPy 1.18, Matplotlib 3.11, JAX 0.12, and BlackJAX 1.7.

The folder `code/` of the course material contains the modules of Table B.1. Start Python in this folder, or add it to the search path with `sys.path.insert(0, "path/to/code")`, and use `help(samplers.gibbs_hierarchical)` and the like for the documentation. Every figure of the notes is reproduced by the script of its chapter, e.g., `python ch04_figures.py` for all figures of Chapter 4 or `python ch05_figures.py kl viridge` for two of them; the scripts are also the most complete examples of how the modules are used.

Table B.1: The modules of the course code.

module	contents	chapters
`running_examples.py`	the deconvolution operator and its two test signals; the grid, the steady shallow-water model in NumPy, and the two test bottoms of the bathymetry problem	all
`swe_jax.py`	the bathymetry forward model in JAX, `make_forward(x_cells, x_obs)`, differentiable and compiled	2, 4, 5
`solvers.py`	difference matrices, Tikhonov, CGLS, ℓ^1 regularization by FISTA, Gauss–Newton, the linear-Gaussian posterior, and the IAS	2, 3
`priors.py`	precision matrices and samplers of the Gaussian, Laplace, and hierarchical priors	3
`samplers.py`	random-walk Metropolis, perturb-then-optimize, Gibbs samplers for the hierarchical and the Laplace prior, HMC in NumPy and in JAX, elliptical slice sampling; autocorrelation, integrated autocorrelation time, effective sample size, $\widehat{R}$	4
`approximations.py`	the low-rank Laplace approximation, Gaussian variational inference, the Pareto $\hat{k}$ and importance sampling, variational Bayes for the hierarchical prior, ensemble Kalman updates, ES-MDA, and EKI	5
`ch0*_figures.py`, `appA_figures.py`	the figures of the chapters and of Appendix A	all

B.2 A short JAX primer

JAX looks like NumPy, `jax.numpy` has almost the same functions as `numpy`, and it adds a few *transformations* that take a Python function and return a new one. The four that the course uses fit into a dozen lines, here for the bent ridge of Example 4.3:

```
import jax
jax.config.update("jax_enable_x64", True)   # 64-bit floats: set this first
import jax.numpy as jnp

def U(x):                                   # pure function of jnp arrays
    r = 1.0 - x[0] - 0.5 * x[1] ** 2
    return 0.5 * (x @ x) + r**2 / (2 * 0.3**2)

grad_U = jax.jit(jax.grad(U))               # gradient by AD, compiled
U_batch = jax.vmap(U)                       # U for points of shape (S, 2)
x = jnp.array([0.5, 1.0])
print(U(x), grad_U(x), U_batch(jnp.ones((4, 2))))
key = jax.random.PRNGKey(0)                 # an explicit key ...
key, sub = jax.random.split(key)            # ... split, never reused
print(jax.random.normal(sub, (3,)))
```

`jax.grad` returns the gradient of a scalar function by reverse-mode automatic differentiation (Section 2.5), at the cost of a few evaluations of the function, and `jax.jacfwd` and `jax.jacrev` return Jacobians. `jax.jit` traces the function once with abstract arrays of the given shapes, compiles the trace, and runs the compiled code from the second call on; the first call is slow, the following ones are fast. `jax.vmap` turns a function of one point into a function of a batch, without a Python loop. Random numbers come from explicit keys, which makes them reproducible and is the price of having no global state.

Tracing is the reason for the one rule of JAX: functions must be *pure*, i.e., their result may depend only on their arguments, and they may not modify anything. Python loops inside a jitted

function are unrolled during tracing, which is fine for a few iterations and slow to compile for many; `jax.lax.fori_loop` and `jax.lax.scan` compile the body once. The leapfrog integrator of HMC (Section 4.5) with unit mass matrix, which returns the energy after every step, is a typical use:

```
@jax.jit
def leapfrog(x, p, eps):                    # 20 leapfrog steps of size eps
    def step(carry, _):
        x, p = carry
        p = p - 0.5 * eps * grad_U(x)
        x = x + eps * p
        p = p - 0.5 * eps * grad_U(x)
        return (x, p), U(x) + 0.5 * p @ p   # new carry, output per step
    (x, p), energy = jax.lax.scan(step, (x, p), None, length=20)
    return x, p, energy
```

B.3 Running the samplers

A sampler needs the potential $U(\boldsymbol{x}) = -\log p(\boldsymbol{x} \mid \boldsymbol{y}) + \text{const}$, written in `jax.numpy` so that it can be differentiated. For the bathymetry problem with 20 sensors and the prior on increments with $\gamma = 1\,\text{cm}$ per cell of Example 5.4, the potential, the MAP estimate, and HMC with the Laplace covariance as inverse mass matrix (Example 4.18) look as follows:

```
import numpy as np
import jax
jax.config.update("jax_enable_x64", True)
import jax.numpy as jnp
from running_examples import make_grid, bottom_smooth, L
from solvers import difference_matrix, gauss_newton
from samplers import make_hmc_jax, elliptical_slice, ess
import swe_jax

xg, dx = make_grid(200)
xo = np.linspace(0.5, L - 0.5, 20)          # 20 sensors
F = swe_jax.make_forward(xg, xo)            # bottom (199,) -> surface (20,)
sigma, gamma = 0.01, 0.01                   # noise 1 cm, increments 1 cm
rng = np.random.default_rng(4)
b_true = bottom_smooth(xg)[:-1]
y = np.asarray(F(jnp.asarray(b_true))) + sigma * rng.standard_normal(20)
Lw = difference_matrix(200)[:, :-1] / gamma  # prior: Lw b ~ N(0, I)
Lw_j = jnp.asarray(Lw)

def U(b):                                   # -log posterior + const
    r = (F(b) - y) / sigma
    z = Lw_j @ b
    return 0.5 * (r @ r) + 0.5 * (z @ z)

J = jax.jit(jax.jacfwd(F))
F_np = lambda b: np.asarray(F(b))
J_np = lambda b: np.asarray(J(b))
b_map, _ = gauss_newton(F_np, J_np, y, np.zeros(199), lam=sigma**2, L=Lw)
Jm = J_np(b_map)
H = Jm.T @ Jm / sigma**2 + Lw.T @ Lw        # Gauss-Newton Hessian
C = np.linalg.cholesky(np.linalg.inv(H))    # Laplace covariance C C^T
step = make_hmc_jax(U, 0.3, 5, C)           # step size 0.3, 5 leapfrog steps
```

```
key, b, chain = jax.random.PRNGKey(0), jnp.asarray(b_map), []
for k in range(2000):
    key, sub = jax.random.split(key)
    b, accepted, dH = step(sub, b)
    chain.append(np.asarray(b))
chain = np.array(chain)
print(min(ess(chain[:, i]) for i in range(199)))   # about 950 of 2000
```

With the same potential, the NUTS sampler of BlackJAX [17] adapts the step size and a mass matrix during a warm-up phase and then chooses the number of leapfrog steps itself:

```
import blackjax
logdensity = lambda b: -U(b)
warmup = blackjax.window_adaptation(blackjax.nuts, logdensity)
key, wkey, skey = jax.random.split(jax.random.PRNGKey(1), 3)
(state, params), _ = warmup.run(wkey, jnp.asarray(b_map), num_steps=500)
nuts = blackjax.nuts(logdensity, **params)

def one_step(state, key):
    state, info = nuts.step(key, state)
    return state, (state.position, info.num_integration_steps)
keys = jax.random.split(skey, 1000)
_, (draws, n_leap) = jax.lax.scan(one_step, state, keys)
```

The comparison is instructive. The listing uses $\gamma = 1\,\mathrm{cm}$, not the $\gamma = 3\,\mathrm{cm}$ of Example 4.18, so that its numbers differ from those of Chapter 4. HMC with the Laplace covariance needs 5 gradients per iteration and about 10 per effective sample. NUTS with the diagonal mass matrix that `window_adaptation` adapts by default takes about 100 leapfrog steps per draw and about 300 gradients per effective sample, because a diagonal mass matrix cannot undo the strong correlations of this posterior; with `is_mass_matrix_diagonal=False`, which adapts a dense one, it takes about 24 steps per draw and 45 gradients per effective sample. The mass matrix matters more than the sampler.

Diagnostics. `samplers.ess` returns the effective sample size of a scalar chain, or of several chains of shape (C, N) together, and `samplers.rank_rhat` the rank-normalized $\widehat{R}$ of several chains (Section 4.6). Run at least four chains from different starting points, discard a warm-up, and check $\widehat{R} < 1.01$ and the effective sample sizes of every quantity that you report, not only of the components.

Elliptical slice sampling. The function `samplers.elliptical_slice` samples $p(\boldsymbol{w} \mid \boldsymbol{y}) \propto L(\boldsymbol{w})\,\mathcal{N}(\boldsymbol{w}; \mathbf{0}, \boldsymbol{I})$ and needs only the logarithm of the likelihood L, which may return `-inf` or NaN for infeasible points. For the bathymetry problem, $\boldsymbol{b} = \boldsymbol{R}\boldsymbol{w}$ with $\boldsymbol{R} = \boldsymbol{L}_w^{-1}$ turns the prior into $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$:

```
R = np.linalg.inv(Lw)
log_lik = lambda w: -0.5 * np.sum(((F(R @ w) - y) / sigma) ** 2)
chain, evals = elliptical_slice(log_lik, np.linalg.solve(R, b_map), 2000)
```

The reference can be changed to the Laplace approximation, as in Figure 4.15, which needs far fewer likelihood evaluations per effective sample.

Reproducibility. Seed every random number generator, `np.random.default_rng(seed)` in NumPy and `jax.random.PRNGKey(seed)` in JAX; save the chains together with the settings that produced them, e.g., with `np.savez_compressed`; and record the versions of the packages, since results can change with them.

B.4 Pitfalls

Most hours lost with JAX are lost to a few mistakes.

- *Single precision.* JAX computes in 32-bit floats unless `jax_enable_x64` is set before the first array is created. In single precision, Cholesky factorizations of ill-conditioned matrices fail, Gauss–Newton stalls, and HMC rejects for no visible reason. Check `x.dtype`.
- *In-place updates.* `x[i] = v` fails for JAX arrays, which are immutable; write `x = x.at[i].set(v)` or `x.at[i].add(v)`.
- *Python control flow on traced values.* Inside a jitted function, `if x > 0:` fails, because the value of `x` is not known during tracing. Use `jnp.where`, `jax.lax.cond`, or the loops `fori_loop`, `while_loop`, and `scan`.
- *NaN gradients behind `jnp.where`.* Both branches of `jnp.where` are evaluated and differentiated, so that `jnp.where(x > 0, jnp.sqrt(x), 0.0)` has a NaN gradient at negative `x`. Make the argument safe in the branch that is not taken as well, `jnp.where(x > 0, jnp.sqrt(jnp.where(x > 0, x, 1.0)), 0.0)`.
- *Recompilation and dispatch.* A jitted function is recompiled for every new shape or dtype of its arguments; keep the shapes fixed. Many small calls from a Python loop cost more in overhead than in computation; move the loop into `scan` or batch the work with `vmap`. JAX returns before the computation has finished, so call `.block_until_ready()` on the result before timing, and time the second call, not the first.
- *Random keys.* The same key gives the same numbers. Split the key before every use and never reuse a subkey.
- *NumPy inside JAX.* NumPy functions cannot handle traced arrays; inside a jitted or differentiated function, use `jax.numpy` throughout, and convert with `np.asarray` only outside, e.g., for plotting.
- *Densities instead of log densities.* A posterior density of 199 unknowns underflows to zero; work with log densities, `scipy.special.logsumexp`, and `log1p`, and evaluate $\log \Phi$ with `jax.scipy.special.log_ndtr` rather than `jnp.log(norm.cdf(x))`.
- *Inverses.* Solve linear systems with `np.linalg.solve` or a Cholesky factorization instead of forming the inverse; the listing above inverts a 199×199 matrix once, which is fine, but not in every iteration.
- *Infeasible points.* The bathymetry model returns NaN where no subcritical steady state exists. The samplers of the course treat NaN and infinite potentials as rejections; a potential of your own must do the same, and an optimizer must be kept away from such points, e.g., by step halving.

Appendix C

Solutions to the Exercises

This appendix gives solutions to the exercises of Chapters 1 to 5; the solutions to the exercises of Appendix A are in Appendix A.7. Try each exercise before reading its solution.

The solutions of the computer exercises state the approach, give a short code sketch, and report the results that the course code produces. Where the result depends on the noise realization, we say so and give the range over several realizations; your numbers will differ in the last digits. The code sketches assume that they are run in the folder `code/` after `import numpy as np` and the imports from the course modules that they name.

C.1 Inverse problems: the basics

Exercise 1.1. (a) With the substitution $\tau = s - t$ and the addition theorem,

$$\begin{aligned}(k * x)(s) &= \int_{\mathbb{R}} k(\tau)\sin\bigl(2\pi\nu(s-\tau)\bigr)\,\mathrm{d}\tau \\ &= \sin(2\pi\nu s)\int_{\mathbb{R}} k(\tau)\cos(2\pi\nu\tau)\,\mathrm{d}\tau - \cos(2\pi\nu s)\int_{\mathbb{R}} k(\tau)\sin(2\pi\nu\tau)\,\mathrm{d}\tau.\end{aligned}$$

The second integral vanishes because k is even. For the first, let $I(\omega) = \int k(\tau)\cos(\omega\tau)\,\mathrm{d}\tau$ and use $\tau k(\tau) = -\gamma^2 k'(\tau)$ and integration by parts:

$$I'(\omega) = -\int_{\mathbb{R}} \tau k(\tau)\sin(\omega\tau)\,\mathrm{d}\tau = \gamma^2\int_{\mathbb{R}} k'(\tau)\sin(\omega\tau)\,\mathrm{d}\tau = -\gamma^2\omega\, I(\omega).$$

With $I(0) = \int k = 1$ this gives $I(\omega) = \mathrm{e}^{-\gamma^2\omega^2/2}$, and $\omega = 2\pi\nu$ gives $\widehat{k}(\nu) = \mathrm{e}^{-2\pi^2\gamma^2\nu^2}$, which is (1.6).

(b) For $k \in L^1(\mathbb{R})$ let $\widehat{k}(\nu) = \int k(\tau)\mathrm{e}^{-2\pi\mathrm{i}\nu\tau}\,\mathrm{d}\tau$. The same substitution gives $(k*x)(s) = \operatorname{Im}\bigl(\widehat{k}(\nu)\,\mathrm{e}^{2\pi\mathrm{i}\nu s}\bigr) = \bigl|\widehat{k}(\nu)\bigr|\sin\bigl(2\pi\nu s + \arg\widehat{k}(\nu)\bigr)$: the response is a sine of the same frequency with amplitude $\bigl|\widehat{k}(\nu)\bigr|$. It remains to show $\widehat{k}(\nu)\to 0$. For the indicator function of an interval $[a,b]$ we have $\widehat{k}(\nu) = (\mathrm{e}^{-2\pi\mathrm{i}\nu a} - \mathrm{e}^{-2\pi\mathrm{i}\nu b})/(2\pi\mathrm{i}\nu)$, so $\bigl|\widehat{k}(\nu)\bigr| \le 1/(\pi\,|\nu|)$, and by linearity every step function has a transform that tends to zero. Given $\epsilon > 0$, choose a step function g with $\|k-g\|_{L^1} < \epsilon$. Since $\bigl|\widehat{k}(\nu) - \widehat{g}(\nu)\bigr| \le \|k-g\|_{L^1}$ for every ν, we get $\limsup_{|\nu|\to\infty}\bigl|\widehat{k}(\nu)\bigr| \le \epsilon$, and ϵ is arbitrary.

(c) Let $Kx = k*x$ on $L^2(\mathbb{R})$; it is bounded, $\|Kx\|_{L^2} \le \|k\|_{L^1}\,\|x\|_{L^2}$ by Young's inequality. By the convolution theorem and Plancherel's theorem (the Fourier transform with the normalization of (b) is unitary on L^2), $\|Kx\|_{L^2} = \bigl\|\widehat{k}\,\widehat{x}\bigr\|_{L^2}$. For $\nu > 0$ let x_ν be the function with $\widehat{x}_\nu = 2^{-1/2}\bigl(\mathbb{1}_{[\nu,\nu+1]} + \mathbb{1}_{[-\nu-1,-\nu]}\bigr)$, a real wave packet of frequency about ν with $\|x_\nu\|_{L^2} = 1$. Then $\|Kx_\nu\|_{L^2} \le \sup_{|\xi|\ge\nu}\bigl|\widehat{k}(\xi)\bigr| \to 0$ by (b). If there were a bounded operator B with $BKx = x$ for

all x, we would get $1 = \|x_\nu\| \le \|B\|\,\|Kx_\nu\| \to 0$, a contradiction. So no bounded inverse exists, not even on the range of K. Note that K is not compact on $L^2(\mathbb{R})$ (it is unitarily equivalent to the multiplication by $\widehat{k}$), so Theorem 1.8 would not even apply: the decay of $\widehat{k}$ alone makes the inverse unbounded.

Exercise 1.2. Number the cells $j = 1, \dots, n$ from the inflow, with centers x_j and surface values $\eta_j = h_j + b_j$. The energy $E = q^2/(2h_{\text{out}}^2) + g(h_{\text{out}} + b_n)$ depends only on the bottom b_n of the outflow cell, so h_j depends only on b_j and b_n, and $h_n = h_{\text{out}}$. A sensor at $x_{\text{obs}} \in [x_j, x_{j+1}]$ reads $(1-w)\eta_j + w\eta_{j+1}$ with $w = (x_{\text{obs}} - x_j)/(x_{j+1} - x_j)$.

(a) The reading depends on the bottom of the two neighboring cells j and $j+1$ (of one cell if the sensor sits at a center) and of the outflow cell n, through E; on no other cell.

(b) Write the forward map as $F(\boldsymbol{b}) = \boldsymbol{W}\boldsymbol{\eta}(\boldsymbol{b})$ with the cellwise surface $\boldsymbol{\eta}(\boldsymbol{b}) \in \mathbb{R}^n$ and the interpolation matrix $\boldsymbol{W} \in \mathbb{R}^{m\times n}$, which depends only on the geometry. Then $\boldsymbol{J}(\boldsymbol{b}) = \boldsymbol{W}\boldsymbol{S}(\boldsymbol{b})$ with $\boldsymbol{S} = \partial\boldsymbol{\eta}/\partial\boldsymbol{b} = \operatorname{diag}(s_1, \dots, s_{n-1}, 1) + \boldsymbol{c}\,\boldsymbol{e}_n^{\mathsf{T}}$, where $s_j = \partial\eta_j/\partial b_j = -\mathrm{Fr}_j^2/(1-\mathrm{Fr}_j^2)$ by (c), $c_j = \partial\eta_j/\partial b_n = g\,\partial h_j/\partial E = 1/(1-\mathrm{Fr}_j^2)$ for $j < n$, and $c_n = 0$. $\boldsymbol{S}$ has nonzero entries only on the diagonal and in the last column, and its diagonal entries are nonzero, so it is invertible, $\operatorname{rank}\boldsymbol{J} = \operatorname{rank}\boldsymbol{W}$, and $\ker\boldsymbol{J}(\boldsymbol{b}) = \boldsymbol{S}(\boldsymbol{b})^{-1}\ker\boldsymbol{W}$. For $m = 20$ equispaced sensors between $x = 0.5$ and 24.5 m the spacing is 1.26 m, about ten cells, so each sensor has its own pair of neighboring cells and each row of $\boldsymbol{W}$ has two nonzeros in its own two columns. Hence $\operatorname{rank}\boldsymbol{W} = 20$ and $\dim\ker\boldsymbol{J} = 200 - 20 = 180$. In words, the null space consists of (i) arbitrary changes of the 159 cells that are neither next to a sensor nor the outflow cell, i.e., everything between the sensors; (ii) at each sensor, the one combination of its two cells that leaves the interpolated surface unchanged (20 directions); and (iii) a change of the outflow bottom, compensated at the sensor cells (1 direction): $159 + 20 + 1 = 180$.

(c) Differentiating (1.4) with respect to b at fixed q and E gives $-(q^2/h^3)\,\partial h/\partial b + g\,(\partial h/\partial b + 1) = 0$, so

$$\frac{\partial h}{\partial b} = -\frac{g}{g - q^2/h^3} = -\frac{1}{1-\mathrm{Fr}^2}, \qquad \frac{\partial \eta}{\partial b} = 1 + \frac{\partial h}{\partial b} = -\frac{\mathrm{Fr}^2}{1-\mathrm{Fr}^2},$$

which is (1.9); it is finite and nonzero for subcritical flow, $0 < \mathrm{Fr} < 1$. The nonlinearity of F acts cell by cell, and the only coupling between cells, the interpolation $\boldsymbol{W}$, is linear and independent of $\boldsymbol{b}$. Therefore $\ker\boldsymbol{J}(\boldsymbol{b}) = \boldsymbol{S}(\boldsymbol{b})^{-1}\ker\boldsymbol{W}$ has dimension 180 for every subcritical $\boldsymbol{b}$, and its part (i) does not depend on $\boldsymbol{b}$ at all, because $\boldsymbol{S}^{-1}$ maps each unit vector $\boldsymbol{e}_j$ of a cell away from the sensors to a multiple of itself. More is true: a change supported on these cells leaves F exactly unchanged, whatever its size, as long as the flow stays subcritical; this part of the null space belongs to the nonlinear map and not only to its linearization. The directions (ii) and (iii) do depend on $\boldsymbol{b}$, through the ratios s_j/s_{j+1} and c_j/s_j, which change with the local Froude numbers; in our example the largest principal angle between $\ker\boldsymbol{J}(\boldsymbol{b})$ for the smooth bottom and for the flat bottom is 6°.

Exercise 1.3. The sketch computes the condition number and the least-squares errors; the least-squares solver of NumPy returns the minimum-norm solution $\boldsymbol{A}^{\dagger}\boldsymbol{y}$.

```
from running_examples import deconvolution_operator, signal_smooth
A, t, s = deconvolution_operator(128, 64, 0.03)
sv = np.linalg.svd(A, compute_uv=False)
print(sv[0] / sv[-1])                                   # kappa(A) = 3.5e7
x, rng = signal_smooth(t), np.random.default_rng(0)
for sig in [0, 1e-6, 1e-4, 1e-2, 0.05]:
    for k in range(20):
        y = A @ x + sig * rng.standard_normal(64)
        err = np.linalg.norm(np.linalg.lstsq(A, y, rcond=None)[0] - x)
```

(a) $\sigma_1 = 0.70$, $\sigma_{64} = 2.0 \cdot 10^{-8}$, and $\kappa(\boldsymbol{A}) = 3.5 \cdot 10^{7}$. For exact data the error is $5 \cdot 10^{-8}$, a rounding error of the size $\kappa(\boldsymbol{A})\, \epsilon_{\text{mach}}\, \|\boldsymbol{x}\|$. For $\sigma > 0$ the error is proportional to σ, about $4.9 \cdot 10^{7}\sigma$ on average over 20 realizations (between $1.9 \cdot 10^{7}\sigma$ and $9 \cdot 10^{7}\sigma$), i.e., about 50, $5 \cdot 10^{3}$, $5 \cdot 10^{5}$, and $2.5 \cdot 10^{6}$ for $\sigma = 10^{-6}, \ldots, 0.05$: a straight line of slope one in a log-log plot. The smooth signal lies in the row space of $\boldsymbol{A}$ up to $5 \cdot 10^{-8}$, so the error is $\boldsymbol{A}^{\dagger}\boldsymbol{\varepsilon}$, and the rectangular version of (1.7) bounds it by $\kappa(\boldsymbol{A})\, \|\boldsymbol{x}\|\, \|\boldsymbol{\varepsilon}\| \,/\, \|\boldsymbol{A}\boldsymbol{x}\| = 5.2 \cdot 10^{7}\, \|\boldsymbol{\varepsilon}\|$, or, with the first step of the proof, by $\|\boldsymbol{\varepsilon}\| \,/\sigma_{64} = 5.0 \cdot 10^{7}\, \|\boldsymbol{\varepsilon}\|$. With $\|\boldsymbol{\varepsilon}\| \approx \sqrt{m}\,\sigma = 8\sigma$ the bound is $4 \cdot 10^{8}\sigma$, eight times the observed error. The bound assumes that all of the noise points along the worst direction $\boldsymbol{u}_{64}$; white noise puts only a share of about $1/\sqrt{m}$ of its norm into each direction, and $\mathbb{E}\left\|\boldsymbol{A}^{\dagger}\boldsymbol{\varepsilon}\right\|_2^2 = \sigma^2 \sum_i \sigma_i^{-2}$ gives a root-mean-square error of $5.8 \cdot 10^{7}\sigma$, dominated by the few smallest singular values. The bound has the right scaling and order of magnitude, and it is attained only for the worst noise.

(b) For exact data, $\boldsymbol{A}^{\dagger}\boldsymbol{A}\boldsymbol{x}$ is the orthogonal projection of $\boldsymbol{x}$ onto the row space of $\boldsymbol{A}$, and the error is the component of $\boldsymbol{x}$ in the 64-dimensional null space. For the piecewise-constant signal it has norm 0.69 and its maximum 0.26 at $t = 0.152$, at the first jump. The rows of $\boldsymbol{A}$ are Gaussian bumps centered at the 64 sensors, and a vector in $\ker \boldsymbol{A}$ is orthogonal to all of them. Numerically, a sine of frequency $\nu \le 28$ lies entirely in the row space, at $\nu = 32$ a share of 77 percent of its squared norm lies in $\ker \boldsymbol{A}$, and for $\nu \ge 36$ it is 97 percent: 64 equispaced samples cannot see frequencies above the Nyquist frequency $m/2 = 32$. The smooth signal has nothing there (error $5 \cdot 10^{-8}$); the Fourier coefficients of a jump decay only like $1/\nu$, and the missing high frequencies show up as oscillations at the edges. This is a failure of uniqueness, not of stability: no method that uses only these data can recover the null-space component; it has to come from prior information.

(c) With `kappa = [np.linalg.cond(deconvolution_operator(n, n // 2, gam)[0]) for n in ns]` one finds

n	32	64	128	256	512	1024
$\gamma = 0.015$	1.0	2.2	65	$5.1 \cdot 10^{7}$	$2.4 \cdot 10^{17}$	$4.9 \cdot 10^{18}$
$\gamma = 0.03$	2.2	62	$3.5 \cdot 10^{7}$	$1.4 \cdot 10^{17}$	$7.9 \cdot 10^{19}$	$3.5 \cdot 10^{18}$
$\gamma = 0.06$	51	$1.0 \cdot 10^{7}$	$7.0 \cdot 10^{16}$	$1.2 \cdot 10^{18}$	$3.3 \cdot 10^{18}$	$4.5 \cdot 10^{19}$

With $m = n/2$ sensors the highest frequency that the data can carry is $\nu \approx m/2 = n/4$, and the smallest singular value is roughly the attenuation there, $\widehat{k}(n/4) = \exp(-\pi^2\gamma^2 n^2/8)$. Hence $\kappa(\boldsymbol{A}_n) \approx \exp(c\, n^2)$ with $c = \pi^2\gamma^2/8$; for $\gamma = 0.03$ and $n = 128$ this estimate gives $8 \cdot 10^{7}$ against the computed $3.5 \cdot 10^{7}$. The condition number depends on n and γ only through γn, the kernel width in cells: $(\gamma, n) = (0.015, 128), (0.03, 64), (0.06, 32)$ give 65, 62, and 51, so doubling γ has the effect of doubling n. Once $\sigma_{\min} < \epsilon_{\text{mach}}\sigma_1$, which happens for γn between 4 and 8, the computed smallest singular values are rounding errors of size about $\epsilon_{\text{mach}}\, \|\boldsymbol{A}\|$ or smaller, and the computed κ stalls at 10^{16} to 10^{20} and fluctuates irregularly; it then says nothing about the operator.

(d) The data live on $m = 64$ equispaced points of $[0, 1]$, so the discrete Fourier transform gives the frequencies $\nu = 0, 1, \ldots, 32$:

```
def lowpass(y, nu_c):                    # keep the frequencies nu <= nu_c
    c = np.fft.rfft(y)
    c[np.fft.rfftfreq(len(y), 1 / len(y)) > nu_c] = 0
    return np.fft.irfft(c, len(y))
nu_star = np.sqrt(2 * np.log(1 / sig)) / (2 * np.pi * 0.03)
err = np.linalg.norm(lowpass(y, nu_star) - A @ x)    # vs the blurred signal
```

For $\sigma = 0.05$ we have $\nu^* = 13.0$. Averaged over 200 realizations, the filtered data at ν^* are closer to the blurred signal than the data themselves (0.25 against $\sqrt{m}\,\sigma = 0.40$), and the error over all cutoffs is smallest at $\nu = 6$ for the smooth and at $\nu = 9$ for the piecewise-constant signal. For $\sigma = 10^{-3}$ we have $\nu^* = 19.7$, and the best cutoffs are 9 and 15. The best cutoff is in each

case the last frequency at which the Fourier coefficient of the blurred signal exceeds that of the noise. This confirms the estimate of Remark 1.9 as an upper bound: ν^* is where a component of unit amplitude disappears in the noise, and a signal whose components have amplitude $a(\nu) < 1$ disappears earlier, where $a(\nu)\widehat{k}(\nu) \approx \sigma$; the smooth signal, whose own spectrum decays fast, loses its information first. Reducing the noise fiftyfold buys six more frequencies for the piecewise-constant signal and three for the smooth one.

Exercise 1.4. We use `jacobian_fd` on the NumPy forward map with all 200 cells unknown; `jax.jacfwd` of `swe_jax.make_forward(x, xo, fixed_outflow=False)` gives the same Jacobian up to 10^{-9}.

```
from running_examples import (make_grid, bottom_smooth, surface_at_sensors,
                              jacobian_fd, L)
x, dx = make_grid(200); b = bottom_smooth(x)
for m in [20, 50, 200]:
    xo = np.linspace(0.5, L - 0.5, m)
    F = lambda bb: surface_at_sensors(bb, x, xo)
    J = jacobian_fd(F, b)
    sv = np.linalg.svd(J, compute_uv=False)
    print(m, np.sum(sv > 1e-8 * sv[0]))          # numerical rank
```

(a) The numerical ranks are 20, 50, and 194. In each case there is one large singular value, $\sigma_1 = 6.2$, 9.8, and 19.7: it belongs to the outflow cell, which changes the surface at every sensor by $c_j = 1/(1 - \mathrm{Fr}_j^2) \approx 1.3$ to 1.7, so that its column has norm about $1.4\sqrt{m}$. The other nonzero singular values lie between 0.23 and 0.50 for $m = 20$, between 0.23 and 0.62 for $m = 50$, and between 0.070 and 0.67 for $m = 200$; they are products of the local sensitivities $|s_j| \in [0.33, 0.67]$ and the singular values of the interpolation matrix $\boldsymbol{W}$ of Exercise 1.2. For $m = 200$ the sensors are 0.121 m apart on cells of width 0.125 m, so that some sensors sit midway between two cell centers and average them; $\boldsymbol{W}$ then nearly annihilates the oscillation from cell to cell, $\sigma_{\min}(\boldsymbol{W}) = 0.21$, which explains the smallest values. With sensors at the cell centers, $\boldsymbol{J}$ would be diagonal up to the outflow column, with singular values $|s_j| \in [0.33, 0.67]$ (Example 1.11). For $m = 200$ six singular values vanish (below $3 \cdot 10^{-15}$ with JAX; about 10^{-10} with finite differences, which is the finite-difference error): cells 1 to 3 lie upstream of the first sensor at $x = 0.5$ m, cells 198 and 199 lie between the last sensor at $x = 24.5$ m and the outflow cell, and the sixth direction is the outflow change (iii) of Exercise 1.2.

(b) Any change supported on cells strictly between two sensors works, e.g., the bump of Figure 1.9c:

```
xo = np.linspace(0.5, L - 0.5, 20); F = lambda bb: surface_at_sensors(bb, x, xo)
lo, hi = xo[9], xo[10]
db = 0.1 * np.exp(-((x - (lo + hi) / 2) / (0.18 * (hi - lo))) ** 2) \
     * ((x > lo + dx) & (x < hi - dx))
print(np.abs(F(b + db) - F(b)).max())                    # 0.0
```

The change in the data is exactly zero. A null vector of $\boldsymbol{J}$ that touches the two cells next to a sensor, a direction of type (ii), is invisible only to first order: scaled to 10 cm it changes the data by $3.8 \cdot 10^{-3}$ m, scaled to 1 cm by $3.6 \cdot 10^{-5}$ m.

(c) We set $\delta\boldsymbol{y} = \boldsymbol{\varepsilon}$ and compute the minimum-norm solution $\delta\boldsymbol{b} = \boldsymbol{J}^{\dagger}\boldsymbol{\varepsilon}$ by least squares, with a relative cutoff of 10^{-10} for the singular values. Over 100 realizations, the error in the bed has a root-mean-square value of 4.1 cm per cell (between 3.2 and 5.1 cm) and a maximum of about 15 cm; $\|\delta\boldsymbol{b}\|_2 \approx 4.2\,\|\boldsymbol{\varepsilon}\|_2$. The worst-case bound $\|\delta\boldsymbol{b}\|_2 \le \|\boldsymbol{\varepsilon}\|_2 / \sigma_{\min} = 14\,\|\boldsymbol{\varepsilon}\|_2$ from the proof of Lemma 1.10 (with the smallest nonzero singular value) is pessimistic by a factor of about three, while the average-case value $\sigma\big(\sum_i \sigma_i^{-2}/n\big)^{1/2} = 4.2$ cm matches the observation. The relative form of (1.7), with $\kappa = \sigma_1/\sigma_r = 19.7/0.070 \approx 280$, is of no use here, because $\|F(\boldsymbol{b})\|_2$

is dominated by the 2 m water level and says nothing about the bed. The six invisible directions are not recovered at all: the minimum-norm solution sets them to zero.

Exercise 1.5. This exercise is the first paragraph of your project proposal and has no model solution.

C.2 Deterministic regularization

Exercise 2.1. (a) For $\boldsymbol{h} \in \mathbb{R}^n$, expanding (2.2) gives

$$J_\lambda(\boldsymbol{x}+\boldsymbol{h}) = J_\lambda(\boldsymbol{x}) + \boldsymbol{h}^\mathsf{T}(\boldsymbol{A}^\mathsf{T}(\boldsymbol{A}\boldsymbol{x}-\boldsymbol{y}) + \lambda \boldsymbol{L}^\mathsf{T}\boldsymbol{L}\boldsymbol{x}) + \tfrac{1}{2}\boldsymbol{h}^\mathsf{T}\boldsymbol{H}\boldsymbol{h}, \quad \boldsymbol{H} = \boldsymbol{A}^\mathsf{T}\boldsymbol{A} + \lambda \boldsymbol{L}^\mathsf{T}\boldsymbol{L}.$$

If the gradient $\boldsymbol{g}$ in the middle term is nonzero, $\boldsymbol{h} = -\tau \boldsymbol{g}$ decreases J_λ for small $\tau > 0$; if it vanishes, $J_\lambda(\boldsymbol{x}+\boldsymbol{h}) - J_\lambda(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{h}^\mathsf{T}\boldsymbol{H}\boldsymbol{h} \geq 0$. So $\boldsymbol{x}$ is a minimizer if and only if $\boldsymbol{g} = \boldsymbol{0}$, which is (2.3). Minimizers always exist: $\ker \boldsymbol{H} = \ker \boldsymbol{A} \cap \ker \boldsymbol{L} \subset \ker \boldsymbol{A}$, hence $\operatorname{range} \boldsymbol{A}^\mathsf{T} \subset (\ker \boldsymbol{H})^\perp = \operatorname{range} \boldsymbol{H}$, and the normal equations are solvable. Sufficiency of $\ker \boldsymbol{A} \cap \ker \boldsymbol{L} = \{\boldsymbol{0}\}$ for uniqueness is Lemma 2.3. Necessity: if $\boldsymbol{z} \neq \boldsymbol{0}$ lies in both null spaces, then $J_\lambda(\boldsymbol{x}_\lambda + \tau \boldsymbol{z}) = J_\lambda(\boldsymbol{x}_\lambda)$ for all $\tau \in \mathbb{R}$, and the minimizers form at least a line.

(b) On the whole line the convolution maps $\mathrm{e}^{2\pi \mathrm{i} \nu t}$ to $\widehat{k}(\nu)\mathrm{e}^{2\pi \mathrm{i} \nu t}$ (Exercise 1.1), and so does its adjoint, the convolution with $k(-\cdot) = k$; $\widehat{k}$ is real because k is even. The normal equations with $\boldsymbol{L} = \boldsymbol{I}$ therefore read $(\widehat{k}(\nu)^2 + \lambda)\,\widehat{x}_\lambda(\nu) = \widehat{k}(\nu)\,\widehat{y}(\nu)$ for every frequency, and inserting $\widehat{y} = \widehat{k}\,\widehat{x}^\dagger + \widehat{\varepsilon}$ gives (2.4). For $a = \widehat{k}(\nu) \geq 0$ the inequality $a^2 + \lambda \geq 2a\sqrt{\lambda}$, i.e., $(a - \sqrt{\lambda})^2 \geq 0$, gives $\varphi(\nu)/\widehat{k}(\nu) = a/(a^2+\lambda) \leq 1/(2\sqrt{\lambda})$, with equality at the cutoff frequency, where $\widehat{k}(\nu)^2 = \lambda$.

(c) On the grid with spacing $1/n$, the difference matrix maps $\mathrm{e}^{2\pi \mathrm{i} \nu t_j}$ to $(\mathrm{e}^{2\pi \mathrm{i} \nu/n} - 1)\,\mathrm{e}^{2\pi \mathrm{i} \nu t_j}$, so $\boldsymbol{L}^\mathsf{T}\boldsymbol{L}$ multiplies the component at frequency ν by $\widehat{\ell}(\nu)^2 = \left|\mathrm{e}^{2\pi \mathrm{i} \nu/n} - 1\right|^2 = 4\sin^2(\pi\nu/n) \approx (2\pi\nu/n)^2$ (boundaries ignored). The analogue of (2.4) is

$$\widehat{x}_\lambda(\nu) = \varphi_{\boldsymbol{L}}(\nu)\,\widehat{x}^\dagger(\nu) + \frac{\varphi_{\boldsymbol{L}}(\nu)}{\widehat{k}(\nu)}\,\widehat{\varepsilon}(\nu), \quad \varphi_{\boldsymbol{L}}(\nu) = \frac{\widehat{k}(\nu)^2}{\widehat{k}(\nu)^2 + \lambda\,\widehat{\ell}(\nu)^2}.$$

Now $\varphi_{\boldsymbol{L}}(0) = 1$ and $1 - \varphi_{\boldsymbol{L}}(\nu) \approx \lambda(2\pi\nu/n)^2$ for small ν: constants pass unchanged and low frequencies have almost no bias, whereas $\boldsymbol{L} = \boldsymbol{I}$ shrinks every frequency, the constant one included, by the factor $1/(1+\lambda)$ at least. The cutoff lies where $\widehat{k}(\nu) = \sqrt{\lambda}\,\widehat{\ell}(\nu)$, and since $\widehat{\ell} \ll 1$ at low frequencies, the values of λ for the two penalties are not comparable (the discrepancy principle picks $\lambda = 3$ for the difference matrix and 0.045 for the identity in Exercise 2.3).

Exercise 2.2. Write $D(\boldsymbol{x}) = \frac{1}{2}\,\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2$, $P(\boldsymbol{x}) = \frac{1}{2}\,\|\boldsymbol{L}\boldsymbol{x}\|_2^2$, and $D_\lambda = D(\boldsymbol{x}_\lambda)$, $P_\lambda = P(\boldsymbol{x}_\lambda)$, and let $\lambda < \mu$. Minimality gives $D_\lambda + \lambda P_\lambda \leq D_\mu + \lambda P_\mu$ and $D_\mu + \mu P_\mu \leq D_\lambda + \mu P_\lambda$. Adding the two inequalities gives $(\mu - \lambda)(P_\mu - P_\lambda) \leq 0$, i.e., $P_\mu \leq P_\lambda$, and then the first one gives $D_\lambda - D_\mu \leq \lambda(P_\mu - P_\lambda) \leq 0$, i.e., $D_\lambda \leq D_\mu$. The square roots inherit the monotonicity; the argument uses neither linearity nor the quadratic form of P and holds for every regularizer, total variation included.

For the discrepancy principle we need continuity and strict monotonicity. The map $\lambda \mapsto \boldsymbol{x}_\lambda = \boldsymbol{H}_\lambda^{-1}\boldsymbol{A}^\mathsf{T}\boldsymbol{y}$ is continuous on $(0,\infty)$, and so is the discrepancy. Suppose $D_\lambda = D_\mu$ for some $\lambda < \mu$. Then the first inequality gives $P_\lambda \leq P_\mu$, so $P_\lambda = P_\mu$ and $J_\lambda(\boldsymbol{x}_\mu) = J_\lambda(\boldsymbol{x}_\lambda)$; by uniqueness $\boldsymbol{x}_\mu = \boldsymbol{x}_\lambda =: \boldsymbol{x}$. Subtracting the two normal equations gives $(\mu - \lambda)\boldsymbol{L}^\mathsf{T}\boldsymbol{L}\boldsymbol{x} = \boldsymbol{0}$, so $\boldsymbol{L}\boldsymbol{x} = \boldsymbol{0}$ and $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^\mathsf{T}\boldsymbol{y}$; but then $\boldsymbol{x}$ solves the normal equations for every $\lambda > 0$, and the discrepancy is constant on all of $(0,\infty)$. So the discrepancy is either constant or strictly increasing, and it is constant exactly when the normal equations $\boldsymbol{A}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^\mathsf{T}\boldsymbol{y}$ have a solution

in $\ker \boldsymbol{L}$. For $\boldsymbol{L} = \boldsymbol{I}$ the SVD shows this directly: with $\beta_i = \boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{y}$, $\boldsymbol{x}_\lambda = \sum_{\sigma_i>0} \sigma_i\beta_i/(\sigma_i^2+\lambda)\,\boldsymbol{v}_i$ and $\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2^2 = \sum_{\sigma_i>0}(\lambda/(\sigma_i^2+\lambda))^2\beta_i^2 + \left\|\boldsymbol{y} - \boldsymbol{A}\boldsymbol{A}^\dagger\boldsymbol{y}\right\|_2^2$. Every term of the sum is strictly increasing in λ when $\beta_i \neq 0$, so the discrepancy is strictly increasing if and only if $\beta_i \neq 0$ for some $\sigma_i > 0$, i.e., $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{y} \neq \boldsymbol{0}$, which is the condition above for $\ker \boldsymbol{L} = \{\boldsymbol{0}\}$. For general $\boldsymbol{L}$ the generalized SVD of $(\boldsymbol{A}, \boldsymbol{L})$ gives the same sum with $\lambda/(\gamma_i^2+\lambda)$, γ_i the generalized singular values, and the components in $\ker \boldsymbol{L}$ drop out of it. As $\lambda \to 0$ it tends to the discrepancy of the naive (least-squares) solution, which is zero for $m < n$ and full row rank; as $\lambda \to \infty$ it tends to that of the best fit in $\ker \boldsymbol{L}$, which for $\boldsymbol{L} = \boldsymbol{I}$ is $\boldsymbol{x} = \boldsymbol{0}$ with discrepancy $\|\boldsymbol{y}\|_2$. By the intermediate value theorem, every level $\tau\sqrt{m}\,\sigma$ strictly between the two limits is attained, and by strict monotonicity at exactly one λ.

Exercise 2.3. We sweep λ, record the error, the discrepancy, $\|\boldsymbol{L}\boldsymbol{x}_\lambda\|_2$, and the GCV function (2.6), find the discrepancy-principle value by root finding, and take the L-curve corner as the point of maximal curvature of $(\log\|\boldsymbol{A}\boldsymbol{x}_\lambda - \boldsymbol{y}\|_2, \log\|\boldsymbol{L}\boldsymbol{x}_\lambda\|_2)$ as a function of $\log\lambda$, as `ch02_figures.py` does.

```
from running_examples import deconvolution_operator, signal_smooth
from solvers import tikhonov, difference_matrix
A, t, s = deconvolution_operator(); x = signal_smooth(t)
y = A @ x + 0.05 * np.random.default_rng(1).standard_normal(64)
for L in [np.eye(128), difference_matrix(128)]:
    for lam in np.logspace(-7, 2, 181):
        xl = tikhonov(A, y, lam, L)
        H = A.T @ A + lam * L.T @ L
        trS = np.trace(A @ np.linalg.solve(H, A.T))      # tr S_lambda
        err, res = np.linalg.norm(xl - x), np.linalg.norm(A @ xl - y)
        gcv = 64 * res**2 / (64 - trS) ** 2
```

(a) For the data of the chapter (the first noise vector of `default_rng(1)`), $\boldsymbol{L} = \boldsymbol{I}$: the optimum is $\lambda = 0.033$ with error 0.58; the discrepancy principle gives $\lambda = 0.045$ by root finding (0.044 on the grid of Figure 2.5) and error 0.60, three to four percent above the optimum; the L-curve corner lies at $\lambda \approx 0.0028$ with error 1.4 (the corner moves slightly with the λ-grid); GCV gives $\lambda = 0.0071$ and 0.98. These are the values of Figure 2.5.

(b) With the difference matrix the optimum is $\lambda = 0.77$ with error 0.30, half of the best error for $\boldsymbol{L} = \boldsymbol{I}$: the smoothness penalty does not pull the signal towards zero but only penalizes its oscillation, which is the right grammar for the smooth signal. Here the discrepancy principle gives $\lambda = 3.0$ by root finding and error 0.49; Chapter 3 uses the rounded value $\lambda = 2.9$ ($\gamma = 0.029$), with error 0.48, while the L-curve and GCV both give $\lambda = 0.45$ and error 0.32.

(c) Over ten noise realizations (our seeds; the numbers vary with them), for $\boldsymbol{L} = \boldsymbol{I}$ the optimal error is 0.65 on average, and the rules reach on average 1.06 times the optimal error (discrepancy principle, between 1.00 and 1.19), 1.85 times (L-curve, between 1.47 and 2.19), and 5.4 times (GCV). The GCV average is spoiled by one realization in which its curve is flat and its minimum lies at a tiny λ, with 42 times the optimal error; its median is 1.41. For the difference matrix the factors are 1.22 (discrepancy), 1.13 (L-curve), and 1.23 (GCV, median 1.11, at most 1.97). No rule is best in both settings, the discrepancy principle is the most reliable one for $\boldsymbol{L} = \boldsymbol{I}$ when σ is known, and GCV occasionally fails badly.

(d) With 2σ in place of σ, the discrepancy principle oversmooths: $\lambda = 0.15$ and error 1.19 for $\boldsymbol{L} = \boldsymbol{I}$, $\lambda = 17$ and error 1.28 for the difference matrix, two and four times the optimum. With $\sigma/2$ it has to fit the data to $\sqrt{m}\,\sigma/2 = 0.2$, half the norm of the actual noise, and it can only do so by fitting the noise: $\lambda = 4.5 \cdot 10^{-6}$ and $5.9 \cdot 10^{-6}$, and errors of 34 in both cases. Underestimating the noise level is much worse than overestimating it, because the error curve is much steeper on the side of small λ, where the amplified noise grows like $1/\sqrt{\lambda}$, than on the side

of large λ, where the bias saturates.

Exercise 2.4. We use the data of Exercise 2.3 and choose λ by root finding on the discrepancy:

```
from solvers import l1_regularization
from scipy.optimize import brentq
tv = lambda y, lam: l1_regularization(A, y, lam, transform="differences",
                                      n_iter=4000)[0]
f = lambda v: np.linalg.norm(A @ tv(y, 10**v) - y) - np.sqrt(64) * sig
lam = 10 ** brentq(f, -4, 1.5, xtol=1e-3)              # discrepancy principle
x_tv = tv(y, lam)
```

(a) For the piecewise-constant data of the chapter, the error of the total-variation solution is smallest on the grid `np.logspace(-3, 0.5, 15)` at $\lambda = 0.1$ (0.749), and the discrepancy principle gives $\lambda = 0.11$ with error 0.75. The best Tikhonov solutions have errors 1.51 ($\boldsymbol{L} = \boldsymbol{I}$, $\lambda = 0.018$; the discrepancy principle gives the same) and 1.45 (difference matrix, $\lambda = 0.089$): total variation halves the error. Tikhonov smears every edge over about eight cells and overshoots and rings next to it. Total variation puts every edge within one cell of its true position, spread over two to three cells: the jumps at $t = 0.15$, 0.4, and 0.85 have 75, 74, and 66 percent of their height in a single cell, the jump of height 0.5 at $t = 0.55$ is reconstructed one cell late, and the jump of height 0.3 at $t = 0.65$ is reconstructed with height 0.2. The plateaus are flat, and their levels are within 0.07 of the truth, the high ones slightly too low and the zero levels slightly too high: the loss of contrast that the ℓ^1 penalty causes.

(b) For the smooth signal the discrepancy principle gives $\lambda = 0.17$ and an error of 1.06, against 0.60 for Tikhonov with $\boldsymbol{L} = \boldsymbol{I}$ and 0.49 with the difference matrix. The solution is a staircase with 13 nonzero increments, grouped into seven steps: the narrow bump becomes a box of height 0.79 (truth: 1) between $t \approx 0.22$ and 0.37, the wide bump a plateau at 0.48 (truth: 0.6) with flanks of two steps each. The penalty charges a rise by its height, not by its shape, so it does not prefer a ramp over a staircase, and the corners of the ℓ^1 ball make most increments exactly zero; peaks are cut off, because lowering a peak by δ saves $2\lambda\delta$ of penalty.

(c) We repeat (a) for 20 noise realizations per noise level and call the plateau of height 0.5 on $(0.55, 0.65]$ resolved if the median of the reconstruction on its interior lies at least 0.15 above the level to its left and at least 0.1 below the plateau of height 0.8 to its right. It is resolved in 20, 19, 10, 7, and 4 of 20 realizations for $\sigma = 0.05$, 0.1, 0.15, 0.2, and 0.3, with mean errors 0.93, 1.49, 1.90, 2.24, and 2.85; the discrepancy-principle λ grows roughly in proportion to σ. So total variation loses the smaller plateau at about $\sigma \approx 0.15$, three times the default noise level, and at $\sigma = 0.2$ in two thirds of the cases: the plateau is merged with the plateau of height 0.8 or with the zero level next to it, and the reconstruction has a single jump near $t = 0.55$. The reason is the price of a short plateau: keeping it costs two jumps, merging it costs one, and the misfit saved by keeping it shrinks relative to the noise.

Exercise 2.5. The sketch follows `ch02_figures.py`, which keeps $b_n = 0$ fixed and has 199 unknowns:

```
import jax, jax.numpy as jnp, swe_jax
from running_examples import make_grid, bottom_smooth, L
from solvers import difference_matrix, gauss_newton
xg, dx = make_grid(200); b_true = bottom_smooth(xg)
xo = np.linspace(0.5, L - 0.5, 20)                     # or 200 sensors
fwd = swe_jax.make_forward(xg, xo); jac = jax.jit(jax.jacfwd(fwd))
F = lambda b: np.asarray(fwd(jnp.asarray(b)))
Jf = lambda b: np.asarray(jac(jnp.asarray(b)))
y = F(b_true[:-1]) + 0.01 * np.random.default_rng(0).standard_normal(20)
for lam in np.logspace(-2, 1, 13):                     # discrepancy principle
```

```
b, hist = gauss_newton(F, Jf, y, np.zeros(199), lam=lam,
                       L=difference_matrix(199), n_iter=40)
```

(a) With the noise of `ch02_figures.py` (another seed changes the numbers slightly), the discrepancy principle picks $\lambda = 5.6$ for $m = 200$ and $\lambda = 1$ for $m = 20$; Gauss–Newton stops after 5 and 4 iterations, and almost all of the decrease of J_λ happens in the first one ($0.101 \to 0.020 \to 0.0177$ for $m = 200$). The root-mean-square errors are 1.33 cm and 2.06 cm, against 3.84 cm and 6.61 cm without the penalty, as in Example 2.9. For $m = 20$ the error is about the same at the sensor cells (1.9 cm) and between the sensors (2.1 cm): the penalty fills the gaps well because the true bottom is smooth.

(b) With $\boldsymbol{L} = \boldsymbol{I}$ and $m = 20$ the discrepancy principle gives $\lambda = 0.056$ and an error of 6.6 cm, as large as without any penalty. The reconstruction consists of spikes of up to 18 cm at the 40 cells next to the sensors and is exactly zero at the other 159 cells: the data do not see those cells, and the penalty $\|\boldsymbol{b}\|_2^2$ sets every invisible component to its cheapest value, zero, which is the minimum-norm solution of Chapter 1. The difference penalty couples neighboring cells, so that the bottom between the sensors is interpolated from the values at the sensors. For $m = 200$, where every cell is seen, $\boldsymbol{L} = \boldsymbol{I}$ works (2.3 cm against 1.3 cm) but shrinks the hills towards zero.

(c) The starts $\boldsymbol{b}_0 = \boldsymbol{b}^\dagger - 0.5\,\mathrm{m}$, a flat bottom at $-0.5\,\mathrm{m}$, the hills shifted 4 m downstream, and the two hills swapped (0.12 and 0.20 m high instead of 0.20 and 0.12 m) all converge in 4 to 6 iterations to the same minimizer (to within 10^{-5} m, with the same J_λ), for both sensor layouts. The start $\boldsymbol{b}^\dagger + 0.5\,\mathrm{m}$ is infeasible: above the crest the energy E is smaller than the energy of critical flow, no subcritical steady state exists, the forward map returns NaN, and Gauss–Newton cannot start; the starting point must lie in the domain of F. That every feasible start ends in the same minimum is plausible, since F acts cell by cell, is monotone in every cell ($s_j < 0$), and is only mildly nonlinear.

(d) For $m = 200$ and the true bottom, the finite-difference Jacobian of the same (JAX) forward map differs from `jax.jacfwd` by at most $4 \cdot 10^{-10}$ (relative 10^{-9}). Its error is the sum of the truncation error, $O(\epsilon^2)$, and the rounding error, $O(\epsilon_{\mathrm{mach}}/\epsilon)$: $5 \cdot 10^{-8}$, $4 \cdot 10^{-10}$, $6 \cdot 10^{-8}$, and $4 \cdot 10^{-6}$ for $\epsilon = 10^{-4}$, 10^{-6}, 10^{-8}, and 10^{-10}; automatic differentiation is exact up to rounding. On our machine the jitted `jacfwd` takes 3 ms per Jacobian after a compilation of about 3 s, while finite differences need $2 \cdot 199$ forward evaluations and take 0.5 s with the jitted JAX forward map and 0.8 s with the NumPy one (timings depend on the machine). These finite-difference timings are dominated by the overhead of 398 separate calls from Python; measured inside one jitted loop, a Jacobian by `jacfwd` costs about as much as 200 forward solves, one per unknown, like finite differences (Section 4.7). The cost of finite differences grows linearly with n, and for an expensive forward model only the adjoint or reverse mode is affordable.

Exercise 2.6. This exercise belongs to your project proposal and has no model solution.

C.3 Bayesian inverse problems and MAP estimation

Exercise 3.1. Let $Q(\boldsymbol{x})$ be the left-hand side, $\boldsymbol{H} = \boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{A} + \boldsymbol{\Gamma}_{\mathrm{prior}}^{-1}$, and $\boldsymbol{g} = \boldsymbol{A}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{y} + \boldsymbol{\Gamma}_{\mathrm{prior}}^{-1}\boldsymbol{\mu}$. Expanding both quadratic forms,

$$Q(\boldsymbol{x}) = \tfrac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{H}\boldsymbol{x} - \boldsymbol{x}^\mathsf{T}\boldsymbol{g} + \tfrac{1}{2}(\boldsymbol{y}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{y} + \boldsymbol{\mu}^\mathsf{T}\boldsymbol{\Gamma}_{\mathrm{prior}}^{-1}\boldsymbol{\mu}).$$

$\boldsymbol{H}$ is symmetric positive definite, so with $\boldsymbol{\mu}_{\mathrm{post}} = \boldsymbol{H}^{-1}\boldsymbol{g}$ we have $\tfrac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{H}\boldsymbol{x} - \boldsymbol{x}^\mathsf{T}\boldsymbol{g} = \tfrac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_{\mathrm{post}})^\mathsf{T}\boldsymbol{H}(\boldsymbol{x} - \boldsymbol{\mu}_{\mathrm{post}}) - \tfrac{1}{2}\boldsymbol{\mu}_{\mathrm{post}}^\mathsf{T}\boldsymbol{H}\boldsymbol{\mu}_{\mathrm{post}}$, which proves the identity with $\boldsymbol{\Gamma}_{\mathrm{post}} = \boldsymbol{H}^{-1}$ and

$$c = \tfrac{1}{2}(\boldsymbol{y}^\mathsf{T}\boldsymbol{\Gamma}^{-1}\boldsymbol{y} + \boldsymbol{\mu}^\mathsf{T}\boldsymbol{\Gamma}_{\mathrm{prior}}^{-1}\boldsymbol{\mu} - \boldsymbol{\mu}_{\mathrm{post}}^\mathsf{T}\boldsymbol{\Gamma}_{\mathrm{post}}^{-1}\boldsymbol{\mu}_{\mathrm{post}}) = \tfrac{1}{2}(\boldsymbol{y} - \boldsymbol{A}\boldsymbol{\mu})^\mathsf{T}(\boldsymbol{\Gamma} + \boldsymbol{A}\boldsymbol{\Gamma}_{\mathrm{prior}}\boldsymbol{A}^\mathsf{T})^{-1}(\boldsymbol{y} - \boldsymbol{A}\boldsymbol{\mu}).$$

For the second form, substitute $\boldsymbol{x} = \boldsymbol{\mu} + \boldsymbol{z}$ and $\boldsymbol{r} = \boldsymbol{y} - \boldsymbol{A}\boldsymbol{\mu}$, so that $Q = \tfrac{1}{2}\boldsymbol{z}^{\mathsf{T}}\boldsymbol{H}\boldsymbol{z} - \boldsymbol{z}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{r} + \tfrac{1}{2}\boldsymbol{r}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1}\boldsymbol{r}$, whose minimum is $c = \tfrac{1}{2}\boldsymbol{r}^{\mathsf{T}}(\boldsymbol{\Gamma}^{-1} - \boldsymbol{\Gamma}^{-1}\boldsymbol{A}\boldsymbol{H}^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{\Gamma}^{-1})\boldsymbol{r}$; the matrix in parentheses is $(\boldsymbol{\Gamma} + \boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^{\mathsf{T}})^{-1}$ by the Sherman–Morrison–Woodbury formula. Hence $p(\boldsymbol{x} \mid \boldsymbol{y}) \propto \mathrm{e}^{-Q(\boldsymbol{x})} \propto \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_{\text{post}}, \boldsymbol{\Gamma}_{\text{post}})$, which is Theorem 3.10, and the constant is the evidence: integrating e^{-Q} with the normalizations of the two Gaussians and the identity $\det(\boldsymbol{\Gamma} + \boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^{\mathsf{T}}) = \det\boldsymbol{\Gamma}\,\det\boldsymbol{\Gamma}_{\text{prior}}\,\det\boldsymbol{H}$ gives $p(\boldsymbol{y}) = \mathcal{N}(\boldsymbol{y}; \boldsymbol{A}\boldsymbol{\mu}, \boldsymbol{\Gamma} + \boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^{\mathsf{T}})$.

For Figure 3.2a, $\boldsymbol{A} = 1$, $\boldsymbol{\Gamma} = 0.25$, $\boldsymbol{\Gamma}_{\text{prior}} = 1$, $\boldsymbol{\mu} = 0$, and $y = 1.5$: $\boldsymbol{\Gamma}_{\text{post}} = (1/0.25 + 1)^{-1} = 0.2$, $\mu_{\text{post}} = 0.2 \cdot 1.5/0.25 = 1.2$, posterior standard deviation 0.45, and $c = \tfrac{1}{2} \cdot 1.5^2/1.25 = 0.9$, so that $p(y) = \mathrm{e}^{-0.9}/\sqrt{2\pi \cdot 1.25} = 0.145$. The posterior mean is 80 percent of the way from the prior mean to the datum, because the datum is four times as precise as the prior.

Exercise 3.2. (a) By (3.7) and (3.9),

$$-\log p(\boldsymbol{x} \mid \boldsymbol{y}) = \frac{1}{2\sigma^2}\,\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \frac{1}{\gamma}\,\|\boldsymbol{x}\|_1 + \text{const.}$$

Multiplying by $\sigma^2 > 0$ does not change the minimizer, so

$$\boldsymbol{x}_{\text{MAP}} = \arg\min_{\boldsymbol{x}} \left\{ \tfrac{1}{2}\,\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2 + \frac{\sigma^2}{\gamma}\,\|\boldsymbol{x}\|_1 \right\},$$

which is (2.7) with $\boldsymbol{L} = \boldsymbol{I}$ and $\lambda = \sigma^2/\gamma$; the total-variation prior gives the same with the difference matrix.

(b) With $a = \beta + \tfrac{1}{2}$ and $b = \vartheta + x^2/2$, and the substitution $u = 1/\theta$ in $\int_0^\infty \theta^{-a-1}\mathrm{e}^{-b/\theta}\,\mathrm{d}\theta = \Gamma(a)\,b^{-a}$,

$$\begin{aligned} p(x) &= \int_0^\infty \frac{\mathrm{e}^{-x^2/(2\theta)}}{\sqrt{2\pi\theta}}\,\frac{\vartheta^\beta}{\Gamma(\beta)}\,\theta^{-\beta-1}\mathrm{e}^{-\vartheta/\theta}\,\mathrm{d}\theta = \frac{\vartheta^\beta\,\Gamma(\beta+\tfrac{1}{2})}{\Gamma(\beta)\sqrt{2\pi}}\Big(\vartheta + \frac{x^2}{2}\Big)^{-\beta-1/2} \\ &= \frac{\Gamma(\beta+\tfrac{1}{2})}{\Gamma(\beta)\sqrt{2\pi\vartheta}}\Big(1 + \frac{x^2}{2\vartheta}\Big)^{-\beta-1/2}, \end{aligned}$$

which is (3.12). The Student-t density with ν degrees of freedom and scale s is proportional to $(1 + x^2/(\nu s^2))^{-(\nu+1)/2}$; matching the exponents gives $\nu = 2\beta$, and matching $\nu s^2 = 2\vartheta$ gives $s = \sqrt{\vartheta/\beta}$. The normalizing constants agree as well. For $\beta = 1$ and $\vartheta = 10^{-3}$ this is a t_2 density with scale 0.032; for $\beta = \tfrac{1}{2}$ it is the Cauchy density with scale $\sqrt{2\vartheta}$.

(c) For fixed x_i the bracket in (3.16) is $f(\theta) = a/\theta + b\log\theta$ with $a = x_i^2/2 + \vartheta > 0$ and $b = \beta + \tfrac{3}{2} > 0$. Its derivative $f'(\theta) = (b\theta - a)/\theta^2$ is negative for $\theta < a/b$ and positive for $\theta > a/b$, and $f \to \infty$ for $\theta \to 0$ and $\theta \to \infty$, so $\theta_i = a/b = (x_i^2/2 + \vartheta)/(\beta + \tfrac{3}{2})$ is the unique minimizer. Inserting it gives $f(\theta_i) = b\log(1 + x_i^2/(2\vartheta)) + \text{const}$, the logarithmic penalty used in Example 3.15.

Exercise 3.3. With `A`, the smooth signal `x`, and its data `y` as in Exercise 2.3:

```
from solvers import gaussian_posterior
from priors import gaussian_precision
gam = 0.05 / np.sqrt(2.9)       # lambda = 2.9: discrepancy principle, L = diff.
P = gaussian_precision(128, gam, "differences", gamma0=1.0)
mu, C = gaussian_posterior(A, y, 0.05**2, P)
sd = np.sqrt(np.diag(C))
err = np.linalg.norm(mu - x)
coverage = np.mean(np.abs(mu - x) < 1.96 * sd)
```

(a) With the data of the chapter, $\gamma = 0.029$, the error of the posterior mean is 0.48, the posterior standard deviation lies between 0.037 and 0.058, and the truth lies inside the 95 % band at 93 percent of the points, as in Example 3.12.

(b) Over 100 realizations (with the same γ), the coverage is 89 to 90 percent for the smooth signal, depending on the seed, and 58 percent for the piecewise-constant one (between 51 and 62 percent for individual realizations). A 95 % band covers 95 percent of the points on average over truths drawn from the prior; for one fixed truth it does so only if the truth looks like a typical prior sample. The smooth signal nearly does: its increments are at most about 0.04, comparable to γ, and the slight undercoverage is the bias of the posterior mean at the peaks, which the band does not fully account for (the discrepancy principle smooths more than the optimum). The piecewise-constant signal does not: its jumps are 10 to 34 times γ, increments that the prior considers impossible, so the posterior mean smears every edge while the band stays 0.15 to 0.23 wide. For the data of the chapter the truth lies outside the band on seven to thirteen cells around each jump (51 of 128 points). The band accounts for the noise, but not for a wrong prior.

(c) With γ three times too large (0.088, $\lambda = 0.32$), the posterior standard deviation is 0.08 to 0.12, the coverage is 100 percent for the smooth signal (100 percent on average over 100 realizations) and 84 percent on average for the piecewise-constant one, and the error even drops to 0.35, because λ moves towards the optimum 0.77 of Exercise 2.3. With γ three times too small (0.0098, $\lambda = 26$), the standard deviation is 0.019 to 0.029, the error rises to 1.53, and the coverage falls to 23 percent (21 percent on average) for the smooth and to 3 percent for the piecewise-constant signal. Too large a prior scale gives conservative bands; too small a scale gives bands that are narrow and wrong, which is the dangerous direction.

Exercise 3.4. All samples come from `priors.py`:

```
import priors
rng = np.random.default_rng(0)
X_lap = priors.sample_laplace(128, 0.1, "identity", 1000, rng)
X_hie = priors.sample_gsbl(128, 1.0, 1e-3, "identity", 1000, rng)
print(np.mean(np.abs(X_lap) < 1e-3), np.mean(np.abs(X_hie) < 1e-3))
Z = np.diff(priors.sample_gsbl(128, 1.0, 1e-3, "differences", 1000, rng))
print(np.mean(np.abs(Z) > 0.1), np.mean(np.sum(np.abs(Z) > 0.1, axis=1)))
X2 = priors.sample_gaussian(128, 1e-3, "second-differences", 3, rng)
Xse = priors.sample_gaussian_cov(priors.se_covariance(t, 1.0, 0.1), 3, rng)
```

(a) Below 10^{-3} in absolute value are 1.0 percent of the Laplace components (exactly $1 - \mathrm{e}^{-0.01} = 0.995$ percent) and 2.2 percent of the hierarchical ones (exactly 2.24 percent for the t_2 marginal of Exercise 3.2 with scale $\sqrt{\vartheta} = 0.032$): this count hardly tells the priors apart. The share of the squared norm does: the largest 5 percent of the components (six of 128) carry on average 64 percent of it for the hierarchical prior, against 40 percent for the Laplace and 26 percent for the Gaussian prior, while the median component of a hierarchical sample is only 0.026 in absolute value. The hierarchical prior is sparse in the sense of Figure 3.5, a few loud components on a quiet background. Neither prior produces exact zeros, because both have a density on $\mathbb{R}$, and a random variable with a density takes any single value, zero included, with probability zero; exact zeros occur only in MAP estimates (Remark 3.16).

(b) Increments larger than 0.1 in absolute value make up 31.7 percent of the Gaussian increments ($\gamma = 0.1$; exactly $2(1 - \Phi(1)) = 0.317$), 36.9 percent of the Laplace increments ($\mathrm{e}^{-1} = 0.368$), and 8.8 percent of the hierarchical ones ($1 - \tau/\sqrt{2 + \tau^2} = 0.087$ with $\tau = 0.1/\sqrt{\vartheta}$). Per sample of 127 increments these are 40 (between 32 and 49 for 90 percent of the samples), 47 (39 to 55), and 11 (6 to 17) "jumps". For the Gaussian and the Laplace prior these jumps are simply the steps of a random walk and occur everywhere; for the hierarchical prior they are few and large, and the five largest increments carry 26 percent of the total variation, against 12 and 16 percent.

(c) Samples of the prior on second differences are integrated random walks: smooth, but not stationary. They start at zero with zero slope, and their standard deviation grows like $t^{3/2}$: for $\gamma = 10^{-3}$ it is 0.10, 0.29, and 0.81 at $t = 0.25$, 0.5, and 1, in agreement with $\gamma\big(\sum_{k=1}^{n-2} k^2\big)^{1/2} \approx \gamma n^{3/2}/\sqrt{3} = 0.82$ at the end, so that γ has to shrink like $n^{-3/2}$ to keep the amplitude fixed under refinement. Samples with the squared-exponential covariance ($\gamma = 1$, $\ell = 0.1$) are stationary, with standard deviation 1 everywhere, and vary on the length scale ℓ. Both look smooth, unlike the random walks of Figure 3.4d: the ratio of the mean absolute second difference to the mean absolute first difference is 0.18 and 0.14, against 1.40 for the random walk, and the static of Figure 3.4a is rougher still.

(d) With the scale $\gamma = 0.1$ of each increment fixed, the samples of the total-variation prior for $n = 1024$ are random walks whose amplitude grows like $\gamma\sqrt{2n}$ (standard deviation of x_n: 1.6 for $n = 128$, 4.5 for $n = 1024$). The occasional larger step disappears from view: the five largest increments carry 16 percent of the total variation for $n = 128$ and 3 percent for $n = 1024$, and the largest increment shrinks from 0.26 to 0.12 of the range of the sample. With $\gamma = 0.1\,(127/(n-1))^{1/2}$, i.e., at fixed amplitude, the largest increment shrinks in the same way (to 0.12 of the range), and the samples converge to Brownian motion by the central limit theorem (x_n is then Gaussian, with excess kurtosis 0.08 and -0.06); this is the loss of the edge-preserving character found by Lassas and Siltanen [74] (Remark 3.9). The hierarchical prior keeps its jumps under the same refinement: its largest increment is 0.42 and 0.36 of the range.

Exercise 3.5. With the piecewise-constant signal `x_true` and its data `y` as in Example 3.15:

```
from solvers import ias
for beta in [0.5, 1.0, 2.0]:
    for vt in [1e-4, 1e-3, 1e-2]:
        x, th, hist = ias(A, y, 0.05, beta, vt, "differences", n_iter=500)
        e = np.linalg.norm(x - x_true)
        print(beta, vt, e, len(hist), np.sum(th > 1e-2))
```

(a) For $\beta = 1$ and $\vartheta = 10^{-3}$ the scheme reproduces Example 3.15: 20 iterations, error 0.64, and $\theta_i > 10^{-2}$ at the four increments 18, 50, 70, and 108 (true jumps at 18, 50, 69, 82, and 108). The table lists the error, the number of iterations, and the number of $\theta_i > 10^{-2}$.

	$\vartheta = 10^{-4}$	$\vartheta = 10^{-3}$	$\vartheta = 10^{-2}$
$\beta = 1/2$	1.06, 15, 4	0.63, 18, 4	0.92, 32, 2
$\beta = 1$	1.07, 18, 4	0.64, 20, 4	0.94, 36, 2
$\beta = 2$	1.16, 15, 4	0.81, 23, 4	0.96, 52, 2

ϑ matters much more than β, because it sets the threshold $\sqrt{2\vartheta}$ between the quadratic and the logarithmic regime of the penalty (Example 3.15). For $\vartheta = 10^{-4}$ the threshold is 0.014 and the floor $\vartheta/(\beta + \frac{3}{2})$ very small, so that the small increments are held at zero very firmly: the four jumps are found, the one of height 0.5 three to four cells late, and the data are fitted less well (discrepancy 0.42 to 0.45). For $\vartheta = 10^{-2}$ the threshold is 0.14, and only the two unit jumps are treated as jumps; the others are spread out as ramps. $\vartheta = 10^{-3}$ is the best of the three, and a heavier tail (smaller β) helps slightly. The number of iterations grows with ϑ.

(b) Started from $\boldsymbol{\theta} = \mathbf{1}$, the scheme stops after 23 iterations at $J = -2061.2$, against $J = -2076.4$ from the mode of the hyper-prior: a different and worse local minimum, whose joint posterior density is smaller by a factor of e^{15}. The reconstruction has error 1.57 instead of 0.64, large θ_i at the increments 18, 51, 68, 104, and 112 (the last edge is split into two jumps, and the step at 82 is missing), ramps instead of the two small steps, and 24 increments above 0.01. With "everything may jump", the first $\boldsymbol{x}$-step is almost unregularized and fits the noise, and the $\boldsymbol{\theta}$-step rewards the noise-induced increments with large variances, which the following steps then keep.

(c) For the smooth signal the scheme stops after 16 iterations with error 1.06, against 0.30 for the best Tikhonov solution with the difference matrix. Two increments become jumps, of size 0.7 to 0.8, at the steep flanks of the narrow bump (increments 28 and 46), so that the narrow bump turns into a box; all other increments stay below $\sqrt{2\vartheta} = 0.045$, where the penalty is quadratic, and the wide bump is reconstructed smoothly. The hierarchical prior on increments is the wrong grammar for the narrow bump: it explains steep flanks by jumps.

(d) With $p(\theta) \propto \theta^{\beta-1}\mathrm{e}^{-\theta/\vartheta}$, the bracket of (3.16) becomes $f(\theta) = z^2/(2\theta) + \theta/\vartheta - \eta \log\theta$ with $\eta = \beta - \frac{3}{2}$ and $z = (\boldsymbol{L}\boldsymbol{x})_i$. Setting $f'(\theta) = -z^2/(2\theta^2) + 1/\vartheta - \eta/\theta$ to zero gives $\theta^2 - \eta\vartheta\theta - \vartheta z^2/2 = 0$, whose only positive root is the new $\boldsymbol{\theta}$-step

$$\theta_i = \frac{\vartheta}{2}\Big(\eta + \sqrt{\eta^2 + 2z_i^2/\vartheta}\Big).$$

For $\beta < \frac{3}{2}$, i.e., $\eta < 0$, take any $\boldsymbol{x}$ with $z_i = 0$: the factor of the joint density that belongs to θ_i is $\theta_i^{-1/2}\,\theta_i^{\beta-1}\mathrm{e}^{-\theta_i/\vartheta} = \theta_i^{\eta}\mathrm{e}^{-\theta_i/\vartheta}$, which tends to infinity as $\theta_i \to 0$, while all other factors stay fixed. The joint density is therefore unbounded, and the joint MAP estimate does not exist; the scheme then drives such θ_i to zero and the increments with them. For $\beta = \frac{3}{2}$ the step is $\theta_i = |z_i|\sqrt{\vartheta/2}$, and minimizing f over θ leaves the penalty $\sqrt{2/\vartheta}\,|z_i|$, so that the scheme minimizes the total-variation functional (2.7) with $\lambda = \sigma^2\sqrt{2/\vartheta}$ (0.11 for $\vartheta = 10^{-3}$); where $z_i = 0$ the minimum over θ_i is approached only as $\theta_i \to 0$, so that the iteration needs a floor on θ_i and converges slowly. For $\beta > \frac{3}{2}$ the step never falls below $\eta\vartheta$. Since $z^2/(2\theta)$ is jointly convex in (z, θ) and $\theta/\vartheta - \eta\log\theta$ is strictly convex, the objective is then strictly convex in $(\boldsymbol{x}, \boldsymbol{\theta})$: the joint MAP estimate exists, is unique, and does not depend on the start, and the penalty that remains after minimizing out θ_i is quadratic for small and linear for large increments. In a copy of `solvers.ias` only the $\boldsymbol{\theta}$-step, the objective, and the start (the mode $(\beta-1)\vartheta$ of the hyper-prior) change:

```
eta = beta - 1.5
theta = 0.5 * vartheta * (eta + np.sqrt(eta**2 + 2 * w[pen]**2 / vartheta))
```

Repeating (a) gives the errors and numbers of iterations

	$\vartheta = 10^{-4}$	$\vartheta = 10^{-3}$	$\vartheta = 10^{-2}$
$\beta = 2$	1.43, 285	1.30, 87	1.33, 29
$\beta = 3$	1.61, 64	1.47, 22	1.41, 9
$\beta = 4$	1.68, 34	1.51, 13	1.42, 6

The largest increment of any of these reconstructions is 0.26, although the true jumps are as large as 1: the edges are smeared over many cells (53 to 96 increments above 0.01), and every error is about twice that of the inverse-gamma scheme (0.64) and of total variation (0.75). The larger β, the wider the quadratic regime of the penalty and the closer the result to a Tikhonov solution; the number of θ_i above 10^{-2} is no longer informative, since the floor $\eta\vartheta$ itself exceeds 10^{-2} for $\vartheta = 10^{-2}$ and $\beta \geq 3$. The gamma hyper-prior has an exponentially light tail and penalizes large variances too much to produce sharp, sparse jumps; the polynomial tail of the inverse gamma is what makes (3.10) sparsity-promoting.

Exercise 3.6. This exercise is the second paragraph of your project proposal and has no model solution.

C.4 Uncertainty quantification by sampling

All numbers in the computer parts were obtained by running the course code, with the data sets and seeds of `ch04_figures.py` and `ch05_figures.py` where an exercise reproduces an example.

Numbers that depend on the seed are given as ranges over a few runs; run times depend on the computer and indicate orders of magnitude only. The exercises called "Your own inverse problem" have no model solution.

Exercise 4.1. (a) Write $p(\boldsymbol{x})$ for $p(\boldsymbol{x} \mid \boldsymbol{y})$ and $\alpha(\boldsymbol{x}, \boldsymbol{x}')$ for the acceptance probability (4.11) of a move from $\boldsymbol{x}$ to $\boldsymbol{x}'$. The chain moves to $\boldsymbol{x}' \neq \boldsymbol{x}$ with the density $K(\boldsymbol{x}' \mid \boldsymbol{x}) = q(\boldsymbol{x}' \mid \boldsymbol{x})\,\alpha(\boldsymbol{x}, \boldsymbol{x}')$ and stays at $\boldsymbol{x}$ with the probability $r(\boldsymbol{x}) = 1 - \int q(\boldsymbol{x}' \mid \boldsymbol{x})\,\alpha(\boldsymbol{x}, \boldsymbol{x}')\,\mathrm{d}\boldsymbol{x}'$. For $\boldsymbol{x}' \neq \boldsymbol{x}$,

$$p(\boldsymbol{x})\,K(\boldsymbol{x}' \mid \boldsymbol{x}) = p(\boldsymbol{x})\,q(\boldsymbol{x}' \mid \boldsymbol{x}) \min\Big\{1, \frac{p(\boldsymbol{x}')\,q(\boldsymbol{x} \mid \boldsymbol{x}')}{p(\boldsymbol{x})\,q(\boldsymbol{x}' \mid \boldsymbol{x})}\Big\} = \min\{p(\boldsymbol{x})\,q(\boldsymbol{x}' \mid \boldsymbol{x}),\ p(\boldsymbol{x}')\,q(\boldsymbol{x} \mid \boldsymbol{x}')\},$$

which is symmetric in $\boldsymbol{x}$ and $\boldsymbol{x}'$ and therefore equal to $p(\boldsymbol{x}')\,K(\boldsymbol{x} \mid \boldsymbol{x}')$; if $p(\boldsymbol{x})\,q(\boldsymbol{x}' \mid \boldsymbol{x}) = 0$, both sides vanish. The point mass $r(\boldsymbol{x})$ contributes only for $\boldsymbol{x}' = \boldsymbol{x}$, where (4.7) holds trivially. (b) Take the proposal $q((\boldsymbol{x}', \boldsymbol{\theta}') \mid (\boldsymbol{x}, \boldsymbol{\theta})) = p(\boldsymbol{x}' \mid \boldsymbol{\theta}, \boldsymbol{y})\,\delta(\boldsymbol{\theta}' - \boldsymbol{\theta})$, which keeps $\boldsymbol{\theta}$ and draws $\boldsymbol{x}'$ from its conditional. With $p(\boldsymbol{x}, \boldsymbol{\theta} \mid \boldsymbol{y}) = p(\boldsymbol{x} \mid \boldsymbol{\theta}, \boldsymbol{y})\,p(\boldsymbol{\theta} \mid \boldsymbol{y})$, the ratio in (4.11) is

$$\frac{p(\boldsymbol{x}', \boldsymbol{\theta} \mid \boldsymbol{y})\,p(\boldsymbol{x} \mid \boldsymbol{\theta}, \boldsymbol{y})}{p(\boldsymbol{x}, \boldsymbol{\theta} \mid \boldsymbol{y})\,p(\boldsymbol{x}' \mid \boldsymbol{\theta}, \boldsymbol{y})} = \frac{p(\boldsymbol{x}' \mid \boldsymbol{\theta}, \boldsymbol{y})\,p(\boldsymbol{\theta} \mid \boldsymbol{y})\,p(\boldsymbol{x} \mid \boldsymbol{\theta}, \boldsymbol{y})}{p(\boldsymbol{x} \mid \boldsymbol{\theta}, \boldsymbol{y})\,p(\boldsymbol{\theta} \mid \boldsymbol{y})\,p(\boldsymbol{x}' \mid \boldsymbol{\theta}, \boldsymbol{y})} = 1,$$

so that every proposal is accepted. (c) Counting the pairs (k, l) with $|k - l| = j$, of which there are N for $j = 0$ and $2(N - j)$ for $j \geq 1$,

$$\operatorname{Var}\Big[\frac{1}{N}\sum_{k=1}^{N} f_k\Big] = \frac{\sigma_f^2}{N^2}\sum_{k,l=1}^{N} \rho_f(|k - l|) = \frac{\sigma_f^2}{N}\Big(1 + 2\sum_{j=1}^{N-1}\Big(1 - \frac{j}{N}\Big)\rho_f(j)\Big).$$

If $\sum_j |\rho_f(j)| < \infty$, the bracket converges to τ_f of (4.9) as $N \to \infty$ by dominated convergence, which is (4.10). For the autoregressive chain in equilibrium, $f_k \sim \mathcal{N}(0, 1)$, since $\operatorname{Var}[f_{k+1}] = a^2 + (1 - a^2) = 1$, and $f_{k+j} = a^j f_k + (\text{noise independent of } f_k)$, so that $\rho_f(j) = a^j$ and

$$\tau_f = 1 + 2\sum_{j=1}^{\infty} a^j = 1 + \frac{2a}{1 - a} = \frac{1 + a}{1 - a}.$$

For $a = 0.9$, $\tau_f = 19$: nineteen steps are worth one independent sample. For $a < 0$, $\tau_f < 1$, and the chain average is more accurate than an average of independent samples; we meet such antithetic chains in Exercise 4.5c.

Exercise 4.2. (a) With $a = z^2/2$ and $b = 1/(2\gamma^2)$, the given integral yields

$$\int_0^\infty \frac{e^{-z^2/(2\theta)}}{\sqrt{2\pi\theta}}\,\frac{e^{-\theta/(2\gamma^2)}}{2\gamma^2}\,\mathrm{d}\theta = \frac{1}{2\gamma^2\sqrt{2\pi}}\sqrt{2\pi\gamma^2}\,e^{-2\sqrt{z^2/(4\gamma^2)}} = \frac{1}{2\gamma}\,e^{-|z|/\gamma}$$

for $z \neq 0$, and for $z = 0$ by continuity, or directly from $\int_0^\infty \theta^{-1/2} e^{-b\theta}\,\mathrm{d}\theta = \sqrt{\pi/b}$. (b) In the variables $(x_1, \boldsymbol{z})$ with $\boldsymbol{z} = \boldsymbol{L}\boldsymbol{x}$, a linear change of variables with determinant one, the prior given $\boldsymbol{\theta}$ is $\mathcal{N}(x_1; 0, \gamma_0^2)\prod_i \mathcal{N}(z_i; 0, \theta_i)$, and the likelihood does not depend on $\boldsymbol{\theta}$. Hence $p(\boldsymbol{\theta} \mid \boldsymbol{x}, \boldsymbol{y}) \propto p(\boldsymbol{y} \mid \boldsymbol{x})\,p(\boldsymbol{x} \mid \boldsymbol{\theta})\,p(\boldsymbol{\theta}) \propto \prod_i \mathcal{N}(z_i; 0, \theta_i)\,p(\theta_i)$, a product, so that the θ_i are conditionally independent, and with (3.11),

$$\mathcal{N}(z_i; 0, \theta_i)\,p(\theta_i) \propto \theta_i^{-1/2} e^{-z_i^2/(2\theta_i)}\,\theta_i^{-\beta-1} e^{-\vartheta/\theta_i} = \theta_i^{-(\beta+1/2)-1} \exp\Big(-\frac{\vartheta + z_i^2/2}{\theta_i}\Big),$$

which is (4.13). The logarithm of the density of $\operatorname{InvGamma}(a, b)$ is $-(a + 1)\log\theta - b/\theta + \text{const}$, whose derivative $-(a + 1)/\theta + b/\theta^2$ vanishes at the mode $\theta = b/(a + 1)$. For $a = \beta + \frac{1}{2}$ and $b = \vartheta + z_i^2/2$, the mode is $(\vartheta + z_i^2/2)/(\beta + \frac{3}{2})$, the $\boldsymbol{\theta}$-step of Algorithm 3.14. (c) With the

exponential hyper-prior of (a), $p(\theta_i \mid z_i) \propto \theta_i^{-1/2} \exp(-z_i^2/(2\theta_i) - \theta_i/(2\gamma^2))$. The density of $w = 1/\theta_i$ picks up the Jacobian $|\mathrm{d}\theta_i/\mathrm{d}w| = w^{-2}$:

$$p(w \mid z_i) \propto w^{1/2} \exp\Big(-\frac{z_i^2 w}{2} - \frac{1}{2\gamma^2 w}\Big)\, w^{-2} = w^{-3/2} \exp\Big(-\frac{z_i^2 w}{2} - \frac{1}{2\gamma^2 w}\Big).$$

Expanding the inverse Gaussian exponent, $-\lambda(w-\mu)^2/(2\mu^2 w) = -\lambda w/(2\mu^2) + \lambda/\mu - \lambda/(2w)$, and comparing the coefficients of w and $1/w$ gives $\lambda = 1/\gamma^2$ and $\lambda/\mu^2 = z_i^2$, i.e., $\mu = 1/(\gamma\,|z_i|)$, as stated in Section 4.4.2.

Exercise 4.3. (a) Each chain starts in equilibrium, and τ is estimated from $2 \cdot 10^5$ iterations ($4 \cdot 10^5$ for $n = 128$):

```
for n in (2, 8, 32, 128):
    for c in (0.5, 1, 1.5, 2, 2.38, 3, 4, 5):
        x0 = rng.standard_normal(n)                # a draw from the target
        ch, acc = samplers.rw_metropolis(lambda x: -0.5 * x @ x, x0, 200000,
                                         c / np.sqrt(n), rng=rng)
        print(n, c, acc, samplers.iat(ch[:, 0]) / n)
```

The table lists the acceptance rates and τ/n for $n = 8$ and $n = 128$, averaged over two chains, and the limits for $n \to \infty$, which follow from approximating the chain of x_1 by a diffusion [104]: the acceptance rate is $a(c) = 2\Phi(-c/2)$, the mean squared move of x_1 per iteration is $c^2 a(c)/n$, and an Ornstein–Uhlenbeck process with this diffusion coefficient and unit variance has $\tau \approx 4n/(c^2 a(c))$.

c	0.5	1	1.5	2	2.38	3	4	5
acceptance, $n = 8$	0.81	0.63	0.47	0.35	0.27	0.17	0.08	0.04
acceptance, $n = 128$	0.80	0.62	0.45	0.32	0.24	0.14	0.05	0.01
acceptance, limit $2\Phi(-c/2)$	0.80	0.62	0.45	0.32	0.23	0.13	0.05	0.01
τ/n, $n = 8$	23	7.0	4.4	3.4	3.2	3.5	5.8	11
τ/n, $n = 128$	16	6.8	3.8	3.4	3.4	3.8	5.2	12
τ/n, limit $4/(c^2 a(c))$	20	6.5	3.9	3.2	3.0	3.3	5.5	13

The optimum is flat, between $c = 2$ and $c = 3$, with $\tau \approx 3n$ to $3.4n$ and an acceptance rate of 0.24 for $n \geq 32$, as predicted; the estimates of τ scatter by about ten percent between chains. For $n = 2$ the optimal c is the same, but the acceptance rate there is 0.36 and $\tau = 3.8n$: the value 0.234 is a limit for large n. (b) With σ_j the standard deviations of the target:

```
sd = 10.0 ** (-2 * np.arange(n) / (n - 1))
lp = lambda x: -0.5 * np.sum((x / sd) ** 2)
ch, acc = samplers.rw_metropolis(lp, sd * rng.standard_normal(n), 10**6,
                                 c / np.sqrt(np.sum(sd**-2)), rng=rng)
print(acc, samplers.iat(ch[:, 0]), samplers.iat(ch[:, -1]))  # isotropic
ch, acc = samplers.rw_metropolis(lp, sd * rng.standard_normal(n), 200000,
                                 c / np.sqrt(n), chol=np.diag(sd), rng=rng)
print(acc, samplers.iat(ch[:, 0]), samplers.iat(ch[:, -1]))  # shaped
```

With shaped steps, the chain in the variables x_i/σ_i is the chain of (a), and so are the results for every component: for $n = 32$ and $c = 2.38$, the acceptance rate is 0.24 and $\tau = 2.8n$ for x_1 and $3.0n$ for x_n. With isotropic steps, the acceptance rates are close to those of (a), e.g., 0.63, 0.27, and 0.08 for $n = 32$ and $c = 1$, 2.38, and 4, but the components mix very differently. The diffusion argument of (a), applied to every component in its own units, explains both: the acceptance rate is $2\Phi(-\ell/2)$ with $\ell^2 = s^2 \sum_j \sigma_j^{-2} = c^2$, as in (a), and component i moves by the mean squared step $s^2 a(c)/\sigma_i^2$ in its own units, so that

$$\tau_i \approx \frac{4\,\sigma_i^2 \sum_j \sigma_j^{-2}}{c^2 a(c)} = \frac{\sigma_i^2 \sum_j \sigma_j^{-2}}{n}\, \tau_{(a)},$$

the τ of (a) times a factor that is about 10^3 for the widest component and about 0.1 for the narrowest: the step size is dictated by the narrowest components, between $1.2\sigma_n$ and $2.4\sigma_n$ for $c = 2.38$ and $n \le 32$, and the widest one then needs of the order of $(\sigma_1/s)^2$ steps to move by its own standard deviation. For $c = 2.38$, this predicts $\tau = 3.0 \cdot 10^4$, $4.1 \cdot 10^4$, and $1.2 \cdot 10^5$ for x_1 and $n = 2$, 8, and 32; single chains of 10^6 iterations give $2.3 \cdot 10^4$, $2.7 \cdot 10^4$, and $1.5 \cdot 10^5$, while τ of x_n is between 4 and 14. A chain that is worth only 10 to 40 independent samples of x_1 estimates τ only roughly, and the optimum in c is lost in this noise; 100 parallel chains of 0.6 to $2.3 \cdot 10^6$ iterations each, with $\tau \approx N \operatorname{Var}[\bar{x}_1]$ over the chains of length N, give $1.4 \cdot 10^4$, $3.0 \cdot 10^4$, and $1.0 \cdot 10^5$, and 40 chains of $4 \cdot 10^6$ iterations for $n = 128$ give $4.3 \cdot 10^5$, exactly the prediction: the diffusion limit becomes accurate as n grows. Shaping the steps like the posterior turns a factor $\sigma_1^2 \sum_j \sigma_j^{-2}/n \approx 10^3$ in the cost into one. (c) With the setup of `fig_rwmh` in `ch04_figures.py` and its seed, the two chains reproduce Example 4.8: acceptance rates 0.28 and 0.24, $\tau = 5799$ and $456 = 3.6n$ for $x(0.3)$, and 31 and 394 effective samples of $1.8 \cdot 10^5$. Two further seeds give $\tau = 2662$ and 4140 for the isotropic and 390 and 410 for the shaped chain: an estimate of τ from a chain that is worth only 30 to 70 independent samples is itself uncertain by a factor of two. For 400 effective samples in $x(0.3)$, the isotropic random walk needs $400\,\tau \approx 1$ to $2.3 \cdot 10^6$ iterations after burn-in, and since the slowest component has only 19 to 26 effective samples in $1.8 \cdot 10^5$ iterations, about 3 to $4 \cdot 10^6$ for 400 effective samples in every component, i.e., 20 times the run of the example, each with one evaluation of the posterior.

Exercise 4.4. (a) For the prior $\mathcal{N}(\boldsymbol{\mu}_{\text{prior}}, (\boldsymbol{B}^\mathsf{T}\boldsymbol{B})^{-1})$, perturb the prior mean $\boldsymbol{B}\boldsymbol{\mu}_{\text{prior}}$ of the transformed unknown instead of zero:

$$\boldsymbol{x}^* = \arg\min_{\boldsymbol{x}} \left\{ \frac{1}{2\sigma^2} \left\| \boldsymbol{A}\boldsymbol{x} - (\boldsymbol{y} + \sigma\boldsymbol{\xi}) \right\|_2^2 + \frac{1}{2} \left\| \boldsymbol{B}\boldsymbol{x} - (\boldsymbol{B}\boldsymbol{\mu}_{\text{prior}} + \boldsymbol{\eta}) \right\|_2^2 \right\}.$$

The normal equations give

$$\boldsymbol{x}^* = \boldsymbol{H}^{-1}(\boldsymbol{A}^\mathsf{T}(\boldsymbol{y} + \sigma\boldsymbol{\xi})/\sigma^2 + \boldsymbol{B}^\mathsf{T}(\boldsymbol{B}\boldsymbol{\mu}_{\text{prior}} + \boldsymbol{\eta})),$$

whose mean $\boldsymbol{H}^{-1}(\boldsymbol{A}^\mathsf{T}\boldsymbol{y}/\sigma^2 + \boldsymbol{B}^\mathsf{T}\boldsymbol{B}\boldsymbol{\mu}_{\text{prior}})$ is $\boldsymbol{\mu}_{\text{post}}$ of (3.13) with $\boldsymbol{\Gamma}_{\text{prior}}^{-1} = \boldsymbol{B}^\mathsf{T}\boldsymbol{B}$; the covariance is that of the proof of Theorem 4.10, since the shift is deterministic. In `perturb_optimize`, this is the argument `b = B @ mu`; for $\boldsymbol{\mu}_{\text{prior}}(t) = 0.3\sin(2\pi t)$ and 20000 samples, the sample mean and standard deviations agree with the closed form to 0.018 posterior standard deviations and two percent. For the smooth signal:

```
B = np.linalg.cholesky(P_s).T                          # P_s = B^T B
X = samplers.perturb_optimize(A, y_s, sigma, B, 4000, rng)
first = t < 0.5
height = X[:, first].max(axis=1)                       # of the first bump
pos = t[first][np.argmax(X[:, first], axis=1)]         # and its position
print(height.mean(), np.percentile(height, [2.5, 97.5]))
print(pos.mean(), pos.std(), np.percentile(pos, [2.5, 97.5]))
```

With the data and the seed of `fig_pto`, this reproduces Example 4.12: the largest error of the mean is 0.034 posterior standard deviations, the height has the mean 0.88 and the interval $[0.82, 0.95]$, the peak of the posterior mean is 0.86, the sample band differs from the closed form by at most 0.004, and it contains the truth at 93 percent of the points. The position of the maximum has the posterior mean 0.301 and the standard deviation 0.011, i.e., 1.4 cells, and the 95 % credible interval $[0.277, 0.324]$, seven cells around the true position 0.3; 27 percent of the samples have their maximum in the cell at $t = 0.301$, and 22 and 24 percent in its two neighbors. The data determine where the bump is much better than how high it is, and unlike the height, the position is not biased by the smoothness prior. (b) With the data of the piecewise-constant signal, `L = solvers.difference_matrix(n)`, and `rng = np.random.default_rng(5)` as in `fig_threepriors`:

```
B = np.linalg.cholesky(priors.gaussian_precision(n, 0.1, "differences")).T
X_g = samplers.perturb_optimize(A, y_p, sigma, B, 18000, rng)
X_l = samplers.gibbs_laplace(A, y_p, sigma, 0.1, 20000, rng=rng)[0][2000:]
X_h = samplers.gibbs_hierarchical(A, y_p, sigma, 1.0, 1e-3, 20000,
                                  rng=rng)[0][2000:]
for X in (X_g, X_l, X_h):
    lo, hi = np.percentile(X, [2.5, 97.5], axis=0)     # 95 % band
    Z = X @ L.T                                        # increments
    loud = np.mean(np.abs(Z) > 0.1, axis=0)            # bottom row of the figure
    print(np.linalg.norm(X.mean(0) - x_p), np.mean((x_p >= lo) & (x_p <= hi)),
          np.sum(np.abs(Z) > 0.1, axis=1).mean(), np.abs(Z).sum(1).mean())
```

The output reproduces Example 4.14: errors of the posterior means 1.50, 1.39, and 0.99; coverages 87, 95, and 98 percent, the hierarchical band missing only the three points around the small step; band widths 0.36 (Gaussian) and 0.23 (hierarchical, plateau), with 0.26 to 1.1 at the jumps of the hierarchical band; 44, 46, and 13 loud increments and total variations 10.8, 12.7, and 7.5 per sample; and probabilities of a loud increment of about 0.3 on the plateaus and up to 0.65 at the large edges for the Gaussian and the Laplace prior, against 0.05 on the plateaus, between 0.27 and 0.54 at the four large jumps, and below 0.1 at the small step for the hierarchical prior. (c) The same run of `gibbs_hierarchical` with `vartheta` $= 10^{-4}$ and 10^{-2}, here with `rng = np.random.default_rng(5)` for each value, and the integrated autocorrelation times of $\log\theta_i$ as a check of the mixing:

	$\vartheta = 10^{-4}$	$\vartheta = 10^{-3}$	$\vartheta = 10^{-2}$
error of the posterior mean	0.87	0.97	1.32
coverage of the 95 % band	98 %	98 %	99 %
band width on the first plateau	0.12	0.23	0.54
$\mathbb{P}(\lvert z_i\rvert > 0.1)$ on the plateau, at the small step	0.003, 0.10	0.06, 0.10	0.38, 0.39
loud increments per sample (truth: 5)	5.6	13	51
total variation per sample (truth: 3.6)	5.0	7.5	15.0
largest τ of $\log\theta_i$	2200	97	7

The posterior does not find the small step for any of the three values. For $\vartheta = 10^{-4}$, the samples are almost exactly piecewise constant, with 5.6 loud increments on average, the plateaus are flat, and the band is half as wide there, but the probability of a loud increment at the small step is still 0.1 and the band misses the same three points as for $\vartheta = 10^{-3}$. For $\vartheta = 10^{-2}$, the threshold $\sqrt{2\vartheta} = 0.14$ between quiet and loud increments of Example 3.15 exceeds 0.1, the prior is no longer sparse on the scale of the small step, and the probability of a loud increment is 0.38 everywhere: the samples are rougher than those of the Laplace prior, the bands are wide, and the coverage of 99 percent is bought with them. The price of the sharper prior is the mixing: the largest τ of $\log\theta_i$ grows from 7 sweeps for $\vartheta = 10^{-2}$ to 2200 for $\vartheta = 10^{-4}$, so that the 18000 sweeps of the latter are worth fewer than ten independent samples at the slowest jump, and its numbers are correspondingly uncertain: the funnel of Remark 4.15 deepens as ϑ decreases. (d) With the starts of Example 4.20:

```
runs = [samplers.gibbs_hierarchical(A, y_p, sigma, 1.0, 1e-3, N, theta0=th0,
                                    rng=rng) for th0 in (None, 1.0, 1e-6, 1e-2)]
X = np.array([r[0][1000:] for r in runs])              # (4, N - 1000, n)
T = np.log(np.array([r[1][1000:] for r in runs]))      # (4, N - 1000, n - 1)
Q = np.concatenate([X, T], axis=2)                     # all components
rhat = [samplers.rank_rhat(Q[:, :, i]) for i in range(Q.shape[2])]
neff = [samplers.ess(Q[:, :, i]) for i in range(Q.shape[2])]
print(max(rhat), min(neff))
```

For $N = 6000$ this reproduces Example 4.20: $\widehat{R} \le 1.024$, with eight components above 1.01, all at jumps, and the smallest effective sample size 244, for $\log\theta_i$ at the first jump, against a

median of 9300 of the 20000 retained sweeps. Since N_{eff} grows linearly with the length of a chain in equilibrium, $400/244 \approx 1.6$ times the retained length should suffice, about 9000 sweeps per chain. The estimates of N_{eff} at the slow components are themselves noisy, however: four new chains of 11000 sweeps give $\widehat{R} \le 1.009$ but a smallest N_{eff} of 382, and four of 14000 sweeps give $\widehat{R} \le 1.006$ and 651. About 14000 sweeps per chain, i.e., $5.6 \cdot 10^4$ sweeps or about a minute at the millisecond per sweep of Section 4.4.1, meet both criteria with a margin, and the safe practice is to verify the criteria on the longer run rather than to rely on the extrapolation.

Exercise 4.5. (a) For $U(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu}_{\text{post}})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x}-\boldsymbol{\mu}_{\text{post}})$ and $\boldsymbol{M} = \boldsymbol{H}$, (4.16) reads $\dot{\boldsymbol{x}} = \boldsymbol{H}^{-1}\boldsymbol{r}$ and $\dot{\boldsymbol{r}} = -\boldsymbol{H}(\boldsymbol{x}-\boldsymbol{\mu}_{\text{post}})$, so that $\ddot{\boldsymbol{x}} = \boldsymbol{H}^{-1}\dot{\boldsymbol{r}} = -(\boldsymbol{x}-\boldsymbol{\mu}_{\text{post}})$. Every component of $\boldsymbol{x}(t)-\boldsymbol{\mu}_{\text{post}}$ is a harmonic oscillator with period 2π, whose solution with the initial values $\boldsymbol{x}(0)$ and $\dot{\boldsymbol{x}}(0) = \boldsymbol{H}^{-1}\boldsymbol{r}$ is (4.18). At $t = \pi/2$, $\boldsymbol{x}(\pi/2) = \boldsymbol{\mu}_{\text{post}} + \boldsymbol{H}^{-1}\boldsymbol{r}$ is a function of the fresh momentum $\boldsymbol{r} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{H})$ alone, hence independent of $\boldsymbol{x}(0)$, and it is Gaussian with the mean $\boldsymbol{\mu}_{\text{post}}$ and the covariance $\boldsymbol{H}^{-1}\boldsymbol{H}\boldsymbol{H}^{-1} = \boldsymbol{H}^{-1}$, i.e., distributed according to the posterior. More generally, if $\boldsymbol{x}(0)$ is drawn from the posterior, then $\boldsymbol{x}(t)$ is too, for every t, since $\cos^2 t + \sin^2 t = 1$, and the correlation between $\boldsymbol{x}(0)$ and $\boldsymbol{x}(t)$ in every direction is $\cos t$. (b) The leapfrog scheme in five lines, with `U` and `grad_U` of the bent ridge from Example 4.3:

```
def leapfrog(x, r, h, L):                          # L leapfrog steps, M = I
    for _ in range(L):
        r = r - 0.5 * h * grad_U(x)
        x = x + h * r
        r = r - 0.5 * h * grad_U(x)
    return x, r
H = lambda x, r: U(x) + 0.5 * r @ r                # total energy
x0, r0 = np.array([0.92, 0.0]), np.array([0.3, 1.5])
for h in (0.1, 0.05, 0.025, 0.0125, 0.00625):
    x1, r1 = leapfrog(x0, r0, h, round(1.5 / h))   # trajectory length 1.5
    xb, rb = leapfrog(x1, -r1, h, round(1.5 / h))  # flip the momentum, go back
    print(h, abs(H(x1, r1) - H(x0, r0)), abs(xb - x0).max(), abs(rb + r0).max())
```

Flipping the momentum and integrating again returns to $(\boldsymbol{x}_0, -\boldsymbol{r}_0)$ up to rounding errors of 10^{-15}. The largest energy error along the trajectory is $1.9 \cdot 10^{-2}$, $4.4 \cdot 10^{-3}$, $1.1 \cdot 10^{-3}$, $2.7 \cdot 10^{-4}$, and $6.8 \cdot 10^{-5}$ for the five step sizes: every halving of h divides it by 4.0 to 4.25, i.e., it decreases like h^2, although the number of steps doubles. (c) For $\boldsymbol{\Sigma} = \left(\begin{smallmatrix} 1 & 0.9 \\ 0.9 & 1 \end{smallmatrix}\right)$ and $\boldsymbol{H} = \boldsymbol{\Sigma}^{-1}$, we run 20000 iterations of `samplers.hmc` with `Minv_chol` the Cholesky factor of $\boldsymbol{\Sigma}$, i.e., $\boldsymbol{M} = \boldsymbol{H}$, with `jitter=0.0`, $h = \pi/40$, and $L = 20$, 40, and 80. Nearly all proposals are accepted, and the autocorrelations follow from (a), since the leapfrog trajectories are close to the exact flow:

- $hL = \pi/2$: $|\rho(j)| \le 0.02$ for x_1 and for x_1^2, $\tau = 1.0$; every iteration is an independent sample.
- $hL = \pi$: $\rho(j) = (-1)^j$ for x_1 and $\rho(j) = 1.00$ for x_1^2, to three digits: every iteration maps $\boldsymbol{x} - \boldsymbol{\mu}_{\text{post}}$ to its mirror image $-(\boldsymbol{x} - \boldsymbol{\mu}_{\text{post}})$. The chain average of x_1 is then exact after two iterations ($\tau \approx 0$), but x_1^2 hardly changes, $\tau = 3 \cdot 10^3$, and the chain explores the posterior only through the small leapfrog error: the angle of rotation of L leapfrog steps is $L \arccos(1 - h^2/2) = \pi + 8 \cdot 10^{-4}$ instead of π.
- $hL = 2\pi$: $\rho(j) \approx 1$ for both, $\tau \approx 6 \cdot 10^3$; every trajectory returns to its start, and after 20000 iterations the chain has the mean 0.73 and the variance 0.02 for x_1, instead of 0 and 1: it has barely left its starting point $(1, 0.5)$.

The default random variation of the step size by ± 20 percent repairs the last two cases, with $\tau = 16$ for x_1^2 at $hL = \pi$ and $\tau = 7$ for x_1 at $hL = 2\pi$, and leaves the first unchanged. (d) With the potential `U`, the MAP estimate `b_map`, the Gauss–Newton Hessian `H`, and the jitted forward map `F` of Appendix B, but with $\gamma = 3\,\text{cm}$ and the weighted difference matrix `Lw`:

```
C = np.linalg.cholesky(np.linalg.inv(H))           # C C^T = H^{-1}
```

```
step = samplers.make_hmc_jax(U, 0.3, 5, C)            # HMC with M = H
Li = jnp.asarray(np.linalg.inv(Lw))                   # prior: b = Li w
ll_p = jax.jit(lambda w: -0.5 * jnp.sum((F(Li @ w) - y) ** 2) / 0.01**2)
ll_l = jax.jit(lambda w: -U(b_map + C @ w) + 0.5 * w @ w)   # Laplace
W, evals = samplers.elliptical_slice(lambda w: float(ll_l(w)), np.zeros(199),
                                     6000)            # b = b_map + C w
Wp, evals_p = samplers.elliptical_slice(lambda w: float(ll_p(w)), Lw @ b_map,
                                        20000)        # b = Li w
warmup = blackjax.window_adaptation(blackjax.nuts, lambda b: -U(b),
                                    is_mass_matrix_diagonal=False)
(state, par), _ = warmup.run(jax.random.PRNGKey(1), b_map, num_steps=500)
```

The runs of `fig_hmc` with their seeds reproduce Example 4.18, and the table adds the prior on second differences of (e). For the random walk and for elliptical slice sampling with the prior as reference in (d), the table uses the runs of $2 \cdot 10^5$ iterations of Example 4.18; in (e), these two samplers ran for $2 \cdot 10^4$ iterations only, and their numbers are rough. The gradient-based samplers are converted with the factor of 8 forward solves per gradient of Section 4.5. The last four columns are the effective samples, the smallest number over the 199 components, per thousand forward solves and per second.

		per 10^3 solves		per second	
sampler	per iteration, (d) and (e)	(d)	(e)	(d)	(e)
HMC, $\boldsymbol{M} = \boldsymbol{H}$, $h = 0.3$, $L = 5$	5 gradients	11	12	260	340
random walk, shaped	1 solve	0.9	0.9	2.8	3.9
NUTS, diagonal $\boldsymbol{M}$	45 and 961 gradients	1.3	0.06	150	7
NUTS, dense $\boldsymbol{M}$	51 and 75 gradients	1.9	0.6	150	49
elliptical slice, prior	9.0 and 13.5 solves	0.03	0.02	0.2	0.1
elliptical slice, Laplace	1.65 and 1.39 solves	190	320	700	1500

For (d), HMC needs 11 gradients per effective sample, NUTS 93 and 66 with the diagonal and the dense mass matrix, and elliptical slice sampling with the Laplace reference 5.3 forward solves, the cheapest, about 17 times cheaper than HMC at 90 forward solves, against roughly 30 000 with the prior as reference. The ranking per second is the same, but the factors differ, because NUTS runs in one compiled loop, while the elliptical slice sampler and the random walk call a compiled likelihood from a Python loop, whose overhead counts like extra work; per second is what the user waits for, per forward solve what carries over to a slower forward model. (e) With `Lw = np.vstack([D2 / 2e-3, L1[-1:] / 0.03])` from `fig_hmc`, the prior standard deviation grows to 6.8 m at the inflow: the prior on second differences is an integrated random walk whose slope is pinned only weakly. HMC and elliptical slice sampling with the Laplace reference are unaffected, about 85 and 3.1 forward solves per effective sample, because they work in the coordinates of the Gauss–Newton Hessian, in which this posterior is as close to $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ as the one of (d). Elliptical slice sampling with the prior as reference gets worse, roughly 48 000 forward solves per effective sample, since the prior is now even wider compared with the posterior. NUTS with a diagonal mass matrix needs 961 leapfrog steps per iteration, close to the default limit of ten doublings, and 2089 gradients per effective sample, ten times as many as with a dense mass matrix: the strongly correlated posterior of a smoothness prior on second differences cannot be preconditioned by a diagonal. The mass matrix, not the sampler, decides the cost.

Exercise 4.6. This exercise is about your own problem and has no model solution.

C.5 Beyond MCMC: approximate Bayesian inference

Exercise 5.1. (a) In κ, $U(\kappa) = \log\kappa + (\log\kappa)^2/(2s^2)$ with $s = 0.6$, where $\log\kappa$ is the Jacobian term of the change of variables. Its derivative $U'(\kappa) = (1 + \log\kappa/s^2)/\kappa$ vanishes at $\log\kappa = -s^2$, i.e., $\kappa_{\text{MAP}} = e^{-0.36} = 0.698$, and

$$U''(\kappa) = \frac{1/s^2 - 1 - \log\kappa/s^2}{\kappa^2}, \quad U''(\kappa_{\text{MAP}}) = \frac{1}{s^2\kappa_{\text{MAP}}^2},$$

so that the Laplace approximation is $\mathcal{N}(0.698, (s\kappa_{\text{MAP}})^2) = \mathcal{N}(0.70, 0.42^2)$. Its probability of $\kappa < 0$ is $\Phi(-\kappa_{\text{MAP}}/(s\kappa_{\text{MAP}})) = \Phi(-1/s) = \Phi(-1.67) = 0.048$, independent of κ_{MAP}. In $u = \log\kappa$, $U(u) = u^2/(2s^2)$ is quadratic, $u_{\text{MAP}} = 0$, $U'' = 1/s^2$, and the Laplace approximation $\mathcal{N}(0, 0.6^2)$ is the posterior itself; its image $\kappa = e^u$ is positive, so that the probability of $\kappa < 0$ is zero. (b) With the normalizing constants, $U(\boldsymbol{x}) = \frac{1}{2}\left\|\boldsymbol{\Gamma}^{-1/2}(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y})\right\|_2^2 + \frac{1}{2}\left\|\boldsymbol{\Gamma}_{\text{prior}}^{-1/2}(\boldsymbol{x} - \boldsymbol{\mu})\right\|_2^2 + \frac{1}{2}\log\det(2\pi\boldsymbol{\Gamma}) + \frac{1}{2}\log\det(2\pi\boldsymbol{\Gamma}_{\text{prior}})$ is quadratic with the Hessian $\boldsymbol{H} = \boldsymbol{\Gamma}_{\text{post}}^{-1}$ of (3.13). Its Taylor polynomial is therefore exact, $U(\boldsymbol{x}) = U(\boldsymbol{x}_{\text{MAP}}) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}_{\text{MAP}})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x} - \boldsymbol{x}_{\text{MAP}})$, and

$$\begin{aligned} p(\boldsymbol{y}) = \int e^{-U(\boldsymbol{x})}\,\mathrm{d}\boldsymbol{x} &= e^{-U(\boldsymbol{x}_{\text{MAP}})}\int e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}_{\text{MAP}})^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{x}-\boldsymbol{x}_{\text{MAP}})}\,\mathrm{d}\boldsymbol{x} \\ &= e^{-U(\boldsymbol{x}_{\text{MAP}})}(2\pi)^{n/2}(\det\boldsymbol{H})^{-1/2}, \end{aligned}$$

by the normalization of (3.2). A numerical check against the closed form $p(\boldsymbol{y}) = \mathcal{N}(\boldsymbol{y}; \boldsymbol{A}\boldsymbol{\mu}, \boldsymbol{\Gamma} + \boldsymbol{A}\boldsymbol{\Gamma}_{\text{prior}}\boldsymbol{A}^{\mathsf{T}})$ agrees to all digits. (c) With $\boldsymbol{\Gamma}_{\text{prior}}^{-1} = \boldsymbol{R}^{-\mathsf{T}}\boldsymbol{R}^{-1}$, the Gauss–Newton Hessian factors as $\boldsymbol{H} = \boldsymbol{R}^{-\mathsf{T}}(\boldsymbol{I} + \widetilde{\boldsymbol{H}})\boldsymbol{R}^{-1}$, so that

$$\boldsymbol{H}^{-1} = \boldsymbol{R}(\boldsymbol{I} + \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\mathsf{T}})^{-1}\boldsymbol{R}^{\mathsf{T}} = \boldsymbol{R}\boldsymbol{V}(\boldsymbol{I} + \boldsymbol{\Lambda})^{-1}\boldsymbol{V}^{\mathsf{T}}\boldsymbol{R}^{\mathsf{T}} = \boldsymbol{R}\boldsymbol{R}^{\mathsf{T}} - \boldsymbol{R}\boldsymbol{V}\,\frac{\boldsymbol{\Lambda}}{\boldsymbol{I} + \boldsymbol{\Lambda}}\,\boldsymbol{V}^{\mathsf{T}}\boldsymbol{R}^{\mathsf{T}},$$

using $\boldsymbol{V}\boldsymbol{V}^{\mathsf{T}} = \boldsymbol{I}$ and $(1 + \lambda)^{-1} = 1 - \lambda/(1 + \lambda)$; the last term is the stated sum. Since $\widetilde{\boldsymbol{H}} = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{W}$ with $\boldsymbol{W} = \boldsymbol{\Gamma}^{-1/2}\boldsymbol{J}\boldsymbol{R} \in \mathbb{R}^{m\times n}$, its rank is at most m, and with the singular value decomposition $\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}}$, $\widetilde{\boldsymbol{H}} = \boldsymbol{V}\boldsymbol{\Sigma}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}}$, whose eigenvalues are the squared singular values of $\boldsymbol{W}$. Dropping the terms with $i > r$ replaces the eigenvalues $1/(1 + \lambda_i)$ of $(\boldsymbol{I} + \widetilde{\boldsymbol{H}})^{-1}$ by 1 for $i > r$, which multiplies them by $1 + \lambda_i \le 1 + \lambda_{r+1}$. Hence the truncated covariance $\widehat{\boldsymbol{\Gamma}}$ satisfies $\boldsymbol{H}^{-1} \preceq \widehat{\boldsymbol{\Gamma}} \preceq (1 + \lambda_{r+1})\,\boldsymbol{H}^{-1}$ in the Loewner order, so that the variance $\boldsymbol{c}^{\mathsf{T}}\widehat{\boldsymbol{\Gamma}}\boldsymbol{c}$ of every linear functional, in particular of every component, is overestimated, by a factor less than $1 + \lambda_{r+1} < 1.1$ if all dropped λ_i are below 0.1. For the bathymetry problem of Example 5.4, with the function `bathy` of `ch05_figures.py` for the data, the MAP estimate, and its Jacobian:

```
d = bathy(20)                          # 20 sensors, increments with 1 cm per cell
lam, V = approximations.lowrank_laplace(d["J"], d["R"], 0.01)
print(lam[:12], np.sum(lam > 1), np.sum(lam > 0.1))
sd_r = approximations.lowrank_std(lam, V, d["R"], np.sum(lam > 0.1))
```

With 20 sensors, exactly 20 eigenvalues are nonzero, 252, 25.1, 10.8, 4.8, 3.1, 2.1, 1.5, 1.2, 0.95, and so on down to 0.23; eight exceed one, as stated in Section 5.2, but all twenty exceed 0.1, so that the rule of ten percent keeps all of them, and the truncation after the eight eigenvalues above one overestimates the variances by up to 32 percent. With 200 sensors, 24 eigenvalues exceed one and 73 exceed 0.1, the largest being 2562, 255, and 108; the truncation after 73 terms overestimates the variances by at most 1.2 percent, well within the bound $1 + \lambda_{74} = 1.099$. Ten times more sensors inform three times more directions. (d) The potential is $U(x) = (1 - x^2)^2/(2 \cdot 0.3^2) + (x - 0.4)^2/2$, whose minimizers are $x_1 = 0.987$ (the MAP estimate) and $x_2 = -0.968$, with $U(x_1) = 0.176$, $U(x_2) = 0.958$, $U''(x_1) = 43.7$, and $U''(x_2) = 41.2$, i.e., Laplace standard deviations 0.151 and 0.156. The evidence approximations $Z_j = e^{-U(x_j)}\sqrt{2\pi/U''(x_j)}$ are 0.318

and 0.150, up to the normalizing constants omitted from U, which are common to both, and the mixture $\sum_j w_j \mathcal{N}(x_j, 1/U''(x_j))$ has the weights $w_j = Z_j/(Z_1 + Z_2) = 0.680$ and 0.320, against the true probabilities 0.678 and 0.322 from quadrature. Each Z_j is about five percent smaller than the true probability mass of its peak, because U is not quadratic away from the peaks, but the two errors nearly cancel in the weights.

Exercise 5.2. (a) $\mathrm{KL}(p \,\|\, q) = \int p \log p - \int p \log q$, and only the second term depends on $q = \mathcal{N}(\boldsymbol{m}, \boldsymbol{S})$:

$$-\int p \log q = \tfrac{1}{2} \log\det(2\pi \boldsymbol{S}) + \tfrac{1}{2} \operatorname{tr}(\boldsymbol{S}^{-1}(\boldsymbol{\Sigma}_p + (\boldsymbol{\mu}_p - \boldsymbol{m})(\boldsymbol{\mu}_p - \boldsymbol{m})^\mathsf{T})),$$

with the mean $\boldsymbol{\mu}_p$ and the covariance $\boldsymbol{\Sigma}_p$ of p, assumed finite. The minimum over $\boldsymbol{m}$ is at $\boldsymbol{m} = \boldsymbol{\mu}_p$, since $\boldsymbol{S}^{-1}$ is positive definite. In the precision $\boldsymbol{P} = \boldsymbol{S}^{-1}$, the remaining function $-\tfrac{1}{2}\log\det \boldsymbol{P} + \tfrac{1}{2}\operatorname{tr}(\boldsymbol{P}\boldsymbol{\Sigma}_p)$ is convex, and its gradient $-\tfrac{1}{2}\boldsymbol{P}^{-1} + \tfrac{1}{2}\boldsymbol{\Sigma}_p$ vanishes for $\boldsymbol{S} = \boldsymbol{\Sigma}_p$. For the mean-field family, $\boldsymbol{S} = \operatorname{diag}(s_1^2, \dots, s_n^2)$, and the function separates into $\sum_i (\tfrac{1}{2}\log(2\pi s_i^2) + \mathbb{E}_p[(x_i - m_i)^2]/(2s_i^2))$, which is minimal for $m_i = \mathbb{E}_p[x_i]$ and $s_i^2 = \operatorname{Var}_p[x_i]$, the marginal means and variances. (b) Here $\boldsymbol{H} = \boldsymbol{\Sigma}^{-1} = \frac{1}{1-\rho^2}\left(\begin{smallmatrix} 1 & -\rho \\ -\rho & 1 \end{smallmatrix}\right)$. The minimizer of $\mathrm{KL}(q \,\|\, p)$ is $\mathcal{N}(\boldsymbol{0}, (1-\rho^2)\boldsymbol{I})$ by Theorem 5.6b, with the conditional standard deviation $\sqrt{1-\rho^2}$, and its divergence is

$$\tfrac{1}{2}(\log H_{11} + \log H_{22} - \log\det \boldsymbol{H}) = \tfrac{1}{2}(-2\log(1-\rho^2) + \log(1-\rho^2)) = -\tfrac{1}{2}\log(1-\rho^2).$$

The minimizer of $\mathrm{KL}(p \,\|\, q)$ over the mean-field family is $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ by (a), and with the divergence between Gaussians of Theorem 5.6,

$$\mathrm{KL}(p \,\|\, \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})) = \tfrac{1}{2}(\operatorname{tr}\boldsymbol{\Sigma} - 2 + \log\det \boldsymbol{I} - \log\det \boldsymbol{\Sigma}) = -\tfrac{1}{2}\log(1-\rho^2).$$

The two divergences are equal, 0.83 for $\rho = 0.9$, although the two Gaussians are not: one has the standard deviations 0.44, the other 1, and neither has the correlation. (c) By Theorem 5.6b, $\tfrac{1}{2}(\sum_i \log H_{ii} - \log\det \boldsymbol{H})$ is the divergence of $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{D}^{-1})$ from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{H}^{-1})$, $\boldsymbol{D} = \operatorname{diag}(H_{11}, \dots, H_{nn})$, for any symmetric positive definite $\boldsymbol{H}$ and any $\boldsymbol{\mu}$. A divergence is nonnegative (Gibbs' inequality), so that $\log\det \boldsymbol{H} \le \sum_i \log H_{ii}$, i.e., $\det \boldsymbol{H} \le \prod_i H_{ii}$, with equality if and only if the two Gaussians coincide, i.e., $\boldsymbol{H} = \boldsymbol{D}$ is diagonal.

Exercise 5.3. (a) The exact ELBO of the iterates is computed by Gauss–Hermite quadrature with `elbo_banana` of `ch05_figures.py`, and every run gets the same budget of 96000 gradients:

```
def U(x):                                          # bent ridge, in JAX
    r = 1.0 - x[0] - 0.5 * x[1] ** 2
    return 0.5 * (x[0] ** 2 + x[1] ** 2) + r ** 2 / (2 * 0.3 ** 2)
for S in (1, 4, 16, 64):
    m, C, rec = approximations.gaussian_vi(U, np.zeros(2), n_iter=96000 // S,
                    n_samples=S, lr=1e-2, lr_final=1e-3, keep_params=True)
    elbo = [elbo_banana(mk, Ck) for mk, Ck in zip(rec["m"], rec["C"])]
    plt.semilogx(S * (rec["iter"] + 1), elbo)       # exact ELBO vs gradients
```

The best Gaussian, found by maximizing the quadrature ELBO with Nelder–Mead, has the mean $(0.784, 0)$, the standard deviations 0.287 and 0.539, and the divergence 0.551, and the run of Example 5.8 ($S = 16$, 6000 iterations, seed 2) is reproduced: within 0.01 of the best ELBO after 560 iterations, and the mean $(0.782, -0.004)$ and the standard deviations 0.29 and 0.53 at the end. Over three seeds, the number of gradients until the exact ELBO is within 0.01 of the best one is 2200 to 10000 for $S = 1$, 2400 to 2600 for $S = 4$, 7200 to 10600 for $S = 16$, and 3 to $7 \cdot 10^4$ for $S = 64$, and the final gap is 0.002 to 0.005, 0.001, 0.0005 to 0.0017, and 0.001 to 0.007. In the plot, the curves for $S \ge 4$ have nearly the same shape against the iteration, about

500 iterations to the plateau, so that against the number of gradients they are shifted by the factor S: the number of iterations is set by the step sizes, which limit how far Adam moves per iteration, and not by the noise of the gradients. On the plateau, the exact ELBO of the iterates fluctuates with the standard deviations 0.006, 0.003, and 0.001 for $S = 1$, 4, and 16 over the second half of the budget (seed 0), and only the decreasing step sizes damp these fluctuations at the end. Few draws per iteration reach the plateau with the fewest gradients, more draws give smoother iterates, and $S = 64$ has too few iterations within the budget; $S = 4$ to 16 is a good compromise here. For $S = 16$ and 6000 iterations, the step sizes decaying from 10^{-1} to 10^{-2} reach the plateau after 160 to 1400 iterations but end 0.003 to 0.02 below the best ELBO, because the final steps are too large for the noise; from 10^{-3} to 10^{-4} they never get within 0.01; and from 10^{-1} to 10^{-4} they combine both advantages, 140 to 700 iterations to the plateau and a final gap of 0.0004 to 0.0013. Large steps early and small steps late is the rule. (b) With the precision $\boldsymbol{H}$ and $\boldsymbol{b} = \boldsymbol{A}^\mathsf{T}\boldsymbol{y}/\sigma^2$ of the Gaussian posterior, the potential is $U(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{H}\boldsymbol{x} - \boldsymbol{b}^\mathsf{T}\boldsymbol{x}$, and the divergence of every iterate is exact, $\mathrm{KL} = \log p(\boldsymbol{y}) - \mathrm{ELBO}$, with both terms in closed form as in `fig_vimf`:

```
U = lambda x: 0.5 * x @ Hj @ x - bj @ x              # Hj, bj: jnp arrays
for family, n_iter, lr in (("mean-field", 4000, 2e-2), ("full", 10000, 1e-2)):
    m, C, rec = approximations.gaussian_vi(U, np.zeros(n), C0=0.1 * np.eye(n),
                    family=family, n_iter=n_iter, n_samples=8, lr=lr,
                    lr_final=lr / 1000, keep_params=True)
    sd = np.sqrt(np.sum(C**2, axis=1))               # standard deviations of q
kl_mf = 0.5 * (np.sum(np.log(np.diag(H))) - np.linalg.slogdet(H)[1])
```

For $n = 128$, this reproduces Example 5.9: the mean-field standard deviations are within two percent of $1/\sqrt{H_{ii}}$, between 0.40 and 0.56 times the posterior ones, and the divergence stalls at 29.5; the full-rank standard deviations are within 1.5 percent of the posterior ones, and the divergence is 0.43 after 10^4 iterations; both means are within 0.02 posterior standard deviations. For the other n, with `deconvolution_operator(n, 64)`, data from the same noise realization, and $\gamma = 0.0294\sqrt{128/n}$, and with the settings of Example 5.9 for the full-rank family, $S = 8$ and step sizes decaying from 10^{-2} to 10^{-5} over 10^4 iterations:

	$n = 32$	$n = 64$	$n = 128$
parameters of the full-rank family	560	2144	8384
correlation of neighboring values	0.12	0.48	0.72
iterations until KL < 1, full rank	3550	5950	8625
divergence after 10^4 iterations, full rank	0.02	0.09	0.43
divergence of the best mean-field Gaussian	0.45	8.1	29.5

The number of iterations grows like $n^{2/3}$, much more slowly than the number of parameters, which grows like $n^2/2$; the cost of an iteration, S gradients and products with the $n \times n$ factor $\boldsymbol{C}$, grows as well. The limit of the mean-field divergence grows faster than n, because neighboring values become more strongly correlated as the grid is refined, and each pair of correlated neighbors contributes to $\frac{1}{2}(\sum_i \log H_{ii} - \log\det \boldsymbol{H})$. (c) Draw 4000 times from each q of (b), $\boldsymbol{x}_k = \boldsymbol{m} + \boldsymbol{C}\boldsymbol{\epsilon}_k$, and use the log weights $-U(\boldsymbol{x}_k) - \log q(\boldsymbol{x}_k)$:

```
E = rng.standard_normal((4000, n))
X = m + E @ C.T
log_q = -0.5 * np.sum(E**2, axis=1) - np.sum(np.log(np.diag(C)))
log_w = -(0.5 * np.einsum("si,ij,sj->s", X, H, X) - X @ b) - log_q
print(approximations.pareto_khat(log_w))
```

In five repetitions, $\hat{k}$ is between 1.8 and 2.1 for the mean-field approximation, with effective sample sizes between 1 and 10 of 4000, and between 0.17 and 0.46 for the full-rank approximation, with effective sample sizes between 1350 and 1780. By Theorem 5.6b, the mean-field Gaussian

has the variances $1/H_{ii}$, which are too small, and in the coordinates $\boldsymbol{u} = \boldsymbol{D}^{1/2}(\boldsymbol{x} - \boldsymbol{\mu}_{\text{post}})$ of q, $\boldsymbol{D} = \operatorname{diag}(H_{11}, \dots, H_{nn})$, the weights are $w \propto \exp(\frac{1}{2}\boldsymbol{u}^\mathsf{T}(\boldsymbol{I} - \boldsymbol{D}^{-1/2}\boldsymbol{H}\boldsymbol{D}^{-1/2})\boldsymbol{u})$ with $\boldsymbol{u} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. The matrix $\boldsymbol{D}^{-1/2}\boldsymbol{H}\boldsymbol{D}^{-1/2}$ has unit diagonal and, here, eigenvalues between 0.05 and 2; in the direction of the smallest eigenvalue $\mu_{\min}$, the weight is $e^{(1-\mu_{\min})u^2/2}$ with $u \sim \mathcal{N}(0, 1)$, whose tail is that of a Pareto distribution with the shape $1 - \mu_{\min} = 0.95$. The weights have an infinite variance, and the estimate $\hat{k} > 1$ says that importance sampling with this q is hopeless; in 128 dimensions and with 4000 draws, the estimate exceeds the limit 0.95. For the full-rank approximation, $\boldsymbol{C}^\mathsf{T}\boldsymbol{H}\boldsymbol{C}$ has eigenvalues between 0.79 and 1.26: q is almost the posterior, slightly too narrow in a few directions, which predicts $k = 1 - 0.79 = 0.21$, and the approximation passes the check.

Exercise 5.4. (a) Up to constants, with $\boldsymbol{z} = \boldsymbol{L}\boldsymbol{x}$ and the level prior $\mathcal{N}(x_1; 0, \gamma_0^2)$,

$$\log p(\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{\theta}) = -\frac{\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2}{2\sigma^2} - \frac{x_1^2}{2\gamma_0^2} - \sum_i \Big(\frac{z_i^2}{2\theta_i} + (\beta + \tfrac{3}{2}) \log \theta_i + \frac{\vartheta}{\theta_i} \Big).$$

The average over $q(\boldsymbol{\theta})$ replaces $1/\theta_i$ by $\mathbb{E}[1/\theta_i]$ and turns the $\log \theta_i$ terms into constants:

$$\log q(\boldsymbol{x}) = -\frac{\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{y}\|_2^2}{2\sigma^2} - \frac{x_1^2}{2\gamma_0^2} - \frac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{L}^\mathsf{T}\boldsymbol{D}\boldsymbol{L}\boldsymbol{x} + \text{const},$$

a quadratic function of $\boldsymbol{x}$ with the Hessian $-(\boldsymbol{A}^\mathsf{T}\boldsymbol{A}/\sigma^2 + \boldsymbol{L}^\mathsf{T}\boldsymbol{D}\boldsymbol{L} + \boldsymbol{e}_1\boldsymbol{e}_1^\mathsf{T}/\gamma_0^2)$ and the stationary point $\boldsymbol{m} = \boldsymbol{\Sigma}\boldsymbol{A}^\mathsf{T}\boldsymbol{y}/\sigma^2$, which is the $\boldsymbol{x}$-step. The average over $q(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$ replaces z_i^2 by $\mathbb{E}[z_i^2] = (\boldsymbol{L}\boldsymbol{m})_i^2 + (\boldsymbol{L}\boldsymbol{\Sigma}\boldsymbol{L}^\mathsf{T})_{ii}$:

$$\log q(\boldsymbol{\theta}) = -\sum_i \Big((\beta + \tfrac{3}{2}) \log \theta_i + \frac{\vartheta + \mathbb{E}[z_i^2]/2}{\theta_i} \Big) + \text{const},$$

a product of the densities $\text{InvGamma}(\beta + \frac{1}{2}, \vartheta + \frac{1}{2}\mathbb{E}[z_i^2])$ by (3.11), since $\beta + \frac{3}{2} = (\beta + \frac{1}{2}) + 1$. That $q(\boldsymbol{\theta})$ factorizes over i is a result, not an assumption. For $\theta \sim \text{InvGamma}(a, b)$, $1/\theta \sim \text{Gamma}(a, b)$ with rate b, whose mean is a/b, which gives $\mathbb{E}[1/\theta_i]$. (b) The call is `m, C, (a, b), its = approximations.cavi_hierarchical(A, y_p, sigma, 1.0, vartheta)`, with the data of Example 4.14; the band is $\boldsymbol{m} \pm 1.96\sqrt{\operatorname{diag} \boldsymbol{C}}$, and the probability of a loud increment is computed from the Gaussian marginals of $\boldsymbol{L}\boldsymbol{x}$. For $\vartheta = 10^{-3}$, it reproduces Example 5.11: 38 iterations in 0.02 seconds, the error 0.61, at most 0.02 from the IAS solution, a coverage of 93 percent, bands 0.16 to 0.24 wide at the five jumps, and standard deviations between 0.17 and 0.73 times those of the Gibbs sampler. The comparison for the three values, with the Gibbs sampler of Exercise 4.4c, whose runs use the same seed:

	$\vartheta = 10^{-4}$	$\vartheta = 10^{-3}$	$\vartheta = 10^{-2}$
iterations of variational Bayes	228	38	99
error of the mean, VB	0.88	0.61	0.94
Gibbs	0.87	0.97	1.32
coverage of the 95 % band, VB	84 %	93 %	96 %
Gibbs	98 %	98 %	99 %
band width at the jumps, VB	0.09 to 0.12	0.16 to 0.24	0.36 to 0.55
Gibbs	0.24 to 1.05	0.26 to 1.09	0.52 to 1.10
standard deviations, VB / Gibbs	0.07 to 0.76	0.17 to 0.73	0.49 to 0.71
$\mathbb{P}(\lvert z_i \rvert > 0.1)$ at the small step, VB	0.00	0.01	0.30
Gibbs	0.10	0.10	0.39

Variational Bayes does not find the small step for any ϑ: its mean is a ramp there, like the mean of the Gibbs sampler, and for $\vartheta = 10^{-2}$ the probability of a loud increment near the step is not larger than elsewhere for this weak sparsity prior. Its bands are narrower than the Gibbs

bands everywhere, most of all at the jumps, and the overconfidence grows as ϑ decreases, i.e., as the prior separates quiet from loud increments more sharply and the coupling of an increment and its variance, which the factorization $q(\boldsymbol{x})\,q(\boldsymbol{\theta})$ ignores, becomes stronger. For $\vartheta = 10^{-4}$, the band misses the whole plateau of height 0.5 and the small step, and variational Bayes ends 0.6 away from the IAS solution, at another stationary point. (c) With these two changes, $\mathbb{E}[1/\theta_i] = (\beta + \frac{3}{2})/(\vartheta + (\boldsymbol{L}\boldsymbol{m})_i^2/2)$ is the inverse of the $\boldsymbol{\theta}$-step of Algorithm 3.14 applied to $\boldsymbol{m}$, the covariance $\boldsymbol{\Sigma}$ no longer enters the update, and the $\boldsymbol{x}$-step computes $\boldsymbol{m}$ as the weighted Tikhonov solution of Algorithm 3.14: the result is the IAS, which iterates the mean alone. Numerically, the modified iteration reproduces `solvers.ias` to 10^{-8}.

Exercise 5.5. (a) With the smooth deconvolution data and the precision `P_s` of Example 3.12:

```
Cp = np.linalg.cholesky(np.linalg.inv(P_s))          # prior N(0, P_s^{-1})
for J in (50, 100, 200, 500, 1000, 5000):
    X0 = rng.standard_normal((J, n)) @ Cp.T          # prior ensemble
    X1 = approximations.ensemble_kalman_step(X0, X0 @ A.T, y_s, sigma, rng)
    print(J, np.max(np.abs(X1.mean(0) - mean_s) / sd_s),
          np.max(np.abs(X1.std(0, ddof=1) / sd_s - 1)))
```

Averaged over ten repetitions, the largest error of the mean is 3.7, 2.7, 1.7, 1.2, 0.80, and 0.35 posterior standard deviations for the six ensemble sizes, and the largest relative error of the standard deviations is 0.34, 0.21, 0.15, 0.093, 0.061, and 0.027. Both decay like $J^{-1/2}$, about $25\,J^{-1/2}$ and $2\,J^{-1/2}$, which confirms Theorem 5.13 in the limit $J \to \infty$. The second constant is the sampling error $(2J)^{-1/2}$ of a standard deviation, up to three times that over 128 components. The first is large because the prior standard deviations are 17 to 28 times the posterior ones: the ensemble starts with sampling errors of order $J^{-1/2}$ on the scale of the prior, and the update, whose gain is estimated from the same members, does not remove them. (b) Let $\boldsymbol{Q}$ have orthonormal columns that span the $J - 1 = 9$ deviations $\boldsymbol{x}_0^{(j)} - \bar{\boldsymbol{x}}_0$ (by a QR factorization), and measure $\max_j \left\|(\boldsymbol{I} - \boldsymbol{Q}\boldsymbol{Q}^\mathsf{T})(\boldsymbol{x}^{(j)} - \bar{\boldsymbol{x}}_0)\right\|_2 / \left\|\boldsymbol{x}^{(j)} - \bar{\boldsymbol{x}}_0\right\|_2$. After 50 iterations of `eki` on the deconvolution problem, it is $5 \cdot 10^{-14}$, and after eight updates of `es_mda` or 20 iterations of `eki` on the bathymetry problem, which is nonlinear, $5 \cdot 10^{-15}$: rounding errors. In the linear case the ensemble has collapsed to a spread of 0.002 around a mean that fits the data only to 0.79, twice the noise level 0.4: nine directions are not enough for 128 unknowns. (c) With the functions `bathy`, `bathy_hmc`, and `prior_ensemble` of `ch05_figures.py`:

```
d = bathy(20)
X0, tries = prior_ensemble(d, 400, rng)              # feasible prior draws
X, n_fwd, _ = approximations.es_mda(d["F"], X0, d["y"], 0.01, 8, rng)
print(X.mean(0), X.std(0, ddof=1))                   # compare with HMC
X, recs, n_fwd = approximations.eki(d["F"], X0[:200], d["y"], 0.01, 40, rng,
                                    prior_factor=d["Lw"])   # Tikhonov EKI
```

With the seeds of `fig_enkbathy`, the numbers of Example 5.15 are reproduced; the HMC reference has the smallest effective sample size 2551. For 200 sensors, `bathy(200)` and an HMC reference of the same length give, with the same seeds:

	20 sensors		200 sensors	
	error of the mean	standard deviations	error of the mean	standard deviations
one update, $J = 400$	1.6	0.94 to 1.97	2.5	0.93 to 3.09
ES-MDA, $J = 50$	2.8	0.68 to 1.19	3.1	0.52 to 0.98
ES-MDA, $J = 200$	0.87	0.87 to 1.13	1.19	0.80 to 1.12
ES-MDA, $J = 400$	0.57	0.90 to 1.08	1.01	0.86 to 1.06

The error of the mean is the largest difference from the HMC mean in HMC standard deviations, and the standard deviations are divided by those of HMC. With ten times more data, 24 instead

of 8 eigenvalues of Exercise 5.1c exceed one, the posterior standard deviations drop from 0.9–3.1 to 0.9–2.5 cm, and the posterior is even closer to Gaussian: the Laplace approximation, for the same cost, has standard deviations within 0.97 and 1.04 of HMC and a mean 0.09 HMC standard deviations from the HMC mean. Gauss–Newton with finite differences reaches the MAP estimate to 0.007 cm after three iterations and 610 forward solves, as before. The ensembles get worse. They start from the same prior, so that the straight lines through the forward values of the members are as wrong as before, while the target is narrower and the errors are measured on its smaller scale; a single update overestimates the standard deviations upstream by a factor of three. With $J = 50$, the ensemble now has to represent 24 informed directions with 49 deviations and estimates the 200×200 covariance $\boldsymbol{C}^{ff}$ from them: spurious correlations between the members and the data reduce the spread too much, and the standard deviations are underestimated by up to a factor of two, the collapse that localization and inflation counteract (Remark 5.16). Tikhonov EKI ends 1.15, 1.03, and 0.29 cm from the MAP estimate for $J = 50$, 100, and 200 after 40 iterations, against 2.2, 1.4, and 0.54 cm with 20 sensors, and plain EKI with $J = 200$ fits the data to the noise level 14.1 cm after two iterations, as before, and overfits to 10.1 cm after 30. More data favor the Laplace approximation and Gauss–Newton over the ensembles. (d) A draw $\boldsymbol{b}$ has a subcritical steady state if and only if $\max_i b_i \le E/g - \frac{3}{2}h_c = 0.362\,\mathrm{m}$, with the energy E of the outflow and the critical depth $h_c = (q^2/g)^{1/3}$, since the specific energy $h + q^2/(2gh^2)$ is at least $\frac{3}{2}h_c$; this criterion agrees for every draw with the forward map `F = d["F"]`, which returns NaN if some cell has no subcritical steady state. Of the 10^4 draws of `fig_enkbathy` from the prior with $\gamma = 3$ cm per cell, 36.7 ± 0.5 percent violate it, and of those with $\gamma = 1$ cm, 0.85 ± 0.09 percent:

```
R = np.linalg.inv(solvers.difference_matrix(200)[:, :-1] / 0.03)
B = rng.standard_normal((1000, 199)) @ R.T            # prior draws, b_200 = 0
bad = [np.isnan(F(b)).all() for b in B]              # no steady state somewhere
plt.plot(xg[:-1], B[bad].T, color="tab:red", lw=0.4)   # xg: cell centers
```

The plot shows random walks pinned to zero at the outflow that rise above 0.36 m somewhere; half of them have their maximum within 5 m of the inflow, where the prior standard deviation is 42 cm. The prior thus gives more than a third of its probability to bottoms for which the forward model has no solution: the statement "the bottom changes by 3 cm per cell" is incompatible with the physics over 199 cells, and the effective prior is the Gaussian truncated to feasible bottoms, a truncation that nobody wrote down. The posterior is hardly affected, since the data exclude such bottoms, but every method that draws from the prior is: ensemble methods, elliptical slice sampling with the prior as reference, prior predictive checks, and evidence computations. A smaller γ, as with 1 cm, or a prior that bounds the bottom, removes the conflict.

Exercise 5.6. This exercise is about your own problem and has no model solution.

Bibliography

[1] S. Agrawal, H. Kim, D. Sanz-Alonso, and A. Strang. A variational inference approach to inverse problems with gamma hyperpriors. *SIAM/ASA Journal on Uncertainty Quantification*, 10(4):1533–1559, 2022. doi: 10.1137/21M146209X.

[2] A. Alexanderian. Optimal experimental design for infinite-dimensional Bayesian inverse problems governed by PDEs: a review. *Inverse Problems*, 37(4):043001, 2021. doi: 10.1088/1361-6420/abe10c.

[3] D. F. Andrews and C. L. Mallows. Scale mixtures of normal distributions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(1):99–102, 1974. doi: 10.1111/j.2517-6161.1974.tb00989.x.

[4] J. M. Bardsley, A. Solonen, H. Haario, and M. Laine. Randomize-then-optimize: A method for sampling from posterior distributions in nonlinear inverse problems. *SIAM Journal on Scientific Computing*, 36(4):A1895–A1910, 2014. doi: 10.1137/140964023.

[5] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018. URL https://jmlr.org/papers/v18/17-468.html.

[6] T. Bayes. An essay towards solving a problem in the doctrine of chances. *Philosophical Transactions of the Royal Society of London*, 53:370–418, 1763. doi: 10.1098/rstl.1763.0053. Communicated by Richard Price.

[7] A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009. doi: 10.1137/080716542.

[8] A. Beskos, N. Pillai, G. Roberts, J.-M. Sanz-Serna, and A. Stuart. Optimal tuning of the hybrid Monte Carlo algorithm. *Bernoulli*, 19(5A):1501–1534, 2013. doi: 10.3150/12-BEJ414.

[9] A. Beskos, M. Girolami, S. Lan, P. E. Farrell, and A. M. Stuart. Geometric MCMC for infinite-dimensional inverse problems. *Journal of Computational Physics*, 335:327–351, 2017. doi: 10.1016/j.jcp.2016.12.041.

[10] M. Betancourt. A conceptual introduction to Hamiltonian Monte Carlo. arXiv preprint arXiv:1701.02434, 2017.

[11] C. M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, New York, 2006.

[12] C. M. Bishop and M. E. Tipping. Variational relevance vector machines. In *Proceedings of the 16th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 46–53. Morgan Kaufmann, 2000.

[13] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017. doi: 10.1080/01621459.2017.1285773.

[14] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. doi: 10.1561/2200000016.

[15] T. Bui-Thanh, O. Ghattas, J. Martin, and G. Stadler. A computational framework for infinite-dimensional Bayesian inverse problems Part I: The linearized case, with application to global seismic inversion. *SIAM Journal on Scientific Computing*, 35(6):A2494–A2523, 2013. doi: 10.1137/12089586X.

[16] G. Burgers, P. J. van Leeuwen, and G. Evensen. Analysis scheme in the ensemble Kalman filter. *Monthly Weather Review*, 126(6):1719–1724, 1998. doi: 10.1175/1520-0493(1998)126<1719:ASITEK>2.0.CO;2.

[17] A. Cabezas, A. Corenflos, J. Lao, R. Louf, et al. BlackJAX: Composable Bayesian inference in JAX. arXiv preprint arXiv:2402.10797, 2024.

[18] E. Calvello, S. Reich, and A. M. Stuart. Ensemble Kalman methods: A mean-field perspective. *Acta Numerica*, 34:123–291, 2025. doi: 10.1017/S0962492924000060.

[19] D. Calvetti and E. Somersalo. *Bayesian Scientific Computing*, volume 215 of *Applied Mathematical Sciences*. Springer, Cham, 2023. doi: 10.1007/978-3-031-23824-6.

[20] D. Calvetti, E. Somersalo, and A. Strang. Hierachical Bayesian models and sparsity: ℓ_2-magic. *Inverse Problems*, 35(3):035003, 2019. doi: 10.1088/1361-6420/aaf5ab.

[21] D. Calvetti, M. Pragliola, E. Somersalo, and A. Strang. Sparse reconstructions from few noisy data: analysis of hierarchical Bayesian models with generalized gamma hyperpriors. *Inverse Problems*, 36(2):025010, 2020. doi: 10.1088/1361-6420/ab4d92.

[22] N. K. Chada, A. M. Stuart, and X. T. Tong. Tikhonov regularization within ensemble Kalman inversion. *SIAM Journal on Numerical Analysis*, 58(2):1263–1294, 2020. doi: 10.1137/19M1242331.

[23] Y. Chen and D. S. Oliver. Ensemble randomized maximum likelihood method as an iterative ensemble smoother. *Mathematical Geosciences*, 44(1):1–26, 2012. doi: 10.1007/s11004-011-9376-z.

[24] S. R. Cook, A. Gelman, and D. B. Rubin. Validation of software for Bayesian models using posterior quantiles. *Journal of Computational and Graphical Statistics*, 15(3):675–692, 2006. doi: 10.1198/106186006X136976.

[25] S. L. Cotter, G. O. Roberts, A. M. Stuart, and D. White. MCMC methods for functions: Modifying old algorithms to make them faster. *Statistical Science*, 28(3):424–446, 2013. doi: 10.1214/13-STS421.

[26] K. Cranmer, J. Brehmer, and G. Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, 2020. doi: 10.1073/pnas.1912789117.

[27] T. Cui, K. J. H. Law, and Y. M. Marzouk. Dimension-independent likelihood-informed MCMC. *Journal of Computational Physics*, 304:109–137, 2016. doi: 10.1016/j.jcp.2015.10.008.

[28] P. Del Moral, A. Doucet, and A. Jasra. Sequential Monte Carlo samplers. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(3):411–436, 2006. doi: 10.1111/j.1467-9868.2006.00553.x.

[29] S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth. Hybrid Monte Carlo. *Physics Letters B*, 195(2):216–222, 1987. doi: 10.1016/0370-2693(87)91197-X.

[30] A. A. Emerick and A. C. Reynolds. Ensemble smoother with multiple data assimilation. *Computers & Geosciences*, 55:3–15, 2013. doi: 10.1016/j.cageo.2012.03.011.

[31] H. W. Engl, M. Hanke, and A. Neubauer. *Regularization of Inverse Problems*. Kluwer, Dordrecht, 1996.

[32] O. G. Ernst, B. Sprungk, and H.-J. Starkloff. Analysis of the ensemble and polynomial chaos Kalman filters in Bayesian inverse problems. *SIAM/ASA Journal on Uncertainty Quantification*, 3(1):823–851, 2015. doi: 10.1137/140981319.

[33] G. Evensen. *Data Assimilation: The Ensemble Kalman Filter*. Springer, Berlin, 2nd edition, 2009. doi: 10.1007/978-3-642-03711-5.

[34] H. P. Flath, L. C. Wilcox, V. Akçelik, J. C. Hill, B. van Bloemen Waanders, and O. Ghattas. Fast algorithms for Bayesian uncertainty quantification in large-scale linear inverse problems based on low-rank partial Hessian approximations. *SIAM Journal on Scientific Computing*, 33(1):407–432, 2011. doi: 10.1137/090780717.

[35] A. Garbuno-Inigo, F. Hoffmann, W. Li, and A. M. Stuart. Interacting Langevin diffusions: Gradient structure and ensemble Kalman sampler. *SIAM Journal on Applied Dynamical Systems*, 19(1):412–441, 2020. doi: 10.1137/19M1251655.

[36] A. Gelman and D. B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992. doi: 10.1214/ss/1177011136.

[37] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin. *Bayesian Data Analysis*. CRC Press, Boca Raton, 3rd edition, 2013. doi: 10.1201/b16018.

[38] S. Geman and D. Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-6(6):721–741, 1984. doi: 10.1109/TPAMI.1984.4767596.

[39] A. F. Gessese, M. Sellier, E. Van Houten, and G. Smart. Reconstruction of river bed topography from free surface data using a direct numerical approach in one-dimensional shallow water flow. *Inverse Problems*, 27(2):025001, 2011. doi: 10.1088/0266-5611/27/2/025001.

[40] C. J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992. doi: 10.1214/ss/1177011137.

[41] C. Gilavert, S. Moussaoui, and J. Idier. Efficient Gaussian sampling for solving large-scale inverse problems using MCMC. *IEEE Transactions on Signal Processing*, 63(1):70–80, 2015. doi: 10.1109/TSP.2014.2367457.

[42] M. B. Giles. Multilevel Monte Carlo methods. *Acta Numerica*, 24:259–328, 2015. doi: 10.1017/S096249291500001X.

[43] J. Glaubitz and A. Gelb. Leveraging joint sparsity in hierarchical Bayesian learning. *SIAM/ASA Journal on Uncertainty Quantification*, 12(2):442–472, 2024. doi: 10.1137/23M156255X.

[44] J. Glaubitz and Y. Marzouk. Efficient sampling for sparse Bayesian learning using hierarchical prior normalization. *SIAM/ASA Journal on Uncertainty Quantification*, 14 (3):829–857, 2026. doi: 10.1137/25M1790427. Code: https://github.com/jglaubitz/paper-2025-SBL-priorNormalization.

[45] J. Glaubitz, A. Gelb, and G. Song. Generalized sparse Bayesian learning and application to image reconstruction. *SIAM/ASA Journal on Uncertainty Quantification*, 11(1):262–284, 2023. doi: 10.1137/22M147236X.

[46] J. Glaubitz, T. Li, J. Ryan, and R. Stuhlmacher. The Bayesian SIAC filter. *SIAM Journal on Scientific Computing*, 48(5):A2490–A2516, 2026. doi: 10.1137/25M1800433.

[47] J. Glaubitz, D. Sharp, M. Le Provost, and Y. M. Marzouk. Regularity-informed data assimilation: A hierarchical Bayesian approach to ensemble Kalman filtering for hyperbolic conservation laws. arXiv preprint arXiv:2608.14840, 2026.

[48] G. H. Golub, M. Heath, and G. Wahba. Generalized cross-validation as a method for choosing a good ridge parameter. *Technometrics*, 21(2):215–223, 1979. doi: 10.1080/00401706.1979.10489751.

[49] N. Goutal and F. Maurel. Proceedings of the 2nd workshop on dam-break wave simulation. Technical Report HE-43/97/016/A, EDF, Département Laboratoire National d'Hydraulique, 1997.

[50] R. Gribonval. Should penalized least squares regression be interpreted as maximum a posteriori estimation? *IEEE Transactions on Signal Processing*, 59(5):2405–2410, 2011. doi: 10.1109/TSP.2011.2107908.

[51] A. Griewank and A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, Philadelphia, 2nd edition, 2008. doi: 10.1137/1.9780898717761.

[52] J. Habscheid and J. Glaubitz. Edge-preserving Bayesian neural networks via hierarchical sparsity-promoting priors. arXiv preprint arXiv:2610.17023, 2026.

[53] J. Hadamard. *Lectures on Cauchy's Problem in Linear Partial Differential Equations*. Yale University Press, New Haven, 1923.

[54] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53 (2):217–288, 2011. doi: 10.1137/090771806.

[55] P. C. Hansen. Analysis of discrete ill-posed problems by means of the L-curve. *SIAM Review*, 34(4):561–580, 1992. doi: 10.1137/1034115.

[56] P. C. Hansen. *Discrete Inverse Problems: Insight and Algorithms*. SIAM, Philadelphia, 2010. doi: 10.1137/1.9780898718836.

[57] W. K. Hastings. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. doi: 10.1093/biomet/57.1.97.

[58] T. Helin and R. Kretschmann. Non-asymptotic error estimates for the Laplace approximation in Bayesian inverse problems. *Numerische Mathematik*, 150(2):521–549, 2022. doi: 10.1007/s00211-021-01266-9.

[59] M. D. Hoffman and A. Gelman. The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(47):1593–1623, 2014. URL https://jmlr.org/papers/v15/hoffman14a.html.

[60] M. A. Iglesias. A regularizing iterative ensemble Kalman method for PDE-constrained inverse problems. *Inverse Problems*, 32(2):025002, 2016. doi: 10.1088/0266-5611/32/2/025002.

[61] M. A. Iglesias, K. J. H. Law, and A. M. Stuart. Ensemble Kalman methods for inverse problems. *Inverse Problems*, 29(4):045001, 2013. doi: 10.1088/0266-5611/29/4/045001.

[62] E. T. Jaynes. *Probability Theory: The Logic of Science*. Cambridge University Press, Cambridge, 2003.

[63] W. Ji and S. Deng. Autonomous discovery of unknown reaction pathways from data by chemical reaction neural network. *The Journal of Physical Chemistry A*, 125(4):1082–1092, 2021. doi: 10.1021/acs.jpca.0c09316.

[64] J. Kaipio and E. Somersalo. *Statistical and Computational Inverse Problems*, volume 160 of *Applied Mathematical Sciences*. Springer, New York, 2005. doi: 10.1007/b138659.

[65] J. Kaipio and E. Somersalo. Statistical inverse problems: Discretization, model reduction and inverse crimes. *Journal of Computational and Applied Mathematics*, 198(2):493–504, 2007. doi: 10.1016/j.cam.2005.09.027.

[66] H. Kim, D. Sanz-Alonso, and A. Strang. Hierarchical ensemble Kalman methods with sparsity-promoting generalized gamma hyperpriors. *Foundations of Data Science*, 5(3):366–388, 2023. doi: 10.3934/fods.2023003.

[67] K.-T. Kim, U. Villa, M. Parno, Y. Marzouk, O. Ghattas, and N. Petra. hIPPYlib-MUQ: A Bayesian inference software framework for integration of data with complex predictive models under uncertainty. *ACM Transactions on Mathematical Software*, 49(2):17:1–17:31, 2023. doi: 10.1145/3580278.

[68] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR); arXiv:1412.6980, 2015.

[69] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. International Conference on Learning Representations (ICLR); arXiv:1312.6114, 2014.

[70] D. C. Knill and A. Pouget. The Bayesian brain: the role of uncertainty in neural coding and computation. *Trends in Neurosciences*, 27(12):712–719, 2004. doi: 10.1016/j.tins.2004.10.007.

[71] A. Kucukelbir, D. Tran, R. Ranganath, A. Gelman, and D. M. Blei. Automatic differentiation variational inference. *Journal of Machine Learning Research*, 18(14):1–45, 2017. URL https://jmlr.org/papers/v18/16-107.html.

[72] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, pages 6402–6413, 2017.

[73] P.-S. Laplace. Mémoire sur la probabilité des causes par les événements. *Mémoires de l'Académie royale des sciences de Paris (Savants étrangers)*, 6:621–656, 1774. English translation by S. M. Stigler, Statistical Science 1 (1986), 364–378.

[74] M. Lassas and S. Siltanen. Can one use total variation prior for edge-preserving Bayesian inversion? *Inverse Problems*, 20(5):1537–1563, 2004. doi: 10.1088/0266-5611/20/5/013.

[75] J. Lindbloom, J. Glaubitz, and A. Gelb. Efficient sparsity-promoting MAP estimation for Bayesian linear inverse problems. *Inverse Problems*, 41(2):025001, 2025. doi: 10.1088/1361-6420/ada17f.

[76] J. Lindbloom, M. Pasha, J. Glaubitz, and Y. Marzouk. Priorconditioned sparsity-promoting projection methods for deterministic and Bayesian linear inverse problems. arXiv preprint arXiv:2505.01827, 2025.

[77] C. Louchet and L. Moisan. Posterior expectation of the total variation model: Properties and experiments. *SIAM Journal on Imaging Sciences*, 6(4):2640–2684, 2013. doi: 10.1137/120902276.

[78] M. Markkanen, L. Roininen, J. M. J. Huttunen, and S. Lasanen. Cauchy difference priors for edge-preserving Bayesian inversion. *Journal of Inverse and Ill-posed Problems*, 27(2):225–240, 2019. doi: 10.1515/jiip-2017-0048.

[79] J. Martin, L. C. Wilcox, C. Burstedde, and O. Ghattas. A stochastic Newton MCMC method for large-scale statistical inverse problems with application to seismic inversion. *SIAM Journal on Scientific Computing*, 34(3):A1460–A1487, 2012. doi: 10.1137/110845598.

[80] Y. Marzouk, T. Moselhy, M. Parno, and A. Spantini. Sampling via measure transport: An introduction. In R. Ghanem, D. Higdon, and H. Owhadi, editors, *Handbook of Uncertainty Quantification*, pages 785–825. Springer, Cham, 2017. doi: 10.1007/978-3-319-12385-1_23.

[81] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114.

[82] T. P. Minka. Expectation propagation for approximate Bayesian inference. In *Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 362–369, 2001.

[83] J. L. Mueller and S. Siltanen. *Linear and Nonlinear Inverse Problems with Practical Applications*. SIAM, Philadelphia, 2012. doi: 10.1137/1.9781611972344.

[84] I. Murray, R. P. Adams, and D. J. C. MacKay. Elliptical slice sampling. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of *Proceedings of Machine Learning Research*, pages 541–548, 2010. URL https://proceedings.mlr.press/v9/murray10a.html.

[85] F. Natterer. *The Mathematics of Computerized Tomography*. Classics in Applied Mathematics. SIAM, Philadelphia, 2001. doi: 10.1137/1.9780898719284.

[86] R. M. Neal. Slice sampling. *The Annals of Statistics*, 31(3):705–767, 2003. doi: 10.1214/aos/1056562461.

[87] R. M. Neal. MCMC using Hamiltonian dynamics. In S. Brooks, A. Gelman, G. L. Jones, and X.-L. Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman & Hall/CRC, Boca Raton, 2011.

[88] R. Nishihara, I. Murray, and R. P. Adams. Parallel MCMC with generalized elliptical slice sampling. *Journal of Machine Learning Research*, 15:2087–2112, 2014. URL https://jmlr.org/papers/v15/nishihara14a.html.

[89] J. Nocedal and S. J. Wright. *Numerical Optimization*. Springer, New York, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

[90] D. Norris and J. M. McQueen. Shortlist B: A Bayesian model of continuous speech recognition. *Psychological Review*, 115(2):357–395, 2008. doi: 10.1037/0033-295X.115.2.357.

[91] M. Opper and C. Archambeau. The variational Gaussian approximation revisited. *Neural Computation*, 21(3):786–792, 2009. doi: 10.1162/neco.2008.08-07-592.

[92] F. Orieux, O. Féron, and J.-F. Giovannelli. Sampling high-dimensional Gaussian distributions for general linear inverse problems. *IEEE Signal Processing Letters*, 19(5):251–254, 2012. doi: 10.1109/LSP.2012.2189104.

[93] G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed, and B. Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021. URL https://jmlr.org/papers/v22/19-1028.html.

[94] G. Papandreou and A. L. Yuille. Gaussian sampling by local perturbations. In *Advances in Neural Information Processing Systems 23 (NIPS 2010)*, pages 1858–1866, 2010.

[95] O. Papaspiliopoulos, G. O. Roberts, and M. Sköld. A general framework for the parametrization of hierarchical models. *Statistical Science*, 22(1):59–73, 2007. doi: 10.1214/088342307000000014.

[96] T. Park and G. Casella. The Bayesian lasso. *Journal of the American Statistical Association*, 103(482):681–686, 2008. doi: 10.1198/016214508000000337.

[97] A. Parker and C. Fox. Sampling Gaussian distributions in Krylov spaces with conjugate gradients. *SIAM Journal on Scientific Computing*, 34(3):B312–B334, 2012. doi: 10.1137/110831404.

[98] D. Phan, N. Pradhan, and M. Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro. arXiv preprint arXiv:1912.11554, 2019.

[99] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006. doi: 10.7551/mitpress/3206.001.0001.

[100] D. J. Rezende and S. Mohamed. Variational inference with normalizing flows. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 1530–1538, 2015. URL https://proceedings.mlr.press/v37/rezende15.html.

[101] C. P. Robert and G. Casella. *Monte Carlo Statistical Methods*. Springer, New York, 2nd edition, 2004. doi: 10.1007/978-1-4757-4145-2.

[102] G. O. Roberts and J. S. Rosenthal. Optimal scaling of discrete approximations to Langevin diffusions. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 60 (1):255–268, 1998. doi: 10.1111/1467-9868.00123.

[103] G. O. Roberts and R. L. Tweedie. Exponential convergence of Langevin distributions and their discrete approximations. *Bernoulli*, 2(4):341–363, 1996. doi: 10.2307/3318418.

[104] G. O. Roberts, A. Gelman, and W. R. Gilks. Weak convergence and optimal scaling of random walk Metropolis algorithms. *The Annals of Applied Probability*, 7(1):110–120, 1997. doi: 10.1214/aoap/1034625254.

[105] L. I. Rudin, S. Osher, and E. Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60(1–4):259–268, 1992. doi: 10.1016/0167-2789(92)90242-F.

[106] H. Rue, S. Martino, and N. Chopin. Approximate Bayesian inference for latent Gaussian models by using integrated nested Laplace approximations. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 71(2):319–392, 2009. doi: 10.1111/j.1467-9868.2008.00700.x.

[107] C. Schillings and A. M. Stuart. Analysis of the ensemble Kalman filter for inverse problems. *SIAM Journal on Numerical Analysis*, 55(3):1264–1290, 2017. doi: 10.1137/16M105959X.

[108] C. Schillings, B. Sprungk, and P. Wacker. On the convergence of the Laplace approximation and noise-level-robustness of Laplace-based Monte Carlo methods for Bayesian inverse problems. *Numerische Mathematik*, 145(4):915–971, 2020. doi: 10.1007/s00211-020-01131-1.

[109] S. A. Sisson, Y. Fan, and M. Beaumont, editors. *Handbook of Approximate Bayesian Computation*. Chapman & Hall/CRC Handbooks of Modern Statistical Methods. Chapman and Hall/CRC, 2018. doi: 10.1201/9781315117195.

[110] A. Spantini, A. Solonen, T. Cui, J. Martin, L. Tenorio, and Y. Marzouk. Optimal low-rank approximations of Bayesian linear inverse problems. *SIAM Journal on Scientific Computing*, 37(6):A2451–A2487, 2015. doi: 10.1137/140977308.

[111] L. Stietz, S. Götschel, P. Schleper, and D. Ruprecht. Bathymetry reconstruction by Bayesian inference. arXiv preprint arXiv:2604.18598, 2026.

[112] S. M. Stigler. *The History of Statistics: The Measurement of Uncertainty before 1900*. Harvard University Press, Cambridge, MA, 1986.

[113] S. M. Stigler. Laplace's 1774 memoir on inverse probability. *Statistical Science*, 1(3):359–363, 1986. doi: 10.1214/ss/1177013620.

[114] A. M. Stuart. Inverse problems: A Bayesian perspective. *Acta Numerica*, 19:451–559, 2010. doi: 10.1017/S0962492910000061.

[115] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, and A. Gelman. Validating Bayesian inference algorithms with simulation-based calibration. arXiv preprint arXiv:1804.06788, 2018.

[116] L. Tierney and J. B. Kadane. Accurate approximations for posterior moments and marginal densities. *Journal of the American Statistical Association*, 81(393):82–86, 1986. doi: 10.1080/01621459.1986.10478240.

[117] M. E. Tipping. Sparse Bayesian learning and the relevance vector machine. *Journal of Machine Learning Research*, 1:211–244, 2001. URL https://www.jmlr.org/papers/v1/tipping01a.html.

[118] A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

[119] A. Vehtari, D. Simpson, A. Gelman, Y. Yao, and J. Gabry. Pareto smoothed importance sampling. *Journal of Machine Learning Research*, 25(72):1–58, 2024. URL https://jmlr.org/papers/v25/19-556.html.

[120] Y. Xiao and J. Glaubitz. Sequential image recovery using joint hierarchical Bayesian learning. *Journal of Scientific Computing*, 96:4, 2023. doi: 10.1007/s10915-023-02234-1.

[121] Y. Yao, A. Vehtari, D. Simpson, and A. Gelman. Yes, but did it work?: Evaluating variational inference. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 5581–5590, 2018. URL https://proceedings.mlr.press/v80/yao18a.html.

[122] J. Zhang and M. A. Stephens. A new and efficient estimation method for the generalized Pareto distribution. *Technometrics*, 51(3):316–325, 2009. doi: 10.1198/tech.2009.08017.